



UE24CS352A : Machine Learning

Dr. Shylaja S S

Professor

Department of Computer Science & Engineering

Acknowledgement :

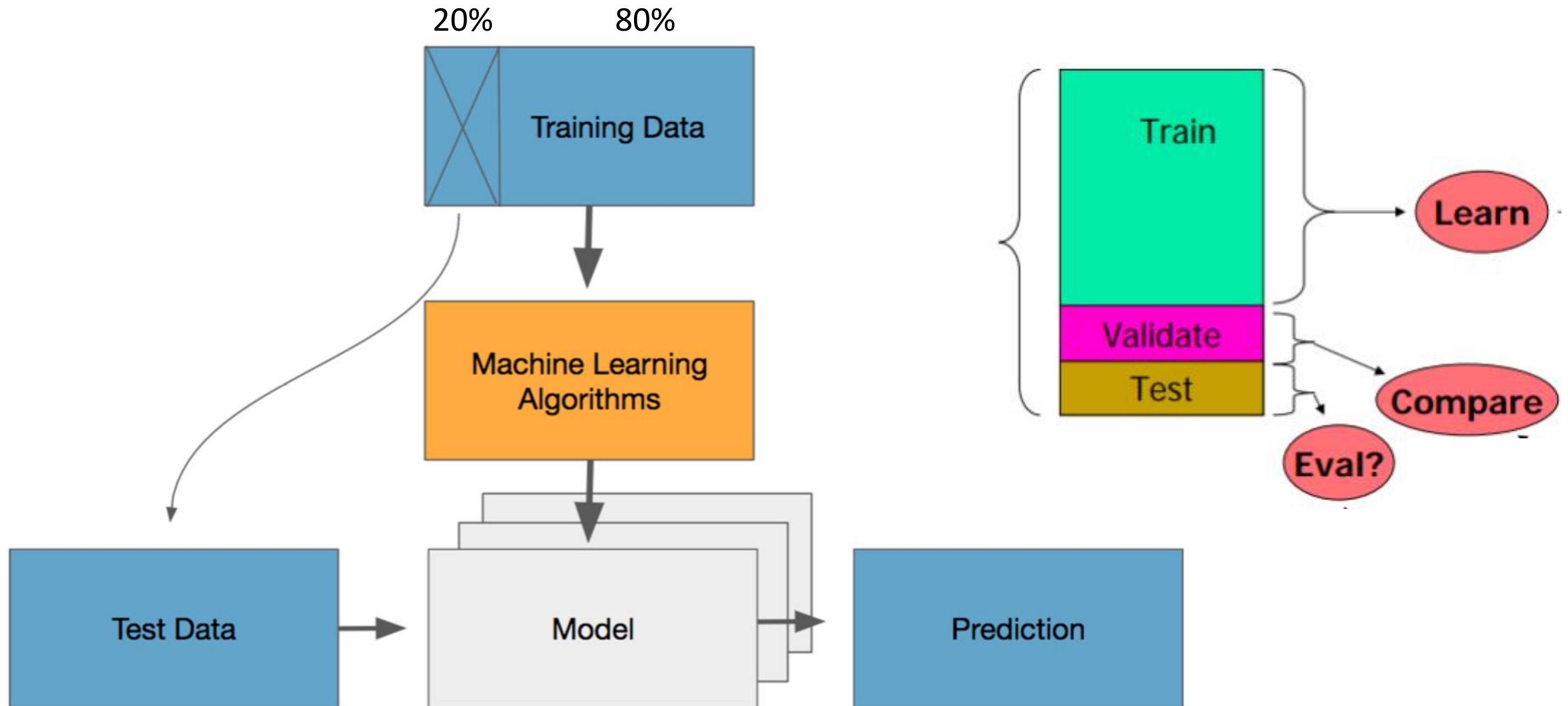
Dr. Rajanikanth K (Academic Advisor, PESU)
Teaching Assistant (Arya Rajiv Chaloli)



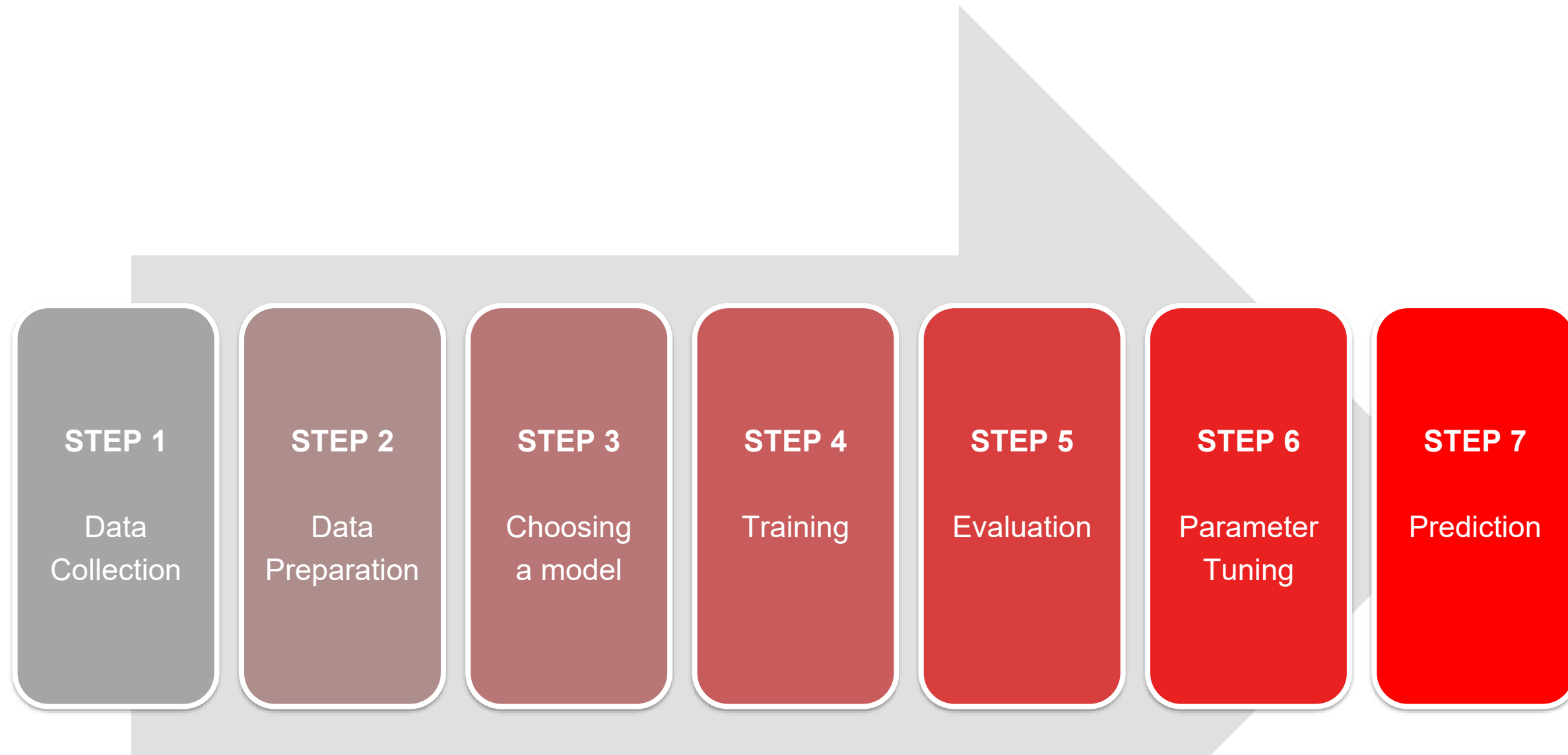
“ Machine learning is the study of computer algorithms that allow computer programs to automatically improve through experience.

~ Tom Mitchell,
Machine Learning, McGraw Hill, 1997

Carnegie Mellon University
Machine Learning



7 steps to Machine Learning



Definition by Tom Mitchell :

Machine Learning is the study of algorithms that

- improve their **performance** P
- at some **task** T
- with **experience** E (equate this to training data).

A well-defined learning task is given by $\langle P, T, E \rangle$

Identify Task(T), Training Experience (E) and Performance Measure (P) for the following problems :

Example 1 :	Learning to play Checkers Game
Example 2 :	Handwriting Recognition Learning Problem
Example 3 :	Self-driven cars (Robot driving problem)
Example 4 :	Text Categorization Problem
Example 5 :	Learning to Check for a spam
Example 6 :	Learning to detect Credit Card Fraud



Well-posed Learning Problem Examples:

Example 1 :	Learning Checkers Game
Task	Playing Checkers
Training Experience	Playing practice games against itself
Performance Measure	Fraction of games won



Well-posed Learning Problem Examples:

Example 2 :	Handwriting Recognition Learning Problem
Task	Recognizing & Classifying handwritten words with images
Training Experience	Database of handwritten words with given classification
Performance Measure	Fraction of correct words identified.



Well-posed Learning Problem Examples:

Example 3 :	Self-driven cars
Task	To drive on the road using vision sensor
Training Experience	Sequence of images & Steering commands recorded while observing human driver
Performance Measure	Average distance travelled before an error is made



Well-posed Learning Problem Examples:

Example 4 :	Text Categorization Problem
Task	Assign a document to its content category
Training Experience	Database of pre-classified documents
Performance Measure	Fraction of documents correctly tagged



Well-posed Learning Problem Examples:

Example 5 :	Learning to Check for a spam
Task	Classify the email as spam or non-spam
Training Experience	Watching the user classify emails as spam or non-spam / Database of emails labelled as spam/ non-spam
Performance Measure	Fraction of emails correctly classified.

Well-posed Learning Problem Examples:

Example 6 :	Learning to detect Credit Card Fraud
Task	Assign label of fraud or not fraud to credit card transaction
Training Experience	Historical credit card transactions labeled as fraud or not
Performance Measure	Accuracy of fraud classifier With higher penalty when fraud is labeled as not fraud

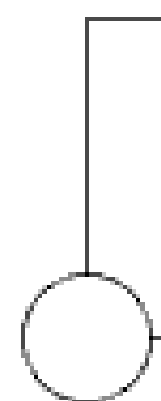
(Data with labels)

Input



Output

(Mapping)



Critic

(Data without labels)

Input



Output

(Classes)

(States and actions)

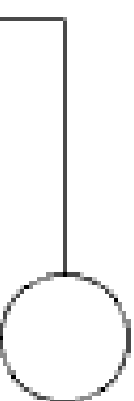
Input



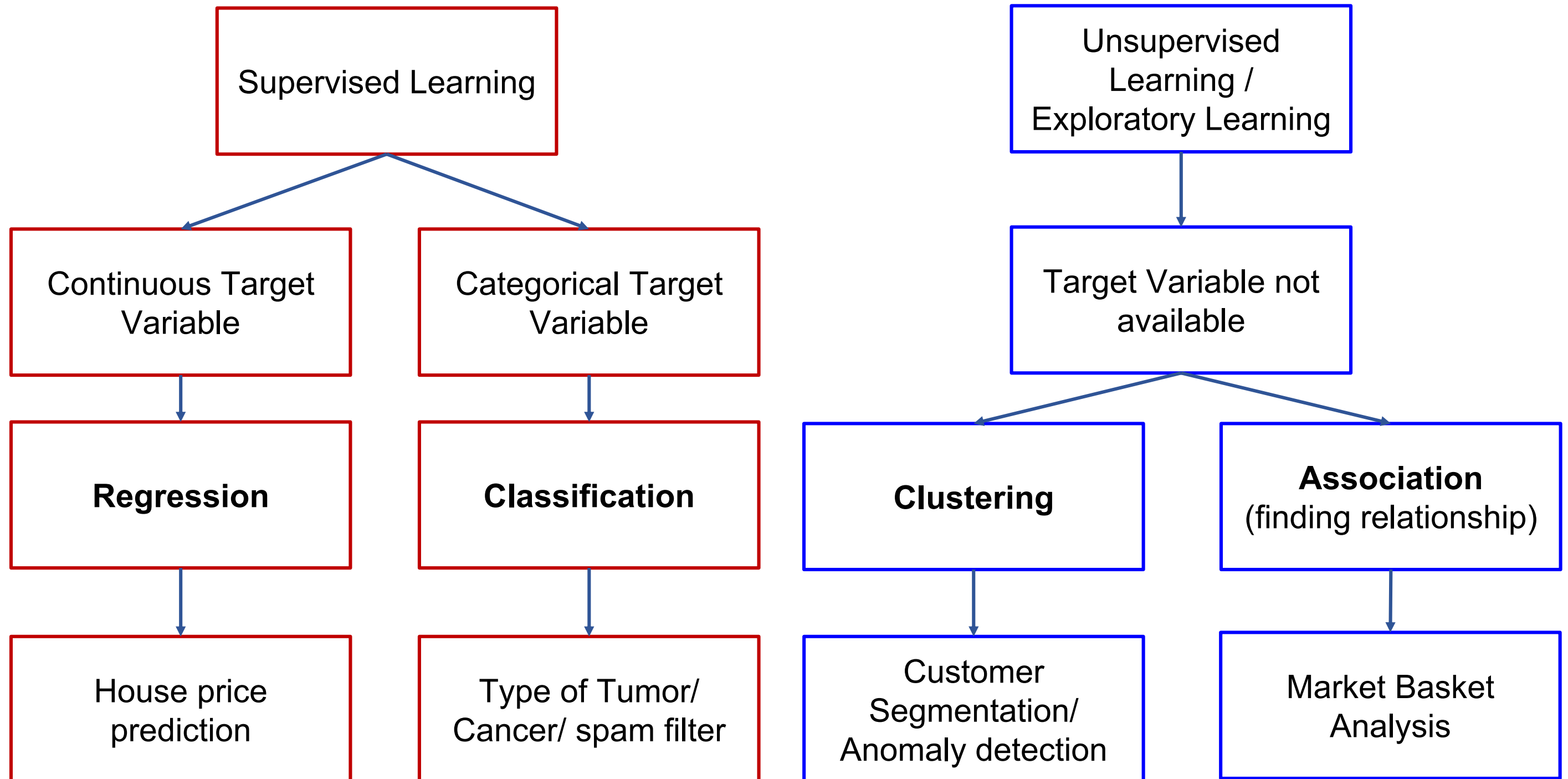
Output

(State/action)

Reinforcement
signal

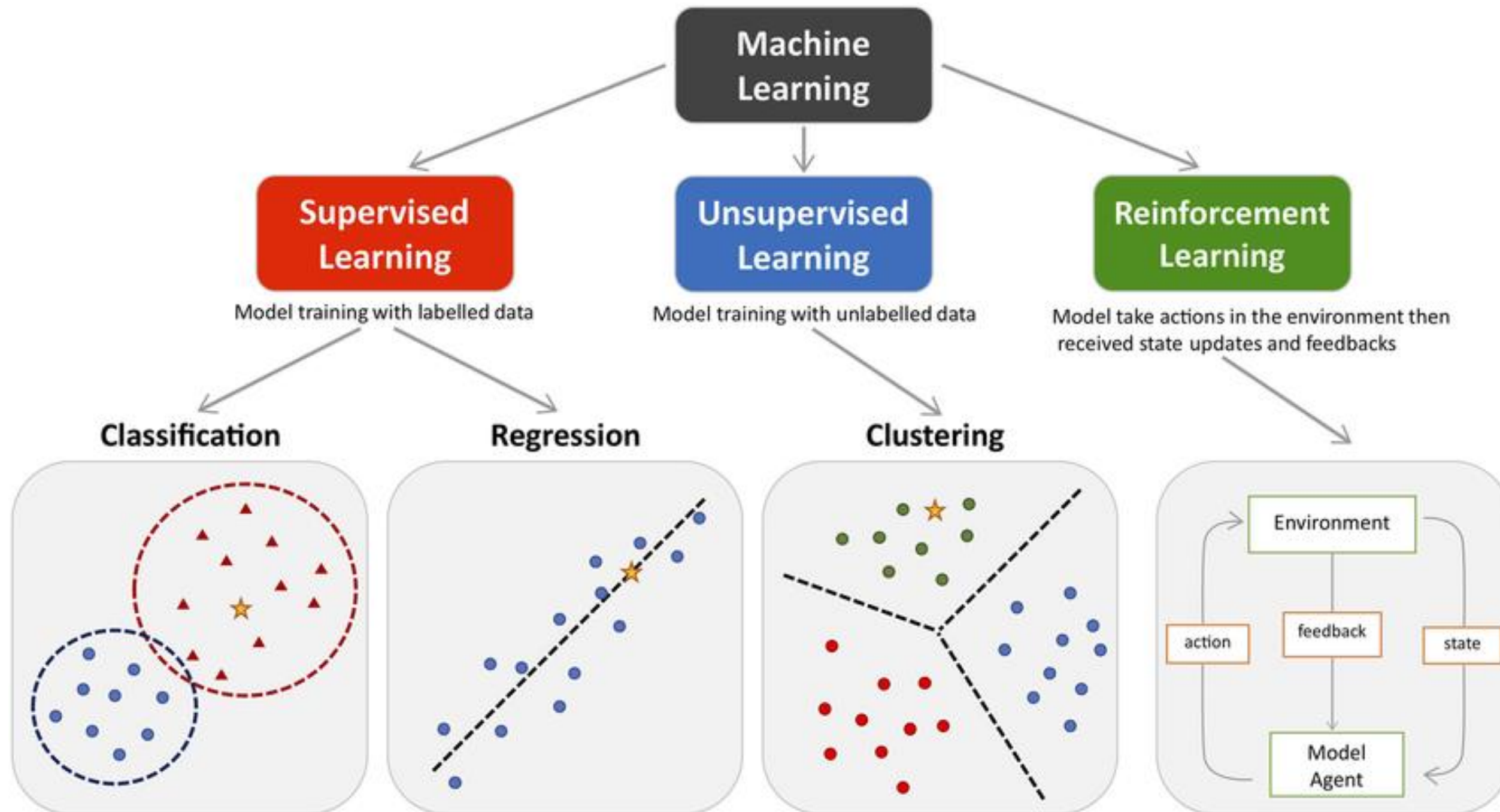


Critic



Machine Learning

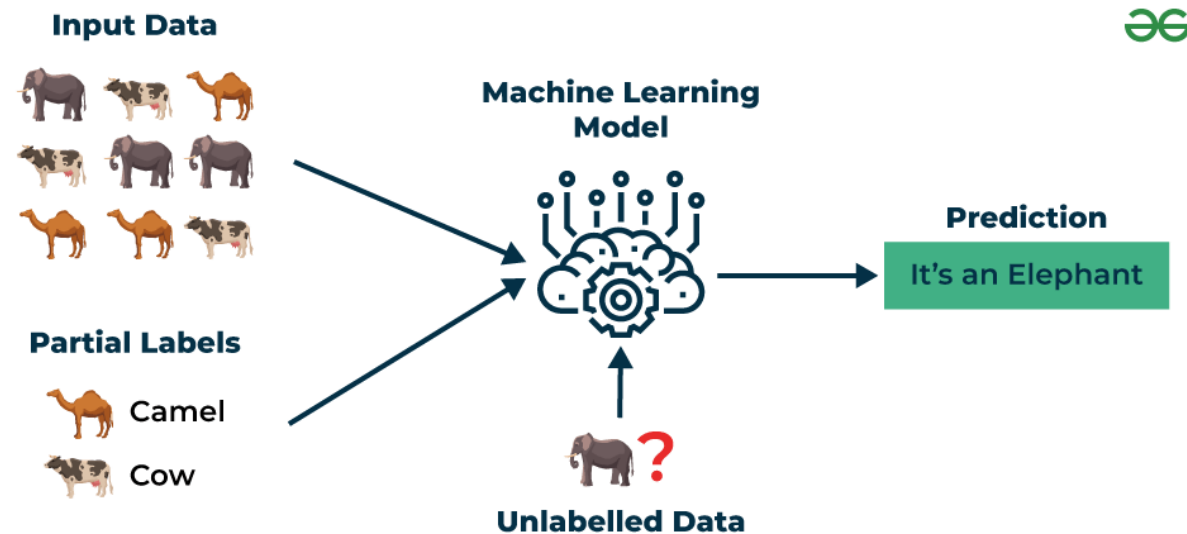
Machine Learning Models



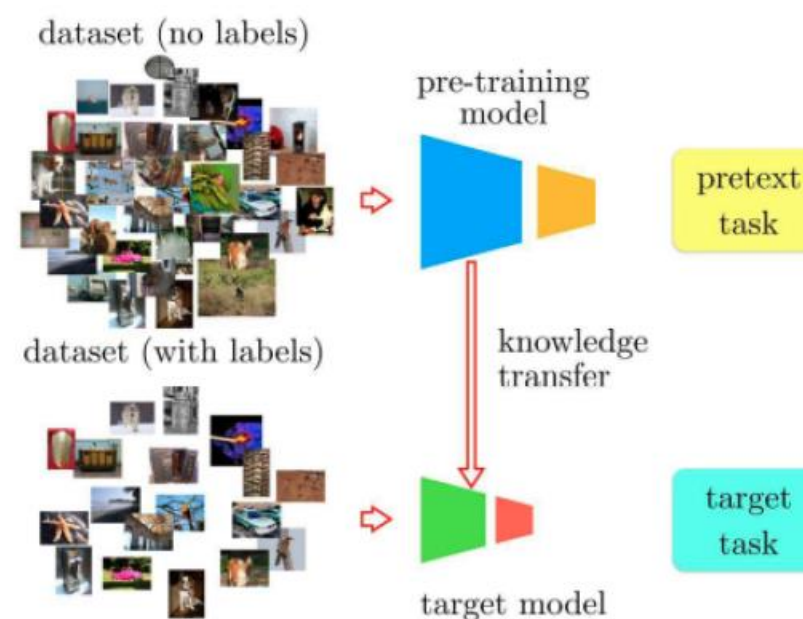
Machine Learning

Other Machine Learning Models

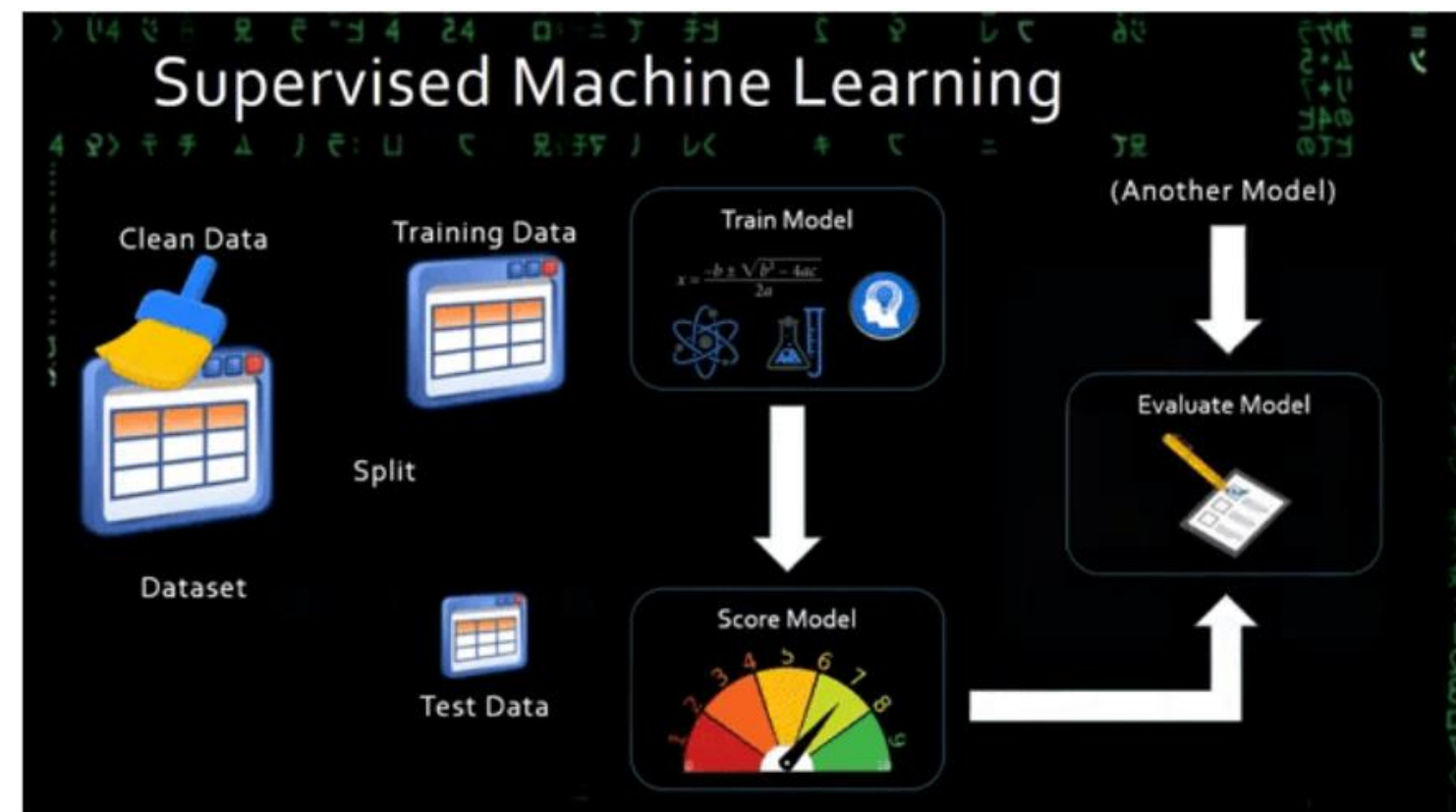
- Semi-Supervised Learning:



- Self-Supervised Learning:



- Given: training data + **desired outputs** (labels)
- Step 1:
Models are trained using **labelled dataset**, where the model learns about each type of data.
- Step 2:
The model is tested on the basis of test data and then it predicts the output



- Supervised learning is where you have input variables (x) and an output variable (Y) and you use an algorithm **to learn the mapping function from the input to the output.**

$$Y = f(X)$$

- The goal is to approximate the mapping function so well that when you have new input data (x) that you can predict the output variables (Y) for that data.
- The function approximation is also called a **hypothesis function** (which means an approximation/claim)

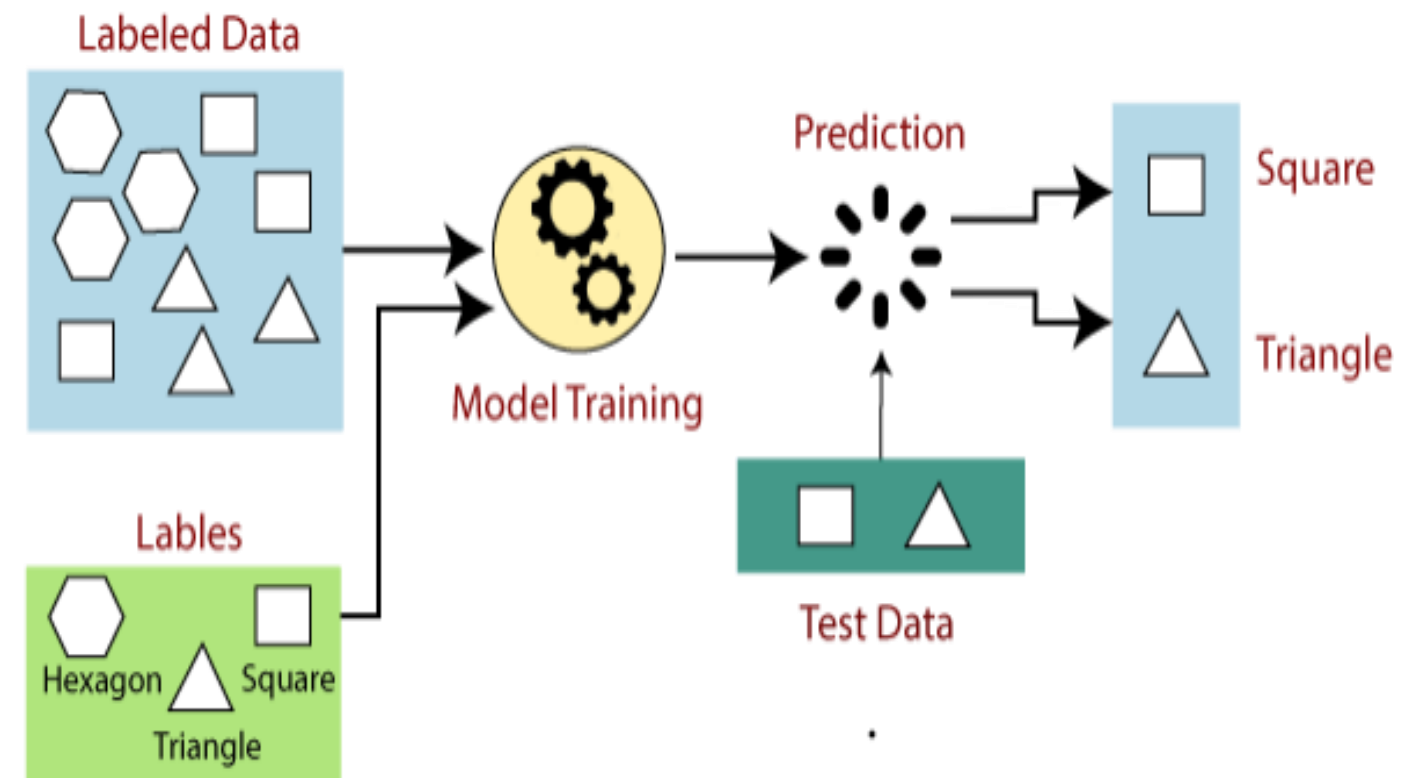
Example: Dataset of shapes which includes square, triangle, and hexagon

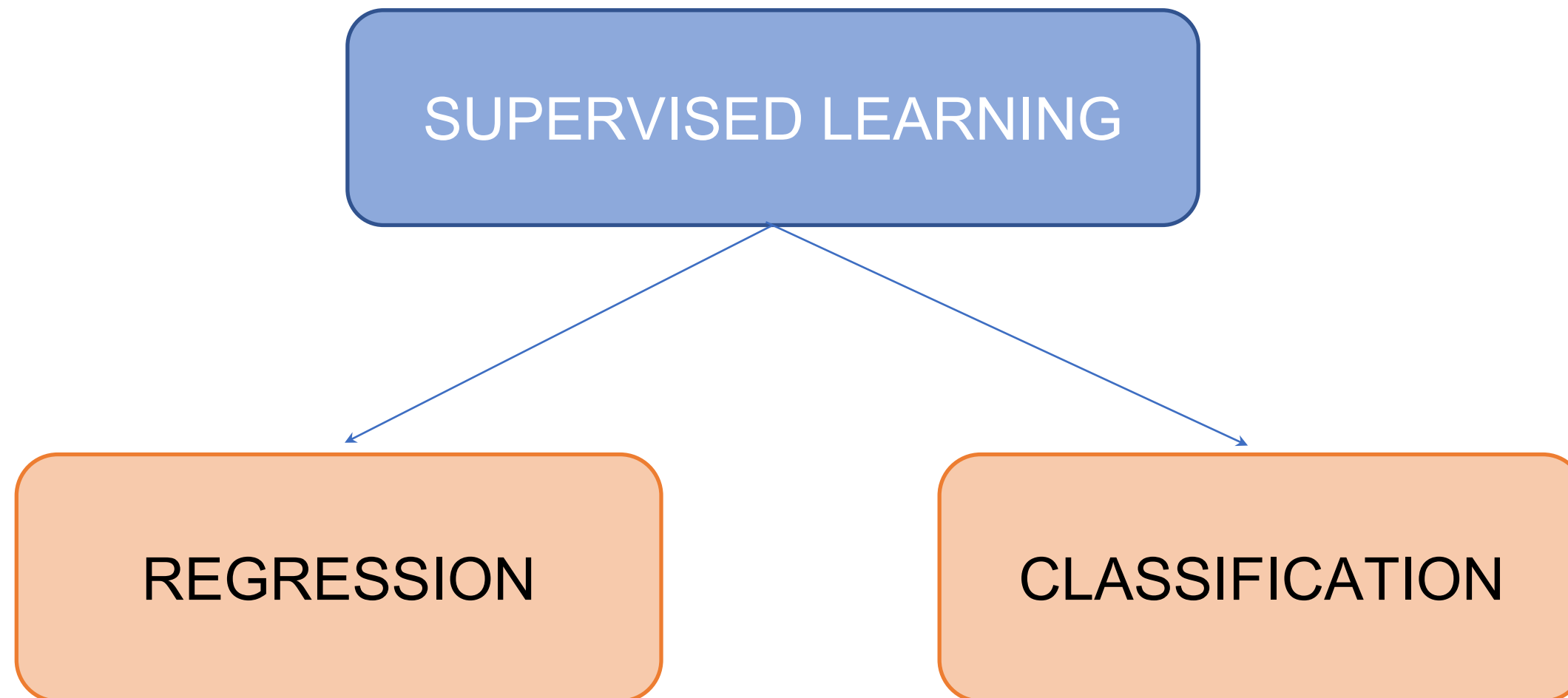
Step 1: Model Training

- If the given shape has four sides, and all the sides are equal, then it will be labelled as a Square.
- If the given shape has three sides, then it will be labelled as a triangle.
- If the given shape has six equal sides then it will be labelled as hexagon.

Step 2: Testing

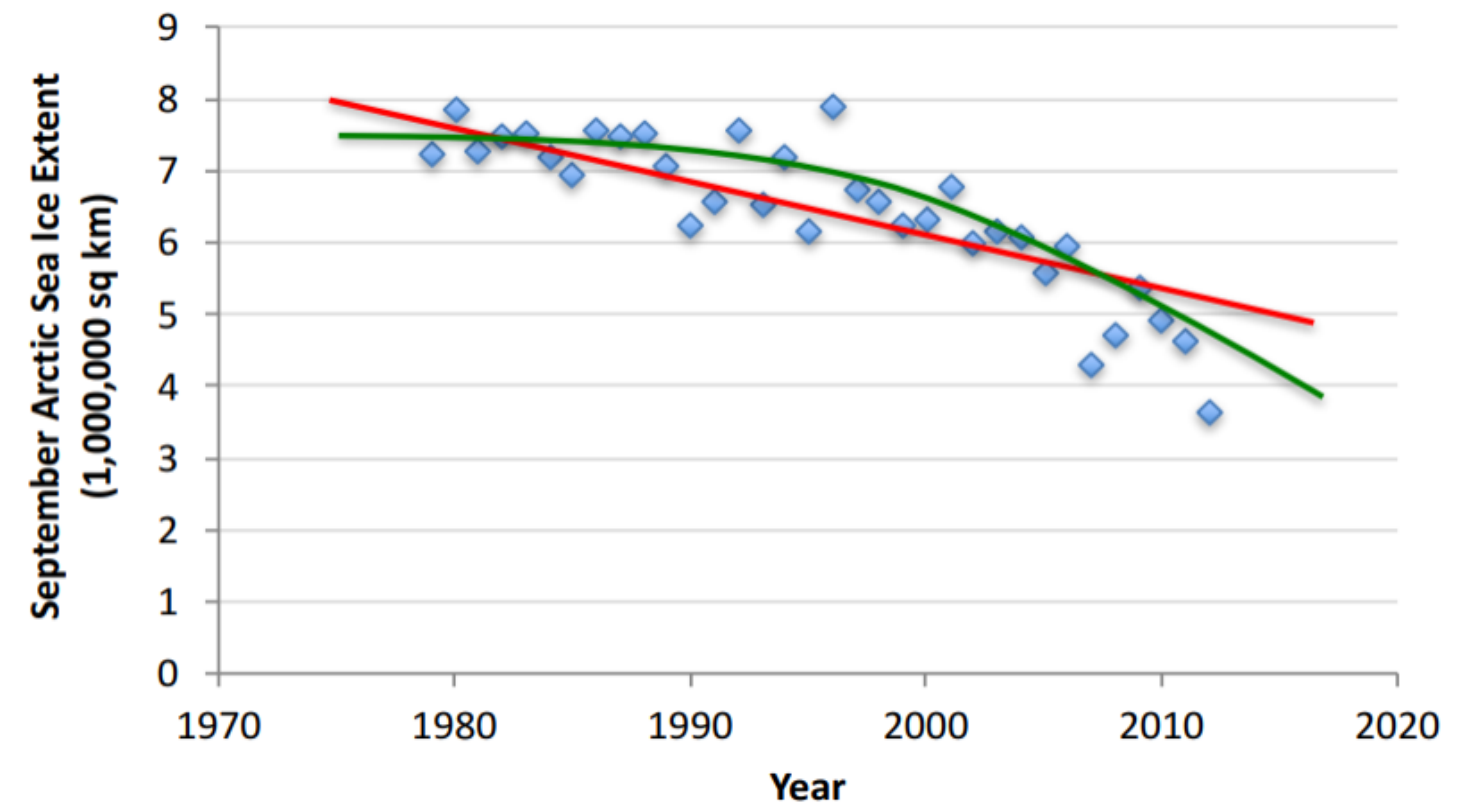
Now, after training, we test our model using the test set, and the task of the model is to identify the shape.





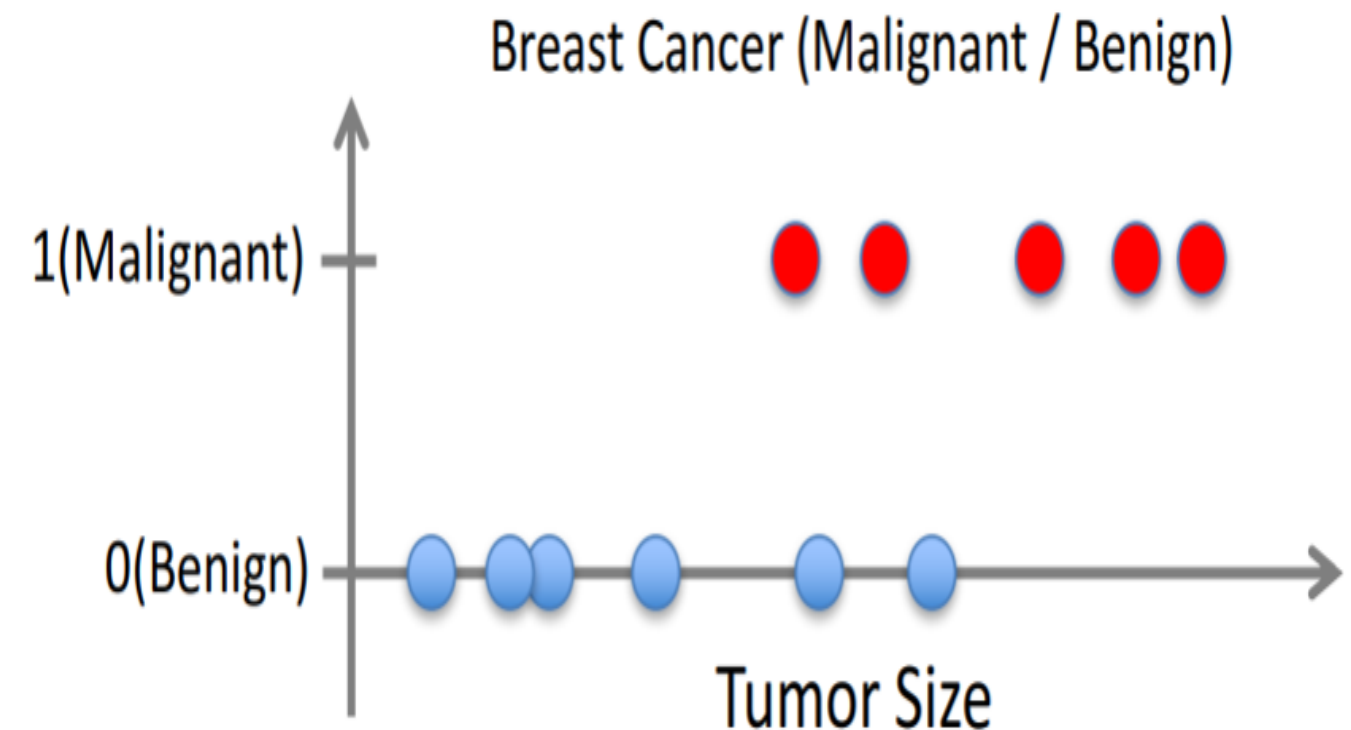
1. REGRESSION

- Regression algorithms are used when the output variable is **continuous**
- Given $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$
- Learn a function $f(x)$ to predict y given x
 - y is real-valued == regression



2. CLASSIFICATION

- Classification algorithms are used when the output variable is **categorical (discrete)**
- Given $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$
- Learn a function $f(x)$ to predict y given x
 - y is categorical == classification



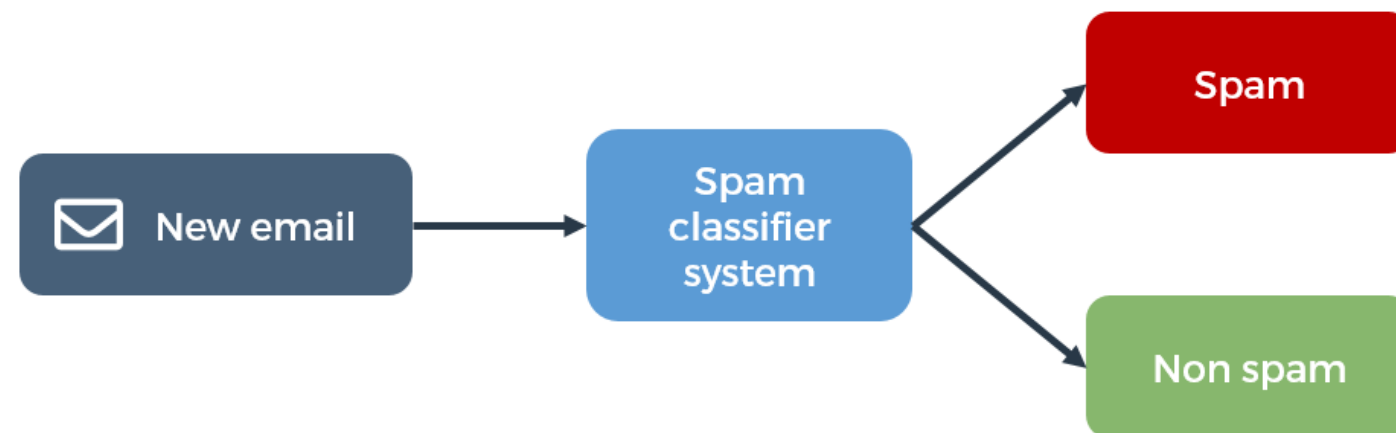
- For **Classification** problems
 - Random Forest
 - Decision Trees
 - Logistic Regression
 - Support vector Machines
- For **Regression** problems
 - Linear Regression
 - Bayesian Linear Regression
- Random forest for classification and regression problems..
- A neural network is said to learn supervised, if the desired output is already known.

Machine Learning

Supervised Learning Applications



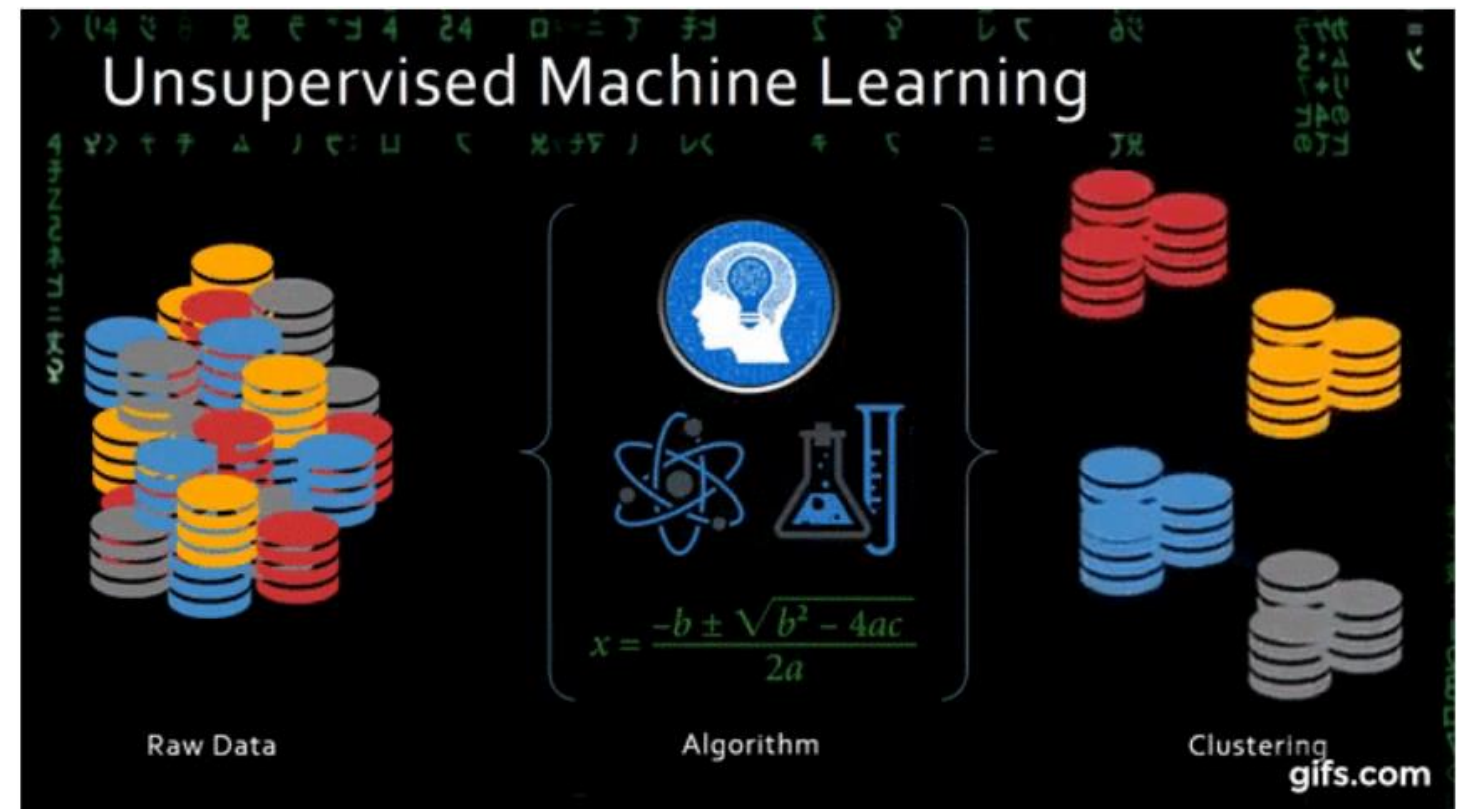
SPAM DETECTION



Machine Learning

Unsupervised Learning

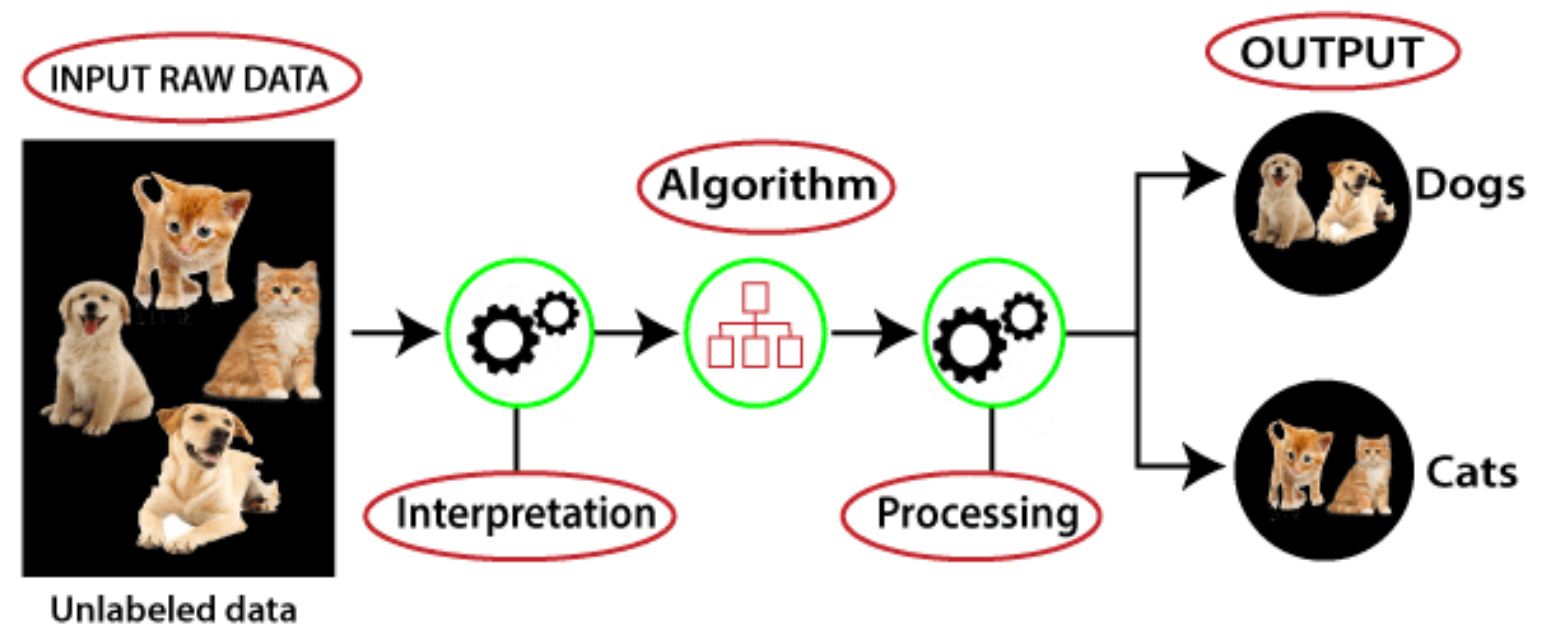
- Given: training data (**without desired outputs**)
- Models are trained using **unlabeled dataset** and are allowed to act on that data without any supervision
- The goal of unsupervised learning is to **find the underlying structure of dataset**, group that data according to similarities, and represent that dataset in a compressed format.

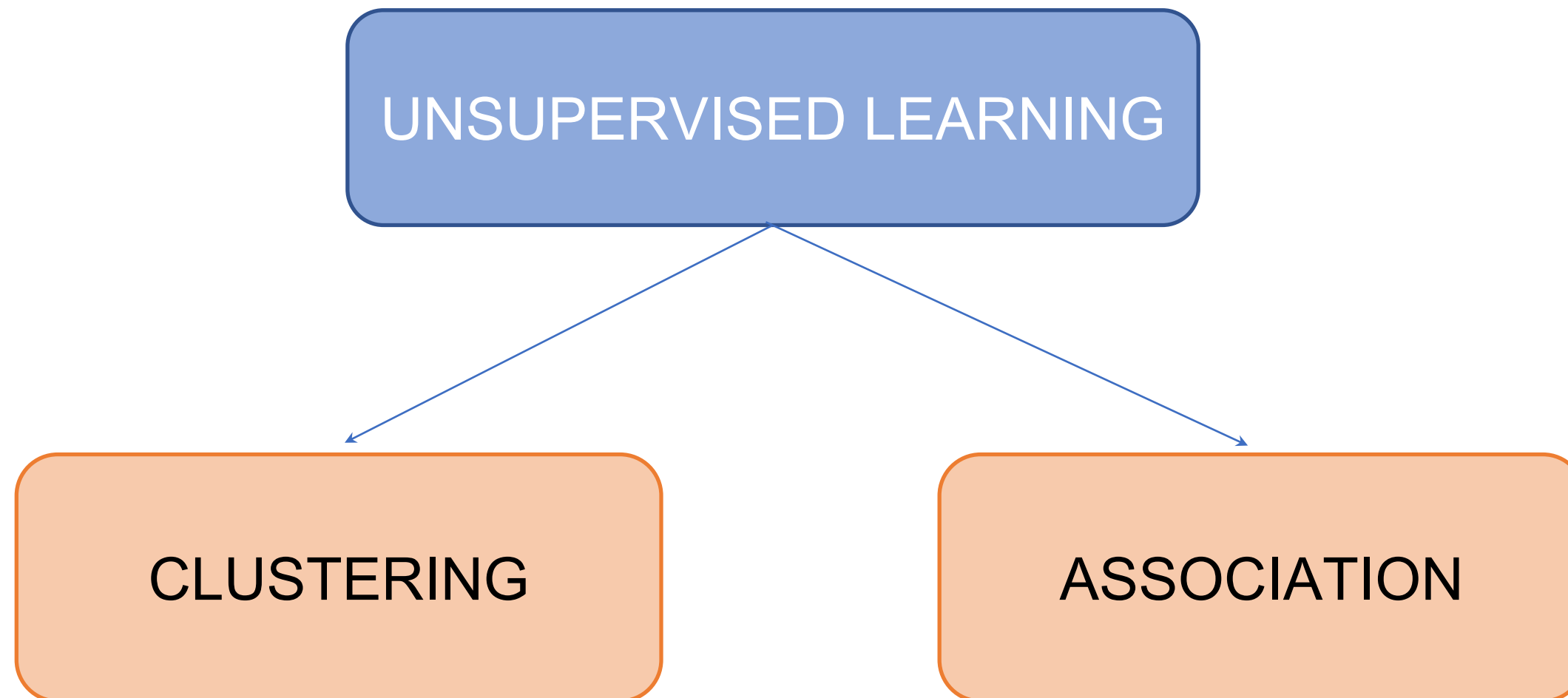


Machine Learning

Unsupervised Learning

- Dataset of **unlabeled input data**, (it is not categorized and corresponding outputs are also not given)
- Unlabeled input data is fed to the machine learning model in order to train it.
- Firstly, it will interpret the raw data to **find the hidden patterns** from the data and then will apply suitable algorithms such as k-means clustering, etc.
- The algorithm divides the data objects into groups according to the similarities and difference between the objects.





Clustering

- Clustering is a method of **grouping objects** into clusters such that objects with most similarities remain in a group and have less or no similarities with the objects of another group.



sample

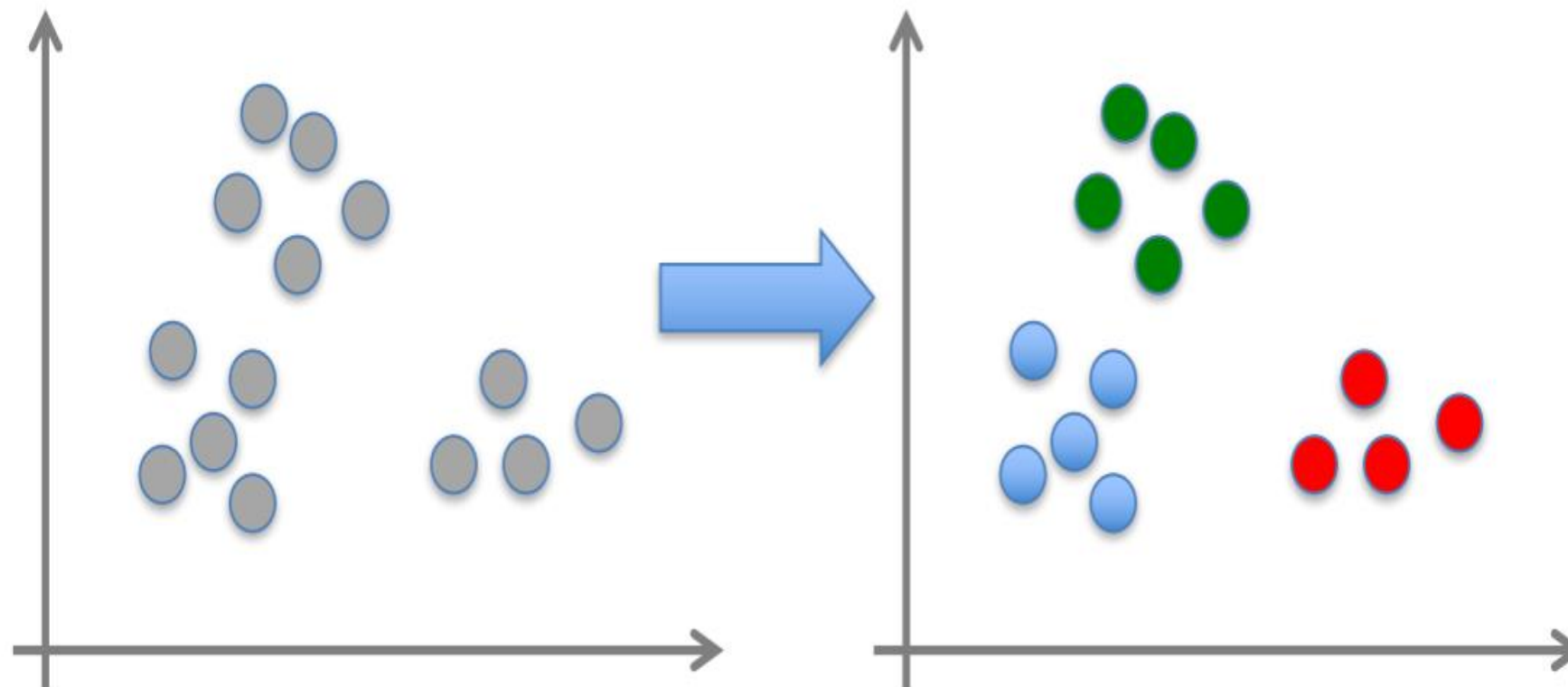


Cluster/group

Machine Learning

Unsupervised Learning - Clustering

- Given $x_1, x_2, \dots, x_n$ (without labels)
- Output hidden structure behind the x 's (data-points)



Association:

- An association rule is an unsupervised learning method which is used for **finding the relationships** between variables in the large database. It determines the set of items that occurs together in the dataset.

Examples:

- People that buy a new home most likely to buy new furniture.
- A subgroup of cancer patients grouped by their gene expression measurements
- Groups of shopper based on their browsing and purchasing histories
- Movie group by the rating given by movies viewers

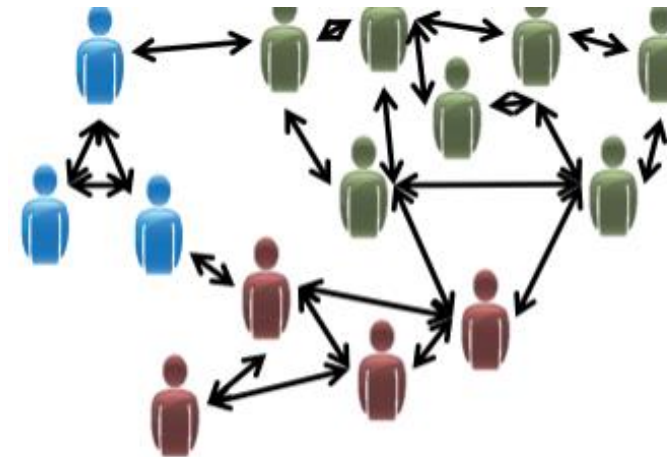
- K-means clustering
- Hierarchical clustering
- Neural Networks (Encoders, Autoencoders)
- Principal Component Analysis (PCA)
- Independent Component Analysis
- Apriori algorithm
- Singular value decomposition (SVD)

Machine Learning

Unsupervised Learning Applications



Organize computing clusters



Social network analysis



Market segmentation



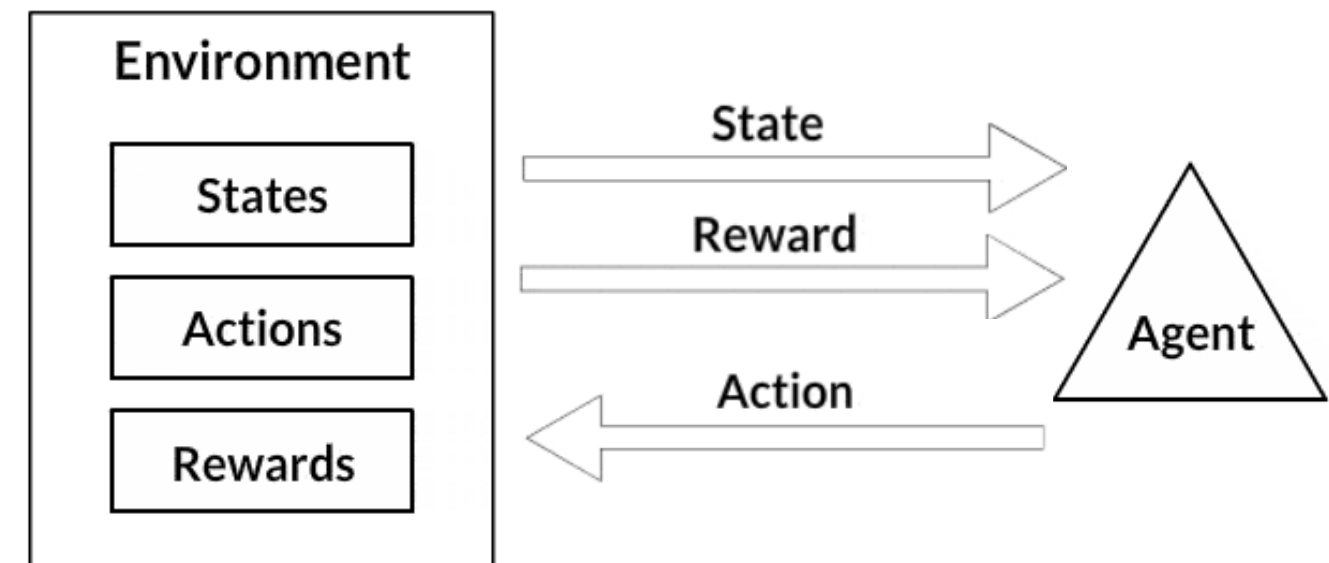
Image credit: NASA/ESA/ESA/Churchill (Univ. of Wisconsin, MA)

Astronomical data analysis

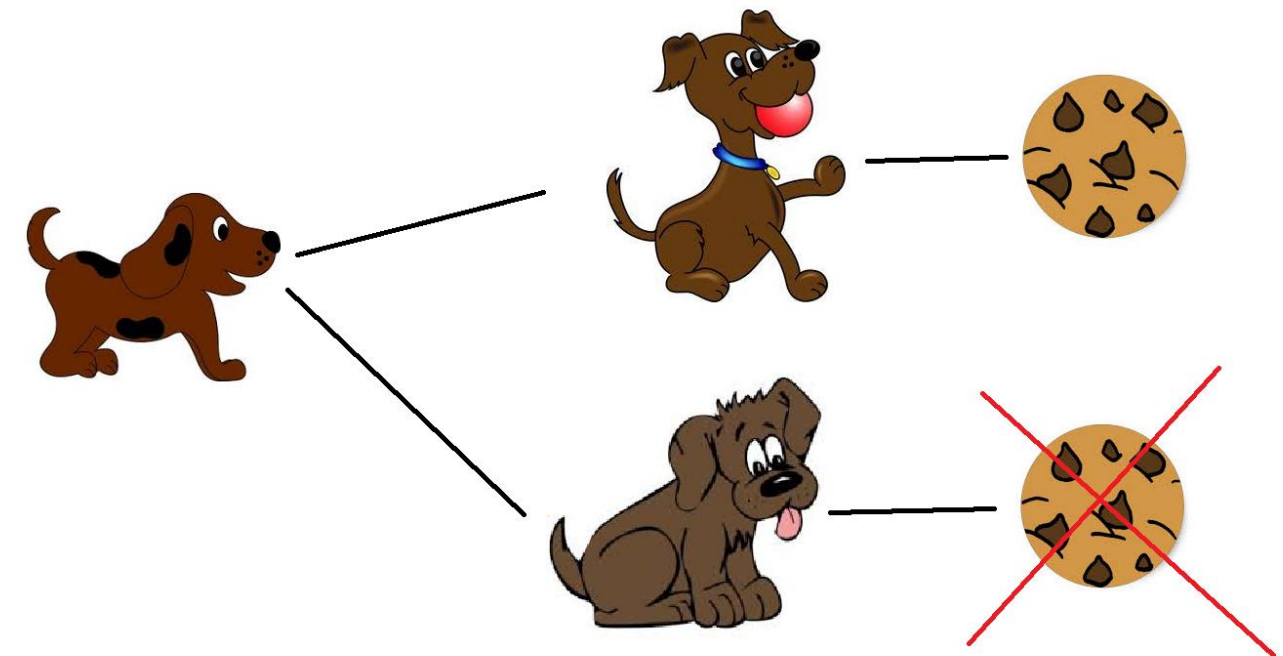
Machine Learning

Reinforcement learning

- Reinforcement Learning is learning **what to do and how to map situations to actions**.
- The algorithm (**agent**) evaluates a current situation (**state**), takes an action, and receives feedback from the environment after each act.
- Positive feedback is a **reward**, and negative feedback is **punishment** for making a mistake.
- The learner must discover which action will yield the **maximum reward**



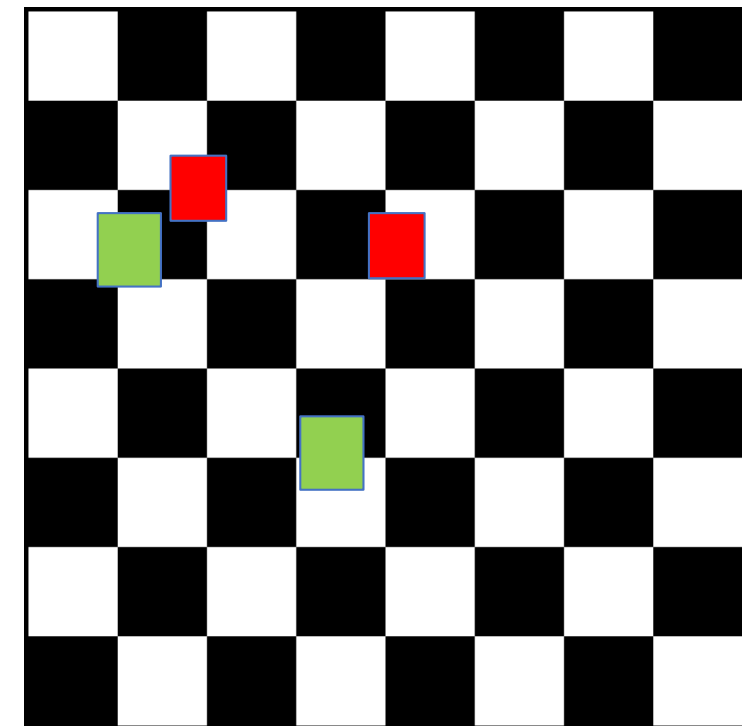
- Now consider a **dog** in a **room** which is **sitting**
- The task is for the dog to learn to fetch the ball, the dog starts **moving** and fetches the ball and hence gets a **cookie**
- if the dog fails to fetch the ball dog gets **no cookie**
- slowly over a period the dog **learns** a **policy** which does fetch ball and gets cookie, such a policy that dog learns is called **control policy**



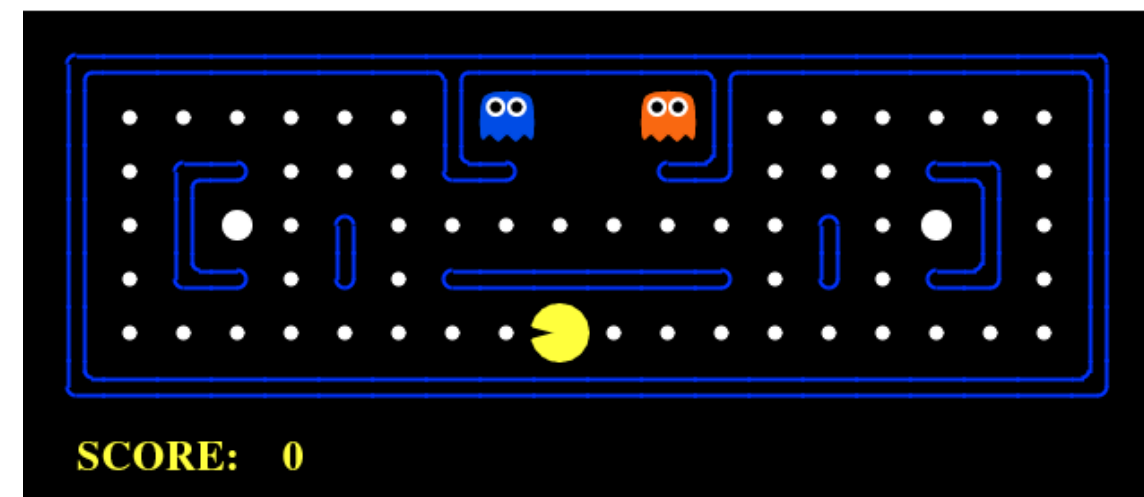
The Chess Board Scene

Let's consider the training of the pawn coin in chess

- Let the green box be the training agent pawn and red be the target and is considered win
- suppose this is state S1 the agent takes a random action A1 and moves two steps forward
- since this is positive as concerned with killing of target the agent gets a reward R1
- In the new state S2 suppose after some calculation the agent decides to go straight, it gets a penalty
- Now suppose after some n scenes the agent sees a similar state
- This time the agent remembers the penalty received in past and goes to the right diagonal and receives a heavy reward, this makes agent learn that this action was superb
- This is just a simple explanation as the actual working of the process involves several steps

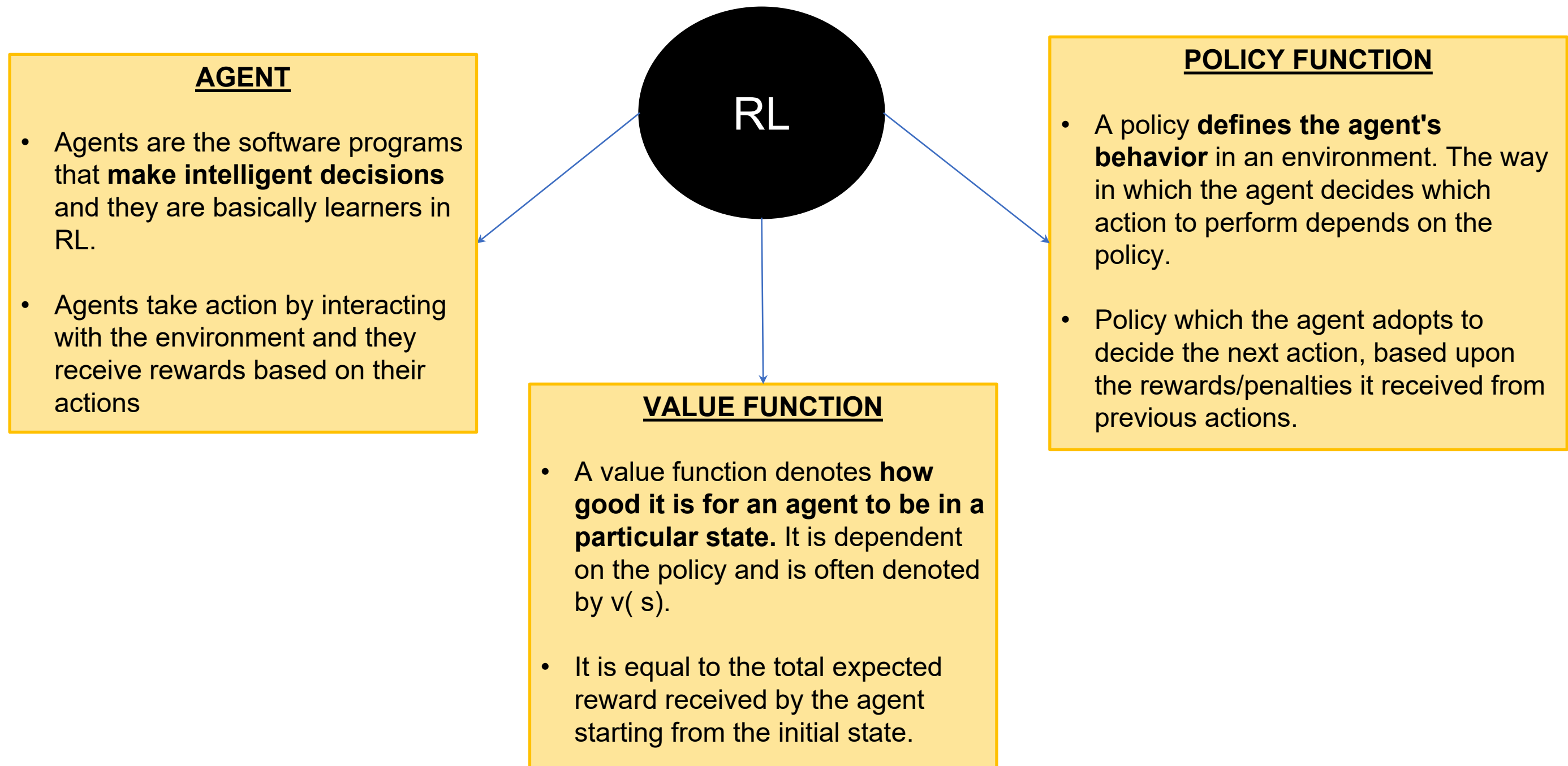


- Let's take another example of - the game of **PacMan** where the goal of the agent is to eat the food in the grid while avoiding the ghosts on its way.
- The grid world is the interactive environment for the agent. PacMan receives a reward for eating food and punishment if it gets killed by the ghost (loses the game).
- The states are the location of PacMan in the grid world and the total cumulative reward is PacMan winning the game.
- In order to build an optimal policy, the agent faces the dilemma of exploring new states while maximizing its reward at the same time.

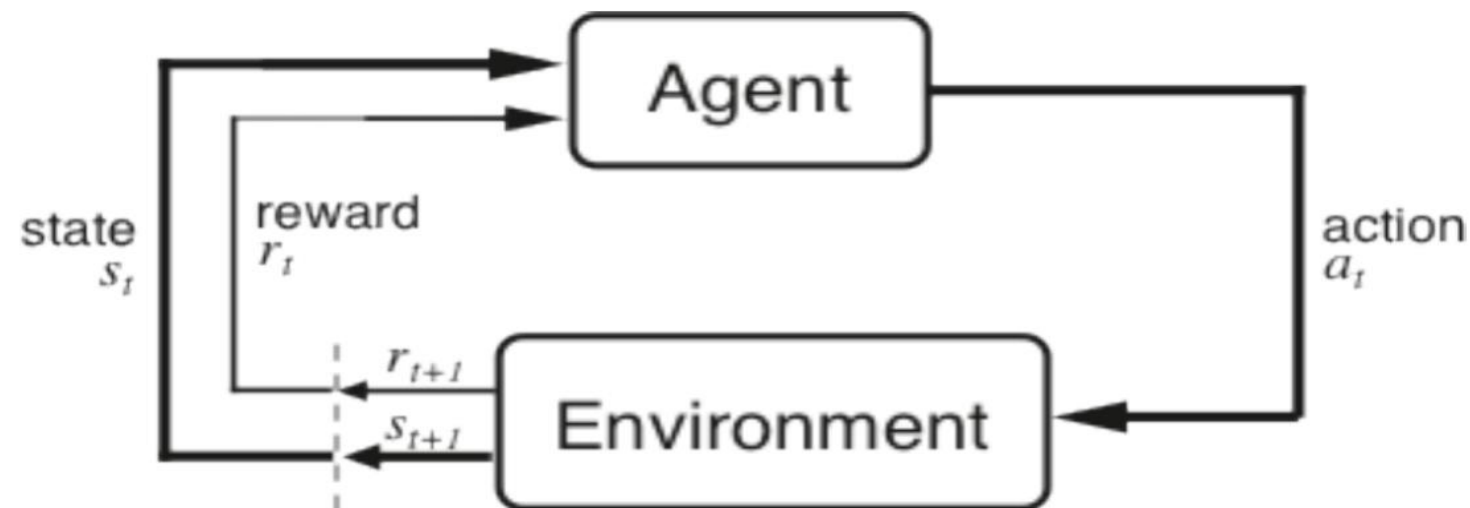


Reinforcement vs Supervised vs Unsupervised Learning

- Though both **supervised** and reinforcement learning use mapping between input and output, unlike supervised learning where feedback provided to the agent is correct set of actions for performing a task, reinforcement learning uses rewards and punishment as signals for positive and negative behavior.
- As compared to **unsupervised** learning, reinforcement learning is different in terms of goals. While the goal in unsupervised learning is to find similarities and differences between data points, in reinforcement learning the goal is to find a suitable action model that would maximize the total cumulative reward of the agent.



- The Agent-Environment Interface

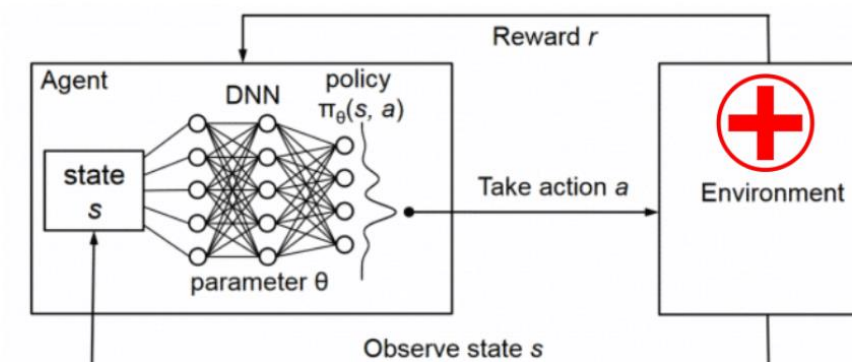


- Agent and environment interact at discrete time steps : $t = 0, 1, 2, \dots, K$
- Agent observes state at step t : $s_t \in S$
- produces action at step t : $a_t \in A(s_t)$
- gets resulting reward : $r_t + 1 \in \mathcal{R}$
- and resulting next state : s_{t+1}

- RL is quite widely used in building AI for playing **computer games**. AlphaGo Zero is the first computer program to defeat a world champion in the ancient Chinese game of Go.
- **Robotics** - RL can be used for high-dimensional control problems as well as various industrial applications. Google, for example, has reportedly cut its energy consumption by about 50% after implementing Deep Mind's technologies.
- **Text mining**. The researchers from Salesforce, a renowned cloud computing company, used RL along with an advanced contextual text generation model to develop a system that's able to produce highly readable summaries of long texts.

- **Trade execution** - Major companies in the financial industry have been using ML algorithms to enhance trading and equity for a while and some of them, such as JPMorgan, have already thrown their hats into the RL ring too. The company announced in 2017 that it would start using a robot for trading execution of large orders.
- **Healthcare** - Recent papers suggest multiple applications for RL in the healthcare industry. Among them are medication dosing, optimization of treatment policies for those suffering from chronic, clinical trials, etc.

The Learning Hospital



Experiments in combining Deep Reinforcement Learning (using PyTorch) and Hospital Simulation (using SimPy)

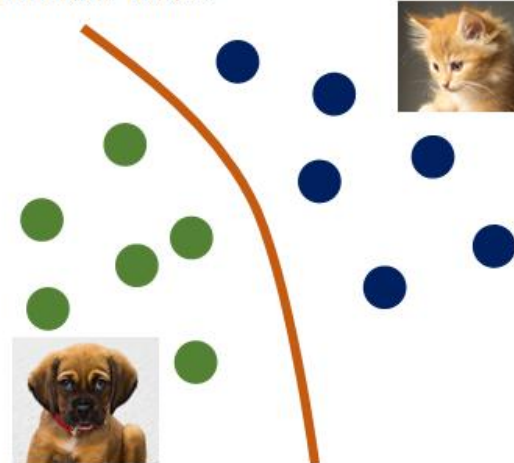


Criteria	Supervised ML	Unsupervised ML	Reinforcement ML
Definition	Learns by using labelled data	Trained using unlabelled data without any guidance.	Works on interacting with the environment
Type of data	Labelled data	Unlabelled data	No – predefined data
Type of problems	Regression and classification	Association and Clustering	Exploitation or Exploration
Supervision	Extra supervision	No supervision	No supervision
Algorithms	Linear Regression, Logistic Regression, SVM, KNN etc.	K – Means, C – Means, Apriori	Q – Learning, SARSA
Aim	Calculate outcomes	Discover underlying patterns	Learn a series of action
Application	Risk Evaluation, Forecast Sales	Recommendation System, Anomaly Detection	Self Driving Cars, Gaming, Healthcare

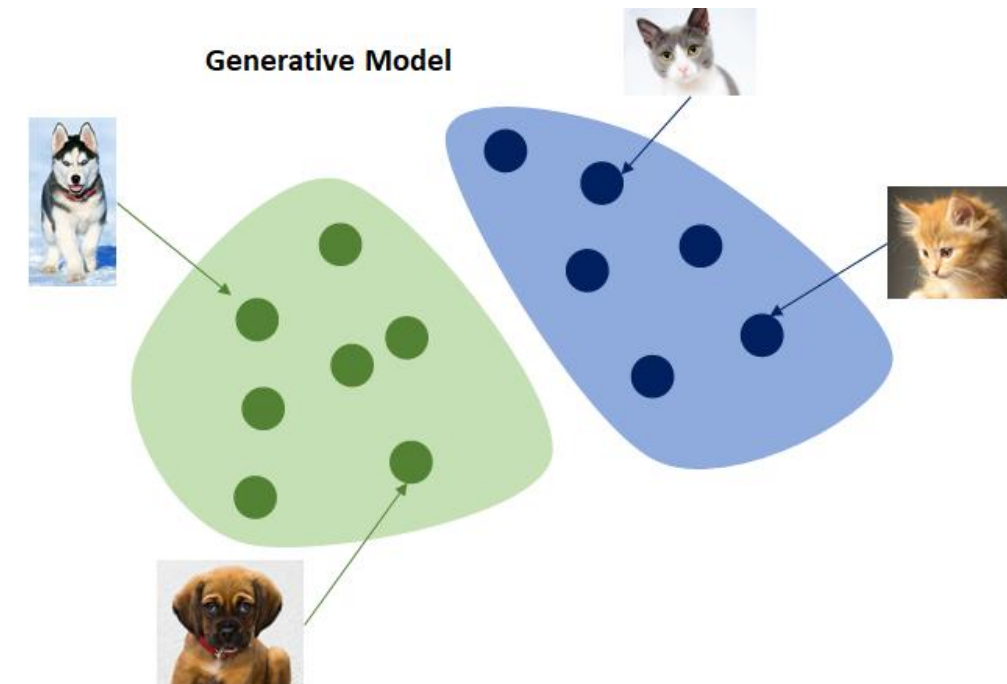
- Machine learning models can be classified into **discriminative and generative models**.
- In simple words, **a discriminative model** makes predictions based on conditional probability and is either used for classification or regression.
- On the other hand, **a generative model** revolves around the distribution of a dataset to return a probability for a given example.

Discriminative models draw boundaries in the data space, while generative ones model how data is placed throughout the space.

Discriminant Model



Generative Model



Let's say you have input data x and you want to classify the data into labels y . A generative model learns the **joint** probability distribution $p(x, y)$ and a discriminative model learns the **conditional** probability distribution $p(y|x)$ - which you should read as "*the probability of y given x* ".

$p(x, y)$ is

	$y=0$	$y=1$
$x=1$	$1/2$	0
$x=2$	$1/4$	$1/4$

$p(y|x)$ is

	$y=0$	$y=1$
$x=1$	1	0
$x=2$	$1/2$	$1/2$

$x(\text{input})$	$y(\text{label})$
1	0
1	0
2	0
2	1

The distribution $p(y|x)$ is the natural distribution for classifying a given example x into a class y , which is why algorithms that model this directly are called discriminative algorithms.

Generative algorithms model $p(x, y)$, which can be transformed into $p(y|x)$ by applying Bayes rule and then used for classification. However, the distribution $p(x, y)$ can also be used for other purposes. For example, you could use $p(x, y)$ to *generate* likely (x, y) pairs.

Machine Learning

Discriminative Model



Types of discriminative models in machine learning include:

- Logistic regression
- Support Vector Machine
- Decision Tree
- Random Forest

Types of generative models in machine learning include:

- Bayesian Network
- Hidden Markov model
- Autoregressive model
- Generative adversarial network (GANs)

A computer program is said to learn from experience E with respect to some task T and some performance measure P if its performance on T , measured by P , improves with experience E .

Suppose we feed a learning algorithm a lot of historical weather data, and have it learn to predict weather. In this setting, what is T ?

- a. The process of the algorithm examining a large amount of historical weather data.
- b. The probability of it correctly predicting a future date's weather.
- c. The weather prediction task.
- d. None of these

A computer program is said to learn from experience E with respect to some task T and some performance measure P if its performance on T , measured by P , improves with experience E .

Suppose we feed a learning algorithm a lot of historical weather data, and have it learn to predict weather. In this setting, what is T ?

- a. The process of the algorithm examining a large amount of historical weather data.
- b. The probability of it correctly predicting a future date's weather.
- c. The weather prediction task.**
- d. None of these

Suppose you are working on weather prediction, and use a learning algorithm to predict tomorrow's temperature (in degrees C/F).

Would you treat this as a classification or regression problem?

- a. Regression
- b. Classification

Suppose you are working on weather prediction, and use a learning algorithm to predict tomorrow's temperature (in degrees C/F).

Would you treat this as a classification or regression problem?

- a. Regression
- b. Classification

Suppose you are working on stock market prediction. Typically tens of millions of shares of Company-XYZ stock are traded (i.e., bought/sold) each day. You would like to predict the number of Company-XYZ shares that will be traded tomorrow.

Would you treat this as a classification or regression problem?

- a. Regression
- b. Classification

Suppose you are working on stock market prediction. Typically tens of millions of shares of Company-XYZ stock are traded (i.e., bought/sold) each day. You would like to predict the number of Company-XYZ shares that will be traded tomorrow.

Would you treat this as a classification or regression problem?

- a. Regression
- b. Classification

Which of the following would you apply supervised learning to? (Select all that apply) In each case, assume some appropriate dataset is available for your algorithm to learn from.

- a. Given genetic (DNA) data from a person, predict the odds of them developing diabetes over the next 10 years.
- b. Have a computer examine an audio clip of a piece of music, and classify whether or not there are vocals (i.e., a human voice singing) in that audio clip.
- c. Given data on how 1000 medical patients respond to an experimental drug (such as effectiveness, side-effects, etc.), discover whether there are different "types" of patients in terms of how they respond to the drug, and if so what these categories are.

Which of the following would you apply supervised learning to? (Select all that apply) In each case, assume some appropriate dataset is available for your algorithm to learn from.

- a. **Given genetic (DNA) data from a person, predict the odds of them developing diabetes over the next 10 years.**
- b. **Have a computer examine an audio clip of a piece of music, and classify whether or not there are vocals (i.e., a human voice singing) in that audio clip.**
- c. Given data on how 1000 medical patients respond to an experimental drug (such as effectiveness, side-effects, etc.), discover whether there are different "types" of patients in terms of how they respond to the drug, and if so what these categories are.

Which of these is a reasonable definition of machine learning?

- a. Machine learning is the field of allowing robots to act intelligently.
- b. Machine learning is the science of programming computers.
- c. Machine learning learns from labeled data.
- d. Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed.

Which of these is a reasonable definition of machine learning?

- a. Machine learning is the field of allowing robots to act intelligently.
- b. Machine learning is the science of programming computers.
- c. Machine learning learns from labeled data.
- d. Machine learning is the field of study that gives computers the ability to learn without being explicitly programmed.**

Machine Learning

QUIZ Links



<https://www.sanfoundry.com/artificial-intelligence-ai-questions-answers-online-quiz/#:~:text=Explanation%3A%20In%20automatic%20vehicle%20set,an%20example%20of%20supervised%20learning.>

<https://github.com/shank885/Machine-Learning-Andrew-Ng/blob/master/Quizzes/Week%201/Quiz%201.pdf>



Machine Learning

Dr. Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu



UE24CS352A : Machine Learning

Dr. Shylaja S S

Professor

Department of Computer Science & Engineering

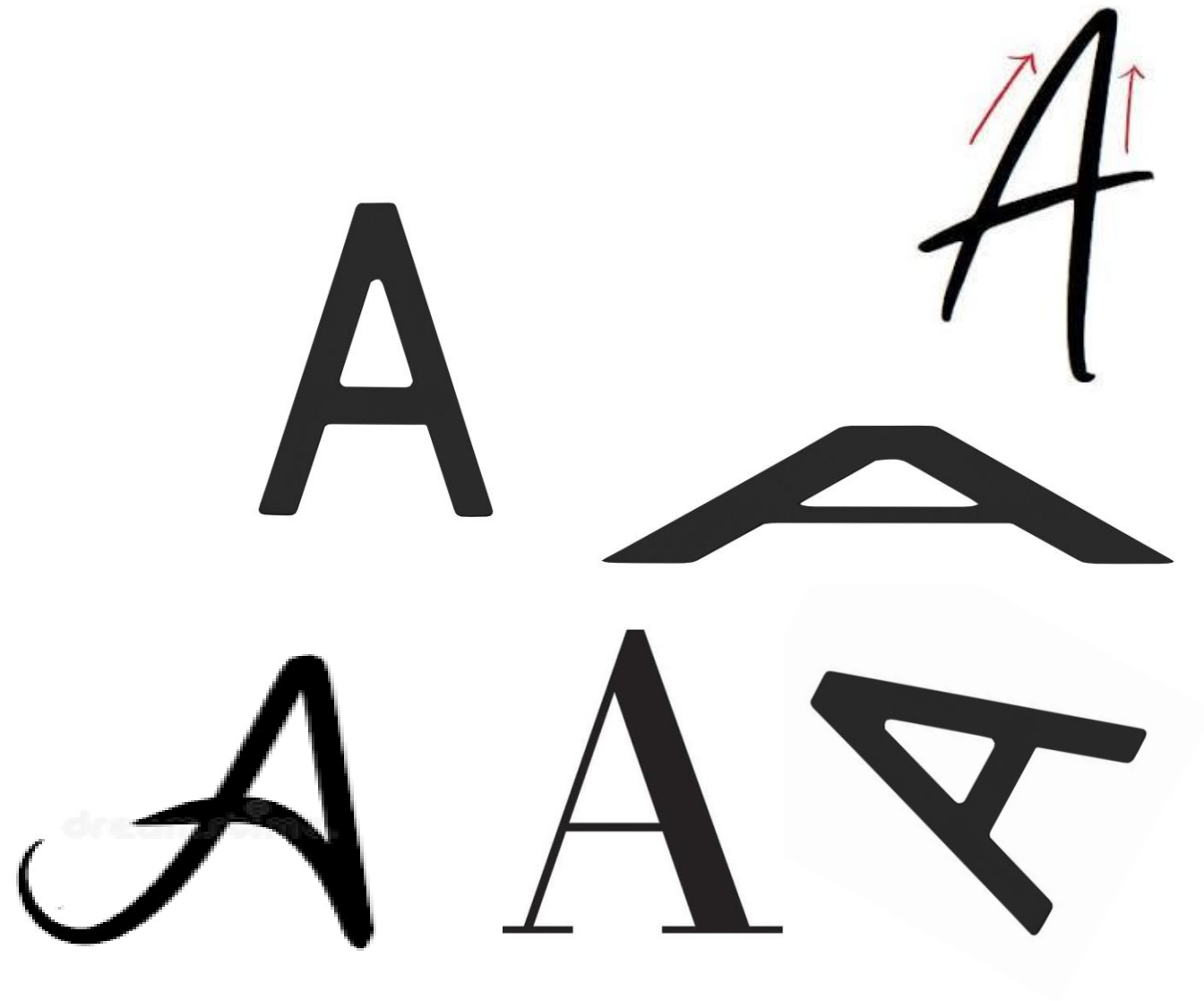
Acknowledgement :

Dr. Rajanikanth K (Academic Advisor, PESU)

Teaching Assistant (Arya Rajiv Chaloli)

Concept defines what belongs to a category and what doesn't.

Concept of the letter 'A':



The **feature space** is the set of all possible combinations of attributes (features) used to describe examples.

(Reducing the scope of the concept of the letter 'A')
We may define the features as:

Feature	Values
Has Crossbar	Yes / No
Has Pointed Top	Yes / No
Symmetrical	Yes / No
Has Two Legs	Yes / No
Is Curved	Yes / No

Each character can be described by such a feature vector

- 'A' might be represented as: [Yes, Yes, Yes, Yes, No]
- '^' might be: [No, Yes, Yes, Yes, No]
- 'a' might be: [Yes, No, No, No, Yes]

The **concept space** is the set of all possible concepts that can be formed using the feature space.

It includes every possible labeling (positive/negative) of every feature vector.

If you have n features and each has 2 values, then you have $m = 2^n$ possible examples, the number of possible concepts is:
 $2^m = 2^{(2^n)}$ (because each example can be labeled Yes or No)
This calculation will be easier to understand in the numerical example we solve next.

So, the concept space is all possible subsets of the instance space.

Concept and Feature Space

Example Concept: A fruit's edibility

Fruit's characteristics:

Color	Shape
Red	Round
Green	Long

Possible fruit feature combinations (**feature space**):

#	Color	Shape
RR	Red	Round
RL	Red	Long
GR	Green	Round
GL	Green	Long

The size of the feature space χ
(the set of all possible values for a
chosen set of features from that data)
 $= 2^2 = 4$: in this example

RR	RL	GR	GL
0	0	0	0
0	0	0	1
0	0	1	0
0	0	1	1
0	1	0	0
0	1	0	1
0	1	1	0
0	1	1	1
1	0	0	0
1	0	0	1
1	0	1	0
1	0	1	1
1	1	0	0
1	1	0	1
1	1	1	0
1	1	1	1

- For each combination, the instance can be labeled $+^{ve}$ (1: edible) or $-^{ve}$ (0: inedible)
- Total number of concepts that can be defined over this set is $2^4 = 16$
- Generally, if there are d binary features then, the **concept space** has 2^{2^d} concepts/functions/hypothesis.

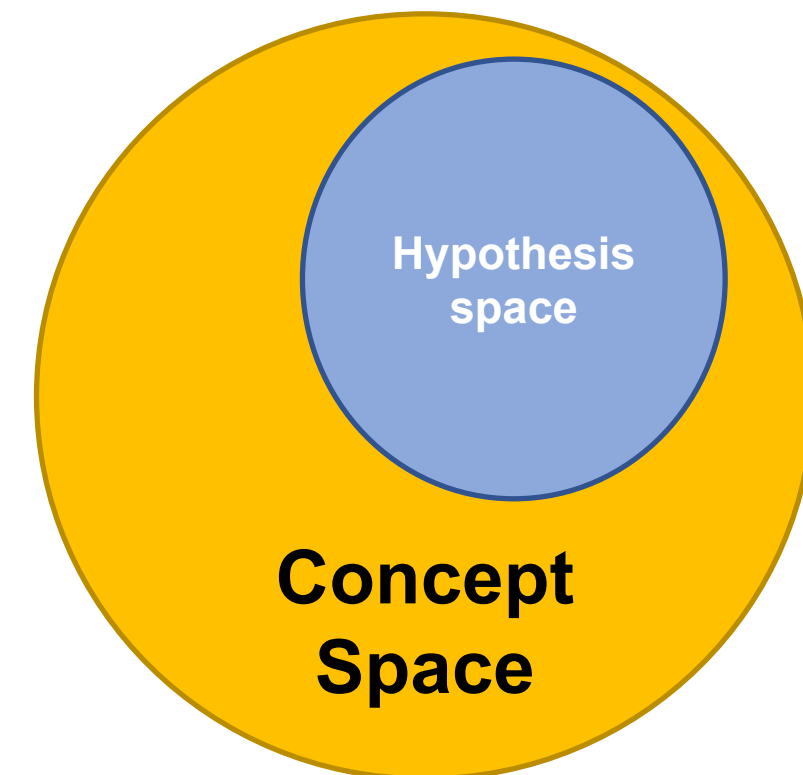
Reducing the Concept Space

Make assumption about the nature of the concept.

The **assumption will reduce the space** of the concept we are going to look at.

This **reduction of the concept space** is what we call as **inductive bias**.

The **shrunk space** is called **hypothesis space**.



- Hypothesis (with induction bias):
From where is this assumption coming?
- Possible values for each attribute:
Number of actual values + ? + $\emptyset$
- Even if one attribute is $\emptyset$, we have to conclude that all instances are -ve!
- So, the modified size of $H = (3 \times 3) + 1 = 10$
10 **semantically distinct hypotheses**
- $H: \{R,R\}, \{R,L\}, \{R,?\}, \{G,R\}, \{G,L\}, \{G,?\}, \{?,R\}, \{?,L\} + \{?,?\}$ (Accept All) + Reject All
- Reduction from 16 to 10

RR	RL	GR	GL	Corresponding Hypothesis
0	0	0	0	$\langle \emptyset \rangle$
0	0	0	1	$\langle G, L \rangle$
0	0	1	0	$\langle G, R \rangle$
0	0	1	1	$\langle G, ? \rangle$
0	1	0	0	$\langle R, L \rangle$
0	1	0	1	$\langle ?, L \rangle$
0	1	1	0	---
0	1	1	1	---
1	0	0	0	$\langle R, R \rangle$
1	0	0	1	---
1	0	1	0	$\langle ?, R \rangle$
1	0	1	1	---
1	1	0	0	$\langle R, ? \rangle$
1	1	0	1	---
1	1	1	0	---
1	1	1	1	$\langle ?, ? \rangle$

6 hypotheses which are excluded because of Induction Bias

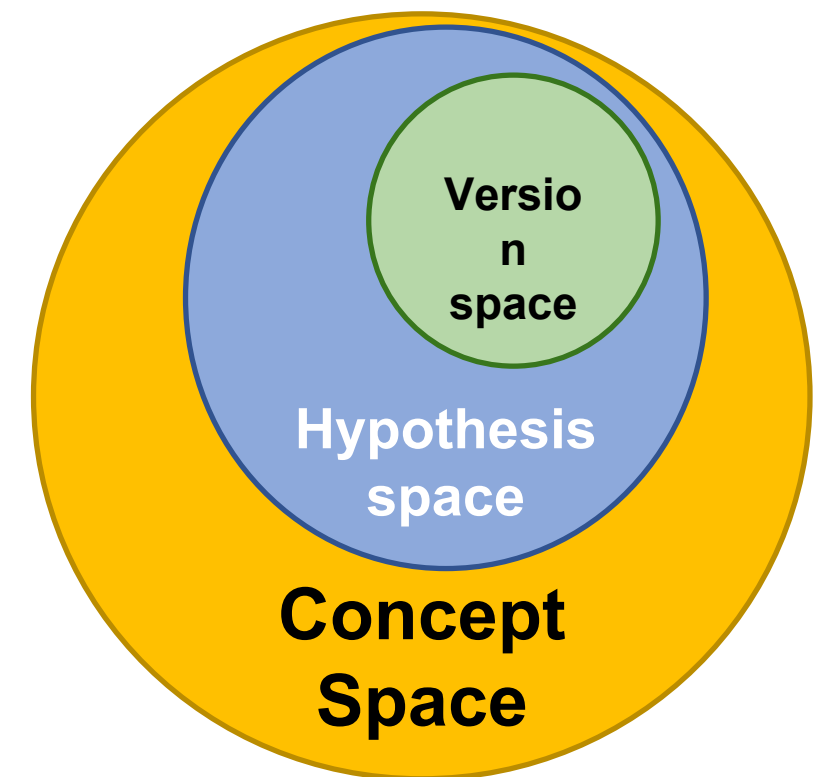
Our Bias - concepts are of a particular kind called **conjunctive concepts** because of which the hypothesis space is restricted.

Hence, it is a **Restriction Bias**. Because of the assumption that hypothesis is a conjunction of attribute values, few of the hypothesis are never considered.

Target concept may or may not (if there are errors in the training data) be present in the hypothesis space.

- Assume a data object is defined by 6 attributes
 - a1 has 3 values;
 - rest (a2, a3, a4, a5, a6) are binary.
- Size of Concept space: $2^{(3 \times 2 \times 2 \times 2 \times 2 \times 2)} = 2^{96}$
- With our Inductive Bias (Hypothesis is a conjunction),
Size of H = $(4 \times 3 \times 3 \times 3 \times 3 \times 3) + 1 = 973$
- Reduction from 2^{96} to 973

- Training data is a subset of the entire data.
- Version space is the set of all hypotheses from the hypothesis space that are consistent with the training data.
- A hypothesis is said to be **consistent wrt to training data** set if it classifies all the objects of training data set to their corresponding classes.
- At any given point let H be our hypothesis space so a version space is a subset of H but all concepts in version space (VS) are consistent wrt to training set i.e.,
 $VS = \{ h : h \in H \text{ and } h \text{ is consistent with } D_{\text{training}} \}$



#	Color	Shape	Edible?
RR	Red	Round	Yes
RL	Red	Long	No
GR	Green	Round	
GL	Green	Long	

→ ● Training Data

→ ● Unknown Data

Version space (VS) is the set of all hypotheses from the hypothesis space (H) that are consistent with the training data.

H: {**R,R**}, {R,L}, {R,?}, {G,R}, {G,L}, {G,?}, {**?,R**}, {?,L} + {?,?} + Reject All

VS: {R,R}, {?,R}

How do we make the machine learn a concept?

- **Concept learning**: Learning to categorize, given a sample of positive and negative training examples of the category.
- We must be able to **provide features** which allow the machine to learn the concept.
- We can learn **multiple concepts based on a dataset**.
- The ML algorithm must learn the one that best fits the training data and works well on the new data instances.
- The concept or function to be learned is called the **target concept**, which we denote by c .

In terms of machine learning, "concept learning" can be defined as:

“The problem of searching through a predefined space of potential hypotheses for the hypothesis that best fits the training examples.”

— Tom Michell

Find S Algorithm - The First ML algorithm

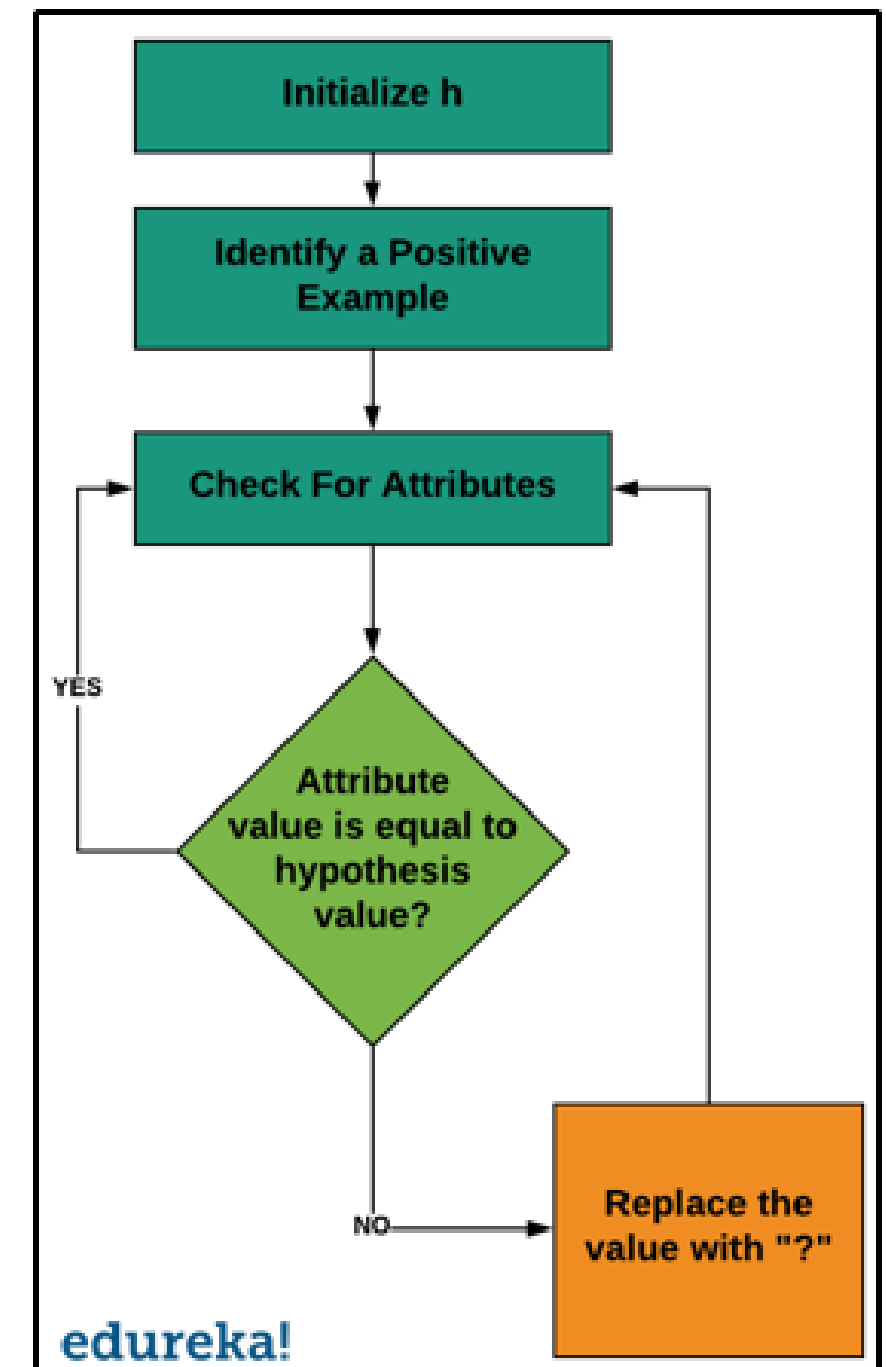
- Find-S algorithm - basic **concept learning algorithm** in machine learning
- Find-S algorithm finds the **most specific hypothesis that fits all the positive examples**
- The algorithm considers **only** the **positive** training examples
- Find-S algorithm moves from the most specific hypothesis to the most general hypothesis.
- It starts with the most specific hypothesis and generalizes this hypothesis each time it fails to classify an observed positive training data.

Machine Learning

Find S Algorithm

Algorithm :

1. Initialize h to the most specific hypothesis in H
2. For each positive training instance x
For each attribute constraint a , in h
If the constraint a , is satisfied by x
Then do nothing
else
replace a , in h by the next more general constraint
that is satisfied by x
3. Output hypothesis h



Find S Algorithm - Example Problem

- Taking a very simple example, one possible target concept may be to Find the day when Mogli enjoys his favorite sport.
- We have some attributes of the day like, Sky, Temperature, Humidity, Wind, Water, Forecast and based on this we have **a target Concept named EnjoySport**.
- We have the following training example available:

sky	temp	humidity	wind	water	forecast	enjoy
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
rainy	cold	high	strong	warm	change	no
sunny	warm	high	strong	cool	same	yes

Machine Learning

Solution



- Initialize $h = \emptyset$ (assume everything is negative)
- Use the first positive instance to update h as:
- $h = \{\text{sunny, warm, normal, strong, warm, same}\}$

sky	temp	humidity	wind	water	forecast	enjoy
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
rainy	cold	high	strong	warm	change	no
sunny	warm	high	strong	cool	same	yes

Use the second positive instance to update h as:

h = {sunny, warm, normal, strong, warm, same}
x = {sunny, warm, high , strong, warm, same}

h = {sunny, warm, ? , strong, warm, same}

sky	temp	humidity	wind	water	forecast	enjoy
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
rainy	cold	high	strong	warm	change	no
sunny	warm	high	strong	cool	same	yes

Use the third positive instance to update h as:

$h = \{\text{sunny, warm, ? , strong, warm, same}\}$
 $x = \{\text{sunny, warm, high , strong, cool , same}\}$

 $h = \{\text{sunny, warm, ? , strong, ? , same}\}$

sky	temp	humidity	wind	water	forecast	enjoy
sunny	warm	normal	strong	warm	same	yes
sunny	warm	high	strong	warm	same	yes
rainy	cold	high	strong	warm	change	no
sunny	warm	high	strong	cool	same	yes

Machine Learning

Find S Algorithm



We are done with all the training data set and we give our final hypothesis as our concept that our machine has learn to label other data sets

$C = \{\text{sunny, warm, ?, strong, ?, same}\}$

Suppose now on a given day we get the following data we use the concept learnt to predict the label that is, if the person will play a sport or no

$x = \{\text{sunny, warm, high, strong, warm, same}\}$

$c(x) = 1$

- Find-S is assuming (by default) that all instances are –ve unless its presently available knowledge shows it to be +ve!
Thus, it ignores –ve instances and needs to examine only +ve instances
- Final hypothesis of Find-S in the previous example:
C={sunny, warm, ?, strong, ?, same}
Is this **consistent** with the –ve example? YES! (*What do we mean by consistent?*)
- In general, the output hypothesis of FIND-S will be consistent with –ve examples also provided:
 - the correct target concept is indeed in H (conjunction of attribute values) and
 - there are no errors in the training examples
- Now consider the hypothesis **C2: {sunny, ?, ?, strong, ?, same} !!**
What do you think about it? - Any reason to prefer C to C2 ?!

Machine Learning

Points to Ponder



- We started with $C = \{\emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset\}$ and progressively generalized it considering only positive examples, and ended with:
 $C = \{\text{sunny, warm, ?, strong, ?, same}\}$
- Can we start with $C_2 = \{?, ?, ?, ?, ?, ?\}$ and progressively specialize it considering only negative examples?
- Can we combine the above 2 ideas?

--- Candidate Elimination Algorithm (Not in syllabus!!)

- Finds the most specific hypothesis within H that is consistent with the positive training examples. The final hypothesis **may give a positive output for a negative example**.
- Output hypothesis is consistent with the training examples, this is just **one of many hypotheses** that might fit the training data equally well.
- **Inconsistent training sets can mislead** the Find-S algorithm, since it ignores the negative examples.
- Find-S algorithm **does not provide a backtracking** technique to determine the best possible changes that could be done to improve the resulting hypothesis.

- Assume a data object is defined 3 attributes, a1 has 3 values, the other two are binary attributes.

a1	a2	a3	value
Low	1	0	YES
Low	0	0	YES
Medium	1	0	NO
High	1	1	YES

- Assume a data object is defined 3 attributes, a1 has 3 values, the other two are binary attributes.

a1	a2	a3	value
Low	1	0	YES
Low	0	0	YES
Medium	1	0	NO
High	1	1	YES

- $h1 = \{\emptyset, \emptyset, \emptyset\};$

- Assume a data object is defined 3 attributes, a1 has 3 values, the other two are binary attributes.

a1	a2	a3	value
Low	1	0	YES
Low	0	0	YES
Medium	1	0	NO
High	1	1	YES

- $h1 = \{\emptyset, \emptyset, \emptyset\};$
 $h2 = \{LOW, 1, 0\};$

- Assume a data object is defined 3 attributes, a1 has 3 values, the other two are binary attributes.

a1	a2	a3	value
Low	1	0	YES
Low	0	0	YES
Medium	1	0	NO
High	1	1	YES

- $h1 = \{\emptyset, \emptyset, \emptyset\};$
 $h2 = \{LOW, 1, 0\};$
 $h3 = \{LOW, ?, 0\};$

- Assume a data object is defined 3 attributes, a1 has 3 values, the other two are binary attributes.

a1	a2	a3	value
Low	1	0	YES
Low	0	0	YES
Medium	1	0	NO
High	1	1	YES

- $h1 = \{\emptyset, \emptyset, \emptyset\};$
 $h2 = \{LOW, 1, 0\};$
 $h3 = \{LOW, ?, 0\};$
 $h4 = \{----, ?, ?\}$

Machine Learning

Practice Problem 1



Experience	Location	Tech Stack	Team Size	Remote
Junior	City	Frontend	Small	No
Senior	Suburb	Fullstack	Medium	Yes
Senior	City	Fullstack	Large	Yes
Mid	Suburb	Backend	Small	No
Senior	Rural	Fullstack	Medium	Yes

1. How many concepts are possible for this instance space?
2. How many hypotheses can be expressed by the hypothesis language?
3. Apply the FIND-S algorithm by hand on the given training set. Consider the examples in the specified order and write down your hypothesis each time after observing an example.

1. Feature/instance Space: $3 * 3 * 3 * 3 = 81$,

Since the concept is boolean-valued, #Concepts = 2^{81}

2. Hypothesis space : Semantically Distinct Hypothesis = $(4 * 4 * 4 * 4) + 1 = 257$

3.

Step 1:

$h = (\emptyset, \emptyset, \emptyset, \emptyset, \emptyset)$

Step 2:

X1: Negative Example Hence Ignore

X2: $h = [\text{Senior}, \text{Suburb}, \text{Fullstack}, \text{Medium}]$

X3: $h = [\text{Senior}, ?, \text{Fullstack}, ?]$

X4: Negative Example Hence Ignore

X5: $h = [\text{Senior}, ?, \text{Fullstack}, ?]$

Step 3:

Final Maximally Specific Hypothesis is: $h = [\text{Senior}, ?, \text{Fullstack}, ?]$

Consider the following data set having the data about “Will the customer buy a subscription”. Find the final hypothesis

Age Group	Device Used	Trial Used	Engagement Level	Buys Subscription
Teen	Mobile	No	Low	No
Adult	Laptop	Yes	High	Yes
Senior	Laptop	Yes	Medium	Yes
Adult	Tablet	No	Low	No
Teen	Laptop	Yes	High	Yes

Solution : Final Hypothesis: $h = \{ ?, \text{Laptop}, \text{Yes}, ? \}$

- We considered several examples of “Learning a concept”
- Can we consider any of them as “Making a Decision”?
Example: Buy or Do not buy a learning Resource?
- Is there any difference between “Concept Learning” and “Decision Making?”



UE24CS352A : Machine Learning

Dr. Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu



UE24CS352A: Machine Learning

Dr. Shylaja S S

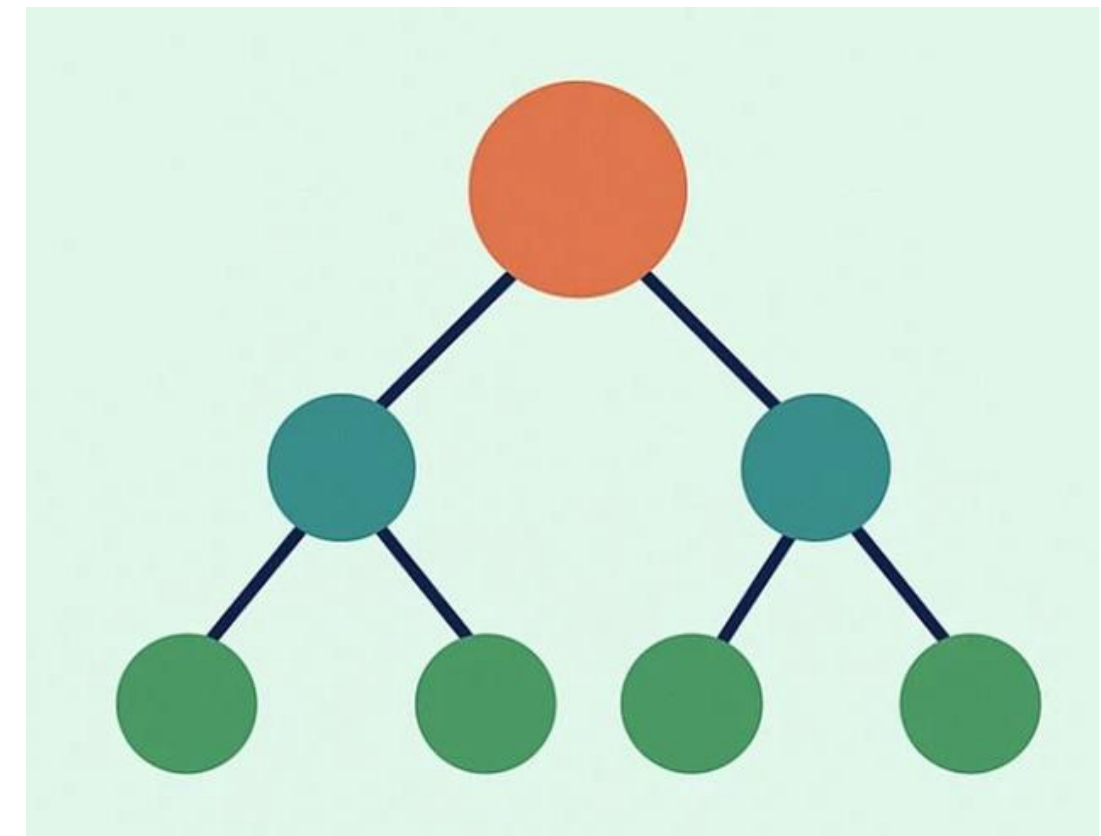
Professor

Department of Computer Science & Engineering

Acknowledgement :

Dr. Rajanikanth K (Academic Advisor, PESU)
Teaching Assistant (Arya Rajiv Chaloli)

- Decision tree is a flowchart-like diagram to make decisions based on a series of conditions.
- The learned function is represented by a decision tree.
- Decision tree can also be re-represented as **if-then rules** to improve human readability.



Machine Learning

Decision Trees



- Decision trees can be used for classification and regression problems
- Decision trees classify instances by sorting them down the tree from the root to some leaf node.
 - **Node:** Specifies some **attribute** of an instance to be tested
 - **Branch:** Corresponds to one of the **possible values** for an attribute



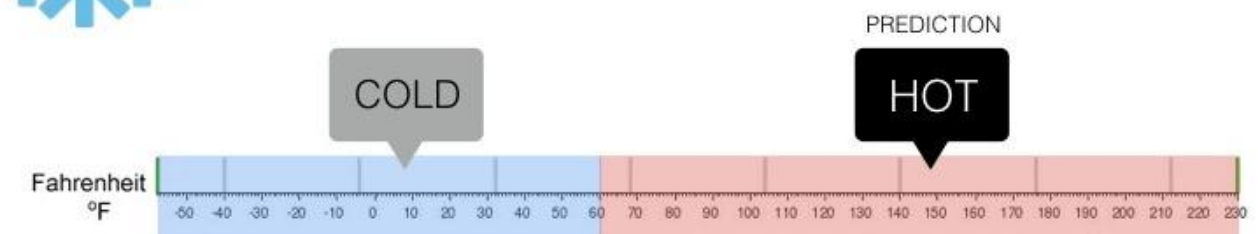
Regression

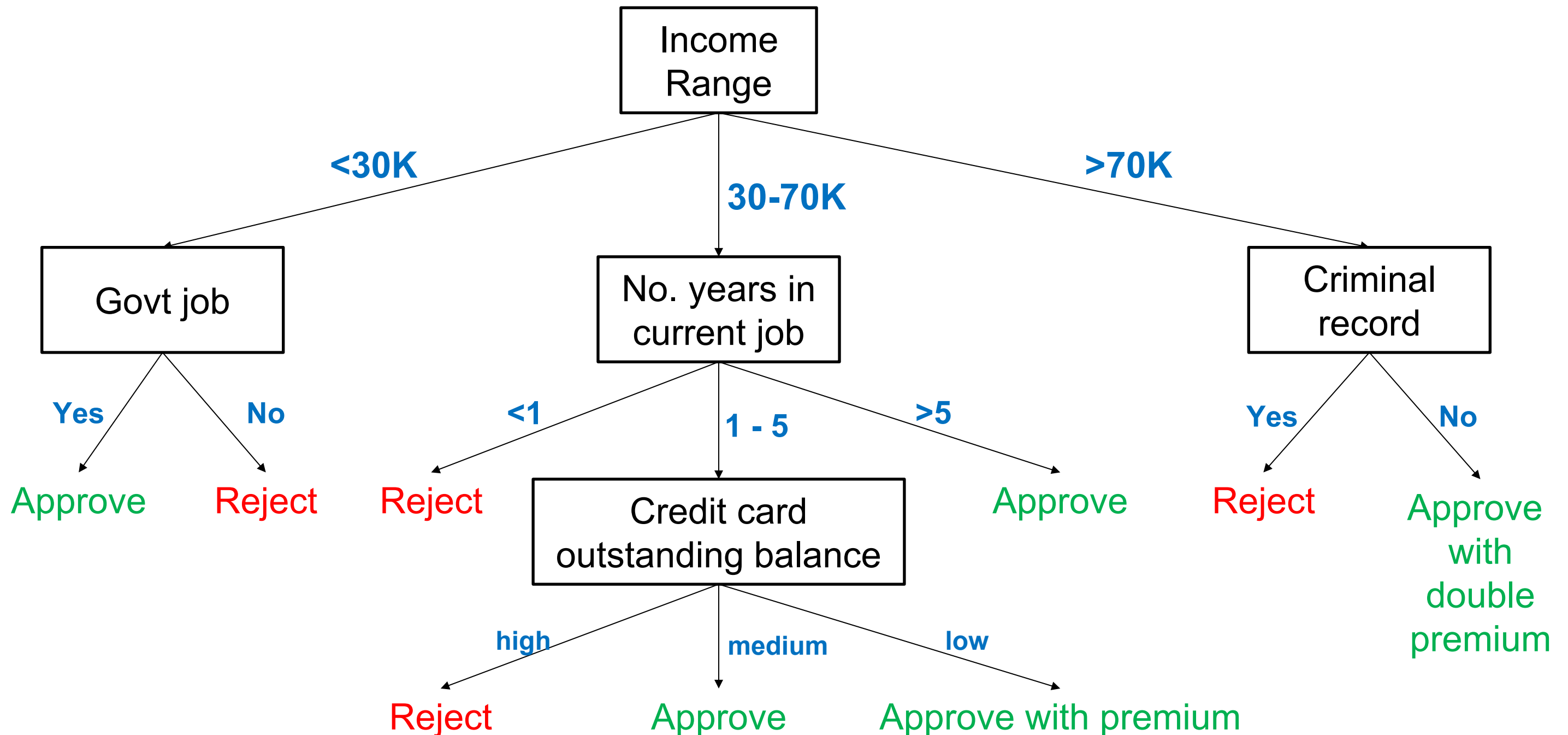
What is the temperature going to be tomorrow?



Classification

Will it be Cold or Hot tomorrow?





Root Node:

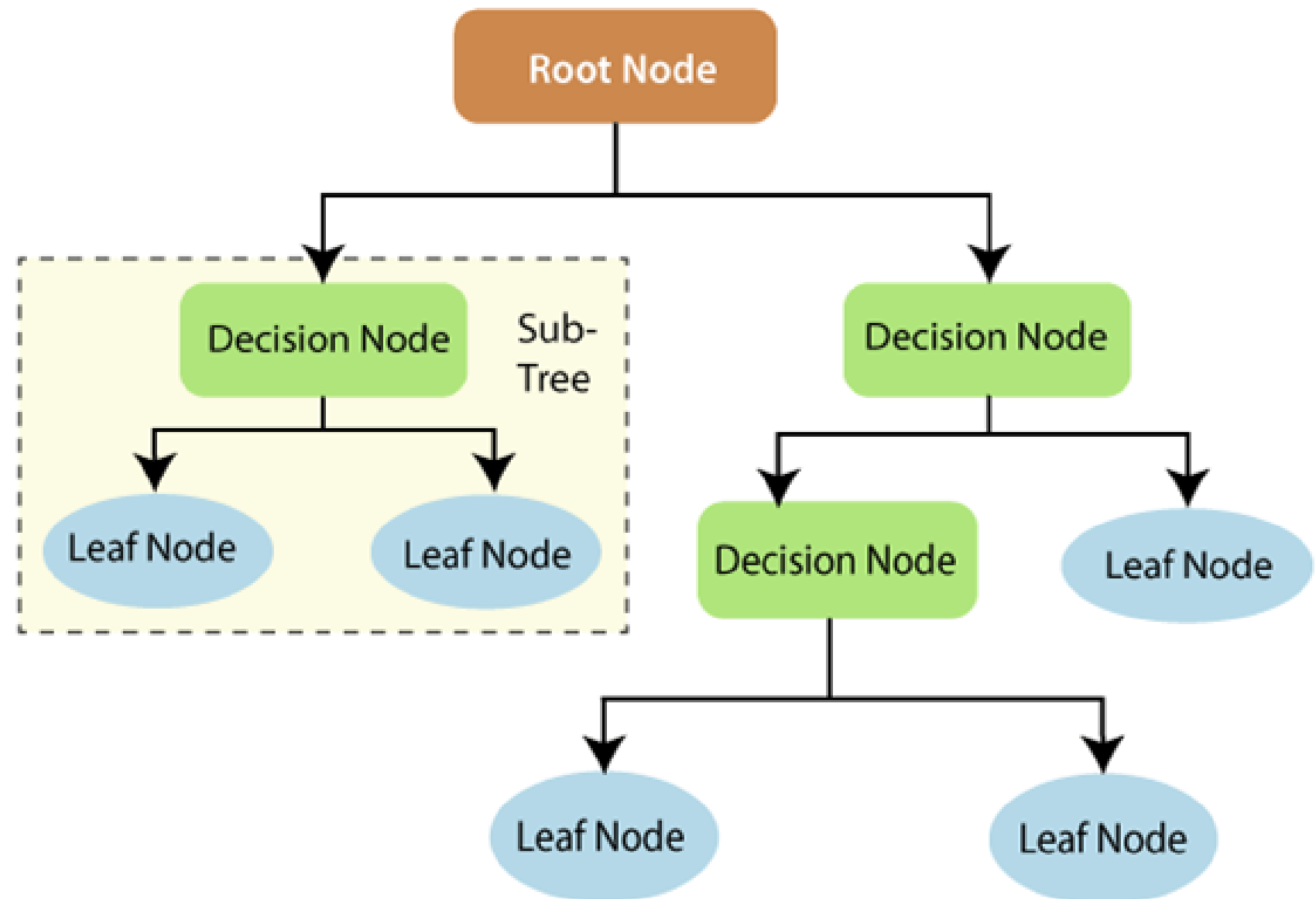
It's the topmost node where we start building the tree.

Leaf node:

It's the end of the decision tree and carries the decision.

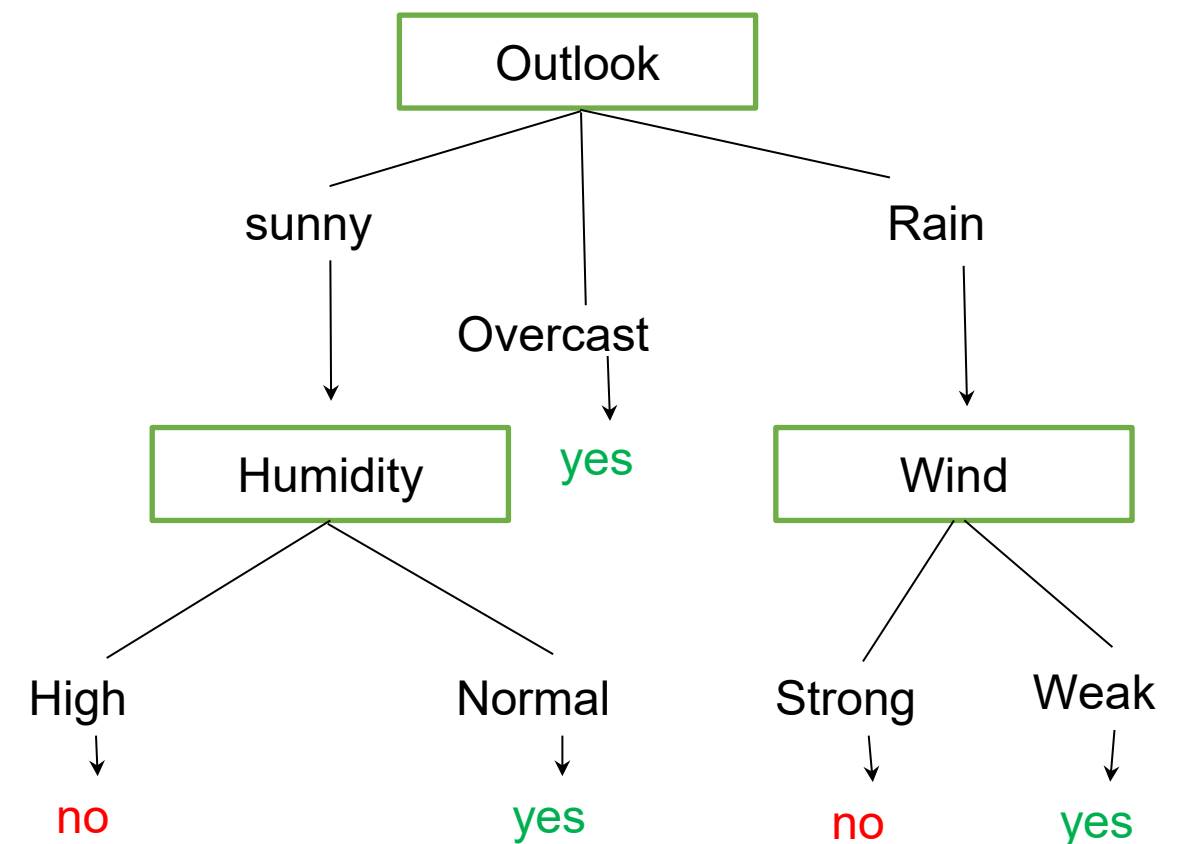
Decision Node:

Is a node where branching (testing) occurs. Certain attribute is tested and based on the attribute value, a particular branch is taken.



- Every node except leaf node is a decision node.
- Each decision node tests for an attribute.
- Each branch corresponds to an attribute value.
- Each leaf node assigns a classification label.
- Decision trees represent a **disjunction of conjunctions** (more expressive than FIND-S).

Example decision tree for concept to play Tennis



For example, the decision is yes whenever: $(\text{Outlook} = \text{Sunny} \wedge \text{Humidity} = \text{Normal}) \text{ OR } (\text{Outlook} = \text{Overcast}) \text{ OR } (\text{Outlook} = \text{Rain} \wedge \text{Wind} = \text{Weak})$

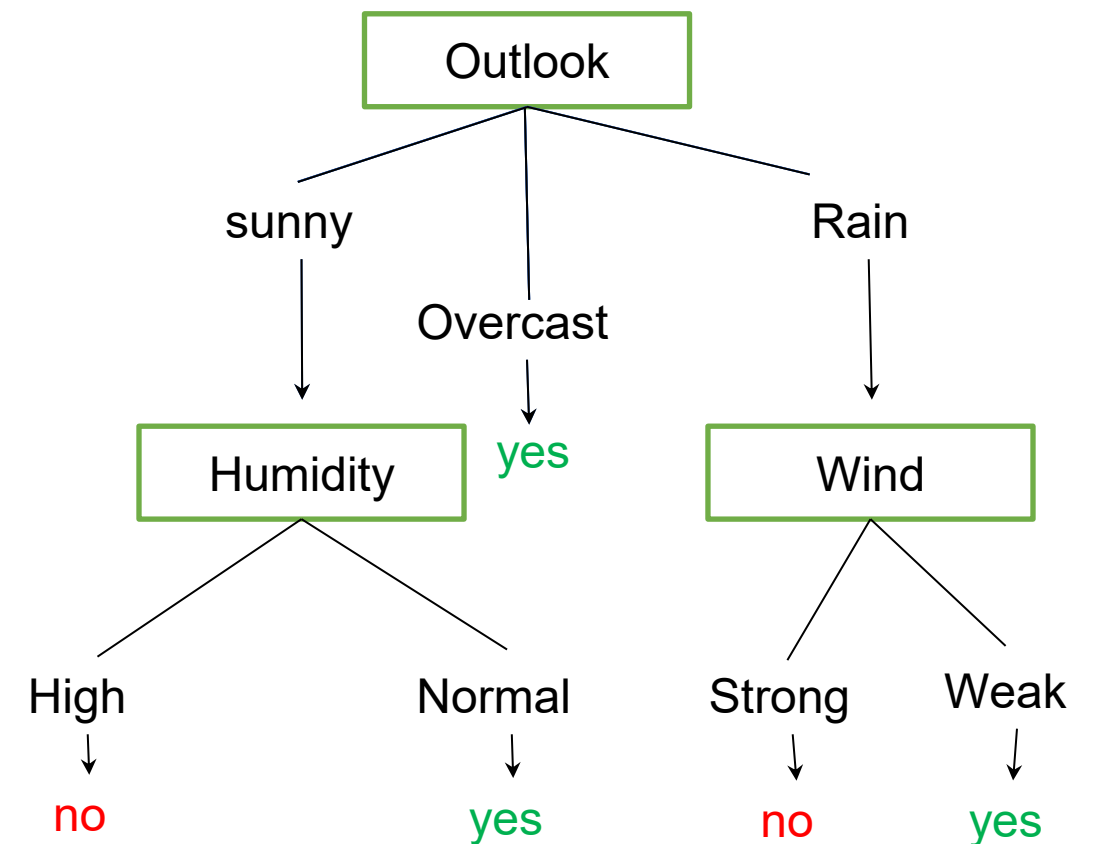
Machine Learning

Building Decision Trees



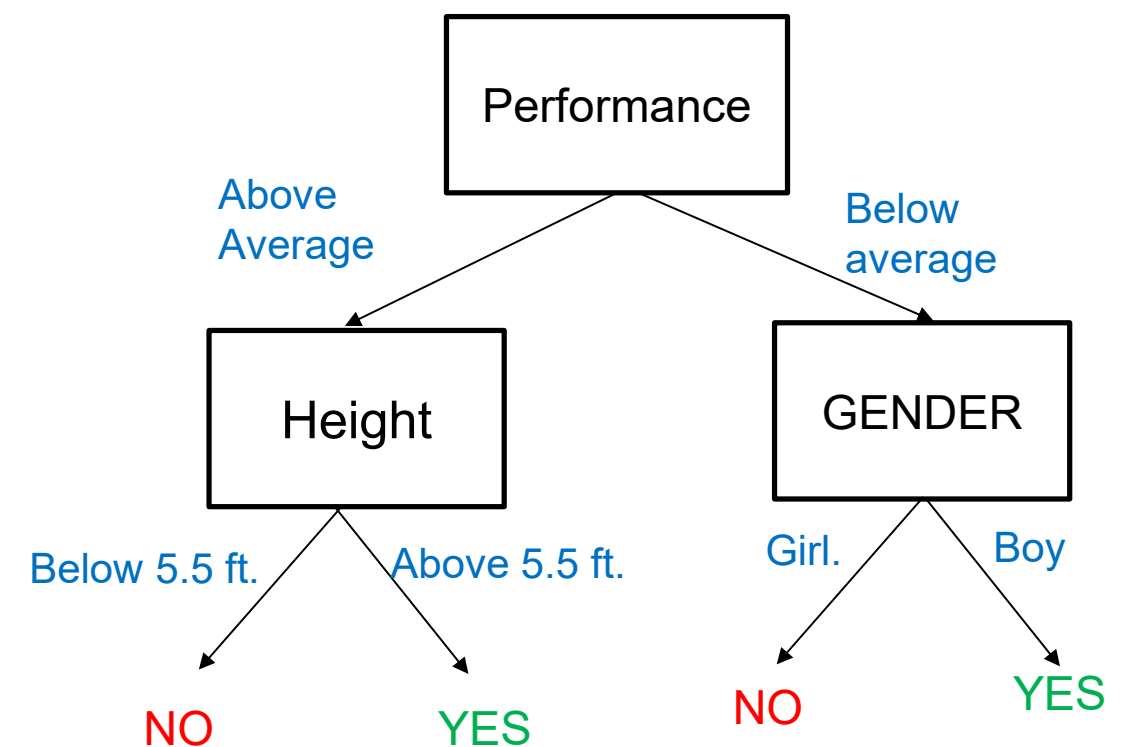
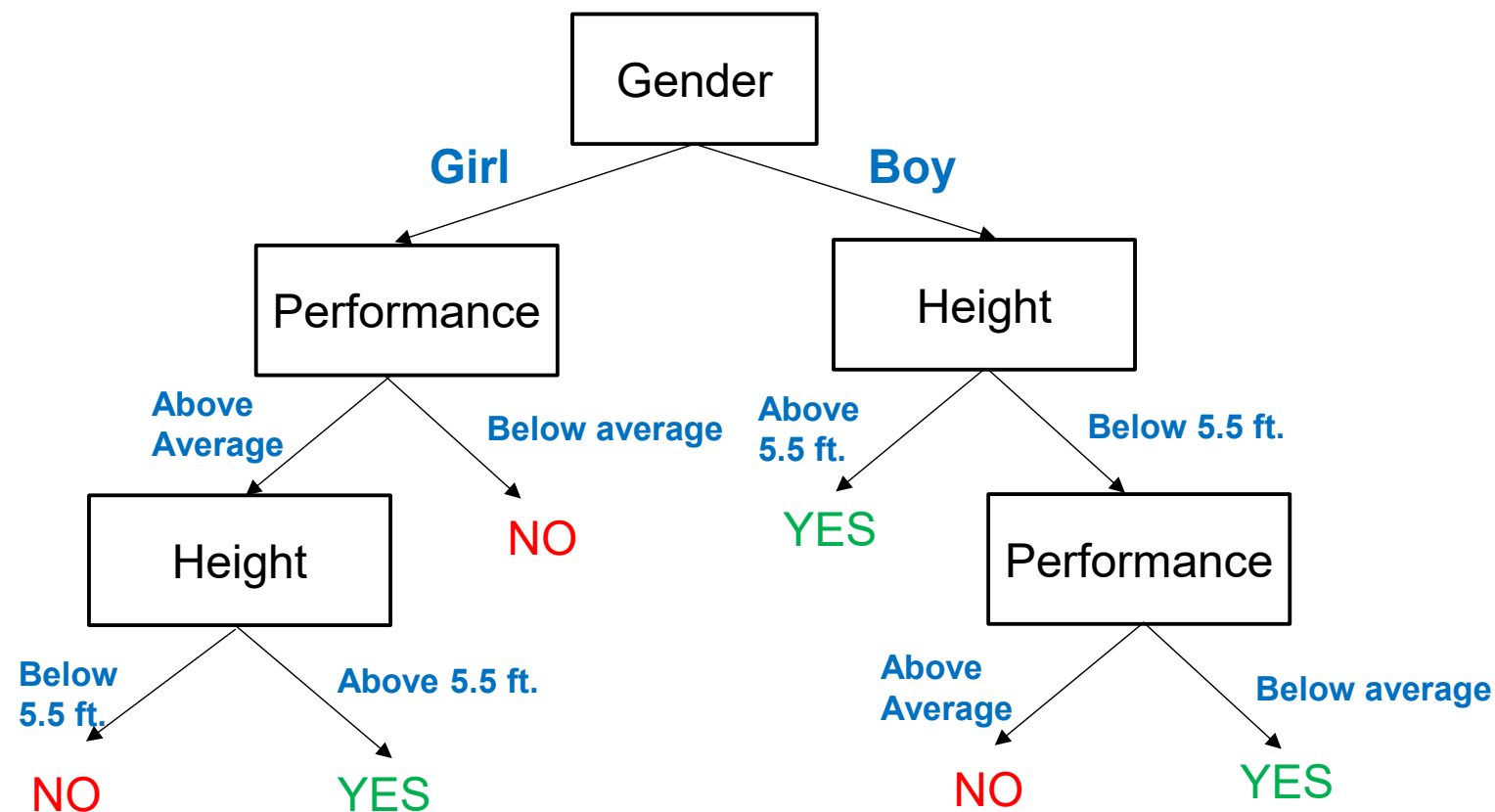
1. Select the “**best**” **decision attribute** for the next node - **crux of the problem**
2. Assign that attribute as the decision attribute for node.
3. For each value of the attribute, create new descendant of node (new branch is opened).
 - It will depend on the subset of instances that fall under that branch.
 - If no further decision making is required, you will reach the leaf node.
4. Sort training examples to new nodes.
5. If training examples perfectly classified, then STOP, else iterate over new leaf nodes.

Decision tree for concept to play Tennis



Decision trees vary based on the algorithm used to select the next “**best**” **decision attribute** node

Example: To find whether a student plays basketball or not



Which one do we prefer and why?

ID3 Algorithm (Iterative Dichotomiser 3)

It was developed in 1986 by Ross Quinlan.

The algorithm creates a multiway tree, finding for each node (i.e. in a greedy manner) the **categorical feature** that will yield the largest **information gain** for the targets.

It can be used for **classification** and regression.



CART

Classification and Regression Trees or CART for short is a term introduced by Leo Breiman. It can be used for classification or regression predictive modeling problems.

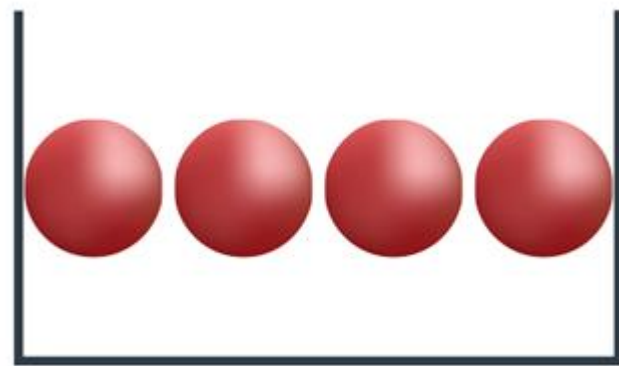
It uses **Gini Impurity** as the heuristic, for classification, and sum of least squares as the heuristic for regression.

- At a given point, we prefer an attribute as the decision node that partitions the set into subsets which are as homogeneous as possible. Ideally, no more decisions are required!
- Entropy is an indicator of non-homogeneity / uncertainty. It is a measure of disorder (indicator of how messy your data is)
 - **Entropy needs to be reduced.**
- Higher the entropy, the more non-homogeneous / uncertain / disordered the data set is.
- If the output y has c possible values, its entropy with respect to a set S is given by:

$$\text{Entropy}(A) = H(A) = - \sum p(x_i) \log_2 (p(x_i)); i = 1 \text{ to } c$$

Machine Learning

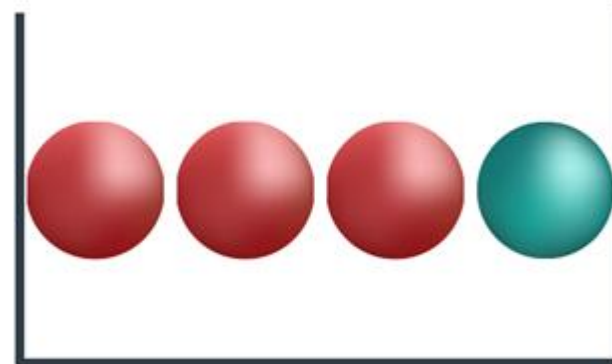
ID3 Algorithm - Entropy



BUCKET 1

LESS ENTROPY

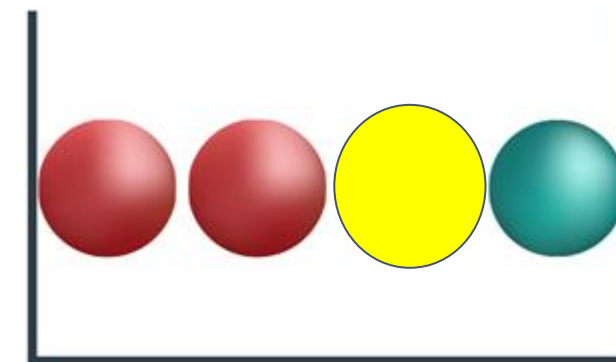
We know for sure that the ball coming out is red.



BUCKET 2

MEDIUM ENTROPY

We know with 75% certainty that the ball is red, and with 25% certainty that it's green.



BUCKET 3

HIGH ENTROPY

We know with 50% certainty that the ball is red, and 25% certainty with yellow and green each.

- So, we have more knowledge regarding bucket 1 and bucket 1 has the least entropy.

- Information gain is a measure of the effectiveness of an attribute in classifying the training data.
- **Information Gain = (Entropy before split – Entropy after split)**
 - Information gain should be as high as possible
- Information gain is the expected reduction in entropy caused by partitioning the examples according to the attribute.

- **Average information entropy: $I(A)$** is the weighted average of entropies of individual subsets.

$$I(A) = \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v)$$

$$\text{Gain}(S, A) \equiv \text{Entropy}(S) - I(A)$$

where $\text{Values}(A)$ is the set of all possible values for attribute A , and S_v is the subset of S for which attribute A has value v (i.e., $S_v = \{s \in S | A(s) = v\}$). Note

Steps to create a decision tree using the ID3 algorithm

1. Compute the **entropy** for data-set **entropy(s)**
1. For each attribute:
 - Calculate entropy for all other values **entropy(a)**
 - Take **average information entropy** for the current attribute
 - Calculate **gain** for the current attribute
3. Pick the **highest gain attribute**
4. **Repeat** until we get the tree we desire.



Outlook	Temp	Humidity	Windy	Play tennis
Sunny	High	High	Weak	No
Sunny	High	High	Strong	No
Overcast	High	High	Weak	Yes
Rainy	Medium	High	Weak	Yes
Rainy	Cool	Normal	Weak	Yes
Rainy	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Medium	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rainy	Medium	Normal	Weak	Yes
Sunny	Medium	Normal	Strong	Yes
Overcast	Medium	High	Strong	Yes
Overcast	High	Normal	Weak	Yes
Rainy	Medium	High	Strong	No

Example 1

Outlook	Temp	Humidity	Windy	Tennis
Sunny	High	High	Weak	No
Sunny	High	High	Strong	No
Overcast	High	High	Weak	Yes
Rainy	Medium	High	Weak	Yes
Rainy	Cool	Normal	Weak	Yes
Rainy	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Medium	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rainy	Medium	Normal	Weak	Yes
Sunny	Medium	Normal	Strong	Yes
Overcast	Medium	High	Strong	Yes
Overcast	High	Normal	Weak	Yes
Rainy	Medium	High	Strong	No

ENTROPY(S) - Calculating for entire Set S

|S| = 14 (training instances provided)

P(play tennis = yes) = p1 = 9/14 = 0.643

P(Play tennis = no) = p2 = 5/14 = 0.357

$$Entropy(S) \equiv \sum_{i=1}^c -p_i \log_2 p_i$$

$$= - (p1 \log_2 p1 + p2 \log_2 p2)$$

$$= 0.94$$

- We will now see, What happens if we split based on:
 - Outlook
 - Temp
 - Humidity
 - Windy
- First, we will see what will happen If we split based on Outlook.
 - Outlook has 3 values that means there will be three branches (three subsets) = sunny, overcast and rain.
 - We will calculate the entropy for each of these subsets.
 - We take a weighted sum of the entropy's of the three subsets which becomes the "Entropy after the split".
 - We then calculate the $\text{gain}(\text{Outlook}) = \text{Entropy before split} - \text{Entropy after split}$
- We repeat this process for each attribute.
- The attribute with the maximum gain is picked as the decision node.

Outlook	Temp	Humidity	Windy	Play tennis
Sunny	High	High	Weak	No
Sunny	High	High	Strong	No
Sunny	Medium	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Sunny	Medium	Normal	Strong	Yes
Overcast	High	High	Weak	Yes
Overcast	Cool	Normal	Strong	Yes
Overcast	Medium	High	Strong	Yes
Overcast	High	Normal	Weak	Yes
Rainy	Medium	High	Weak	Yes
Rainy	Cool	Normal	Weak	Yes
Rainy	Cool	Normal	Strong	No
Rainy	Medium	Normal	Weak	Yes
Rainy	Medium	High	Strong	No

Entropy after Split based on Outlook and Gain Calculation

$$|S| = 14, \text{Entropy}(S) = 0.94$$

- Splitting based on Outlook - There will be three subsets S_{sunny} , S_{overcast} , S_{rain} such that $|S_{\text{sunny}}| = 5$, $|S_{\text{overcast}}| = 4$ and $|S_{\text{rain}}| = 5$
- Entropy of these subsets:
 - $E(S_{\text{sunny}}) = - (\frac{2}{5} \log_2 \frac{2}{5} + \frac{3}{5} \log_2 \frac{3}{5}) = 0.971$
 - $E(S_{\text{overcast}}) = - (1 \log_2 1 + 0 \log_2 0) = 0$
 - $E(S_{\text{rain}}) = - (\frac{3}{5} \log_2 \frac{3}{5} + \frac{2}{5} \log_2 \frac{2}{5}) = 0.971$
- Avg Information of Outlook : weighted avg of the entropies of the subsets:
$$I(\text{Outlook}) = \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v) = \frac{5}{14} * 0.971 + \frac{4}{14} * 0 + \frac{5}{14} * 0.971 = 0.693$$
- $\text{Gain}(S, \text{outlook}) = \text{Entropy}(S) - I(\text{outlook}) = 0.94 - 0.693 = 0.247$

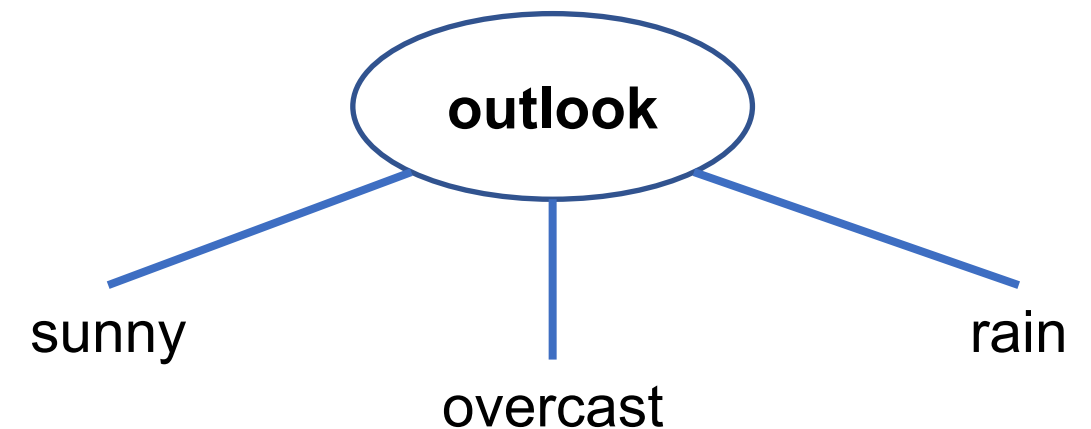
Entropy after Split based on Temp and Gain Calculation

$$|S| = 14, \text{Entropy}(S) = 0.94$$

- Splitting based on Temp - There will be three subsets S_{high} , S_{medium} , S_{cool} such that $|S_{\text{high}}| = 4$, $|S_{\text{medium}}| = 6$ and $|S_{\text{cool}}| = 4$
- Entropy of these subsets:
 - $E(S_{\text{high}}) = - (2/4 \log_2 2/4 + 2/4 \log_2 2/4) = 1$
 - $E(S_{\text{medium}}) = - (4/6 \log_2 4/6 + 2/6 \log_2 2/6) = 0.918$
 - $E(S_{\text{cool}}) = - (3/4 \log_2 3/4 + 1/4 \log_2 1/4) = 0.81125$
- Avg Information of Outlook : weighted avg of the entropies of the subsets:
$$I(\text{Temp}) = \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v) = 4/14 * 1 + 6/14 * 0.918 + 4/14 * 0.81125$$
$$= 0.9109 = 0.911$$
- $\text{Gain}(S, \text{Temp}) = \text{Entropy}(S) - I(\text{Temp}) = 0.94 - 0.911 = 0.029$

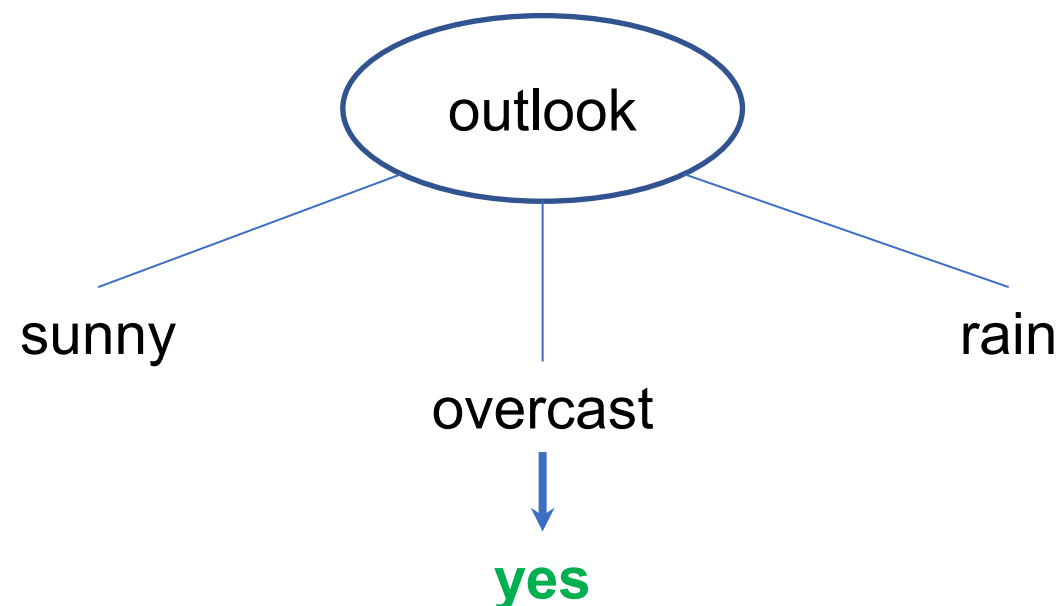
- We do a similar calculation to calculate the following:
 - $\text{Gain}(S, \text{Humidity})$
 - $\text{G}(S, \text{Windy})$
- Hence, we can summarize the calculations as:

property	value
Entropy(S)	0.94
G(S, Outlook)	0.247
G(S, Temp)	0.029
G(S, Humidity)	0.152
G(S, Windy)	0.048



- We pick the **attribute with maximum Gain** as the decision node.

- We choose our root node as outlook and split according to its three values.
- Entropy of overcast was 0 which means it is single valued and hence we can come to a conclusion from overcast .

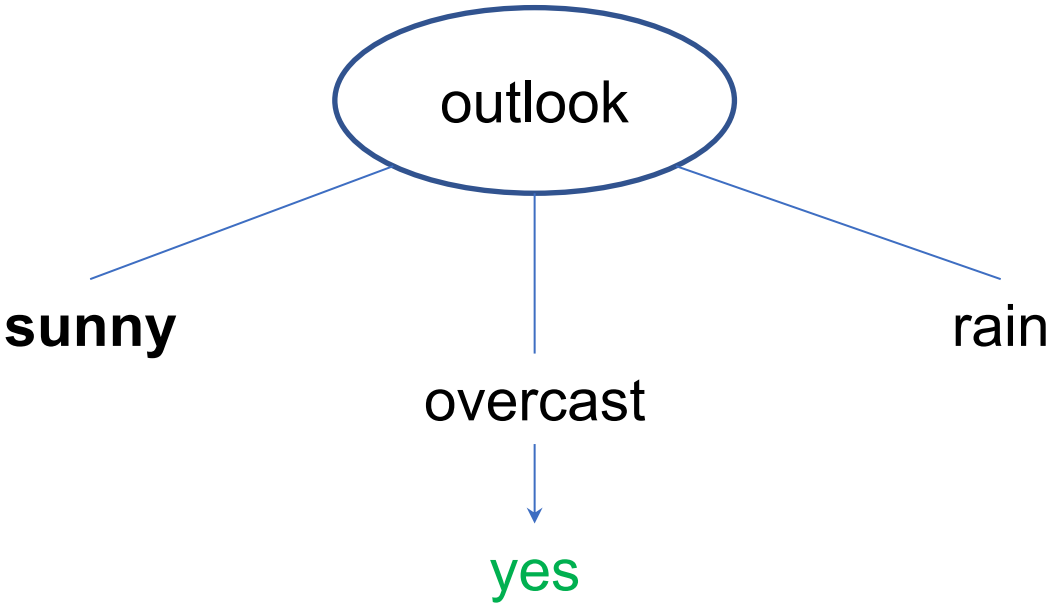


$E(S_{\text{sunny}})$	0.971
$E(S_{\text{Overcast}})$	0
$E(S_{\text{rain}})$	0.971

Repeat the process to pick the descendant node for each branch

With Outlook = Sunny, data would look like this:

Outlook	Temp	Humidity	Windy	Play tennis
Sunny	High	High	Weak	No
Sunny	High	High	Strong	No
Sunny	Medium	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Sunny	Medium	Normal	Strong	Yes



$E(S_{\text{sunny}})$	0.971
$E(S_{\text{Overcast}})$	0
$E(S_{\text{rain}})$	0.971

Entropy after Split based on Temp and Gain Calculation

$$|S_{\text{sunny}}| = 5, \text{Entropy}(S_{\text{sunny}}) = 0.971$$

- Splitting based on Temp - There will be three subsets S_{high} , S_{medium} , S_{cool} such that $|S_{\text{high}}| = 2$, $|S_{\text{medium}}| = 2$ and $|S_{\text{cool}}| = 1$
- Entropy of these subsets:
 - $E(S_{\text{high}}) = - (0 \log_2 0 + 1 \log_2 1) = 0$
 - $E(S_{\text{medium}}) = - (1/2 \log_2 1/2 + 1/2 \log_2 1/2) = 1$
 - $E(S_{\text{cool}}) = - (1 \log_2 1 + 0 \log_2 0) = 0$
- Avg Information of Outlook : weighted avg of the entropies of the subsets:
$$I(\text{Temp}) = \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v) = 2/5 * 1 = 0.4$$
- $\text{Gain}(S_{\text{sunny}}, \text{Temp}) = \text{Entropy}(S_{\text{sunny}}) - I(\text{Temp}) = 0.971 - 0.4 = 0.571$

Entropy after Split based on Humidity and Gain Calculation

$$|S_{\text{sunny}}| = 5, \text{Entropy}(S_{\text{sunny}}) = 0.971$$

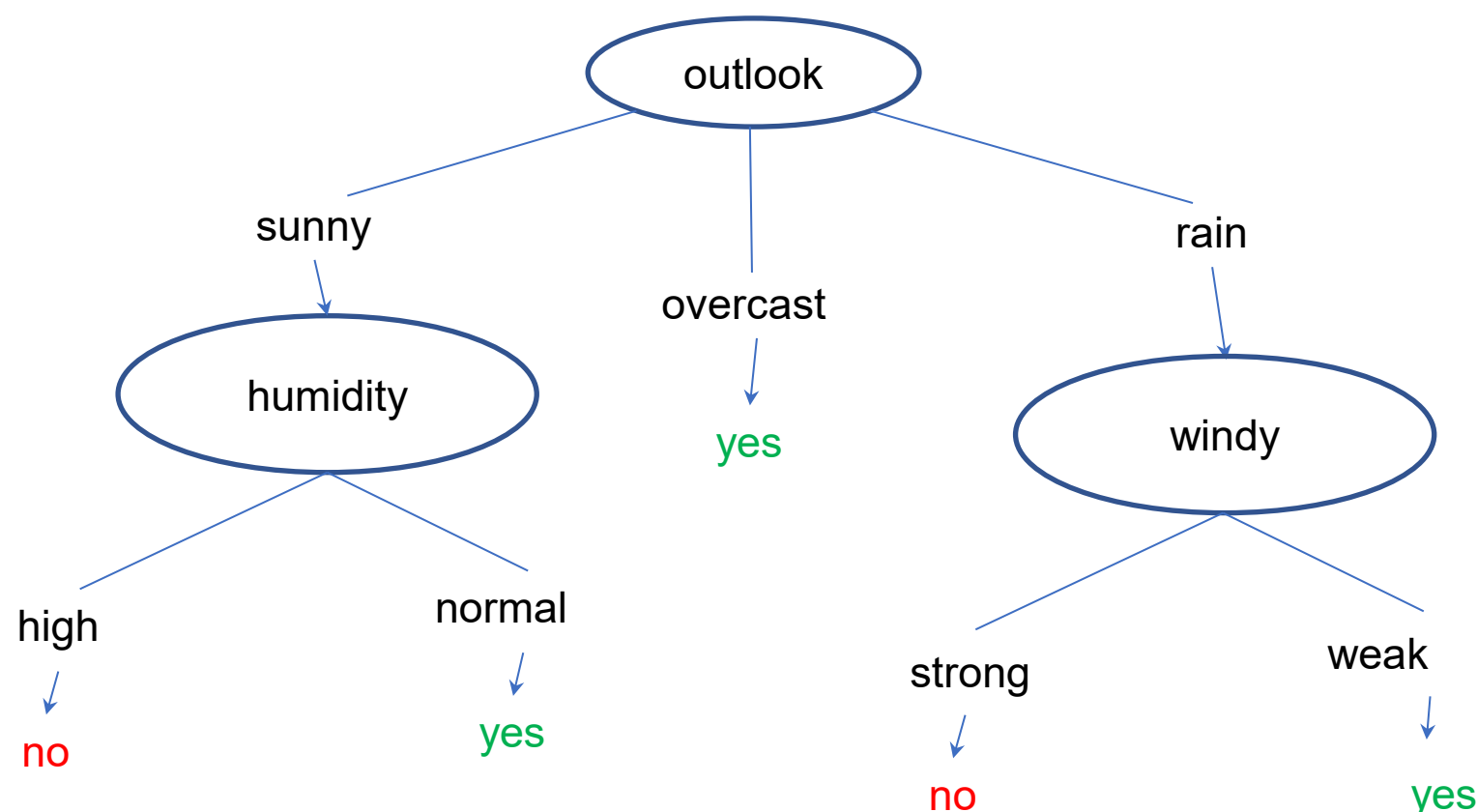
- Splitting based on Humidity - There will be two subsets S_{high} , S_{normal} such that $|S_{\text{high}}|= 2$, $|S_{\text{normal}}|= 3$
- Entropy of these subsets:
 - $E(S_{\text{high}}) = - (0 \log_2 0 + 1 \log_2 1) = 0$
 - $E(S_{\text{normal}}) = - (1 \log_2 1 + 0 \log_2 0) = 0$
- Avg Information of Outlook : weighted avg of the entropies of the subsets:
$$I(\text{Humidity}) = \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v) = 0$$
- $\text{Gain}(S_{\text{sunny}}, \text{Humidity}) = \text{Entropy}(S_{\text{sunny}}) - I(\text{Humidity}) = 0.971 - 0 = 0.971$

Entropy after Split based on Windy and Gain Calculation

$$|S_{\text{sunny}}| = 5, \text{Entropy}(S_{\text{sunny}}) = 0.971$$

- Splitting based on Windy - There will be two subsets S_{weak} , S_{strong} such that $|S_{\text{weak}}|= 3$, $|S_{\text{strong}}|= 2$
- Entropy of these subsets:
 - $E(S_{\text{weak}}) = - (\frac{1}{3} \log_2 \frac{1}{3} + \frac{2}{3} \log_2 \frac{2}{3}) = 0.918$
 - $E(S_{\text{strong}}) = - (\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2}) = 1$
- Avg Information of Outlook : weighted avg of the entropies of the subsets:
$$I(\text{Windy}) = \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \text{Entropy}(S_v) = 0.951$$
- $\text{Gain}(S_{\text{sunny}}, \text{Windy}) = \text{Entropy}(S_{\text{sunny}}) - I(\text{Windy}) = 0.971 - 0.951 = 0.02$

- Humidity has the highest gain hence we choose humidity at second level after outlook as sunny
- Remember that both value of humidity (high and normal) had entropy of 0 which means we have reached the leaf node with this.
- Doing the same procedure for the table with outlook=rain we expand the rain leading node to get the following decision tree.



property	value
Entropy(S_{sunny})	0.97
$G(S_{\text{sunny}}, \text{temp})$	0.571
$G(S_{\text{sunny}}, \text{humidity})$	0.971
$G(S_{\text{sunny}}, \text{windy})$	0.02

Machine Learning

Construct Decision Tree



salary	Location	job acceptance
Tier1	MUM	YES
Tier 1	BLR	YES
Tier 2	BLR	NO
Tier 1	HYD	NO
Tier 2	MUM	YES
Tier 1	HYD	NO

Example 2

consider the data set of concept of accepting a job

Ensure there are no duplicate rows in your data

Machine Learning

Construct Decision Tree



salary	Location	job acceptance
Tier1	MUM	YES
Tier 1	BLR	YES
Tier 2	BLR	NO
Tier 1	HYD	NO
Tier 2	MUM	YES

Solution

Step 1 : Entropy of the entire set:

$$|S| = 5$$

$$P(\text{yes}) = 3/5, P(\text{no}) = 2/5$$

$$\text{Entropy}(S) = 0.971$$

Step 2 : Entropy after Split

a) Based on salary

$$\rightarrow |S_{\text{Tier1}}| = 3$$

$$P(\text{yes}) = 2/3, P(\text{no}) = 1/3$$

$$\text{Entropy}(S_{\text{Tier1}}) = 0.918$$

$$\rightarrow |S_{\text{Tier2}}| = 2$$

$$P(\text{yes}) = 1/2, P(\text{no}) = 1/2$$

$$\text{Entropy}(S_{\text{Tier2}}) = 1$$

$$\rightarrow I(\text{salary}) = 3/5 * 0.918 + 2/5 * 1 =$$

$$0.9508$$

$$\rightarrow \text{Gain}(S, \text{salary}) = 0.971 -$$

$$0.9508 = 0.0202$$

Machine Learning

Construct Decision Tree



salary	Location	job acceptance
Tier1	MUM	YES
Tier 1	BLR	YES
Tier 2	BLR	NO
Tier 1	HYD	NO
Tier 2	MUM	YES

Solution

Step 1 : Entropy of the entire set:

$$|S| = 5$$

$$P(\text{yes}) = 3/5, P(\text{no}) = 2/5$$

$$\text{Entropy}(S) = 0.971$$

Step 2 : Entropy after Split

b)Based on Location

$$\rightarrow |S_{\text{MUM}}| = 2$$

$$P(\text{yes}) = 1, P(\text{no}) = 0$$

$$\text{Entropy}(S_{\text{MUM}}) = 0$$

$$\rightarrow |S_{\text{BLR}}| = 2$$

$$P(\text{yes}) = \frac{1}{2}, P(\text{no}) = \frac{1}{2}$$

$$\text{Entropy}(S_{\text{BLR}}) = 1$$

$$\rightarrow |S_{\text{HYD}}| = 1$$

$$P(\text{yes}) = 0, P(\text{no}) = 1$$

$$\text{Entropy}(S_{\text{HYD}}) = 0$$

$$\rightarrow I(\text{Location}) = \frac{2}{5} * 1 = 0.4$$

$$\rightarrow \text{Gain}(S, \text{Location}) = 0.971 - 0.4 =$$

0.571

Machine Learning

Construct Decision Tree



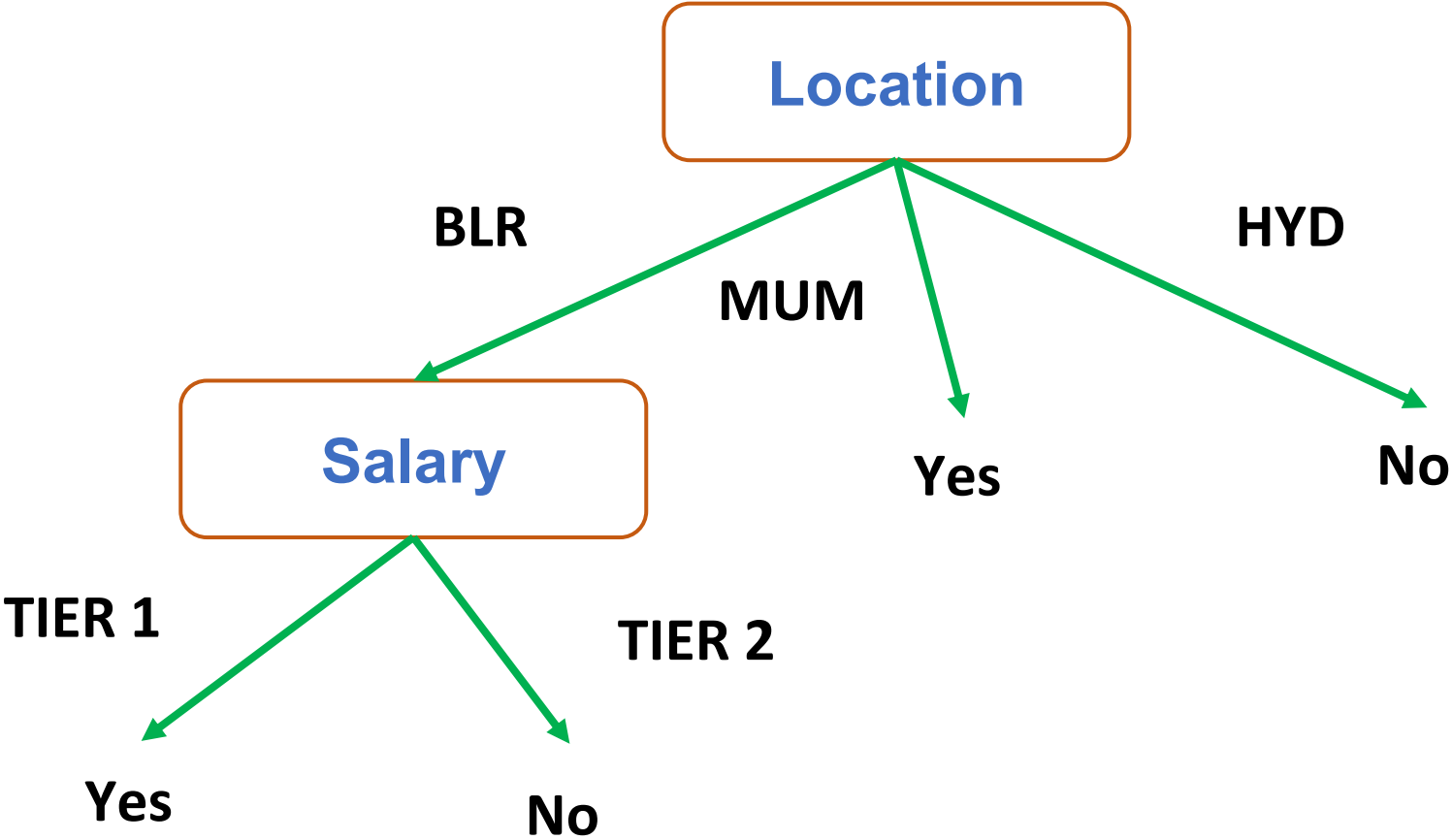
salary	Location	job acceptance
Tier1	MUM	YES
Tier 1	BLR	YES
Tier 2	BLR	NO
Tier 1	HYD	NO
Tier 2	MUM	YES

Decision Node - Max Gain

$\text{Gain}(S, \text{salary}) = 0.0202$

$\text{Gain}(S, \text{location}) = 0.571$

The root node : Location



Machine Learning

Construct Decision Tree



M	N	O	Y
A	C	X	T
A	C	Z	E
A	D	X	T
A	D	Z	T
B	C	X	E
B	C	Z	F
B	D	X	F
B	D	Z	F

Example 3

Machine Learning

Construct Decision Tree



M	N	O	Y
A	C	X	T
A	C	Z	E
A	D	X	T
A	D	Z	T
B	C	X	E
B	C	Z	F
B	D	X	F
B	D	Z	F

Step 1:

Compute entropy of the dataset:

No of points belonging to class T: 3

No of points belonging to class E: 2

No of points belonging to class F: 3

Total no. of points: 8

$$\text{Entropy}(S) = -\frac{3}{8}\log_2\left(\frac{3}{8}\right) - \frac{2}{8}\log_2\left(\frac{2}{8}\right) - \frac{3}{8}\log_2\left(\frac{3}{8}\right)$$

$$\text{Entropy}(S) = 1.56$$

Note that if the target attribute can take on c possible values, the entropy can be as large as $\log_2 c$.



Weather	Mood	Genre	Alone	Watch Movie
Sunny	Happy	Comedy	No	Yes
Sunny	Bored	Drama	Yes	No
Cloudy	Lazy	Thriller	Yes	Yes
Rainy	Sad	Romance	No	Yes
Rainy	Bored	Comedy	Yes	No
Rainy	Happy	Thriller	Yes	No
Cloudy	Happy	Romance	No	Yes
Sunny	Bored	Comedy	No	No
Sunny	Happy	Romance	Yes	Yes
Rainy	Lazy	Drama	No	Yes
Sunny	Lazy	Thriller	Yes	Yes
Cloudy	Bored	Comedy	Yes	Yes
Cloudy	Sad	Romance	No	Yes
Rainy	Sad	Thriller	Yes	No

Example 4

Entropy of the entire set = 0.9403

Step 1. Root Node Selection

Information Gains:

- Mood = **0.2799** (highest)
- Weather = 0.2467
- Genre = 0.2260
- Alone = 0.0903

Root Node = **Mood**

Step 2. Split on Mood

Case 1: Mood = Happy

Entropy = 0.8113

Now calculate IG for attributes in this subset:

- Weather: Gain = 0.311
- Genre: Gain = 0.311
- Alone: Gain = 0.311

All equal. Any can be chosen. Let's pick **Genre**.

Case 2: Mood = Bored

Entropy = 0.8113

- Weather Gain = 0.311
- Genre Gain = 0.311
- Alone Gain = 0.311

Pick any (**Genre**).

Case 3: Mood = Lazy

Entropy = 0.9183

- Weather Gain = **0.918**
- Genre Gain = 0.2516
- Alone Gain = 0.2516

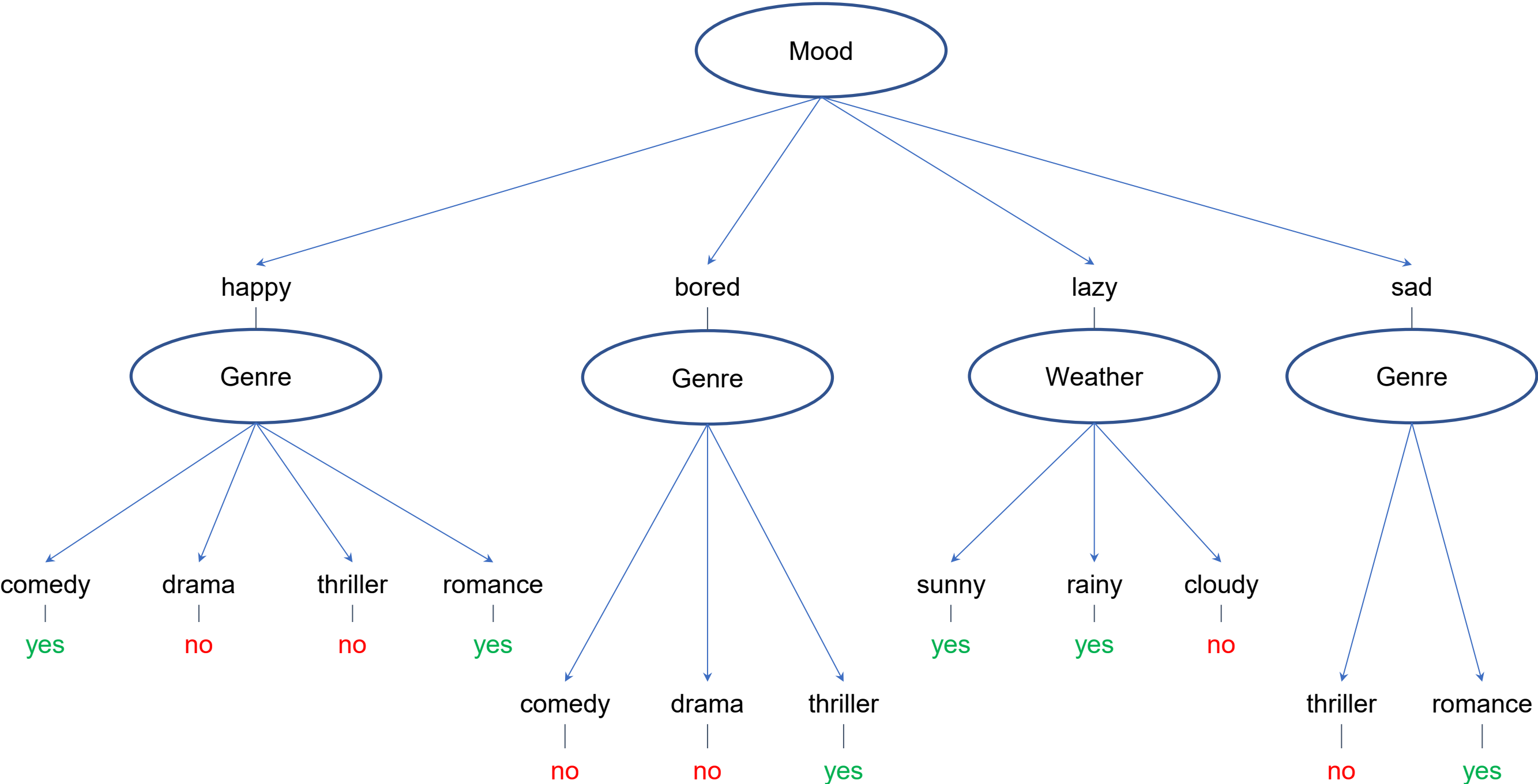
Pick **Weather**.

Case 4: Mood = Sad

Entropy = 0.9183

- Weather Gain = 0.251
- Genre Gain = **0.9183**
- Alone Gain = 0.2516

Pick **Genre**.





UE24CS352A: Machine Learning

Dr. Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu



UE24CS352A: Machine Learning

Dr. Shylaja S S

Professor

Department of Computer Science & Engineering

Acknowledgement :

Dr. Rajanikanth K (Academic Advisor, PESU)
Teaching Assistant (Arya Rajiv Chaloli)

- **Hypothesis space** of ID3 is the **set of all possible decision trees**
- ID3 performs a **simple-to-complex, informed search** through this hypothesis space
 - uses **gain** as the heuristic
 - **greedy search** strategy
- **Outputs a single hypothesis** / decision tree. Does not guarantee an optimal solution.

- **No backtracking** – local minima (accepts the first generated tree it encounters)
 - alternative hypothesis (decision tree) consistent with the data are not explored at all
 - Search performed by ID3 is **incomplete search**: we are not searching all the possible hypotheses
- **Statistically-based search choices**
 - At every step, we look at set of all the instances that fall under that branch. (whereas find-S uses one example at a time)
 - **Robust to noisy data**(replace missing value by average/mode at that level)

Machine Learning

ID3 - Inductive Bias



- Prefers Shortest Trees (Occam's razor) + Prefers attributes with highest gain --- Preference Bias (ID3 Search Strategy)
- Heuristically, ID3 should find the shortest consistent tree as it is biased to favor trees that place attributes with high information gain closest to the root.
- Inductive Bias of ID3 favors attributes with many values as these are more likely to lead to reduced entropy and thus increased gain.

The inductive bias of **ID3** follows from its **search strategy**

The inductive bias of the **FIND-S / CANDIDATE-ELIMINATION** algorithm follows from the **definition of its search space**

The most important issues in decision trees:

- a. **Overfitting of data** (will be discussed in detail)
 - ID3 makes the tree too specific to the given data instances.
- b. Handling continuous attributes
- c. Handling attributes with missing values
- d. Handling attributes with differing costs
- d. Does not guarantee small order of execution time.
 - Sometimes It can take enormously long time (as there could be too many choices at every step)

- Overfitting means becoming too specific to the training set
- Reasons when overfitting turns out to be bad:
 - Noise in data
 - Training set too small
 - Training set biased (not representative of target function)
- Trying to make the tree very specific to the training instances --- the tree may do really bad when tried on the new instances/test data.
 - Model does wonders with training data but does very badly with test data.
 - Overfitting can occur with almost all ML models.

Consider this new test instance:

Sunny, High, Normal, Strong, **No(true label)**

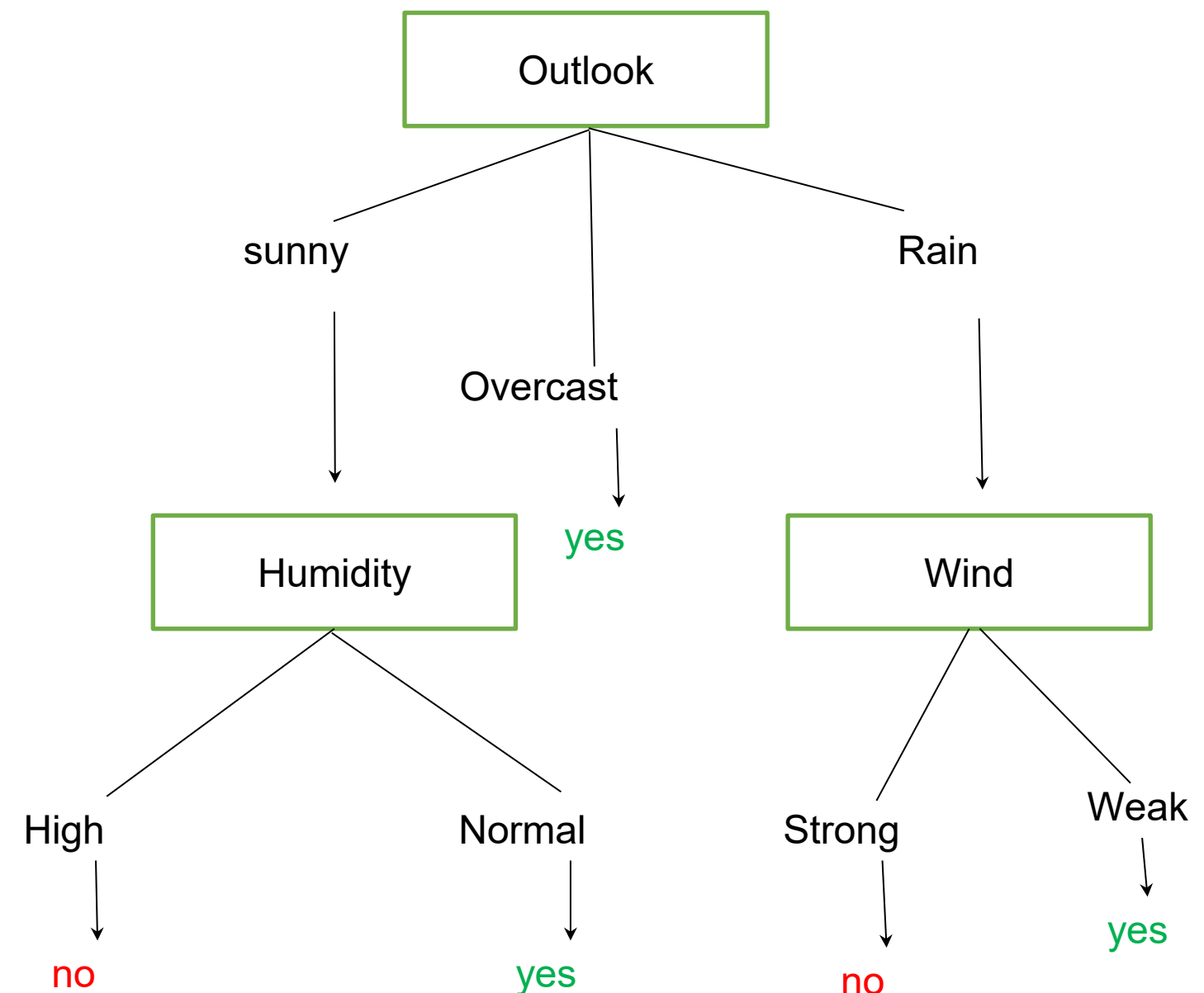
we trace the decision tree and get “**Yes**”

With our decision tree, once it sees sunny it only checks humidity for which if value is normal, it outputs a yes.

--- leads to an error

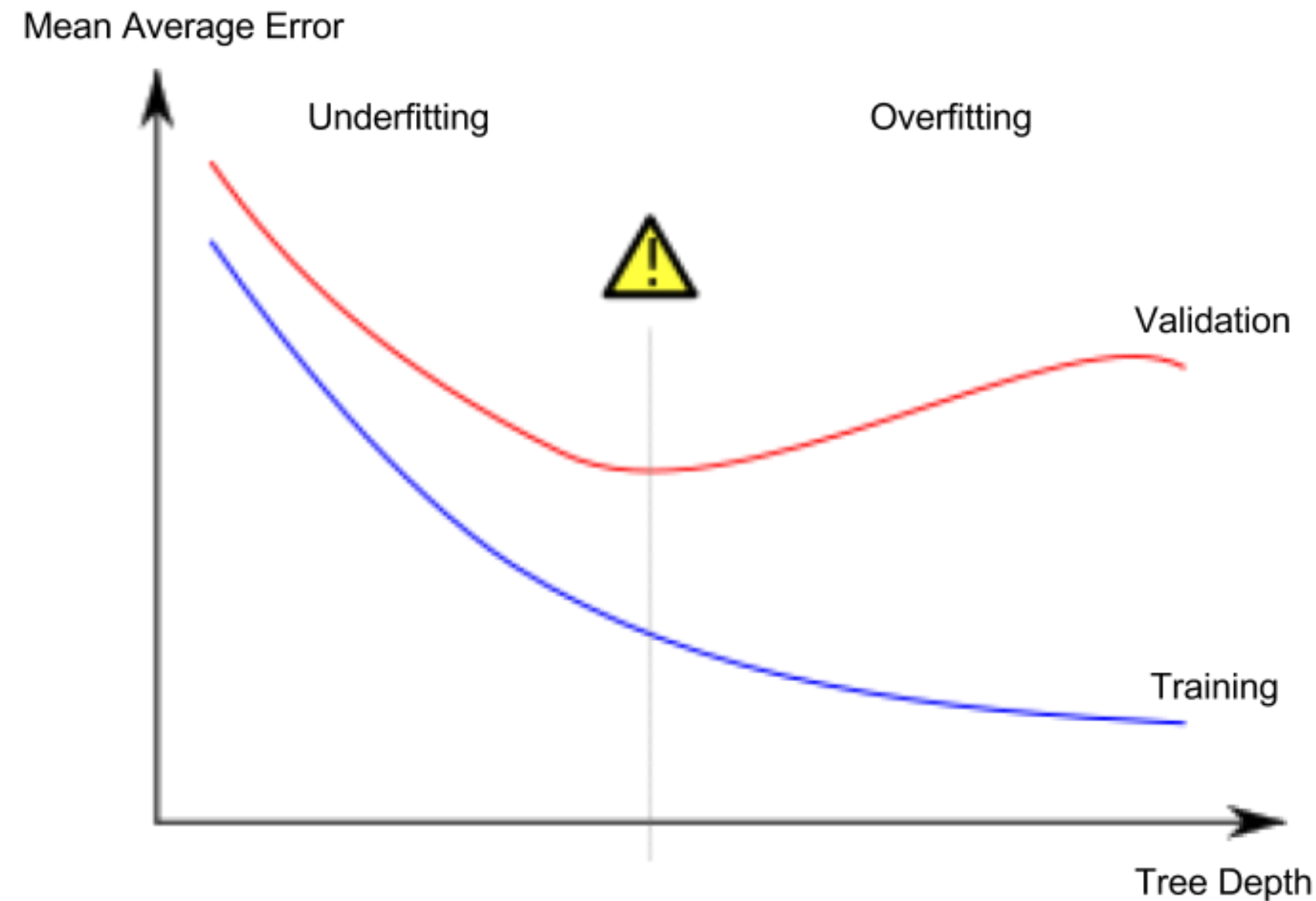
In our training set, if we had even 1 example where for humidity it was a no under this branch, we would have required another decision node (tree would have increased in depth)

Decision tree for concept to play Tennis



Machine Learning

Overfitting of data



What is the depth at which we should stop making the decision tree further?

- not possible to know
- experiment and see

Consider error of hypothesis h over

- training data: $error_{train}(h)$
- entire distribution $\mathcal{D}$ of data: $error_{\mathcal{D}}(h)$

Hypothesis $h \in H$ **overfits** training data if there is an alternative hypothesis $h' \in H$ such that

$$error_{train}(h) < error_{train}(h')$$

and

$$error_{\mathcal{D}}(h) > error_{\mathcal{D}}(h')$$

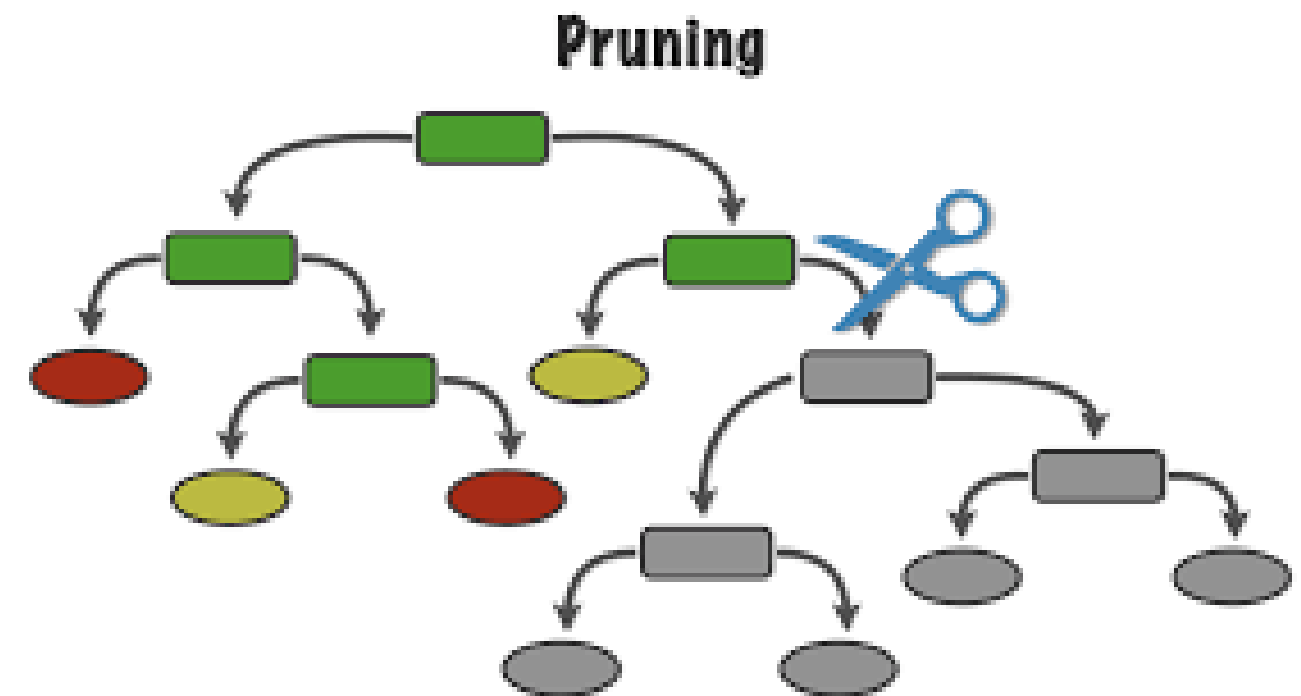
This alternate hypothesis h' must come from:

- either a different algorithm
- or by doing some changes in the Decision tree built by ID3

Definition: Given a hypothesis space H , a hypothesis $h \in H$ is said to **overfit** the training data, if there exists some alternative hypothesis $h' \in H$, such that

- h has **smaller error** than h' **over the training examples**,
- but h' has a **smaller error** than h **over the entire distribution** of instances.

- Use another algorithm to see if you can come up with a better tree
- Use ID3 itself but modify the tree to make it perform better over new instances.
 - There is no guarantee that the change would produce a better tree. You will have to experiment and see.
 - **Pruning**: A way of altering the decision tree - cut off the tree at some point and make that as the leaf node (put the decision as that category to which majority of the instances belong to under that branch).
This implies we are making the tree more general.



Prune the tree at different levels and see if any improvement comes in.

Early Termination

Approaches that **stop growing the tree earlier**, before it reaches the point where it perfectly classifies the training data

Halt the growth of the tree when the goodness of the split falls below a threshold

Ex. Should have a certain minimum information gain or setting up a depth before the algorithm begins

Post Pruning

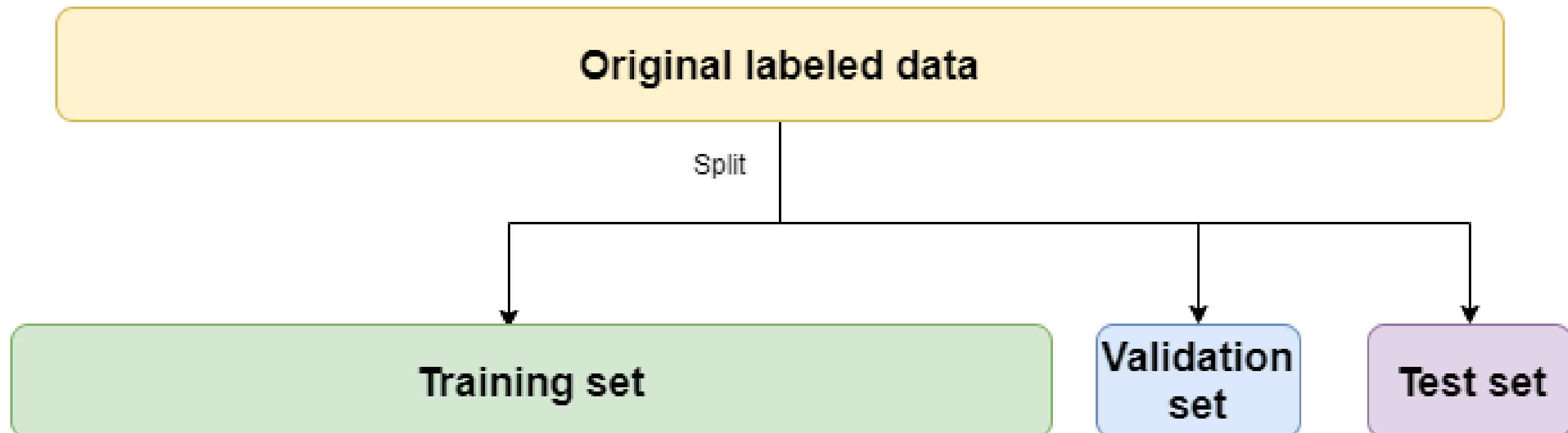
Approaches that allow the tree to overfit the data, and then **post-prune the tree**.

This is found to be more effective due to difficulty in finding when to stop growing the tree

Fully grow the tree and then selectively chop a node and replace it with its most popular class.

The available data has been split into three subsets:

- the **training** examples,
- the **validation** examples **used for pruning the tree**, and
- a set of **test** examples used to provide an unbiased estimate of accuracy over future unseen examples.



Methodical approaches for post-pruning :

Reduced Error pruning :

- Starting at the leaves, **each node is replaced with its most popular class.**
- If the prediction accuracy is not affected/improves then the change is kept. While somewhat naive, reduced error pruning has the advantage of simplicity and speed.

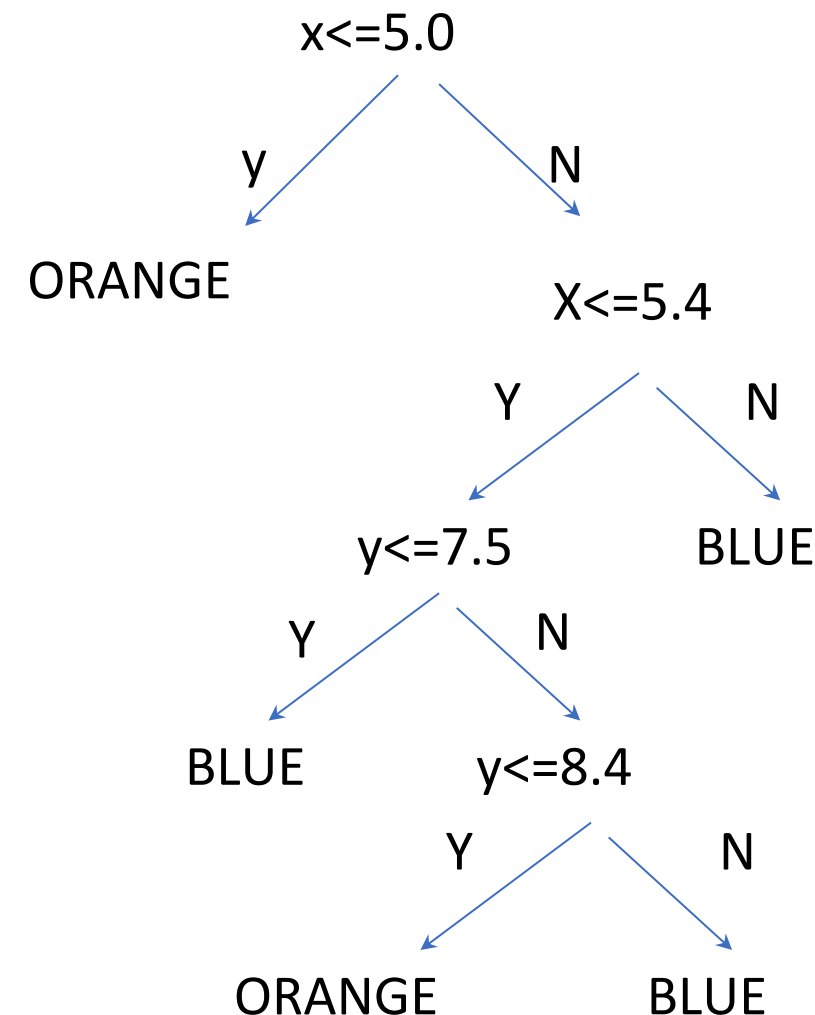
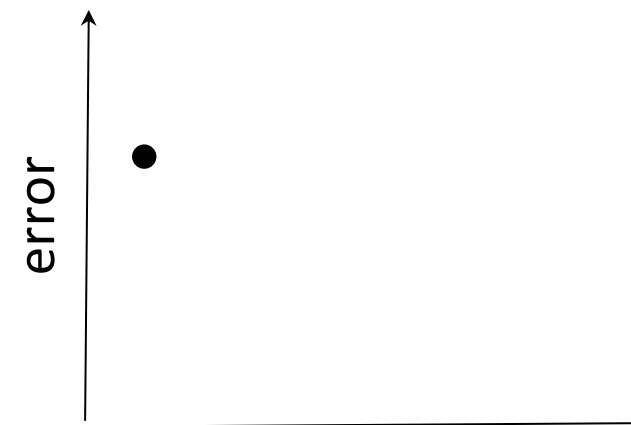
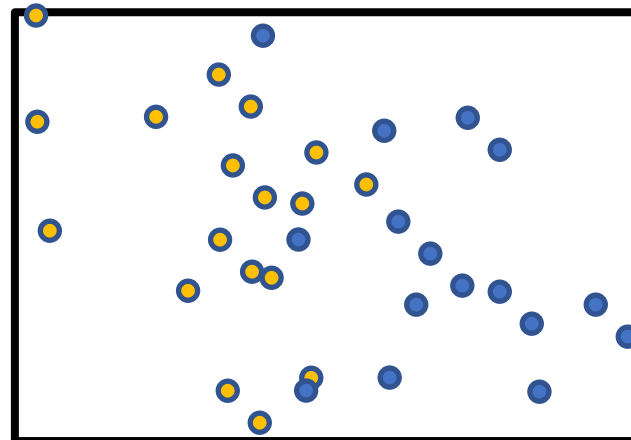
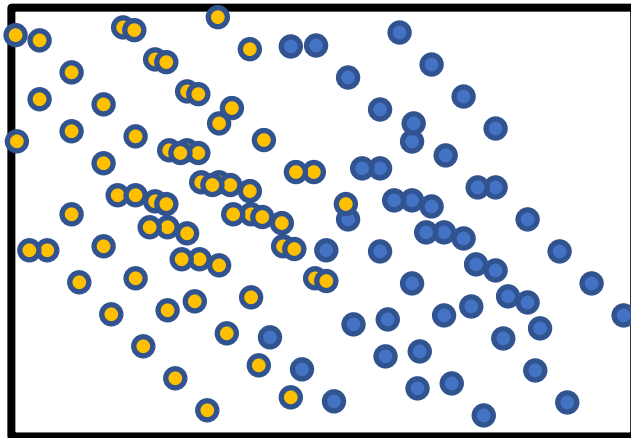
Rule Post-Pruning :

- Prune each rule by **removing preconditions** that result in improved estimated accuracy.

- Consider each of the decision nodes in the tree to be candidates for pruning.
- Pruning a decision node consists of
 - Removing the subtree rooted at that node
 - Making it a leaf node
 - Finally, assigning it the most common classification of the training examples affiliated with that node.
- Nodes are removed only if the resulting pruned tree performs no worse than the original over the validation set
- Nodes are pruned iteratively, always choosing the node whose removal most increases the decision tree accuracy over the validation set.
- Pruning of nodes continues until further pruning is harmful (i.e., decreases accuracy of the tree over the validation set). Or until you feel the improvement is reasonable.

Machine Learning

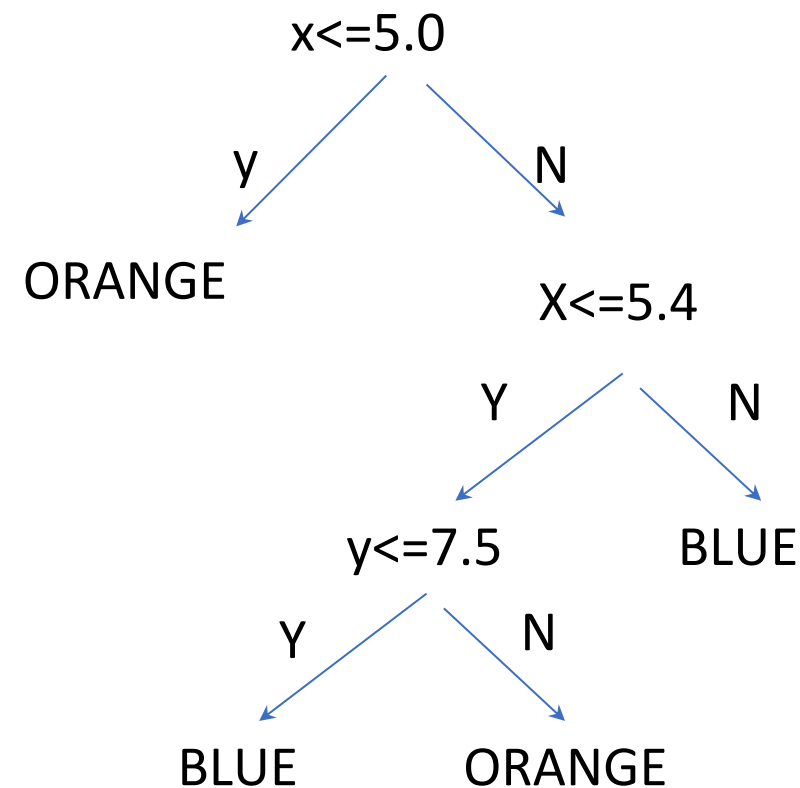
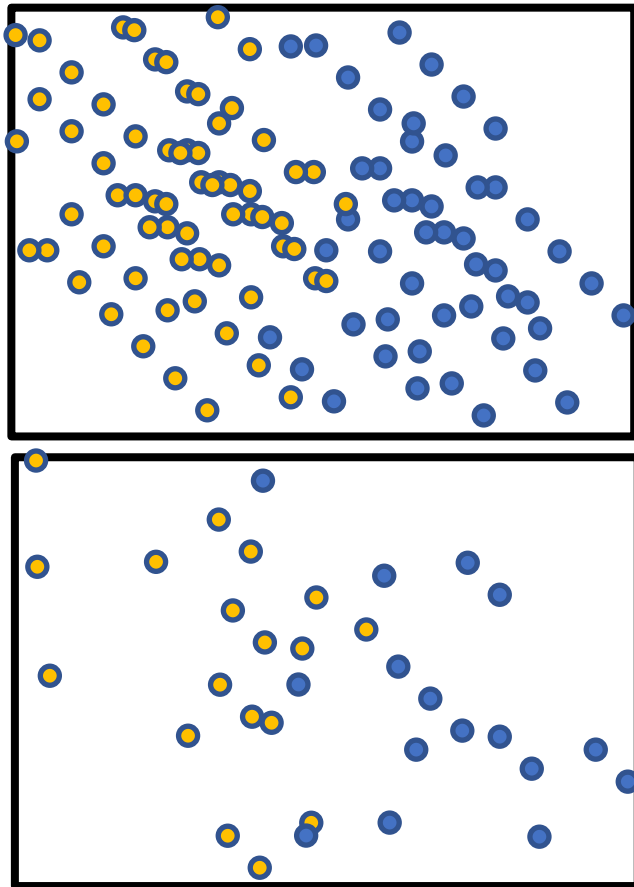
Reduced error pruning



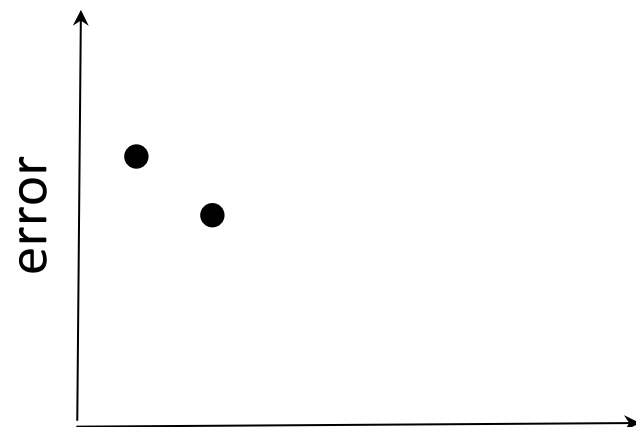
- We split the data set into train and validation and test data set
- First we need to build the complete decision tree
- Then calculate error using validation data set and note it.
- Prune the first node ($y \leq 8.4$) and replace it with the leaf node of the class that is maximum in that particular condition

Machine Learning

Reduced error pruning

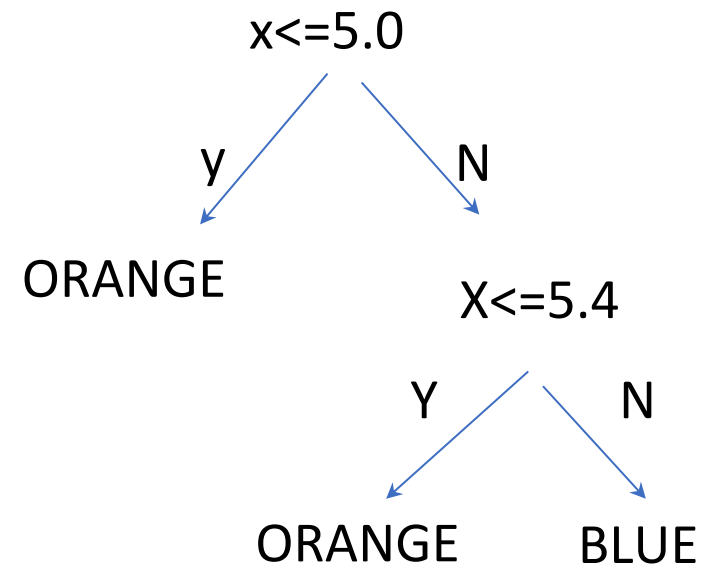
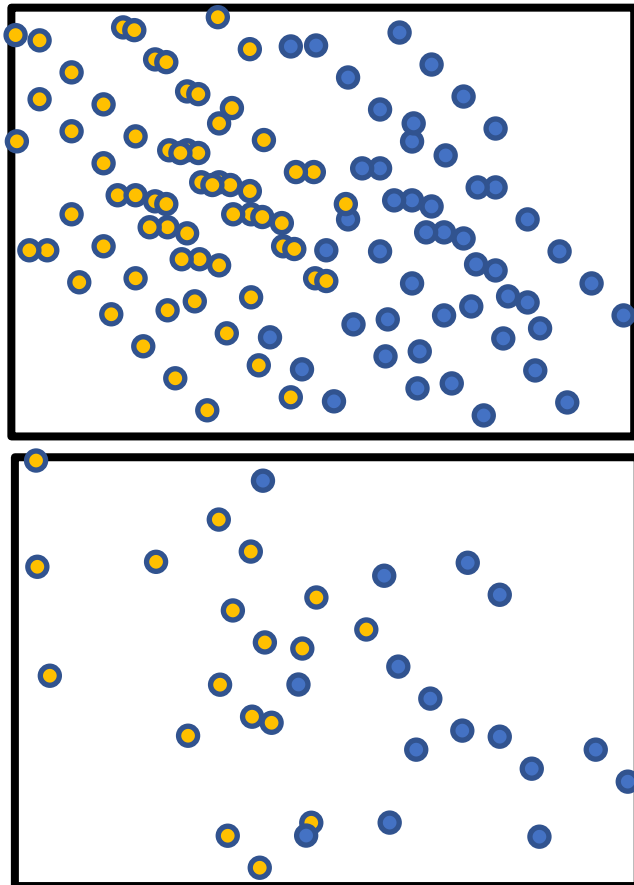


- We calculate the error using the validation set and note it
- Since the error has reduced we keep the changes and further proceed to prune the present tree's last node ($y \leq 7.5$)

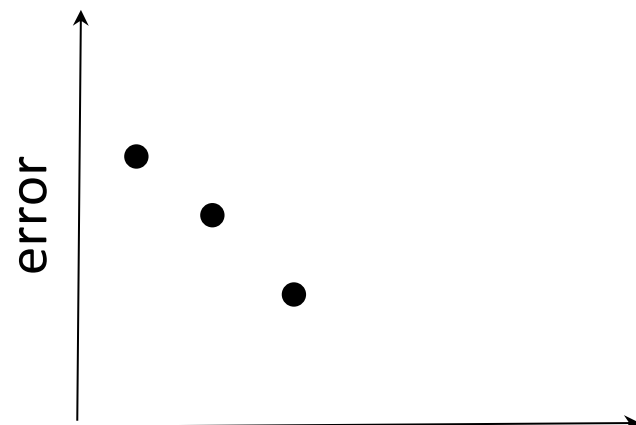


Machine Learning

Reduced error pruning

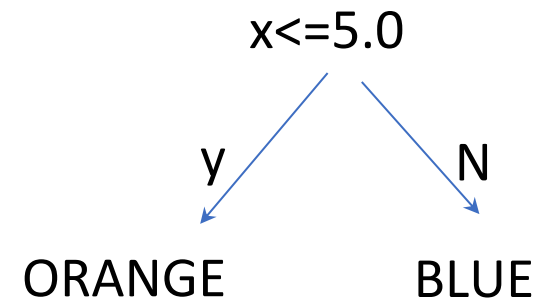
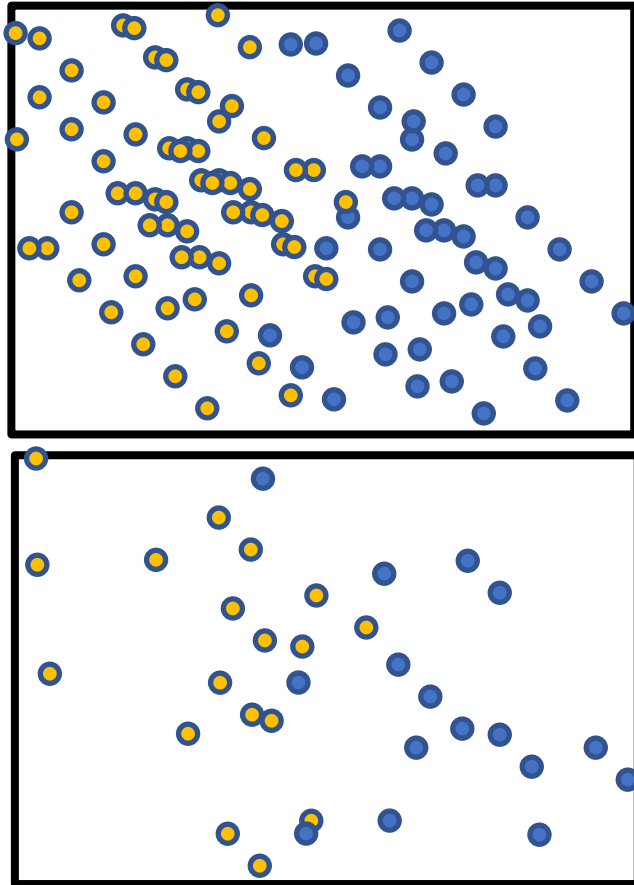


- We again follow same method to decide the leaf node
- Calculate the error and note it
- The error is reduced again, hence we keep the changes and proceed with same steps

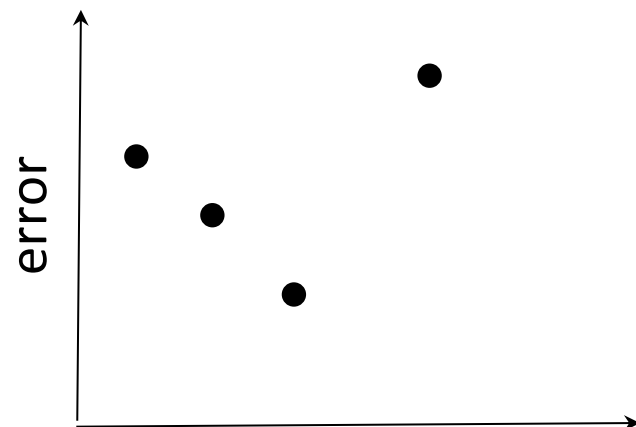


Machine Learning

Reduced error pruning

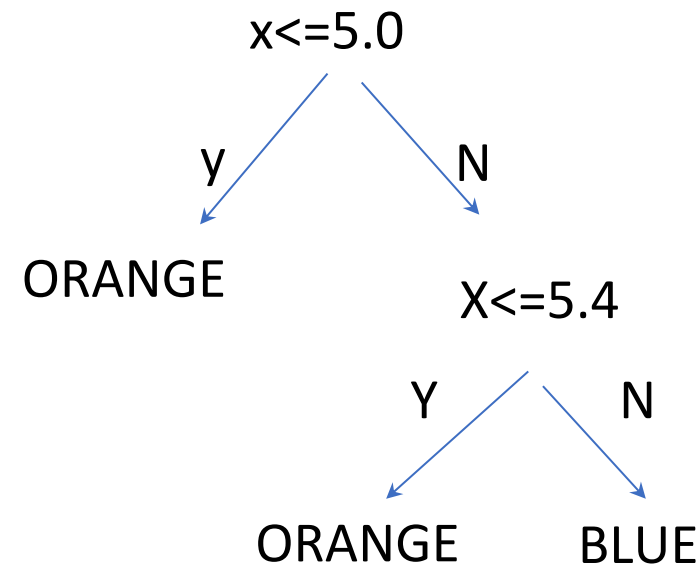
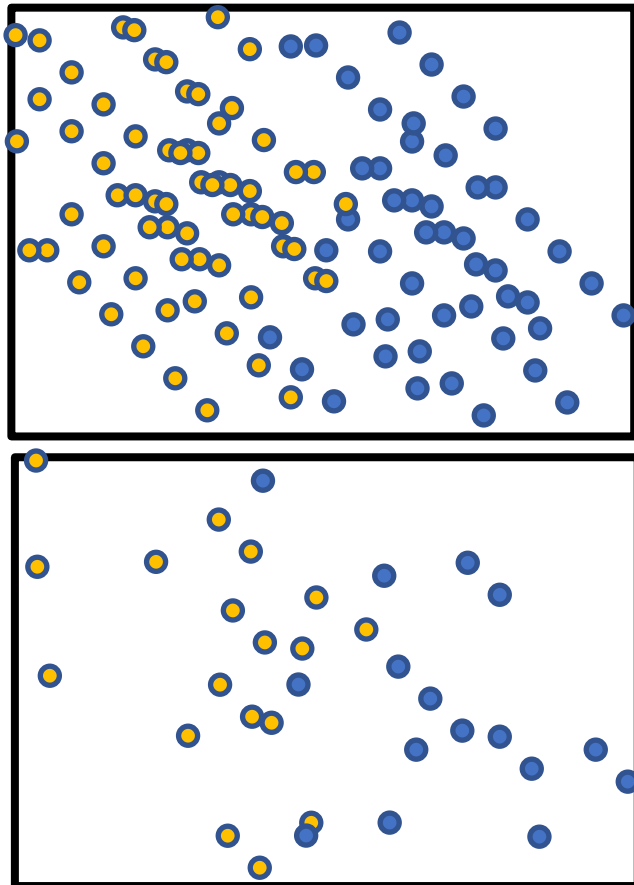


- We repeat the same steps and note down the error
- The error has increased hence we revert our change and stop pruning.



Machine Learning

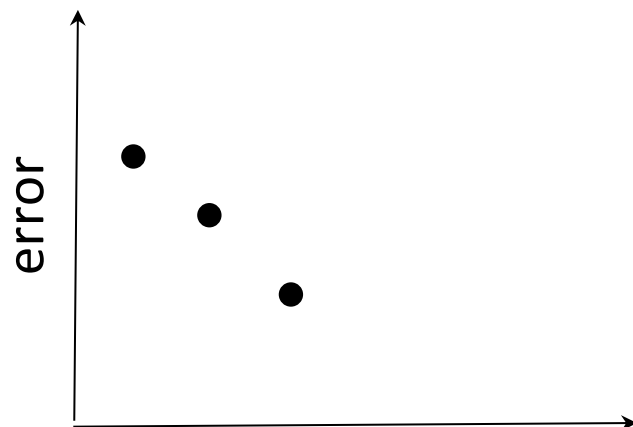
Reduced error pruning



- Stop the pruning since there is no other node to prune at that level.
- This is the final tree we get

When pruning begins, the tree is at its maximum size and lowest accuracy over the validation set.

As pruning proceeds, the number of nodes is reduced and accuracy over the validation set increases.



Idea of Post-pruning

--- Another Method - Rule Post-pruning

Rule post-pruning removes pre-conditions to improve error.

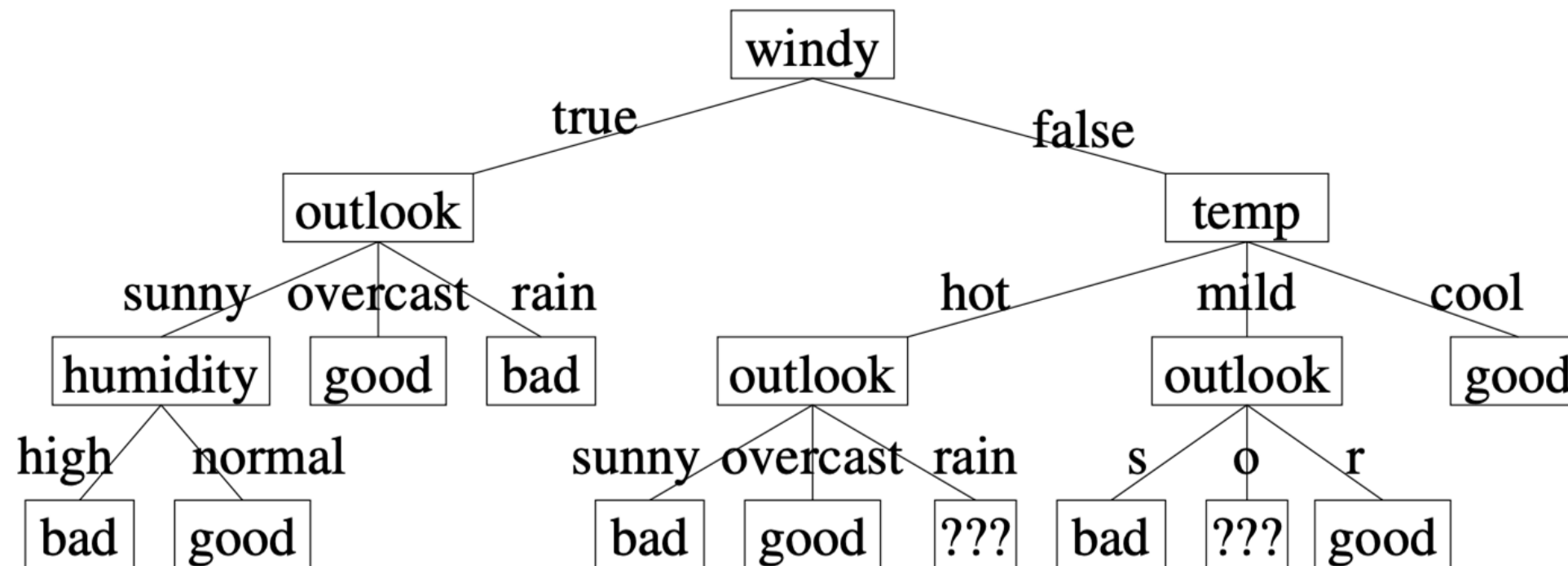
Procedure:

1. Convert tree to equivalent set of rules

Learning rule looks like: if $A1 \wedge A2 \wedge \dots$ then C

where $A1, A2, \dots$ are the preconditions/constraints/body/antecedents of the rule and C is the postcondition/head/consequent of the rule.

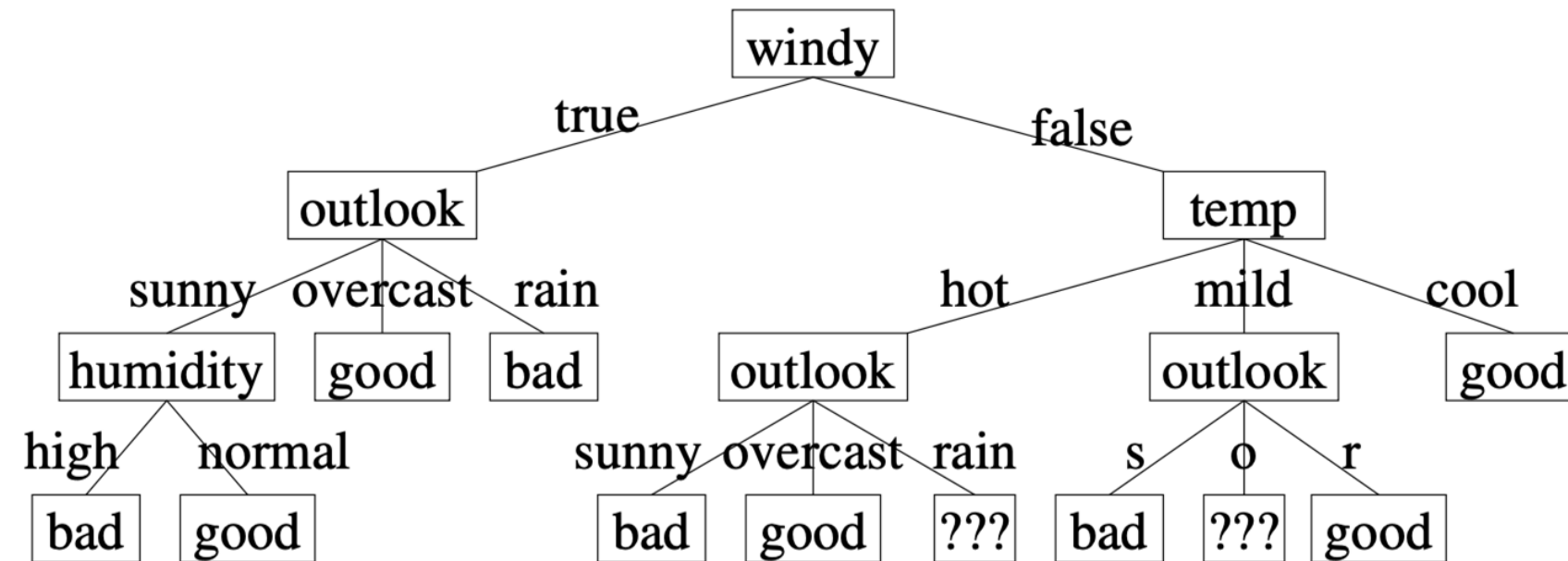
1. Prune each rule by removing preconditions that result in improved estimated accuracy (calculate the rule accuracy over validation set)
1. Sort the pruned rules by their estimated accuracy and consider them in this sequence when classifying unseen instances.



- Each path in a decision tree from root to a leaf can be converted to a rule, e.g.:
if $\neg$ windy $\wedge$ hot $\wedge$ sunny then bad
- Different pruning decisions can be made for different rules.

Machine Learning

Rule post-pruning



~~if windy $\wedge$ sunny $\wedge$ high then bad~~
~~if windy $\wedge$ sunny $\wedge$ normal then good~~
 if windy $\wedge$ overcast then good
 if windy $\wedge$ rain then bad
~~if $\neg$ windy $\wedge$ hot $\wedge$ sunny then bad~~
~~if $\neg$ windy $\wedge$ hot $\wedge$ overcast then good~~
~~if $\neg$ windy $\wedge$ mild $\wedge$ sunny then bad~~
~~if $\neg$ windy $\wedge$ mild $\wedge$ rain then good~~
 if $\neg$ windy $\wedge$ cool then good

Rule Ordering

Order rules by accuracy and coverage.

4 if overcast then good
 3 if $\neg$ windy $\wedge$ rain then good
 3 if sunny $\wedge$ high then bad
 2 if sunny $\wedge$ normal then good
 2 if $\neg$ windy $\wedge$ cool then good
 2 if windy $\wedge$ rain then bad
 2 if hot $\wedge$ sunny then bad
 1 if $\neg$ windy $\wedge$ mild $\wedge$ sunny then bad

Each such rule is pruned by removing any precondition, whose removal does not worsen its estimated accuracy.

Approaches include:

- Either stop growing the tree earlier or prune it afterwards. Post-Pruning has been more effective.
- Use a separate set of examples, distinct from the training examples, to evaluate the post-pruning nodes.
- Apply a statistical test to estimate whether expanding or pruning a node is likely to produce an improvement beyond the training set.
- **Minimum Description Length** principle is another way to minimize the model (combines the tree depth and the cost into a new and improved cost function.
[out of scope of the syllabus to discuss]

- Our initial definition of ID3 is restricted to attributes that take on a discrete set of values.
- This restriction can easily be removed so that continuous-valued decision attributes can be incorporated into the learned tree.
- This can be accomplished by **dynamically defining new discrete-valued attributes** that partition the continuous attribute value into a discrete set of intervals.
- In particular, for an attribute A that is continuous-valued, the algorithm can dynamically create a new Boolean attribute A_c , that is true if $A < c$ and false otherwise. The only question is how to select the best value for the threshold c .

TEMP	40	48	60	72	80	90
Play Sport	no	no	yes	yes	yes	no

- Clearly, we would like to pick a threshold, that produces the greatest information gain.
- By **sorting the examples** according to the continuous attribute A, then identifying adjacent examples that differ in their target classification, we can generate a set of candidate thresholds midway between the corresponding values of A.
- These candidate thresholds can then be evaluated by computing the information gain associated with each.
- In the current example, there are two candidate thresholds, corresponding to the values of Temperature at which the value of Play Tennis changes: $(48 + 60)/2$, and $(80 + 90)/2$. $\text{Temperature}_{>54}$ and $\text{Temperature}_{>85}$, and the best can be selected.

TEMP	40	48	60	72	80	90
Play Sport	no	no	yes	yes	yes	no

Temperature	40	48	60	72	80	90
PlayTennis	No	No	Yes	Yes	Yes	No

Information gain of $Temperature_{>54}$

Values ($Temp_{>54}$) = $< 54, > 54$

$$S = [3+, 3-]$$

$$Entropy(S) = -\frac{3}{6} \log_2 \frac{3}{6} - \frac{3}{6} \log_2 \frac{3}{6} = 1.0$$

$$S_{<54} \leftarrow [0+, 2-]$$

$$Entropy(S_{<54}) = 0.0$$

$$S_{>54} \leftarrow [3+, 1-]$$

$$Entropy(S_{>54}) = -\frac{3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = 0.8113$$

$$Gain(S, Temp_{>54}) = Entropy(S) - \sum_{v \in \{<54, >54\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Temp_{>54}) = Entropy(S) - \frac{2}{6} Entropy(S_{<54}) - \frac{4}{6} Entropy(S_{>54})$$

$$Gain(S, Temp_{>54}) = 1.0 - \frac{2}{6} 0.0 - \frac{4}{6} 0.8113 = 0.4591$$

Temperature	40	48	60	72	80	90
PlayTennis	No	No	Yes	Yes	Yes	No

Information gain of $Temp_{>85}$

Values ($Temp_{>85}$) = $\langle 85, > 85$

$$S = [3+, 3-] \quad Entropy(S) = -\frac{3}{6} \log_2 \frac{3}{6} - \frac{3}{6} \log_2 \frac{3}{6} = 1.0$$

$$S_{<85} \leftarrow [3+, 2-] \quad Entropy(S_{<85}) = -\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} = 0.971$$

$$S_{>85} \leftarrow [0+, 1-] \quad Entropy(S_{>85}) = 0.0$$

$$Gain(S, Temp_{>85}) = Entropy(S) - \sum_{v \in \{<85, >85\}} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$Gain(S, Temp_{>85}) = Entropy(S) - \frac{5}{6} Entropy(S_{<85}) - \frac{1}{6} Entropy(S_{>85})$$

$$Gain(S, Temp_{>85}) = 1.0 - \frac{5}{6} 0.971 - \frac{1}{6} 0.0 = 0.1908$$

$$Gain(S, Temp_{>54}) = 0.4591$$

$$Gain(S, Temp_{>85}) = 0.1908$$

Choose this as threshold

- Example: Blood-test results may only be available for some patients in a hospital.
- The **missing attribute value** can be **estimated** based on other values this attribute has taken in other instances.
- One strategy for dealing with the missing attribute value is to assign it the value that is most common among training examples at the target node (where we are calculating $\text{Gain}(S, A)$, A being the attribute whose value is unknown in one instance)
- Alternatively, we might assign it the most common value among examples at node n that have the classification $c(x)$

- In some learning tasks the instance attributes may have associated costs.
- For example, in learning to classify medical diseases we might describe patients in terms of attributes such as Temperature, BiopsyResult, Pulse, BloodTestResults, etc. These attributes vary significantly in their costs, both in terms of monetary cost and cost to patient comfort.
- In such tasks, we would prefer decision trees that use low-cost attributes where possible, relying on high-cost attributes only when needed to produce reliable classifications.
- ID3 can be modified to take into account attribute costs by introducing a cost term into the attribute selection measure.

Machine Learning

References



<https://medium.com/@pralhad2481/chapter-3-decision-tree-learning-part-2-issues-in-decision-tree-learning-babdfdf15ec3>

<http://www.cs.utsa.edu/~bylander/cs6243/learning-rules.pdf>

https://www.cs.auckland.ac.nz/~pat/706_98/ln/node70.html



UE24CS352A: Machine Learning

Dr. Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu



UE24CS352A: Machine Learning

Dr. Shylaja S S

Professor

Department of Computer Science & Engineering

Acknowledgement :

Dr. Rajanikanth K (Academic Advisor, PESU)
Teaching Assistant (Arya Rajiv Chaloli)

Bias

In statistics, the bias is the difference between this estimator's expected value and the true value of the parameter being estimated.



Variance

In machine learning, variance is the amount by which the performance of a predictive model changes when it is trained on different dataset from the same distribution.



Machine Learning

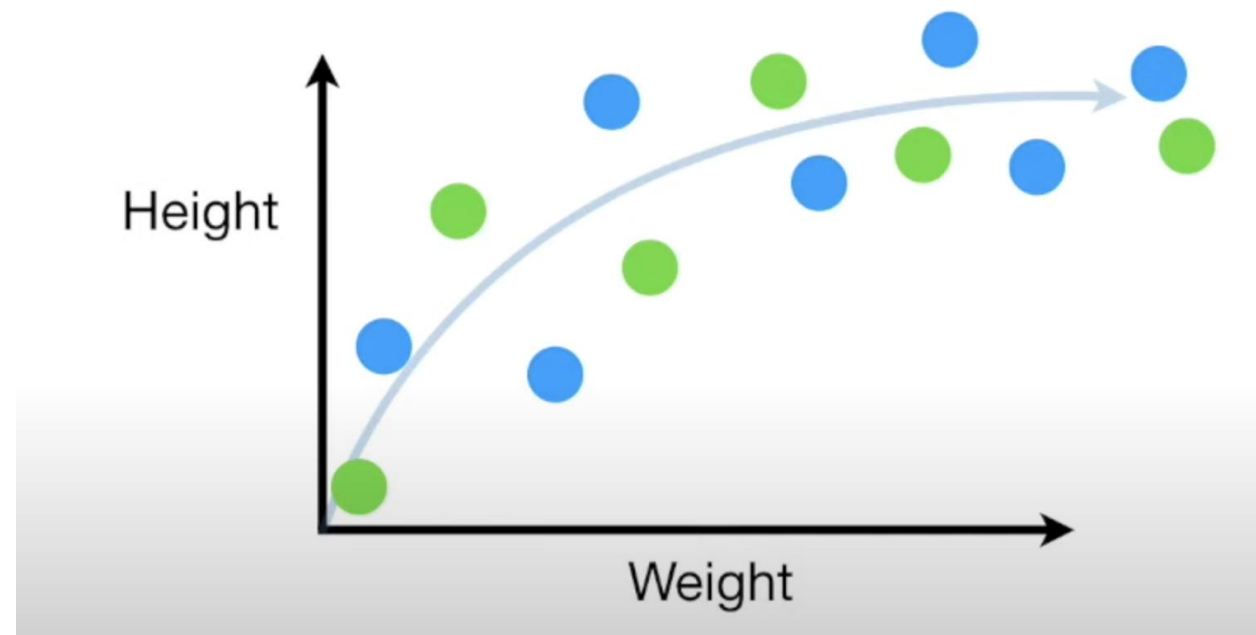
Bias and Variance



Bias and variance in regression problems

Let us assume a regression problem.

The true underlying pattern in the data can be represented by the curve as seen in the image.

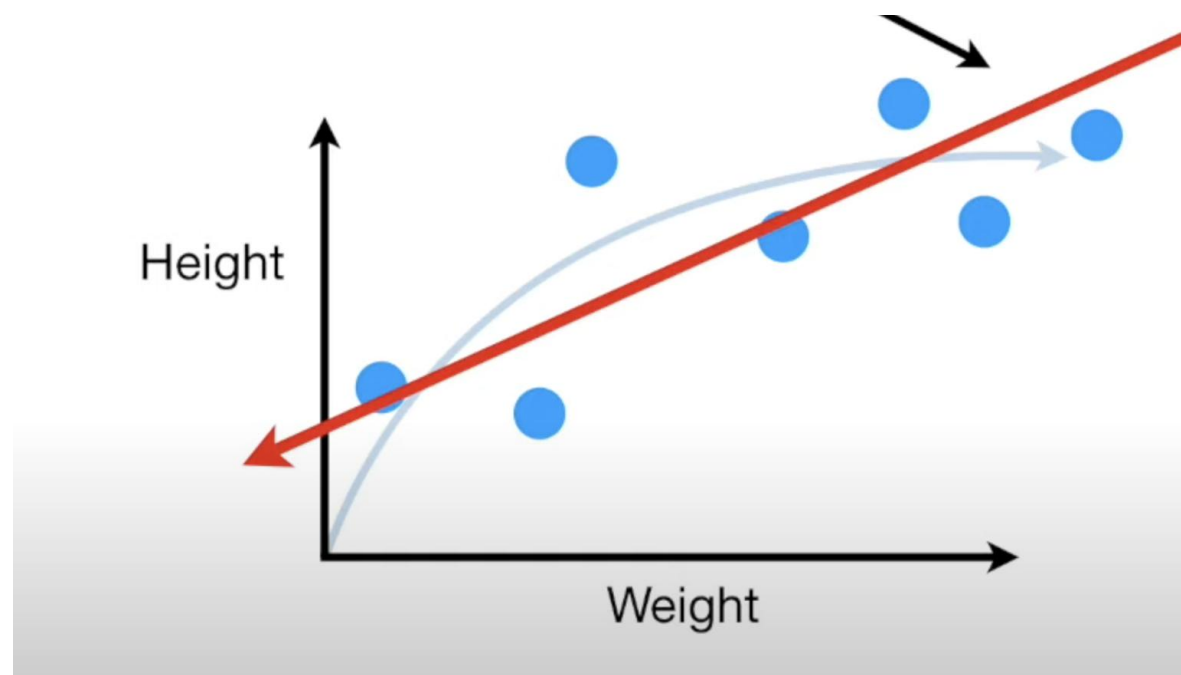


Split this data into a training set and a test set (blue - training, green - testing).

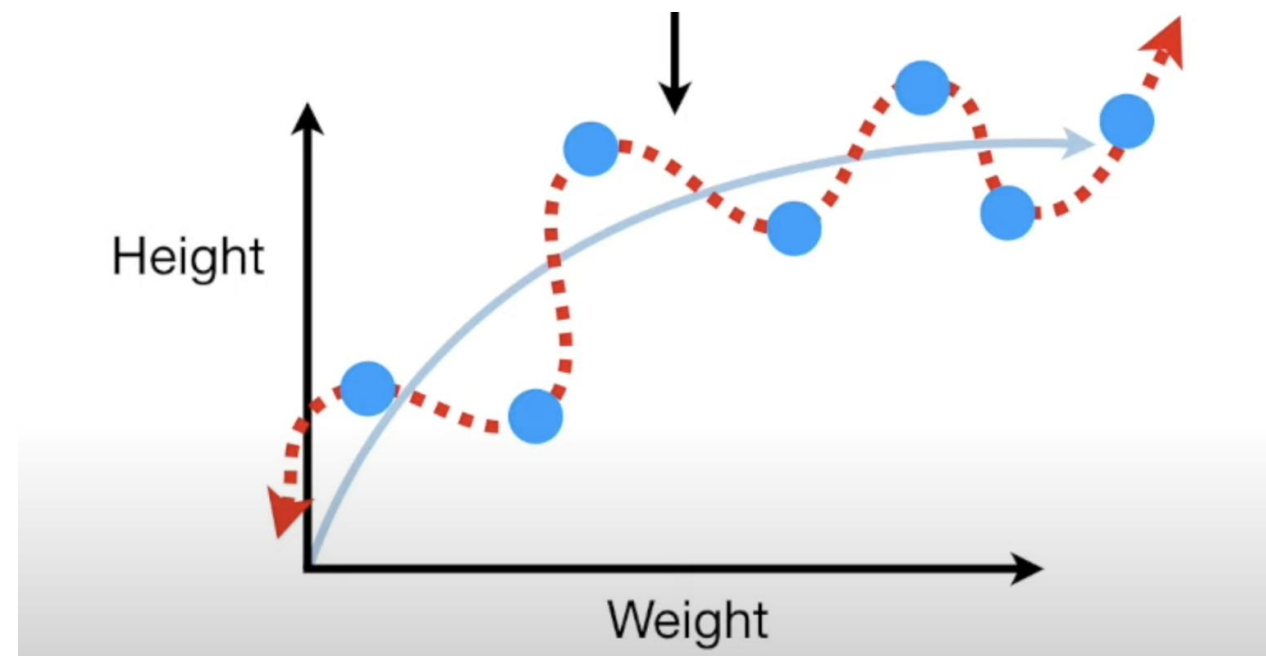
Bias and variance in regression problems

The 2 models we compare are the following:

- Model 1: a linear regression line
- Model 2: a squiggly line that tries to fit all the training data points



Model 1

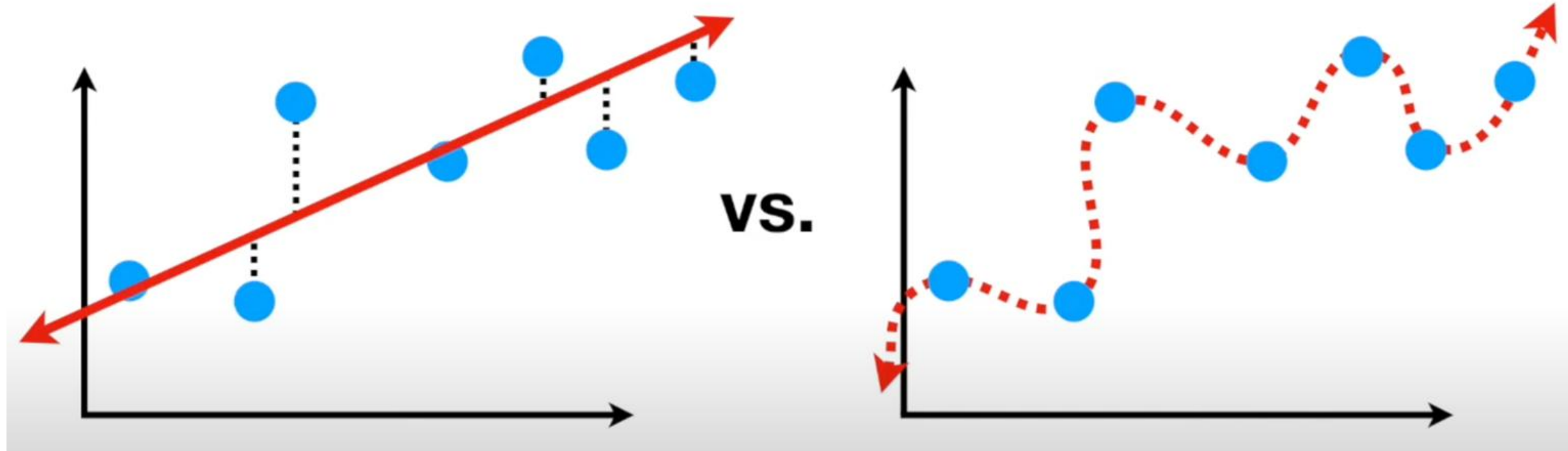


Model 2

Bias and variance in regression problems

Errors on the training set:

- Model 1: has some error
- Model 2: has 0 error



Model 1

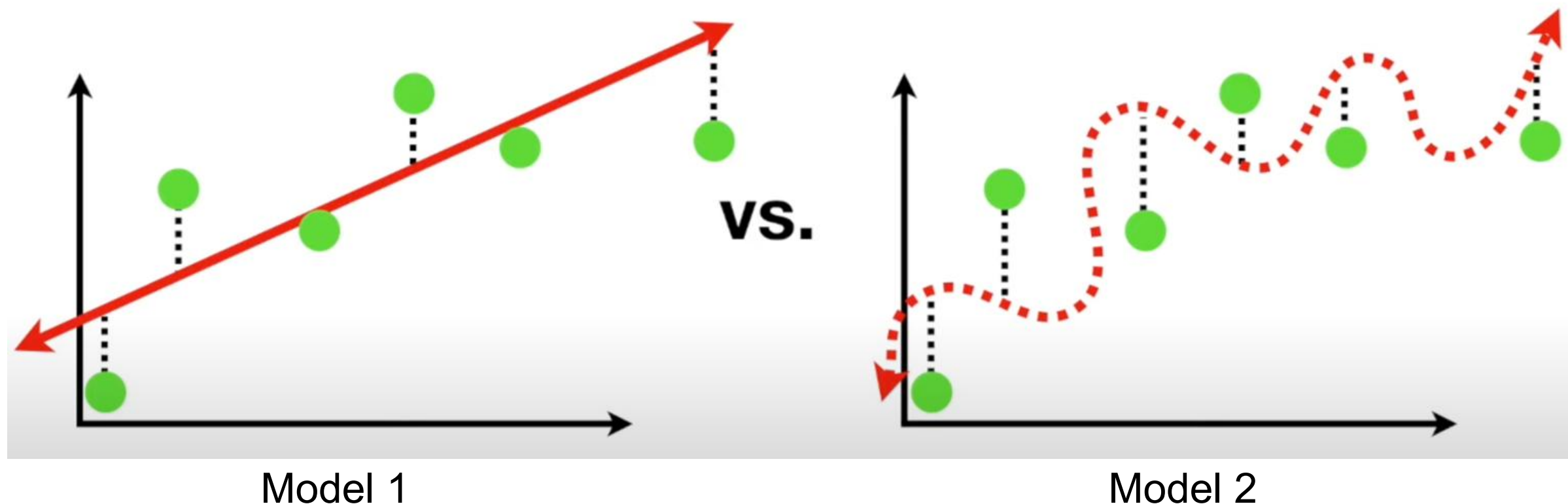
Model 2

Bias and variance in regression problems

However, errors on the test set:

- Model 1: has some error (similar to the training set)
- Model 2: has some error (unlike how it did on the training set)

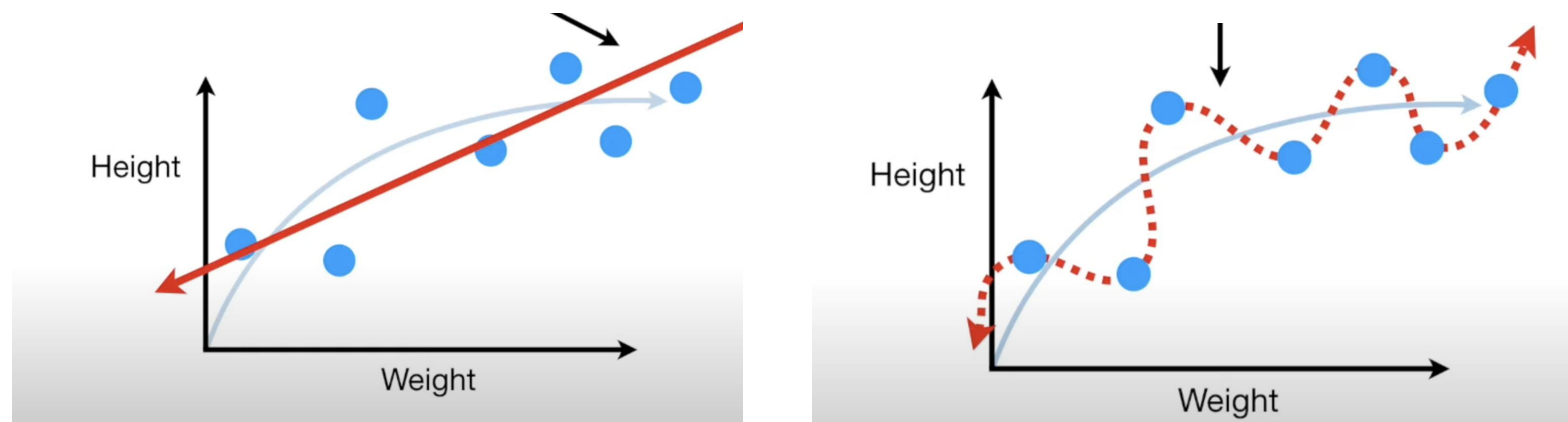
The 2nd model did a great job at fitting the training set, but did a terrible job at fitting the test set.



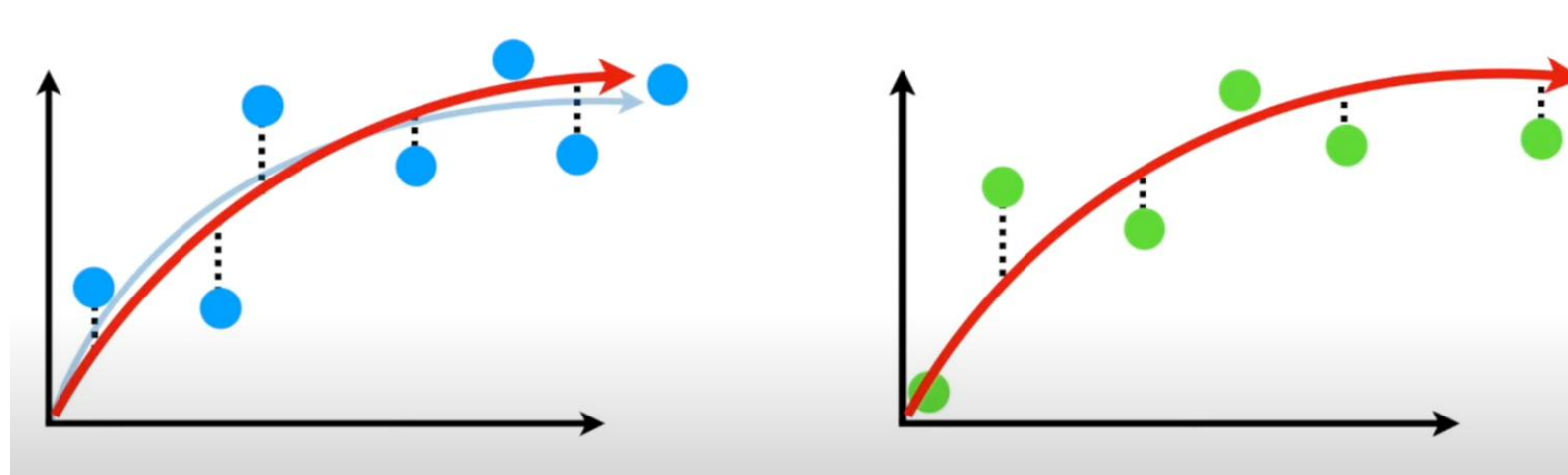
Bias and variance in regression problems

Model 2: **low bias but high variability** because it yields vastly different errors for different datasets taken from the same distribution

Model 1: **high bias but has low variance** because it yields similar errors for different datasets from the same distribution



A possible third model like the one below has **low bias and low variability**



Overfitting and Underfitting

Overfitting

- Occurs when a model captures the noise along with the data pattern.
- A model that fits the the training data well but fails to do so on the testing set is an overfit to the data.
- Overfitted models have **low bias and high variance**.

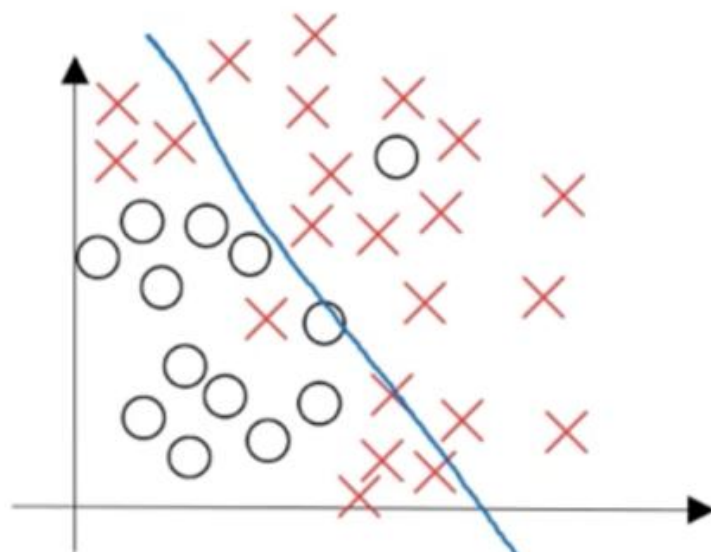
Underfitting

- Occurs when a model fails to even capture the pattern of the data.
- Such models have **high bias and low variance**.

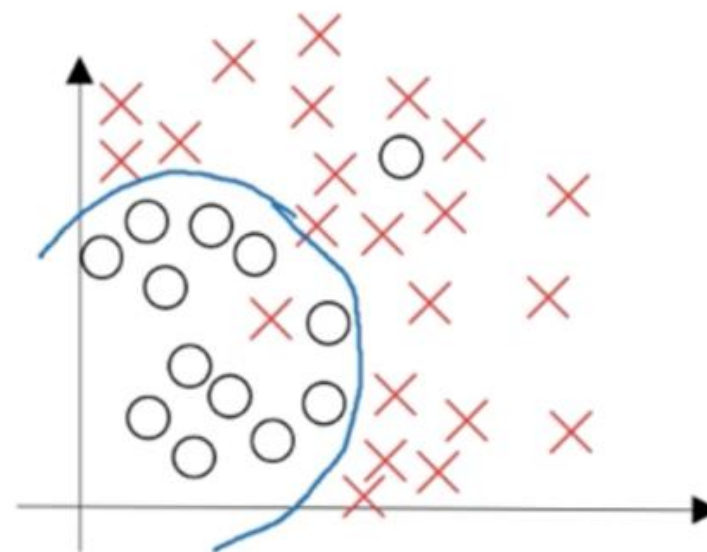
Bias and variance in classification problems

Assume a classifier problem, where we have to determine the decision boundary.

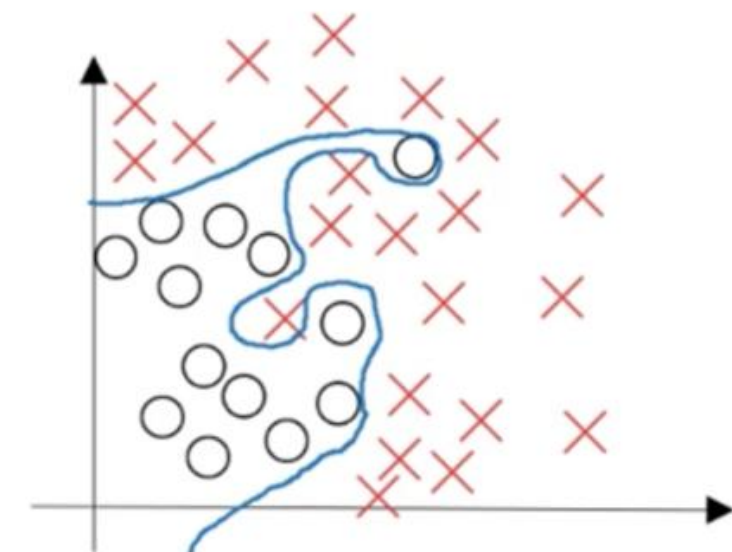
Three (of many) possible models that can be fit are the following.



Model 1



Model 2



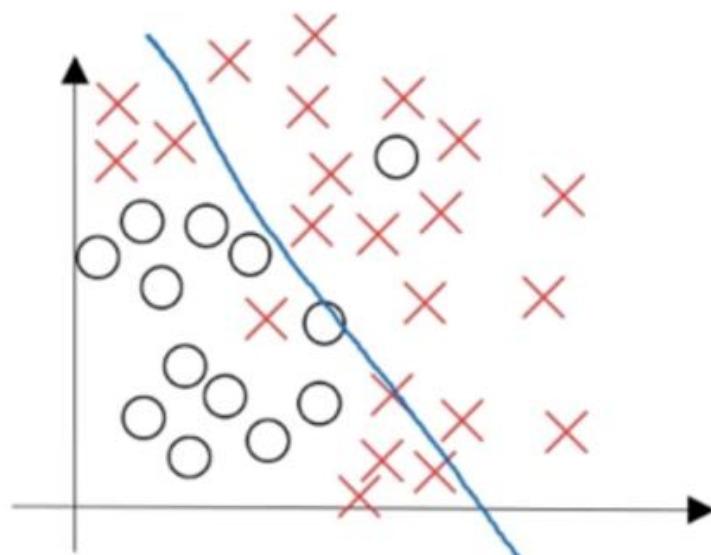
Model 3

Bias and variance in classification problems

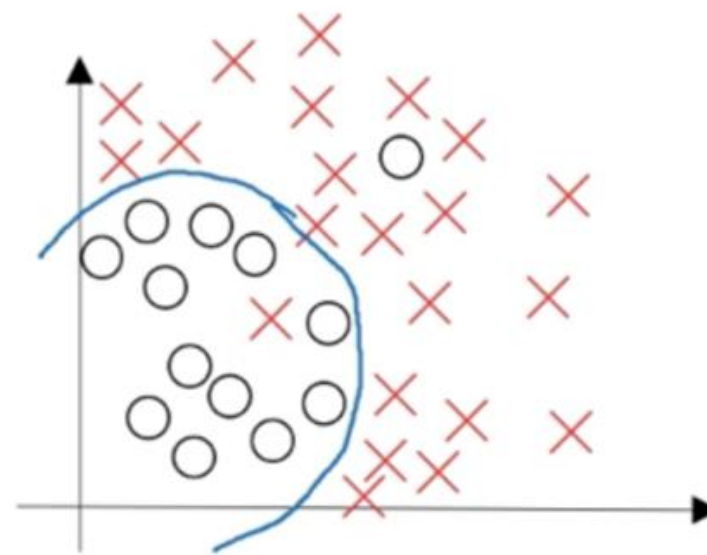
Model 1: Has a high bias - **underfit**

Model 3: Has a high variance - **overfit**

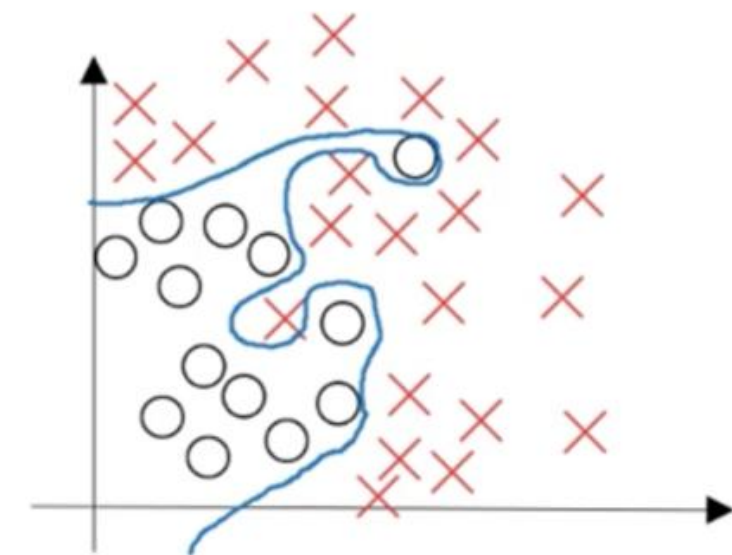
Model 2: Seems like a fair balance



Model 1
High Bias
Underfitting



Model 2
Just Right



Model 3
High Variance
Overfitting

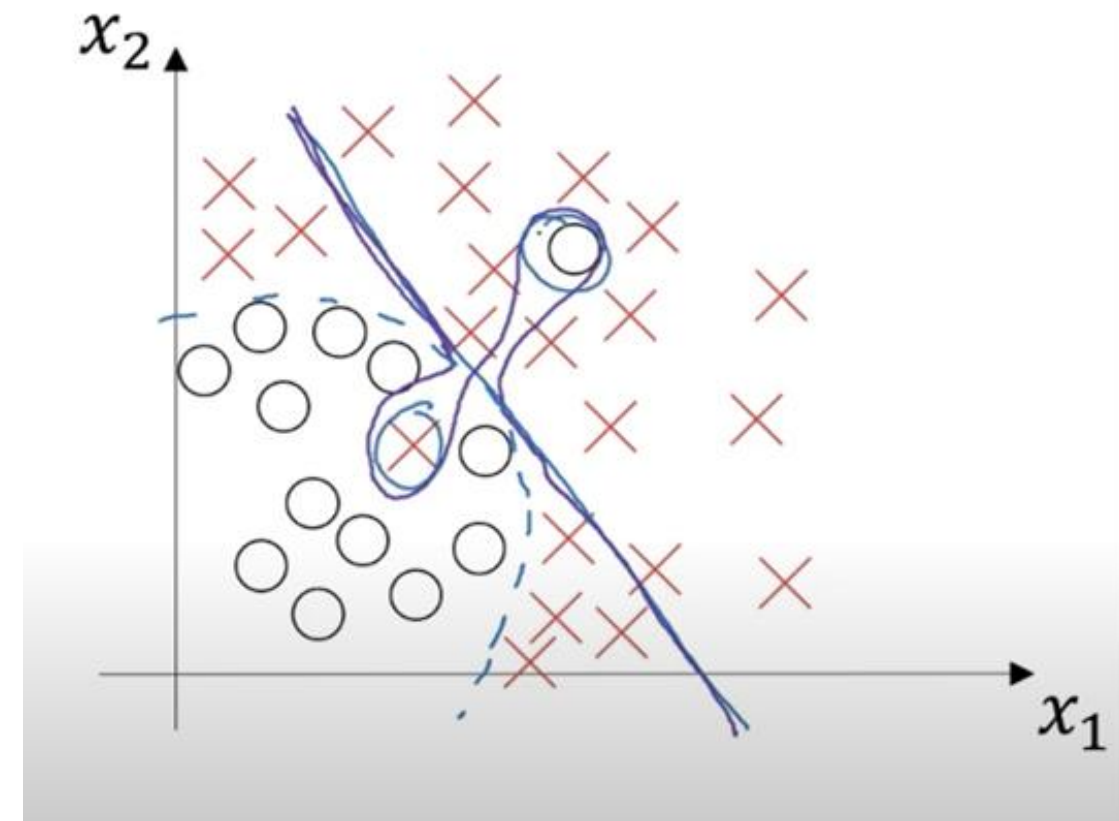
Bias and variance in classification problems

Case – High Bias and High Variance:

Assume that a model classifies the data as shown in the image

Since it is **primarily a linear classifier**, it is not fitting the quadratic like shape very well but having **too much flexibility in the middle**, its overfitting in that area

In higher dimensional input this is more realistic – with high bias in some regions and high variance in some other regions



- In a 2d example it is easy to plot the data and visualize the bias and variance, in higher dimensional data this is not possible.

Example: A cat classifier, where we have to identify cat images from a set of cat and dog images

There would be many features – which is not possible to plot on a graph and identify the bias and variance

Cat classification



Finding the bias and variance

4 different models give the following errors – these are the inferences that we can draw

	Model 1	Model 2	Model 3	Model 4
Train Set Error	1%	15%	15%	0.5%
Test Set Error	11%	16%	30%	1%
	High Variance	High Bias	High Bias and High Variance	Low Bias and Low Variance

This is assuming the base (human) error rate for this problem is 0.

This is called the **optimal (bayes) error**. If it was 15%, then we will not classify Model 2 as high bias (for example, the dataset is full of blurry images).

Finding the bias and variance

	Model 1	Model 2	Model 3	Model 4
Train Set Error	1%	15%	15%	0.5%
Test Set Error	11%	16%	30%	1%
	High Variance	High Bias	High Bias and High Variance	Low Bias and Low Variance

- The training set error helps us identify bias.
- The amount of increase in error from the train-set to the test-set indicates the variance.

(Assuming that the training and test sets are drawn from the same distribution, and the bayes error is quite small)

Bias Variance Identification

High variance can be identified if the model has:

- **Low training error** and **high test error**.

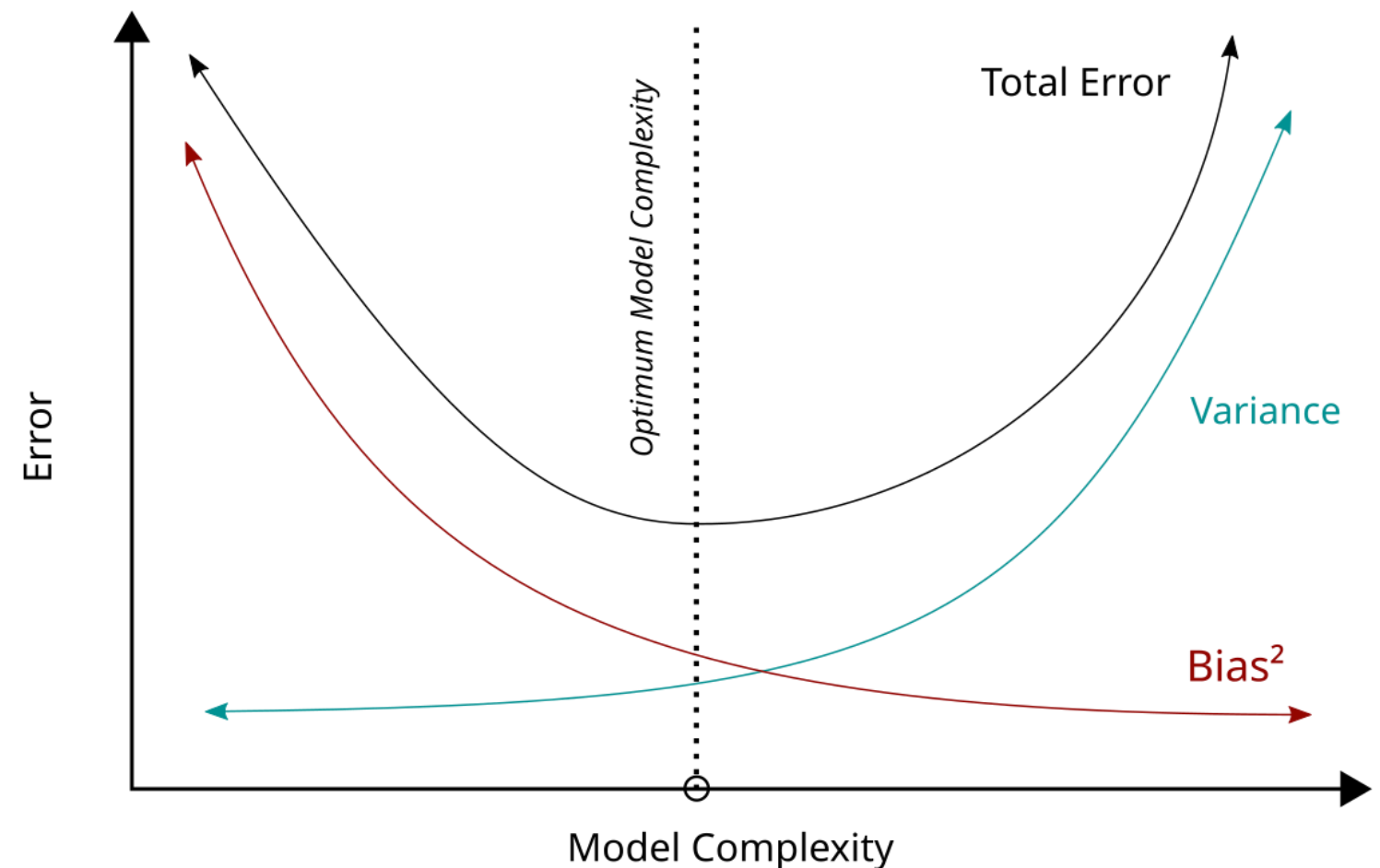
High Bias can be identified if the model has:

- **High training error** and the test error is almost similar to training error



Bias Variance Tradeoff

- **Bias Reduction:** As model complexity increases, bias decreases initially, leading to better performance.
- **Variance Increase:** With more complexity, variance starts to rise, but initially, this increase is slower than the decrease in bias.
- **Total Error:** At first, the total error decreases because the reduction in bias outweighs the increase in variance.
- **Optimal Complexity:** Eventually, increased complexity leads to a rapid rise in variance, causing total error to increase. The point where total error is minimized is the optimal model complexity.



Bias–Variance–Irreducible Error Decomposition

The **total error** of a model can be decomposed into:

- Irreducible Error
- Reducible Error

Irreducible Error

- The error that is not reducible (commonly known as **noise**) is referred to as an irreducible error.
- These errors that cannot be fixed will always exist in even the best models.

Reducible Error

- Both **variance** and **bias** are reducible errors.
- There is a trade-off between bias and variance to decrease the reducible error which in turn would minimise the total error.

Bias–Variance–Irreducible Error Decomposition

For a machine learning model, the **expected prediction error** at a point x can be expressed as:

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

Irreducible Error (σ^2)

- Caused by **inherent noise** in the data (e.g., unpredictable random effects).
- Cannot be reduced by any model.

Bias–Variance–Irreducible Error Decomposition

Bias

$$\text{Bias}(x) = E[\hat{f}(x)] - f(x)$$

- Measures the difference between the **average predicted value** of the model and the **true function** $f(x)$.
- **Interpretation:** Systematic error due to overly simple model assumptions.
- **High Bias** → Model **underfits** (too simple to capture complexity).

Note: E stands for the **expected value** — in other words, the **average** over all possible training datasets that could be drawn from the same data distribution

Bias–Variance–Irreducible Error Decomposition

Variance

$$\text{Variance}(x) = E \left[(\hat{f}(x) - E[\hat{f}(x)])^2 \right]$$

- Measures how much model predictions **fluctuate** when trained on different datasets from the same distribution.
- **Interpretation:** Sensitivity to training data.
- **High Bias** → Model **overfits** (captures noise instead of signal).

Error Decomposition - Example

Assume a machine learning problem, where the true underlying function is: $y = 2x$
x is from: [1, 2, 3] and noise $\epsilon \sim N(0,1)$ – mean 0, variance 1 (normal distribution).

We have 3 training datasets (with some noise).

ACTUALS		Training Set 1		Training Set 2		Training Set 3	
x	y	x	y	x	y	x	y
1	2	1	1.8	1	2.1	1	2.4
2	4	2	3.9	2	3.4	2	3.3
3	6	3	5.7	3	5.6	3	6.1

Error Decomposition - Example

Assume that we train some model that outputs $y = c$
And the following are the fitted values of c for each of the training sets

ACTUALS		Training Set 1		Training Set 2		Training Set 3	
x	y	x	y	x	y	x	y
1	2	1	1.8	1	2.1	1	2.4
2	4	2	3.9	2	3.4	2	3.3
3	6	3	5.7	3	5.6	3	6.1
		$y = 3.8$		$y = 3.7$		$y = 3.93$	

Error Decomposition - Example

We pick a test point $x_0 = 2$

The true value of y :

$$f_{\text{true}}(2) = 4$$

The predictions from each run:

$$\hat{f}_1(2) = 3.8, \quad \hat{f}_2(2) = 3.7, \quad \hat{f}_3(2) = 3.93$$

Training Set 1

x	y
1	1.8
2	3.9
3	5.7

$$y = 3.8$$

Training Set 2

x	y
1	2.1
2	3.4
3	5.6

$$y = 3.7$$

Training Set 3

x	y
1	2.4
2	3.3
3	6.1

$$y = 3.93$$

Error Decomposition - Example

$$\hat{f}_1(2) = 3.8, \quad \hat{f}_2(2) = 3.7, \quad \hat{f}_3(2) = 3.93$$

$$\overline{\hat{f}(2)} = \frac{3.8 + 3.7 + 3.93}{3} = 3.81$$

Calculating Bias

$$\text{Bias} = \overline{\hat{f}(2)} - f_{\text{true}}(2) = 3.81 - 4 = -0.19$$

$$\text{Bias}^2 = (-0.19)^2 = 0.0361$$

Error Decomposition - Example

$$\hat{f}_1(2) = 3.8, \quad \hat{f}_2(2) = 3.7, \quad \hat{f}_3(2) = 3.93$$

$$\overline{\hat{f}(2)} = \frac{3.8 + 3.7 + 3.93}{3} = 3.81$$

Calculating Variance

$$\begin{aligned} \text{Variance} &= \frac{(3.8 - 3.81)^2 + (3.7 - 3.81)^2 + (3.93 - 3.81)^2}{3} \\ &= \frac{0.0001 + 0.0121 + 0.0144}{3} = 0.00887 \end{aligned}$$

Error Decomposition - Example

Since noise is taken from a normal distribution $\varepsilon \sim N(0,1)$, variance = 1:

$$\text{Irreducible Error} = 1$$

Calculating Total Expected Error

$$\begin{aligned}\text{Error} &= \text{Bias}^2 + \text{Variance} + \text{Irreducible Error} \\ &= 0.0361 + 0.00887 + 1 = 1.045\end{aligned}$$



UE24CS352A: Machine Learning

Dr. Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu



UE24CS352A: Machine Learning

Dr. Shylaja S S

Professor

Department of Computer Science & Engineering

Acknowledgement :

Dr. Rajanikanth K (Academic Advisor, PESU)
Teaching Assistant (Arya Rajiv Chaloli)

After the initial feature engineering and implementing a model, we get some output in terms of a class or probability.

Effectiveness is measured by some metrics using our model and test data sets. Some **metrics for classification algorithms** are:

- Accuracy
- Precision
- Recall
- Specificity

- Receiver Operating Characteristics (ROC)

- Area Under Curve (AUC)

- Given a classifier and an instance, there are four possible outcomes.
 - **TP** - True Positive
If the instance is **positive** and is classified as **positive**
 - **FN** - False Negative
If the instance is **positive** and is classified as **negative**
 - **TN** - True Negative
If the instance is **negative** and is classified as **negative**
 - **FP** - False Positive
If the instance is **negative** and is classified as **positive**
- Confusion matrix (also called a contingency table) is a specific table layout that allows visualization of the performance of a classifier

Actual Label	Predicted Label	Result
+	+	True Positive
-	-	True Negative
+	-	False Negative
-	+	False Positive

ACTUAL VALUES

POSITIVE (1)

NEGATIVE (0)

PREDICTIVE VALUES

POSITIVE (1) NEGATIVE (0)

TP	FN
FP	TN

The word **Positive/Negative** is used to indicate the **prediction made** by the model.

The word **True/False** is used to indicate whether the **prediction is right or wrong**.

Let's say we have data sets of 200 objects out of which 100 are from class A and 100 from class B

With predictions from our model as below

- 60 of the data object were correctly predicted positive class A as in the data set
- 30 of the data object were correctly predicted as negative class B as in the dataset

What is the total number of correct and wrong predictions?

N	class A	class B
200	100	100

N	class A	class B
200	100	100

- 60 of the data object were correctly predicted positive class A as in the dataset
- 30 of the data object were correctly predicted as negative class B as in the dataset

Total correct predictions: $60 + 30 = \mathbf{90}$
Total wrong predictions: $70 + 40 = \mathbf{110}$

N = 200

PREDICTIVE VALUES

POSITIVE (1) NEGATIVE (0)

ACTUAL VALUES

POSITIVE (1)

NEGATIVE (0)

TP 60	FN 40
FP 70	TN 30

- Accuracy in classification problems is the number of correct predictions made by the model over all kinds predictions made.
- Accuracy is generally a good measure when the target variable classes are nearly balanced.

- Accuracy =
$$\frac{TP + TN}{(TP + TN + FP + FN)}$$

		PREDICTIVE VALUES	
		POSITIVE (1)	NEGATIVE (0)
ACTUAL VALUES	POSITIVE (1)	TP	FN
	NEGATIVE (0)	FP	TN

	Predicted: (1)	Predicted: (0)
Actual: (1)	98	0
Actual: (0)	02	0

Accuracy is not a very good measure of the classifier when dataset is imbalanced.

Accuracy is generally a good measure when the **target variable classes are nearly balanced**.

Example:

Fraud Detection: Training data distribution

+ve - normal transaction (990)

-ve - fraudulent transaction (10)

In test data of 100 transactions: (if its a naive program where it calls every transaction as +ve), we achieve an accuracy of 98%

However, our interest here is to catch all the fraudulent transactions correctly.

- Precision: How many correct +ve cases did we catch?

- Precision =
$$\frac{TP}{(TP + FP)}$$

↑
In reality, the instance belonged to a negative class but was falsely predicted as +ve.

		PREDICTIVE VALUES	
		POSITIVE (1)	NEGATIVE (0)
ACTUAL VALUES	POSITIVE (1)	TP	FN
	NEGATIVE (0)	FP	TN

Precision is useful when the **cost of false positives** is significantly **higher** than the cost of false negatives

Examples:

- Spam detection:
Misclassifying a legitimate email as spam (false positive) can be highly inconvenient for the user, whereas missing a spam email (false negative) is less critical.
- Product recommendations:
Recommending items a user is unlikely to be interested in (false positive) can lead to user disengagement, while missing a few relevant recommendations (false negative) is less impactful.

- Recall: Sensitivity / True Positive Rate / Hit rate
- How many of the positive cases did we catch correctly?

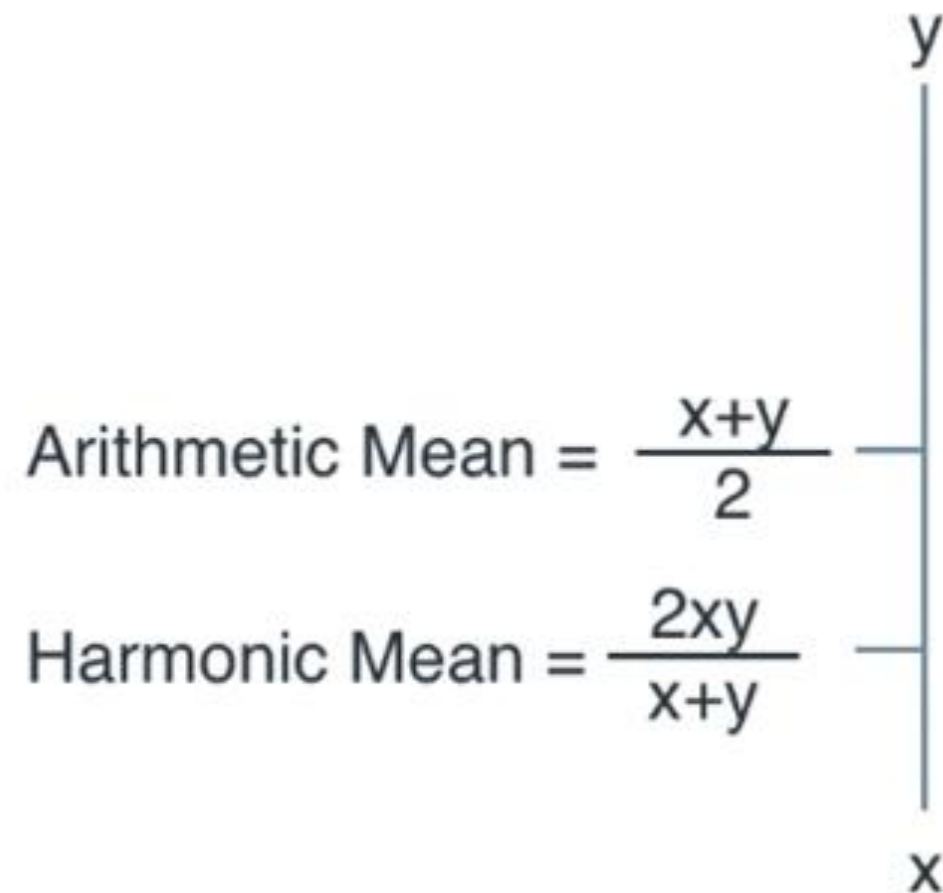
- Recall =
$$\frac{TP}{(TP + FN)}$$

		PREDICTIVE VALUES	
		POSITIVE (1)	NEGATIVE (0)
ACTUAL VALUES	POSITIVE (1)	TP	FN
	NEGATIVE (0)	FP	TN

Recall is useful when the **cost of a false negative** is very high.

Examples:

- Medical diagnoses (especially for life-threatening conditions):
It's crucial to catch all potential cases, even if it leads to some false alarms. For example, a false negative in cancer detection could be devastating, while a false positive might only require further investigation.
- Fraud detection:
Banks may prefer to flag a few legitimate transactions as potentially fraudulent (false positives) than miss even a single fraudulent transaction.



Let us find a measure that combines both recall and precision.

F1 Score: The harmonic mean of precision and recall.

F1 Score = Harmonic Mean(Precision, Recall)

The higher the F1 score the better, with 0 being the worst possible and 1 being the best.

It reaches its optimum 1 only if precision and recall are both at 100%.

Precision - Fraction of True Positives out of *Total Predicted Positives*

Recall - Fraction of True Positives out of *Total Actual Positives*

F1 is a balance between Precision and Recall. Model's capacity to predict positives correctly out of actual as well as predicted positives!

Any performance metric that uses values from both classes will be inherently sensitive to class skews.

Metrics such as accuracy, precision, and F score use values from both +ves and -ves of the confusion matrix.

- Specificity: True Negative Rate
- How many -ve cases did we catch correctly?

- Specificity =
$$\frac{TN}{TN + FP}$$

		PREDICTIVE VALUES	
		POSITIVE (1)	NEGATIVE (0)
ACTUAL VALUES	POSITIVE (1)	TP	FN
	NEGATIVE (0)	FP	TN

- TPR - True positive rate / Recall / Sensitivity = $TP / (TP + FN)$
of +ves correctly classified
- FNR - False negative rate = $FN / (TP + FN)$
of +ves incorrectly classified
- TNR - True negative rate / Specificity = $TN / (TN + FP)$
of -ves correctly classified
- FPR - False positive rate / $1 - \text{Specificity} = FP / (TN + FP)$
of -ves incorrectly classified
- Precision - Fraction of True Positives out of Total Predicted Positives = $TP / (TP + FP)$
- Accuracy - # correct predictions made over all kind of predictions
- F1-Score - combined metric = $\text{harmonic_mean}(\text{recall}, \text{precision})$

Machine Learning

Confusion Matrix



		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

Consider the following confusion matrix for a **binary classifier**
Let us calculate all the metrics we discussed till now

		Predicted Class	
		Positive	Negative
Actual Class	Positive	True Positive (TP) 100	False Negative (FN) Type II Error 5
	Negative	False Positive (FP) Type I Error 10	True Negative (TN) 50

Accuracy - Overall, how often is the classifier correct?

$$(TP+TN)/total = (100+50)/165 = 0.91$$

Precision: When it predicts yes, how often is it correct?

$$TP/predicted\ yes = 100/(100 + 10) = 0.91$$

Recall - When it's actually yes, how often does it predict yes?

$$TP/actual\ yes = 100/(100 + 5) = 0.95$$

Specificity - When it's actually no, how often does it predict no?

$$TN/actual\ no = 50/(10+50) = 0.83$$

$$\begin{aligned} \text{F1 Score} &= 2*(recall*precision)/(recall+precision) \\ &= 2*(0.95*0.91)/(0.95+0.91) = 0.93 \end{aligned}$$

		Predicted Class	
		Positive	Negative
Actual Class	Positive	True Positive (TP) 100	False Negative (FN) Type II Error 5
	Negative	False Positive (FP) Type I Error 10	True Negative (TN) 50

- As the name implies, it is a confusion matrix that deals with **multiple classes** (i.e. more than 2 classes).
- Just like the 2-class confusion matrix, it describes the performance of a multi-class classification model.

		Predicted		
		A	B	C
True labels	A	2	2	0
	B	1	2	0
	C	0	0	3

Actual	Elephant	25	3	0	2
	Monkey	3	53	2	3
	Fish	2	1	24	2
	Lion	1	0	2	71
		Elephant	Monkey	Fish	Lion
		Predicted			

Example - Multi-class confusion matrix

Let's assume that our multi-class classification model is one that classifies images of dogs into the following breeds:

Husky

Labrador

Bulldog



Notation Used :
P(Actual, Prediction)

		Predicted		
		Husky	Labrador	Bulldog
Actual	Husky	P(H,H)	P(H,L)	P(H,B)
	Labrador	P(L,H)	P(L,L)	P(L,B)
	Bulldog	P(B,H)	P(B,L)	P(B,B)

Metrics calculations - Multi-class confusion matrix

For Husky class

True Positive	$P(H,H)$
True Negative	$P(L,L) + P(L,B) + P(B,L) + P(B,B)$
False Positive	$P(L,H) + P(B,H)$
False Negative	$P(H,L) + P(H,B)$



Notation Used :
 $P(\text{Actual}, \text{Prediction})$

		Predicted		
		Husky	Labrador	Bulldog
Actual	Husky	$P(H,H)$	$P(H,L)$	$P(H,B)$
	Labrador	$P(L,H)$	$P(L,L)$	$P(L,B)$
	Bulldog	$P(B,H)$	$P(B,L)$	$P(B,B)$

Metrics calculations - Multi-class confusion matrix

For Labrador class

True Positive	$P(L,L)$
True Negative	$P(H,H) + P(H,B) + P(B,H) + P(B,B)$
False Positive	$P(H,L) + P(B,L)$
False Negative	$P(L,H) + P(L,B)$



Notation Used :
 $P(\text{Actual}, \text{Prediction})$

		Predicted		
		Husky	Labrador	Bulldog
Actual	Husky	$P(H,H)$	$P(H,L)$	$P(H,B)$
	Labrador	$P(L,H)$	$P(L,L)$	$P(L,B)$
	Bulldog	$P(B,H)$	$P(B,L)$	$P(B,B)$

Metrics calculations - Multi-class confusion matrix

For Bulldog class

True Positive	$P(B,B)$
True Negative	$P(H,H) + P(H,L) + P(L,H) + P(L,L)$
False Positive	$P(H,B) + P(L,B)$
False Negative	$P(B,H) + P(B,L)$



Notation Used :
 $P(\text{Actual}, \text{Prediction})$

		Predicted		
		Husky	Labrador	Bulldog
Actual	Husky	$P(H,H)$	$P(H,L)$	$P(H,B)$
	Labrador	$P(L,H)$	$P(L,L)$	$P(L,B)$
	Bulldog	$P(B,H)$	$P(B,L)$	$P(B,B)$

Accuracy calculation - multi-class confusion matrix

$$\text{Accuracy} = \frac{P(H,H) + P(L,L) + P(B,B)}{(P(H,H) + P(L,H) + P(B,H) + P(H,L) + P(L,L) + P(B,L) + P(H,B) + P(L,B) + P(B,B))}$$

**Let us calculate
Accuracy of the
Classifier**

**Notation Used :
P(Actual, Prediction)**

		Predicted		
		Husky	Labrador	Bulldog
Actual	Husky	P(H,H)	P(H,L)	P(H,B)
	Labrador	P(L,H)	P(L,L)	P(L,B)
	Bulldog	P(B,H)	P(B,L)	P(B,B)

Precision Calculation - multi-class confusion matrix

For Husky class

True Positive	$P(H,H)$
True Negative	$P(L,L) + P(L,B) + P(B,L) + P(B,B)$
False Positive	$P(L,H) + P(B,H)$
False Negative	$P(H,L) + P(H,B)$

For Labrador class

True Positive	$P(L,L)$
True Negative	$P(H,H) + P(H,B) + P(B,H) + P(B,B)$
False Positive	$P(H,L) + P(B,L)$
False Negative	$P(L,H) + P(L,B)$

For Bulldog class

True Positive	$P(B,B)$
True Negative	$P(H,H) + P(H,L) + P(L,H) + P(L,L)$
False Positive	$P(H,B) + P(L,B)$
False Negative	$P(B,H) + P(B,L)$

$$\text{Precision} = \frac{\text{True positives}}{\text{True positives} + \text{False Positives}}$$

$$\text{Precision (Husky)} = \frac{P(H,H)}{P(H,H) + P(L,H) + P(B,H)}$$

$$\text{Precision (Labrador)} = \frac{P(L,L)}{P(L,L) + P(H,L) + P(B,L)}$$

$$\text{Precision (Bulldog)} = \frac{P(B,B)}{P(B,B) + P(H,B) + P(L,B)}$$

Recall/TPR Calculation - multi-class confusion matrix

For Husky class

True Positive	$P(H,H)$
True Negative	$P(L,L) + P(L,B) + P(B,L) + P(B,B)$
False Positive	$P(L,H) + P(B,H)$
False Negative	$P(H,L) + P(H,B)$

For Labrador class

True Positive	$P(L,L)$
True Negative	$P(H,H) + P(H,B) + P(B,H) + P(B,B)$
False Positive	$P(H,L) + P(B,L)$
False Negative	$P(L,H) + P(L,B)$

For Bulldog class

True Positive	$P(B,B)$
True Negative	$P(H,H) + P(H,L) + P(L,H) + P(L,L)$
False Positive	$P(H,B) + P(L,B)$
False Negative	$P(B,H) + P(B,L)$

$$\text{Recall} = \frac{\text{True positives}}{\text{True positives} + \text{False Negatives}}$$

$$\text{Recall (Husky)} = \frac{P(H,H)}{P(H,H) + P(H,L) + P(H,B)}$$

$$\text{Recall (Labrador)} = \frac{P(L,L)}{P(L,L) + P(L,H) + P(L,B)}$$

$$\text{Recall (Bulldog)} = \frac{P(B,B)}{P(B,B) + P(B,H) + P(B,L)}$$

Multi-class confusion matrix - Practice Problem

Now you know how to calculate the metric for n classes.

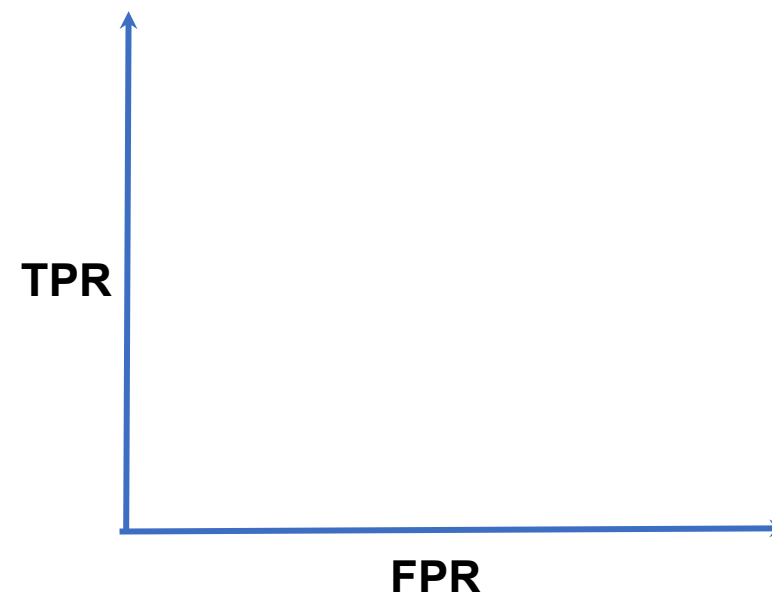
Evaluate the given confusion matrix...

		Predicted			
		A	B	C	D
Actual	A	100	80	10	10
	B	0	9	0	1
	C	0	1	8	1
	D	0	1	0	9

0.601

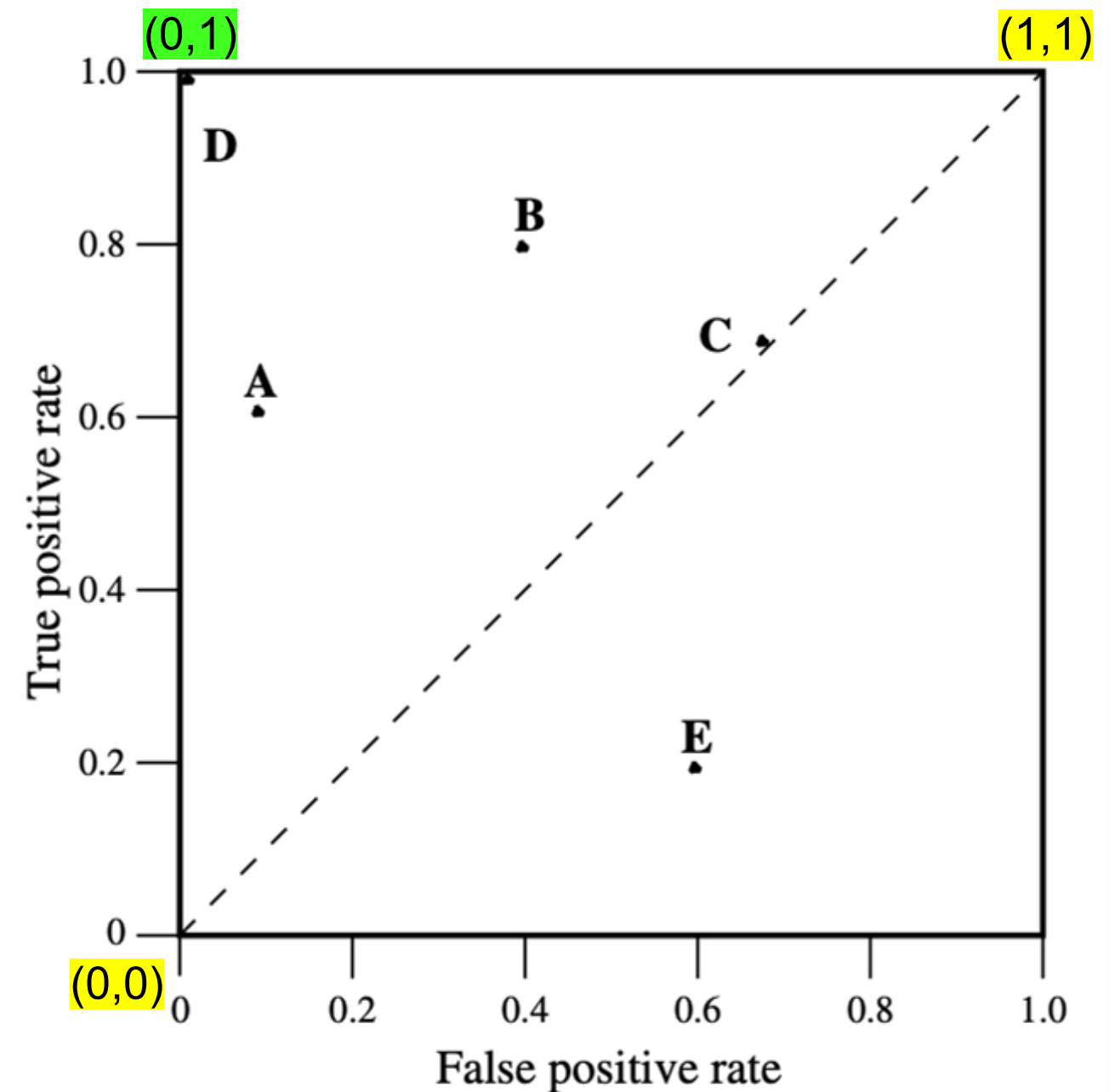
ROC - Receiver Operating Characteristics

- ROC is a probability curve that plots the TPR against FPR at various threshold values and essentially separates the 'signal' from the 'noise'.
- An ROC graph depicts relative tradeoffs between benefits (true positives) and costs (false positives).



Points in ROC space are important to note.

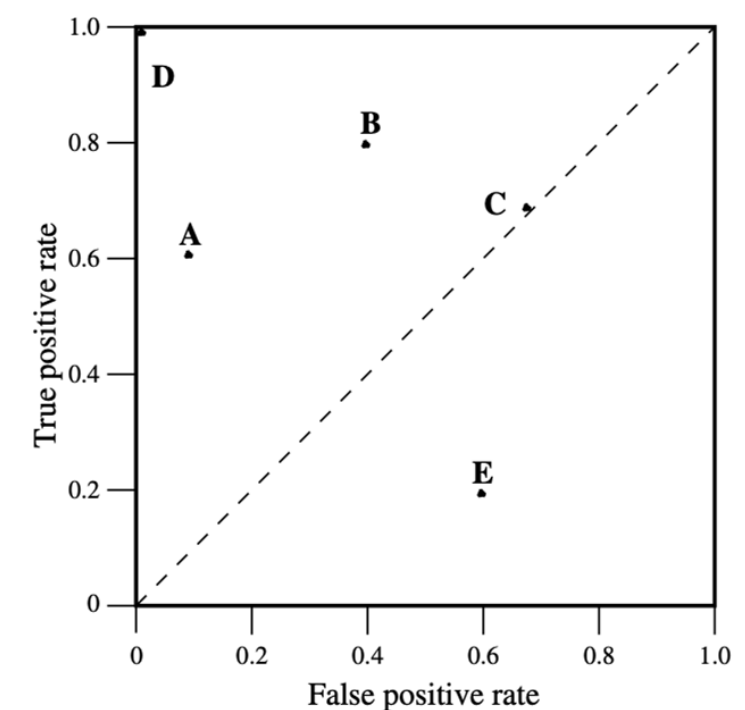
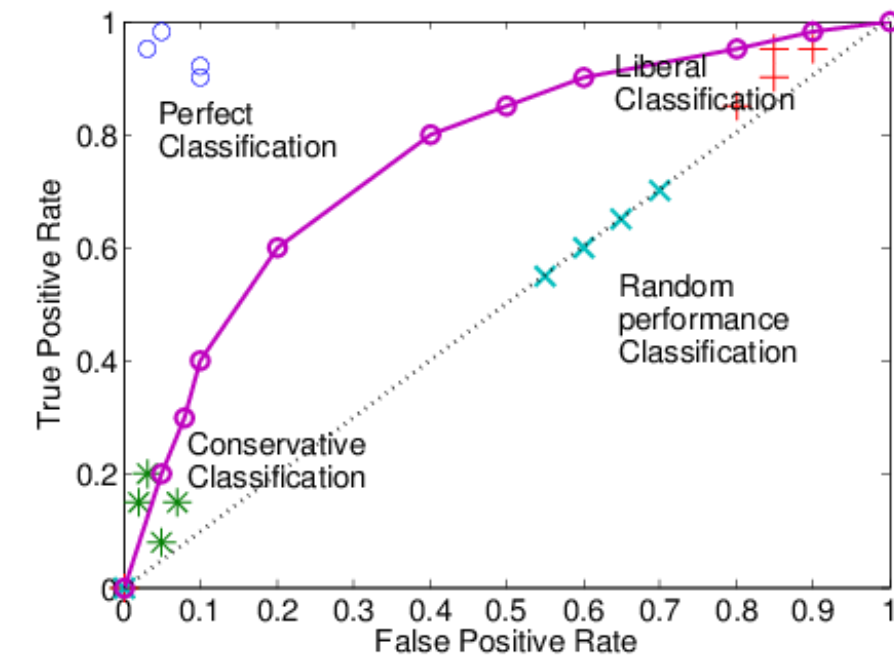
- **(0, 0)** - never issuing a positive classification; such a classifier commits no false positive errors but also gains no true positives.
- **(1, 1)** - always issuing a positive classification
- **(0, 1)** - represents a perfect classification
- Diagonal line **$y=x$** represents the strategy of randomly guessing a class.
- Classifiers in the lower right triangle performs worse than random guessing.



Machine Learning

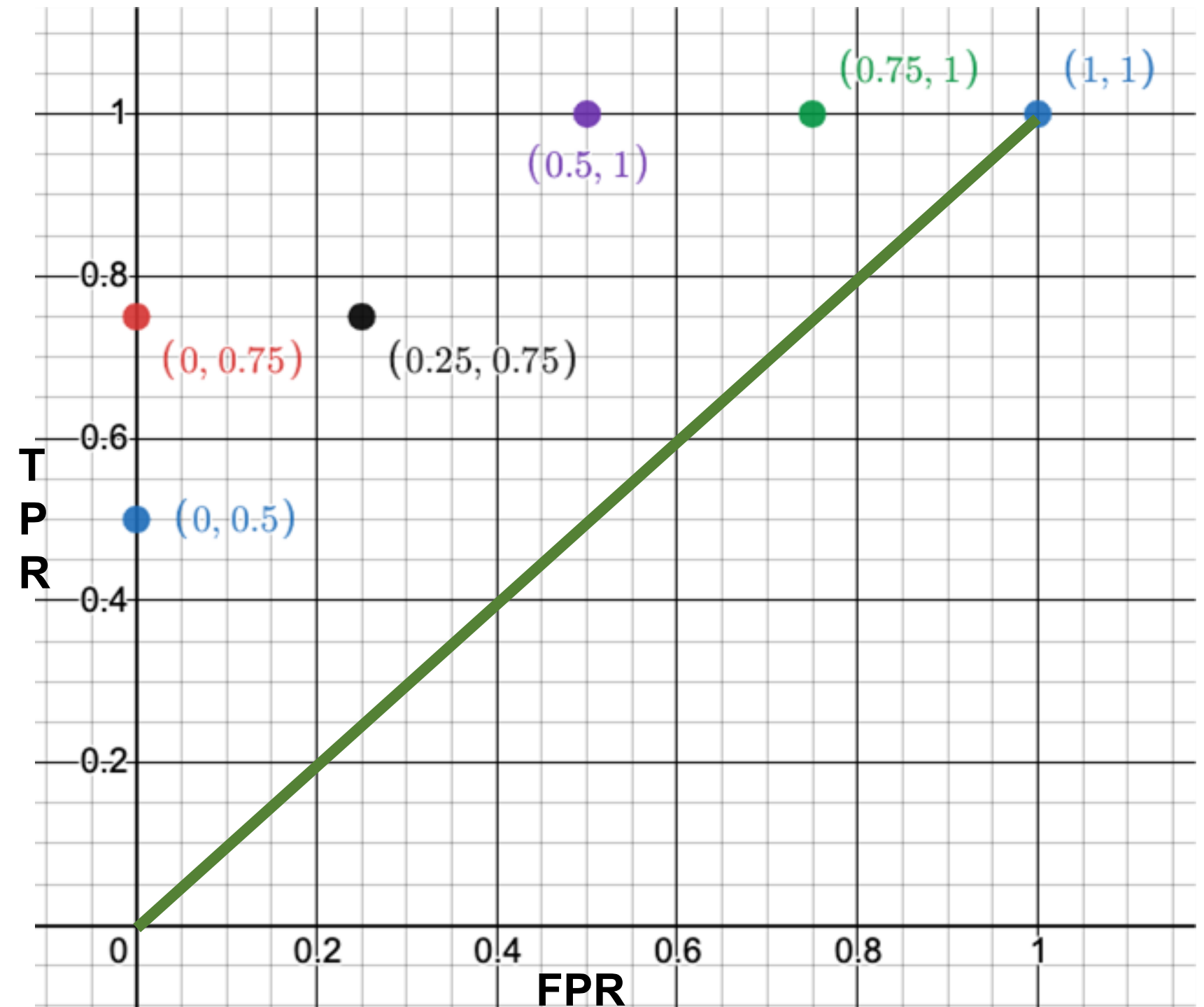
ROC Interpretation

- A point in ROC space is better than another if it is to the northwest (tp rate is higher, fp rate is lower, or both – towards (0,1)) of the first.
- **Conservative classifiers** tend to have lower true positive rates and lower false positive rates, indicating a preference for strong evidence before making positive classifications.
- **Liberal classifiers** exhibit higher true positive rates and higher false positive rates, suggesting a tendency to classify more instances as positive, even with weaker evidence.
- Example: A is more conservative than B.



Take the example of an obesity classifier

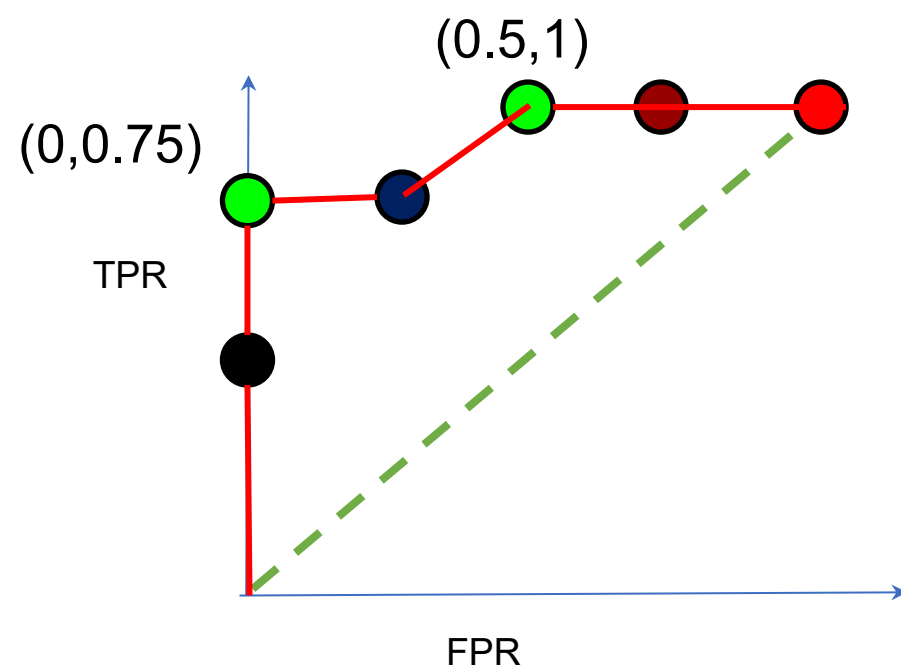
Threshold	FPR	TPR
0 (all sample classified obese)	1	1
0.3	0.75	1
0.4	0.5	1
0.6	0.25	0.75
0.7	0	0.75
0.9	0	0.5

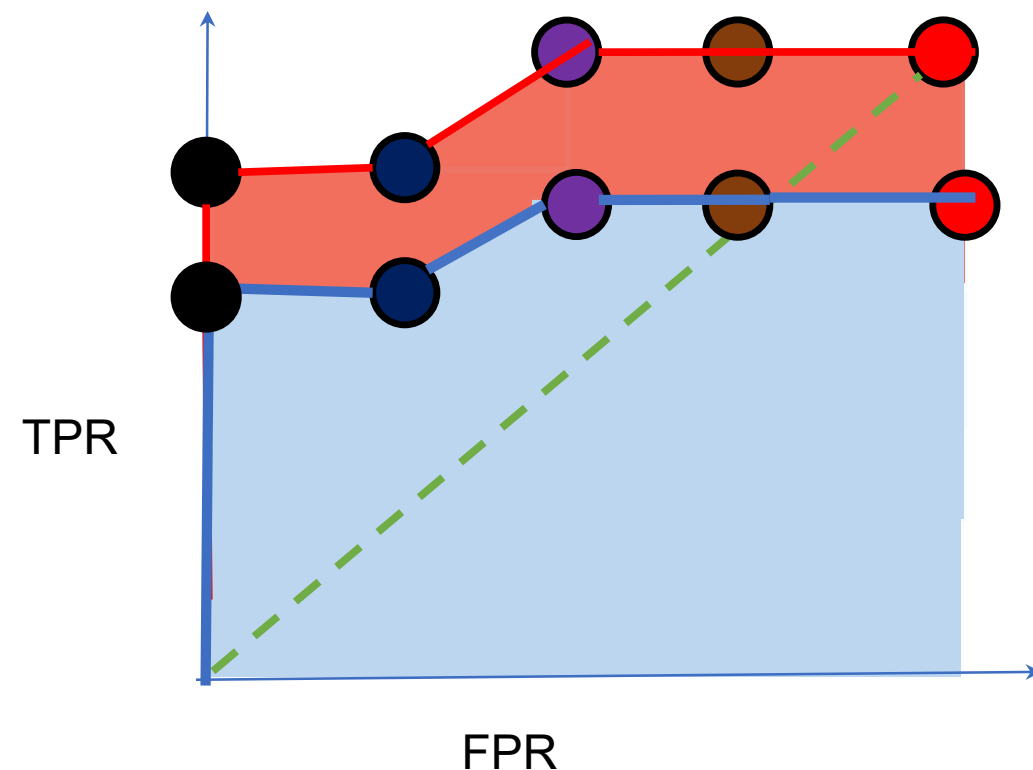


Threshold	FPR	TPR
0 (all sample classified obese)	1	1
0.3	0.75	1
0.4	0.5	1
0.6	0.25	0.75
0.7	0	0.75
0.9	0	0.5

The ROC graph summarizes all of the confusion matrix that each threshold produced

From the graph depending on how many false positive we are willing to accept the optimal one will be the point (0,0.75) or (0.5,1)





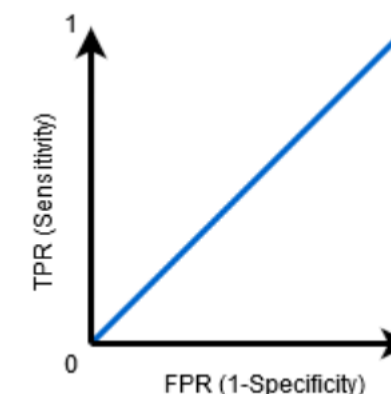
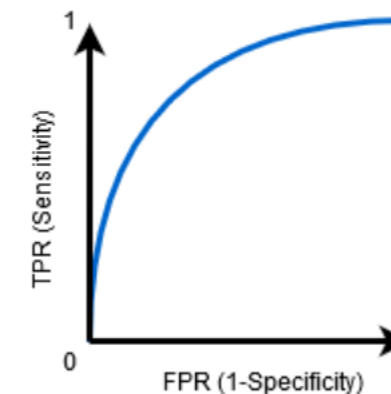
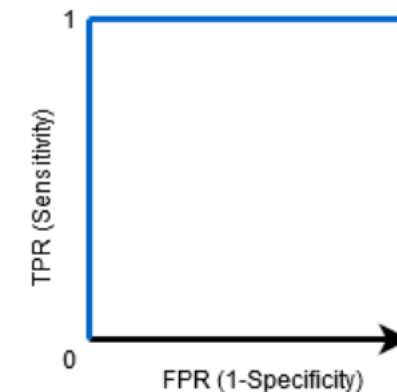
Now that we know what is ROC graph let us discuss the Area Under the Curve (AUC)

The AUC for the red ROC curve is greater than the AUC for the blue which suggests method 1 is better than method two

The AUC defines how good is our model, **more the AUC better is the model**. This helps us in comparing two models and selecting best among them for our problem

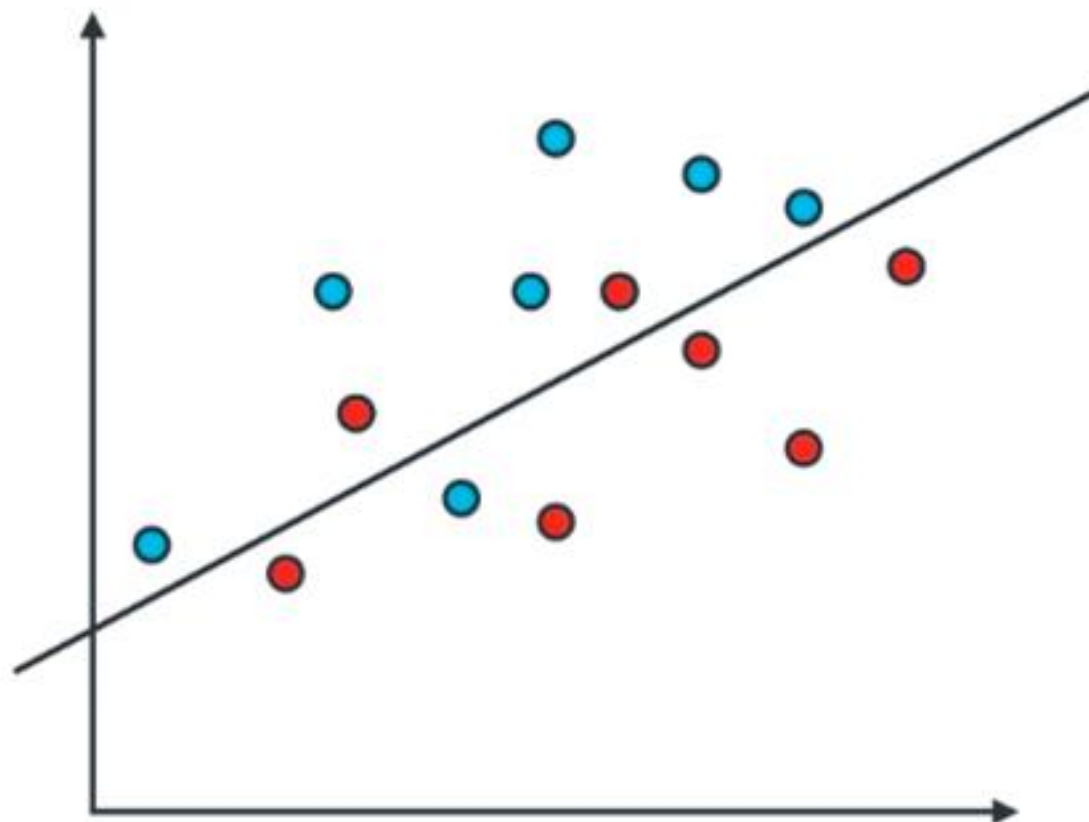
The higher the AUC, the better the performance of the model at distinguishing between the positive and negative classes.

- When **AUC = 1**, then the classifier is able to perfectly distinguish between all the Positive and the Negative class points correctly.
- If, however, the **AUC had been 0**, then the classifier would be predicting all Negatives as Positives, and all Positives as Negatives.
- When **$0.5 < \text{AUC} < 1$** , there is a high chance that the classifier will be able to distinguish the positive class values from the negative class values.
- When **AUC=0.5**, then the classifier is not able to distinguish between Positive and Negative class points. Meaning either the classifier is predicting random class or constant class for all the data points.



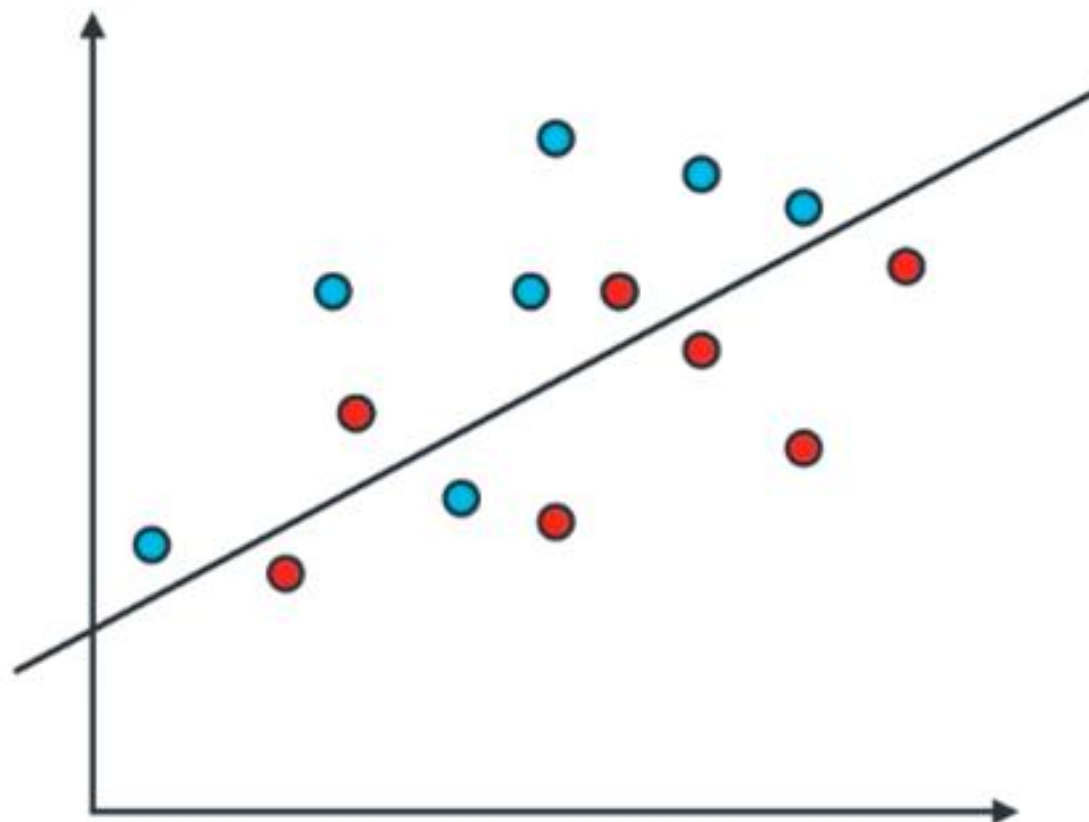
Practice Questions

Find the number of TP, FP, TN, FN cases for the given graph, where blue is positive and red is negative



		Prediction	
		Guessed Positive	Guessed Negative
Data	Positive		
	Negative		

Find the number of TP, FP, TN, FN cases for the given graph, where blue is positive and red is negative



		Prediction	
		Guessed Positive	Guessed Negative
Data	Positive	6 True positives	1 False Negatives
	Negative	2 False Positives	5 True Negatives



Accuracy

Patients	Diagnosis	
	Diagnosed sick	Diagnosed Healthy
	Sick	Healthy
Sick	1000	200
Healthy	800	8000

Accuracy: Out of the all the patients, how many did we classify correctly?

$$\text{Accuracy} = \frac{1,000 + 8,000}{1000 + 200 + 800 + 8000}$$

$$= 90\%$$



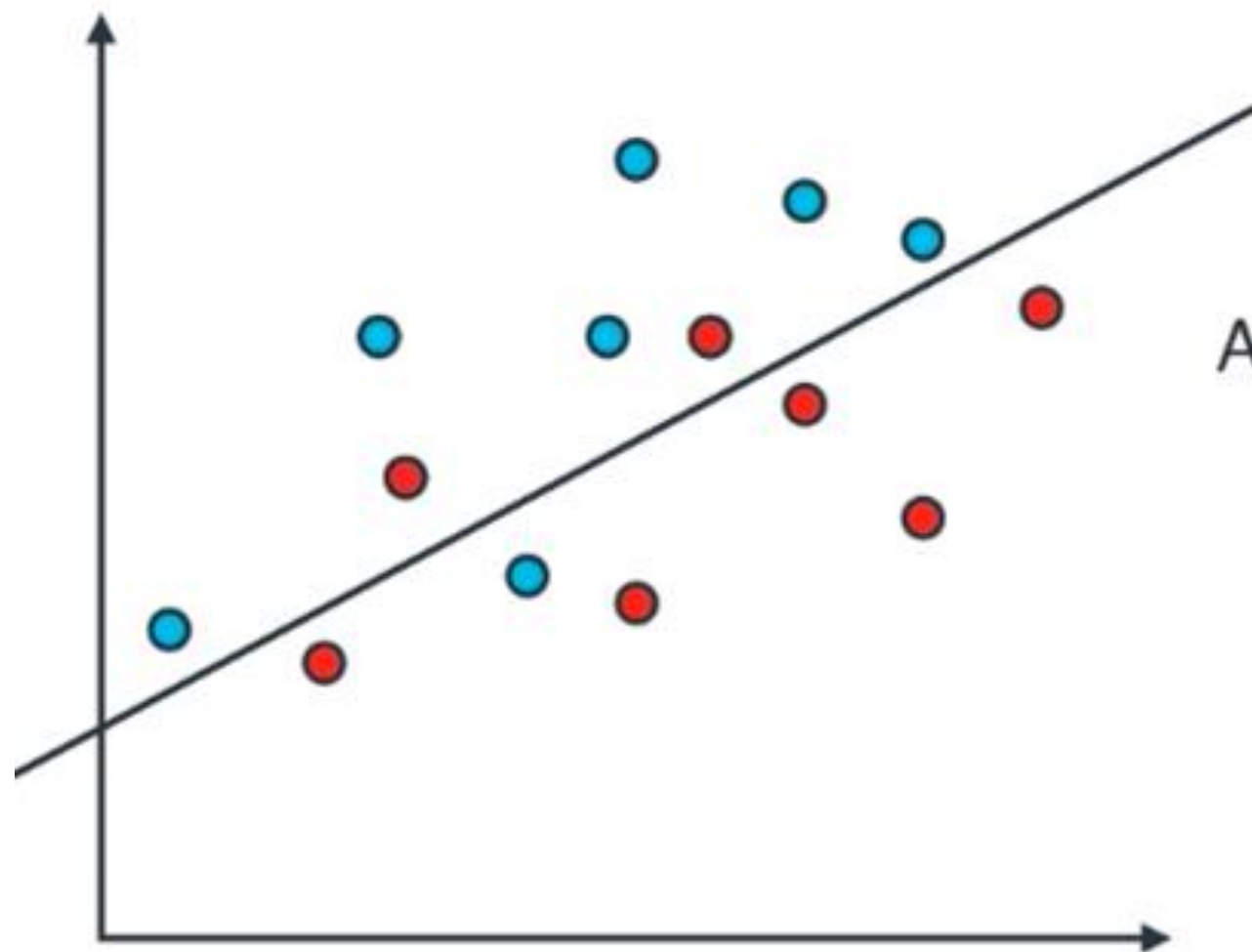
Accuracy

E-mail	Folder	
	Spam Folder	Inbox
Spam	100	170
Not spam	30	700

Accuracy: Out of the all the e-mails, how many did we classify correctly?

$$\text{Accuracy} = \frac{100 + 700}{100 + 170 + 30 + 700}$$

$$= 80\%$$



Precision: Out of all the data, how many points did we classify correctly?

$$\text{Accuracy} = \frac{\text{Correctly Classified points}}{\text{All points}}$$



Precision

Diagnosis

Patients	Diagnosis	
	Sick	Healthy
	Diagnosed sick	Diagnosed Healthy
Sick	1000	200 ❌
Healthy	800	8000

Precision: Out of the patients we diagnosed with an illness, how many did we classify correctly?

$$\text{Precision} = \frac{1,000}{1,000 + 800} = 55.7\%$$

Precision

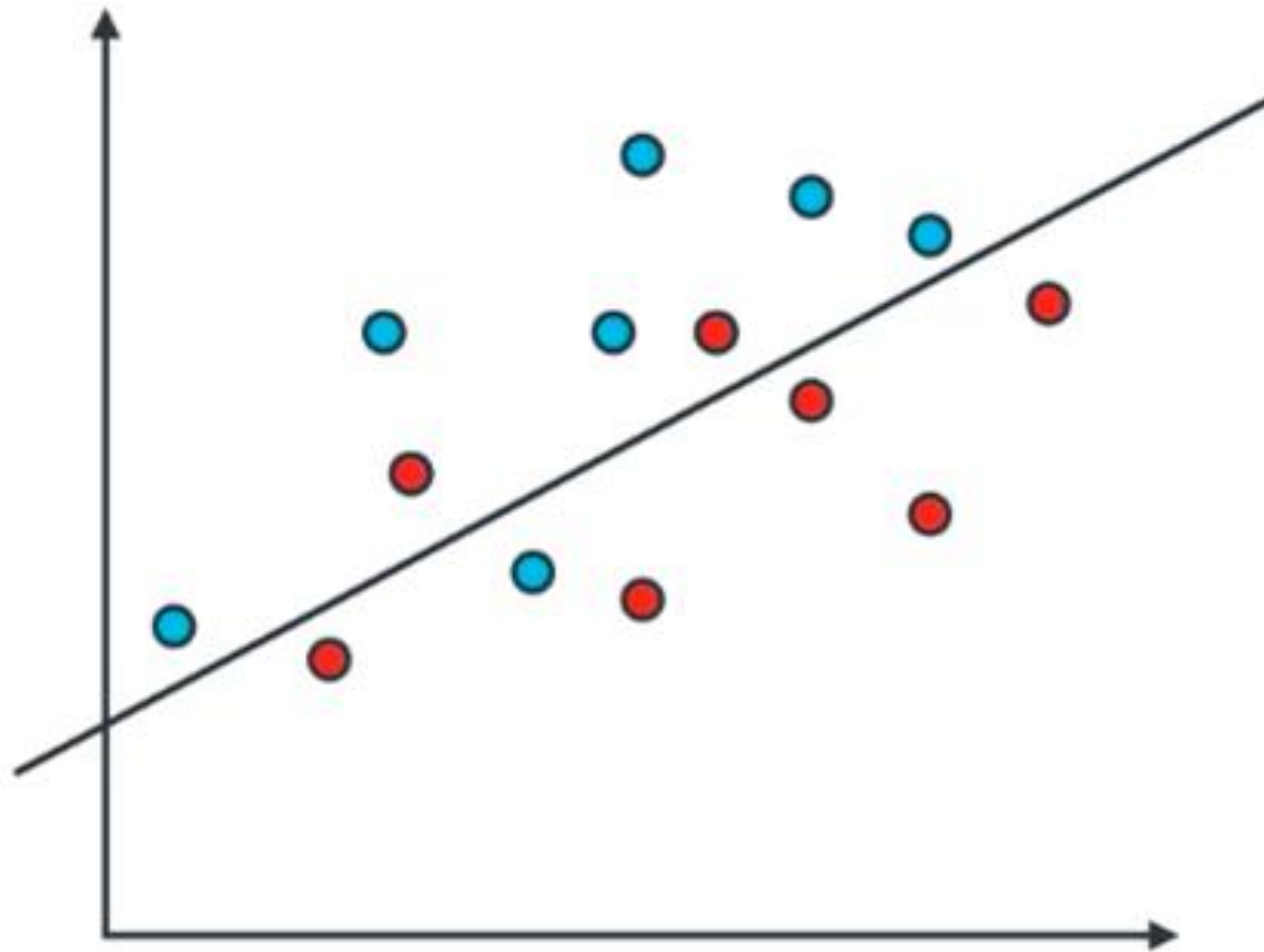


Folder

E-mail	Folder	
	Spam Folder	Inbox
Spam	100	170
Not spam	30 	700

Precision: Out of the all the e-mails, sent to the spam inbox, how many were actually spam?

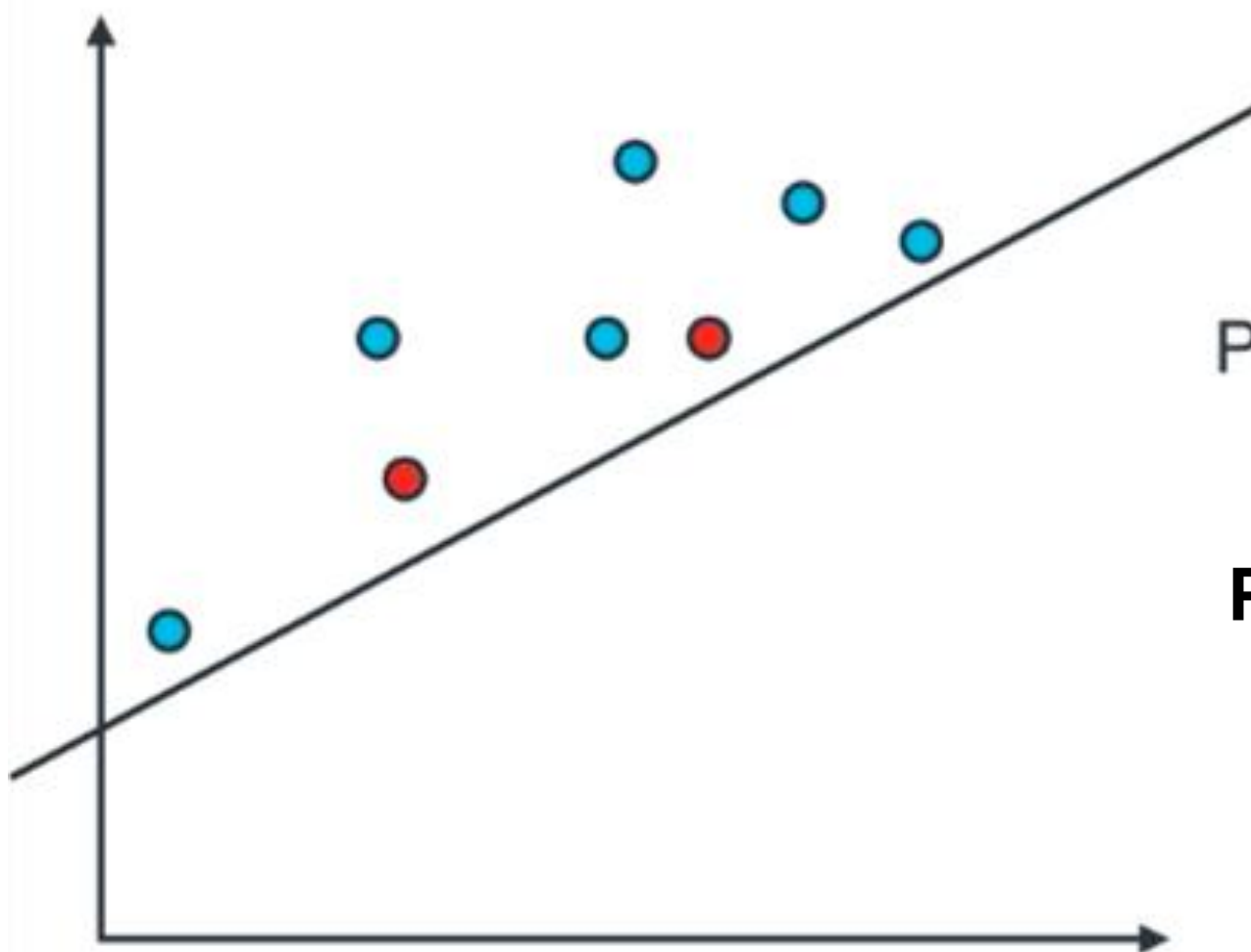
$$\text{Precision} = \frac{100}{100 + 30} = 76.9\%$$



Precision: Out of the points we've predicted to be positive, how many are correct?

Machine Learning

Precision - Solution



Precision: Out of the points we've predicted to be positive, how many are correct?

$$\text{Precision} = \frac{\text{True positives}}{\text{True positives} + \text{False Positives}}$$

$$\text{Precision} = 6/8$$

Machine Learning

Recall - Example

Recall



Patients	Diagnosis	
	Diagnosed Sick	Diagnosed Healthy
	Sick	Is Healthy
	1000	200 ✖
	800	8000

Recall: Out of the sick patients, how many did we correctly diagnose as sick?

$$\text{Recall} = \frac{1,000}{1,000 + 200} = 83.3\%$$

Machine Learning

Recall - Example



Recall

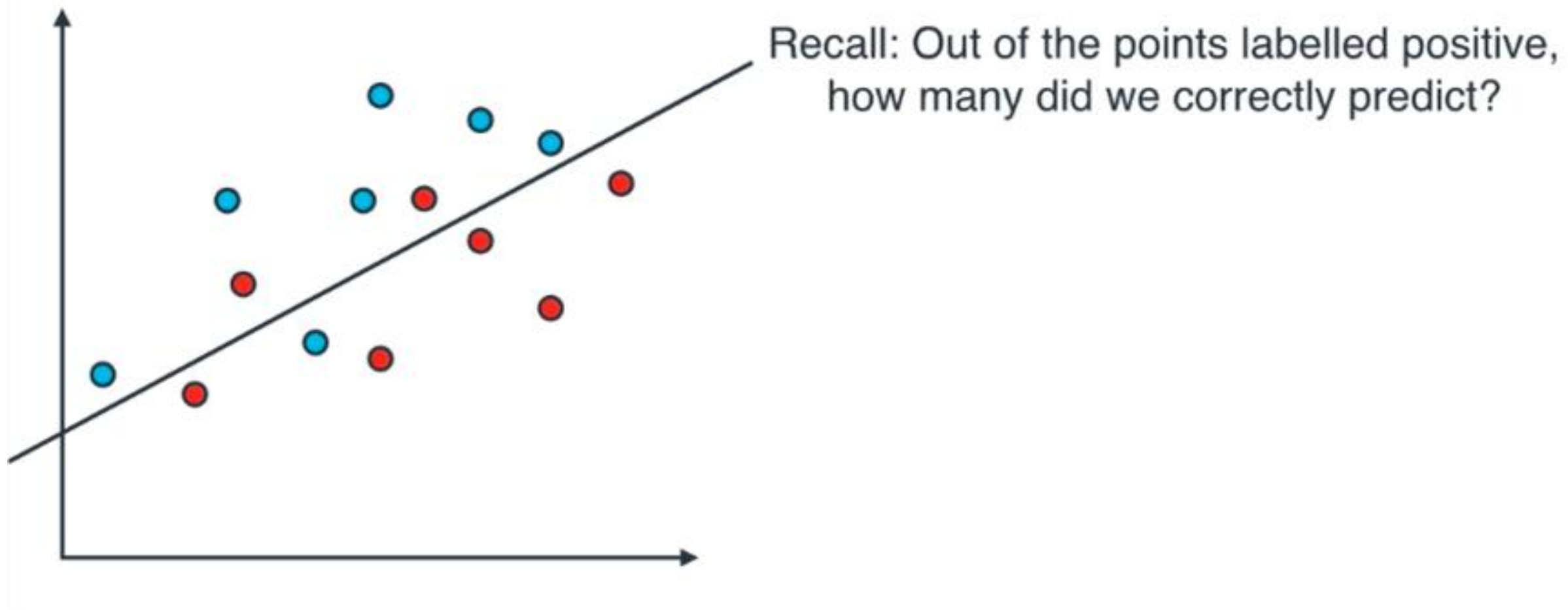
E-mail	Folder	
	Spam Folder	Inbox
Spam	100	170
Not spam	30 	700

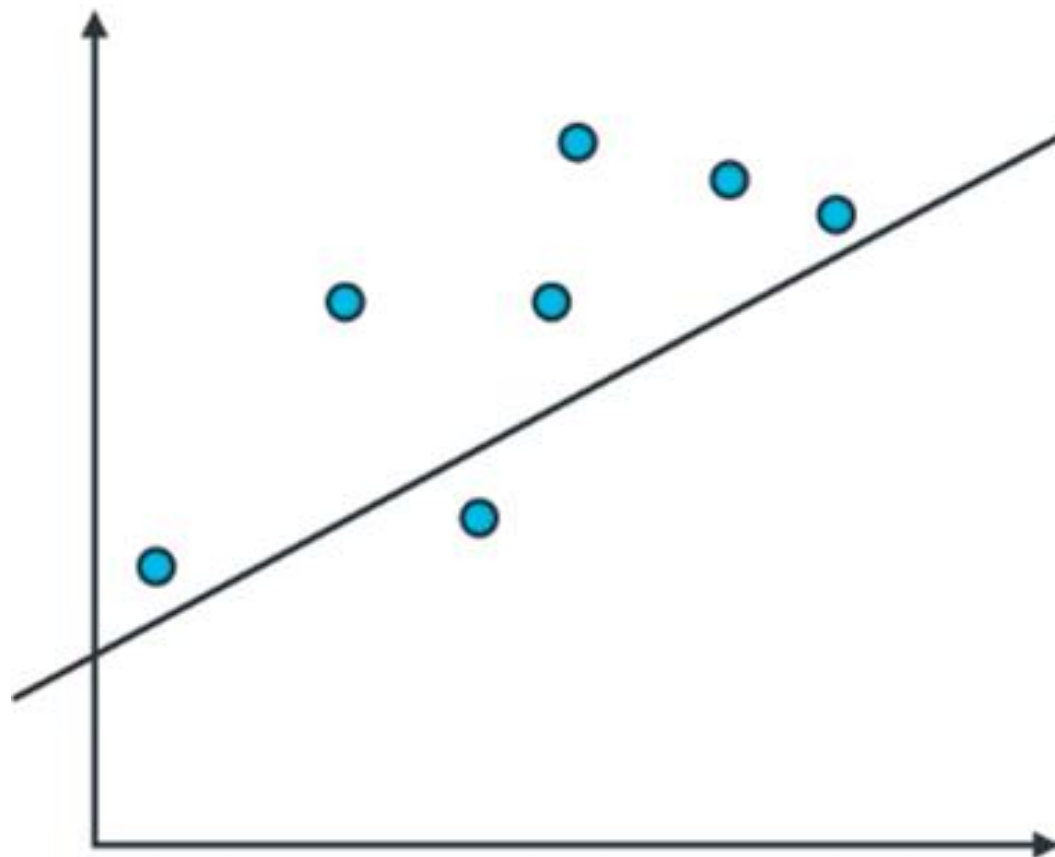
Recall: Out of the all the spam e-mails, how many were correctly sent to the spam folder?

$$\text{Recall} = \frac{100}{100 + 170} = 37\%$$

Machine Learning

Recall - Question





Recall: Out of the points labelled positive, how many did we correctly predict?

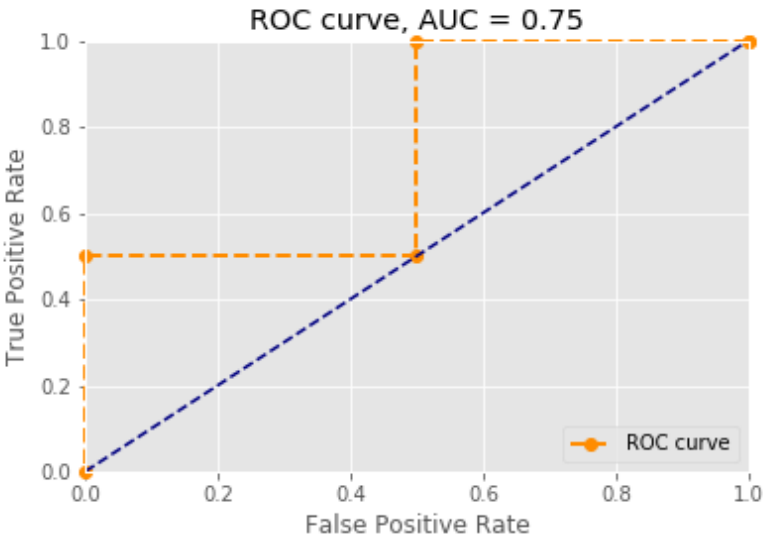
$$\text{Recall} = \frac{\text{True positives}}{\text{True positives} + \text{False Negatives}}$$

$$\text{Recall} = 6/7$$

Given the data, plot ROC Curve

Actual Class (y)	Predicted Probabilities ($\hat{y}$)	Threshold					
		$(\hat{y}_0)$	$(\hat{y}_{0.2})$	$(\hat{y}_{0.4})$	$(\hat{y}_{0.6})$	$(\hat{y}_{0.8})$	$(\hat{y}_1)$
1	0.8	1	1	1	1	1	0
0	0.6	1	1	1	1	0	0
1	0.4	1	1	1	0	0	0
0	0.2	1	1	0	0	0	0

FPR	1	1	0.5	0.5	0	0
TPR	1	1	1	0.5	0.5	0





UE24CS352A: Machine Learning

Dr. Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu



UE24CS352A: Machine Learning

Dr. Shylaja S S

Professor

Department of Computer Science & Engineering

Acknowledgement :

Dr. Andrew Ng (Stanford University)

Dr. Rajanikanth K (Academic Advisor, PESU)

Teaching Assistant (Arya Rajiv Chaloli)

Binary Classification:

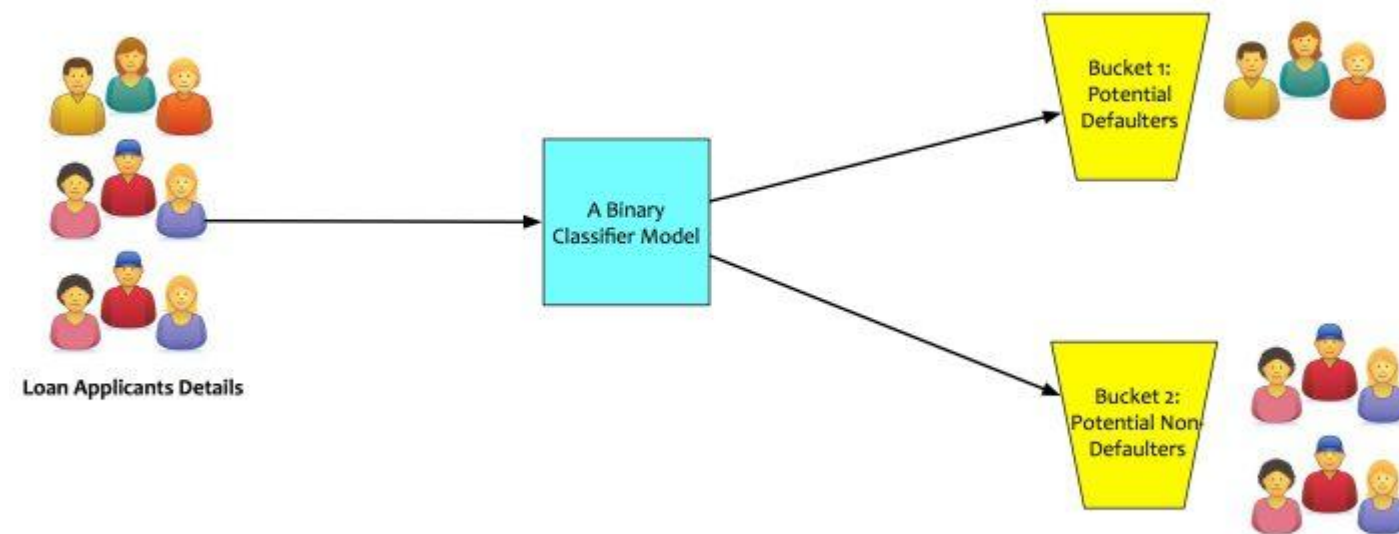
Email: Spam / Not-Spam

Tumor: Malignant / Benign

$y \in \{0, 1\}$

0: "negative class"

1: "positive class"



Multi-class classification problem: $y \in \{0, 1, 2, 3, \dots\}$

Developing a classification algorithm

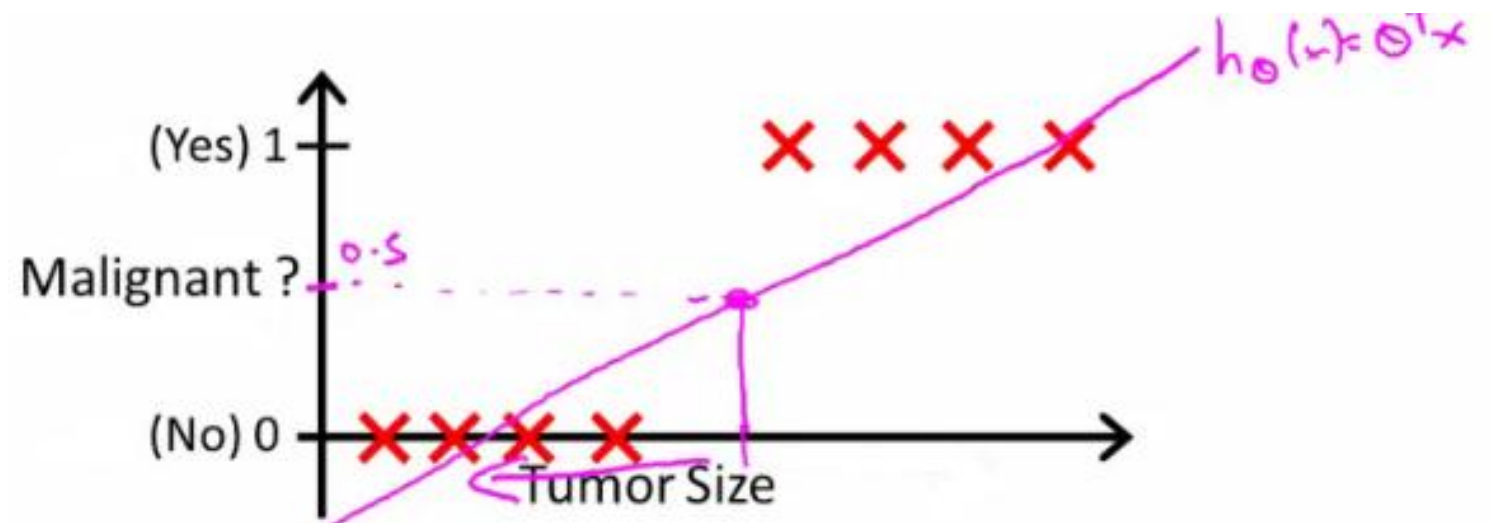
Problem Definition:

Tumour size vs Malignancy (0 or 1)

We could use linear regression

To make predictions the threshold classifier output:

- $h_{\theta}(x) \geq 0.5$, predict $y = 1$
- $h_{\theta}(x) < 0.5$, predict $y = 0$

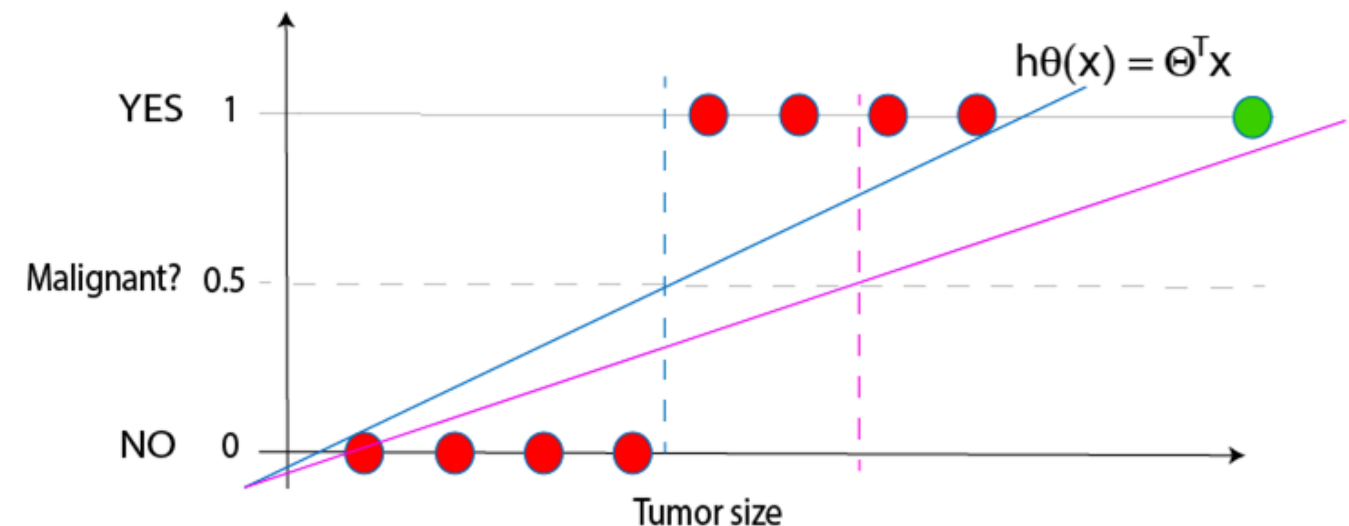


Developing a classification algorithm

Adding an extra point further away.

The linear regression model provides a bad fit.

- Often linear regression is not suitable for classification problems
- It gives a hypothesis function which can output a value much more than 1 or less than 0, when the true value is always either 0 or 1
 - $h_{\theta}(x) > 1$, $h_{\theta}(x) < 0$ are possible
 - but $y \in \{ 0,1 \}$



Thus, develop an algorithm which has output/predictions that are in the range of 0-1, i.e., $0 \leq h_{\theta}(x) \leq 1$



Logistic Regression

Hypothesis representation

The hypothesis (output) of a linear regression is

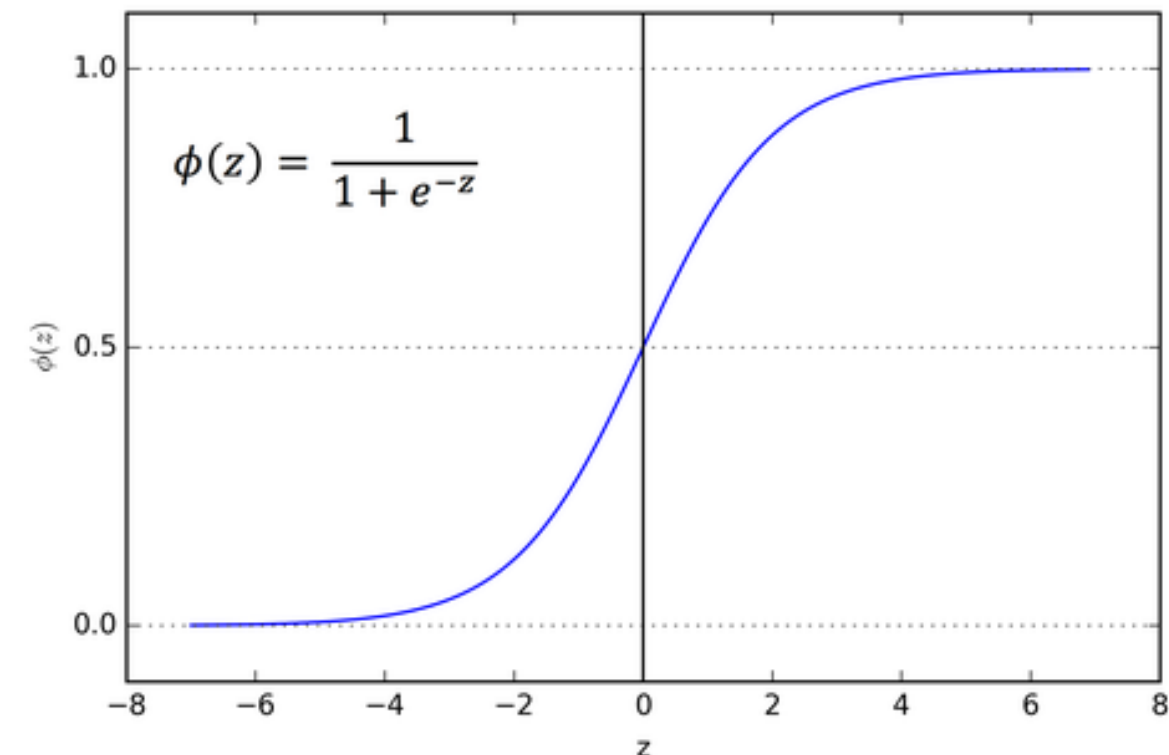
$$h_{\theta}(x) = \theta^T x$$

In logistic regression, for the hypothesis, we use the **sigmoid/logistic function** over the linear regression output to contain it within 0 and 1

$$h_{\theta}(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$



Interpretation of the hypothesis output

- Hypothesis of logistic regression ($h_{\theta}(x)$) is the **estimated probability that $y=1$ on input x**
- From the previous example:
 $h_{\theta}(x) = 0.7 \rightarrow$ means there is a 70% chance that the tumor is malignant
- $h_{\theta}(x) = p(y=1 \mid x; \theta)$ implies "*given x and θ , $h_{\theta}(x)$ is the probability that $y=1$* "
- Since y can only be 0 or 1, $p(y=1 \mid x; \theta) + p(y=0 \mid x; \theta) = 1$

Interpretation of the hypothesis output

$$h_{\theta}(x) \geq 0.5, \text{ predict } y=1$$

$$g(z) \geq 0.5$$

(when $z \geq 0$)

$$h_{\theta}(x) = g(\theta^T x) \geq 0.5$$

(when $\theta^T x \geq 0$)

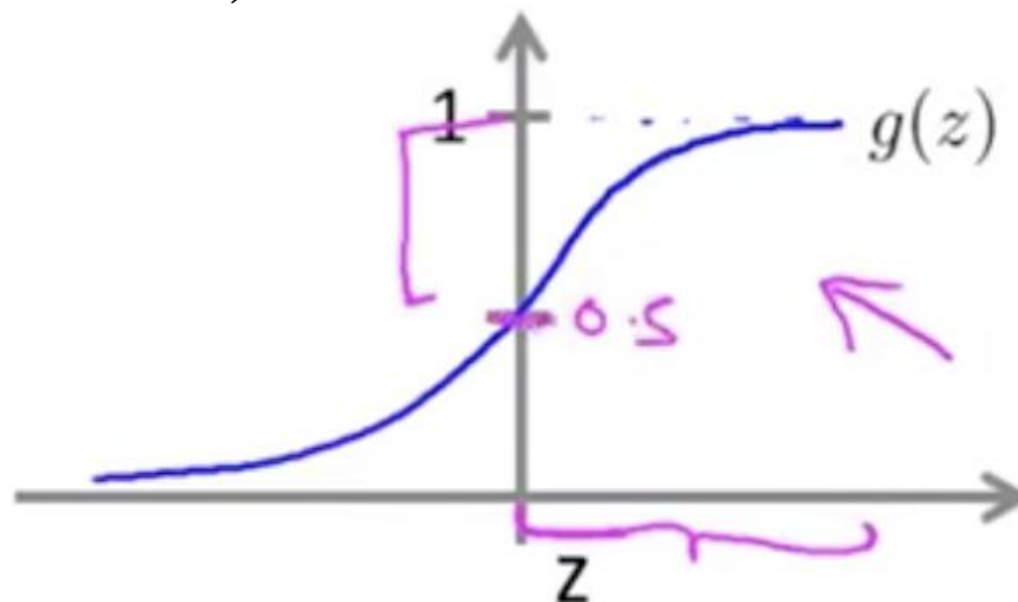
$$h_{\theta}(x) < 0.5, \text{ predict } y=0$$

$$g(z) < 0.5$$

(when $z < 0$)

$$h_{\theta}(x) = g(\theta^T x) < 0.5$$

(when $\theta^T x < 0$)



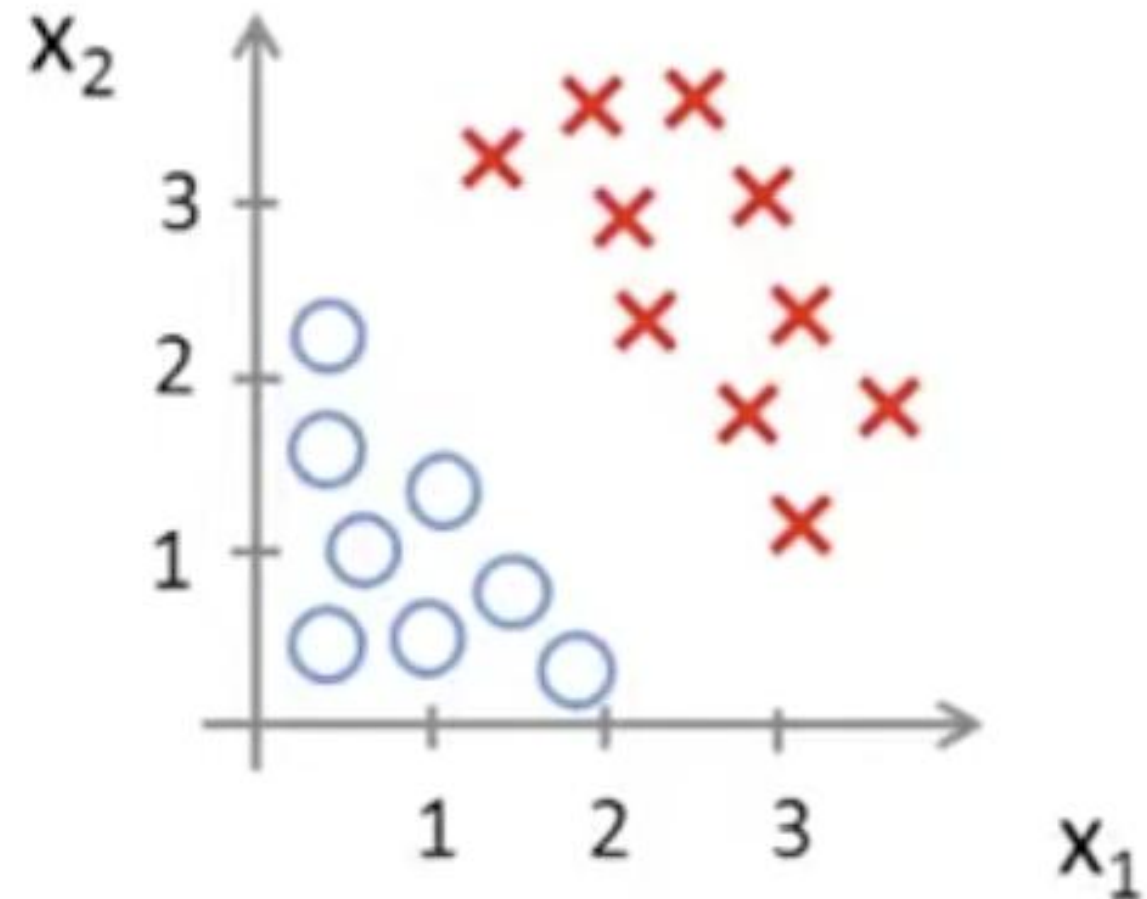
- Let $h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$

- Assume

$$\theta = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$$

- Predict $y=1$ if
...

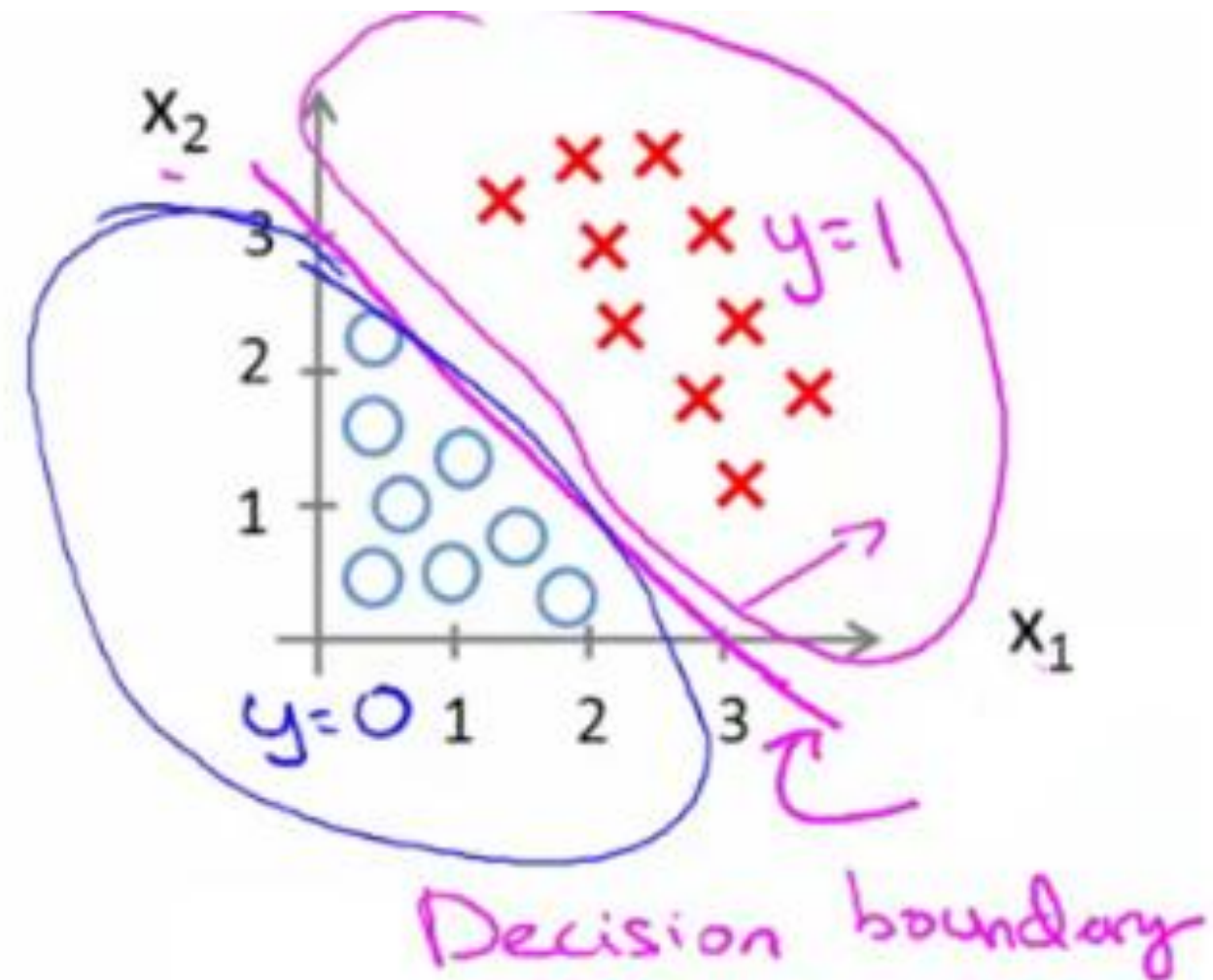
- Predict $y=0$ if
...



- $h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$

$$\theta = \begin{bmatrix} -3 \\ 1 \\ 1 \end{bmatrix}$$

- Predict $y=1$ if
$$\begin{aligned} -3 + x_1 + x_2 &\geq 0 \\ x_1 + x_2 &\geq 3 \end{aligned}$$
- Similarly, predict $y=0$ if
$$\begin{aligned} -3 + x_1 + x_2 &< 0 \\ x_1 + x_2 &< 3 \end{aligned}$$



$x_1 + x_2 = 3 \rightarrow$ graphically plot the **decision boundary**

Non Linear Decision Boundary

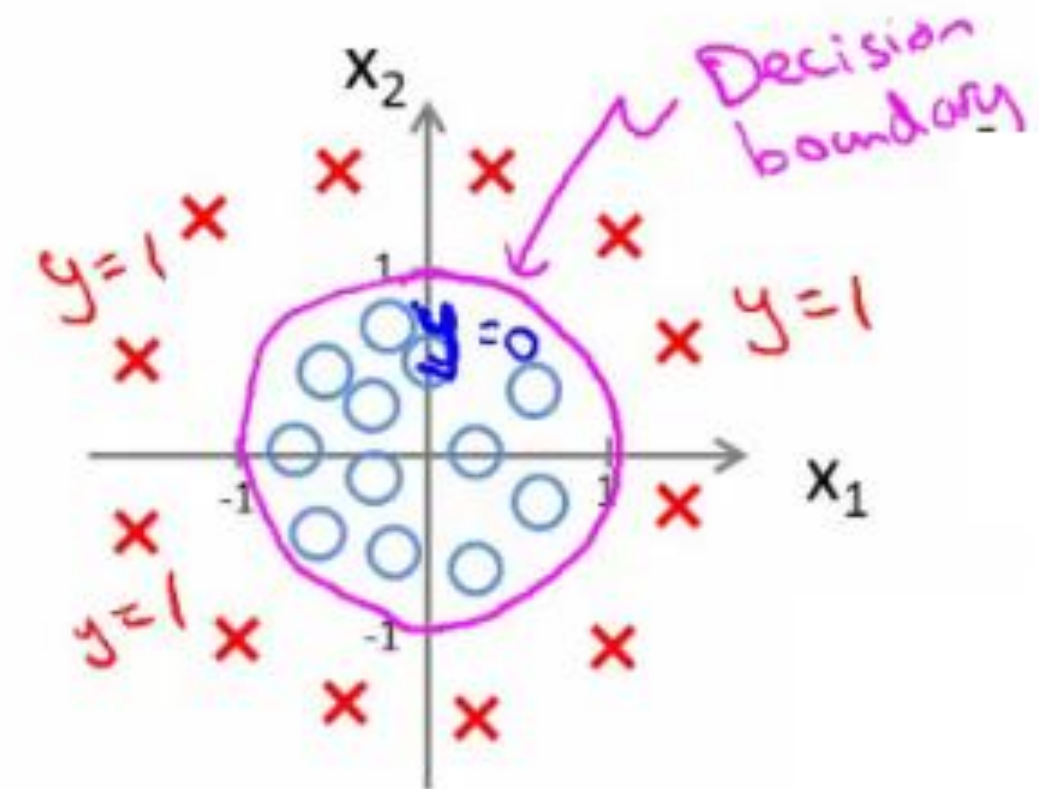
- Get logistic regression to fit a complex non-linear data set

- Add higher order terms

- Let $h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_3 x_1^2 + \theta_4 x_2^2)$

- Assume θ^T was $[-1, 0, 0, 1, 1]$

- Then predict $y=1$ if
$$-1 + x_1^2 + x_2^2 \geq 0$$
or
$$x_1^2 + x_2^2 \geq 1$$



- Plot $x_1^2 + x_2^2 = 1$ gives us a circle with a radius of 1 around 0

- Higher order polynomial terms, yields even more complex decision boundaries

Cost function for fitting a logistic regression model

Defining the optimization object for the cost function used to fit the parameters

Given:

- Training set: m examples $\{ (x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots (x^{(m)}, y^{(m)}) \}$
- n -dimensions: feature vector contains $(n+1)$ elements $\theta^T = [\theta_0 \ \theta_1 \ \dots \ \theta_n]$
- $x_0 = 1, y \in \{ 0, 1 \}$
- Hypothesis is based on parameters (θ)

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

How to choose the parameters (θ) ?

Cost function for fitting a logistic regression model

Linear regression uses the following function to determine θ

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Writing the squared error term as $Cost()$

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m Cost(h_{\theta}(x^{(i)}), y^{(i)})$$

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m Cost(h_{\theta}(x), y)$$

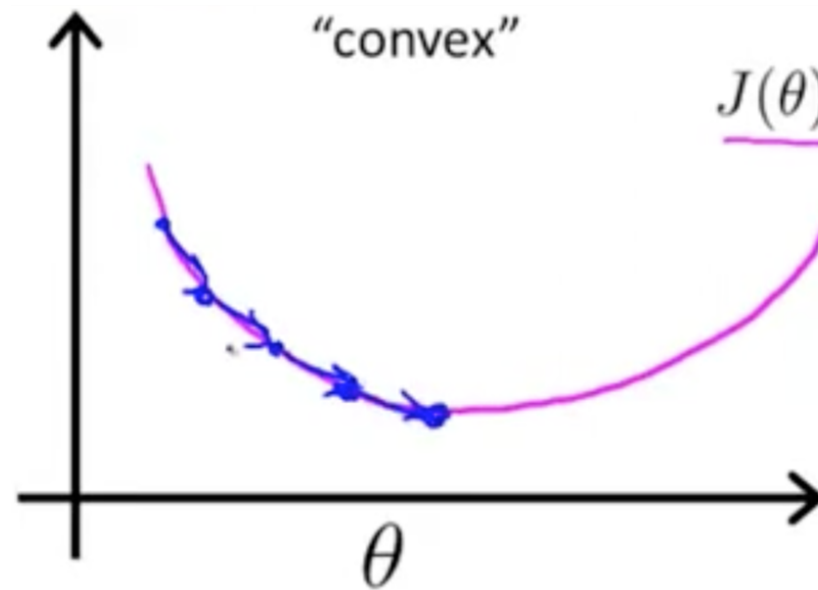
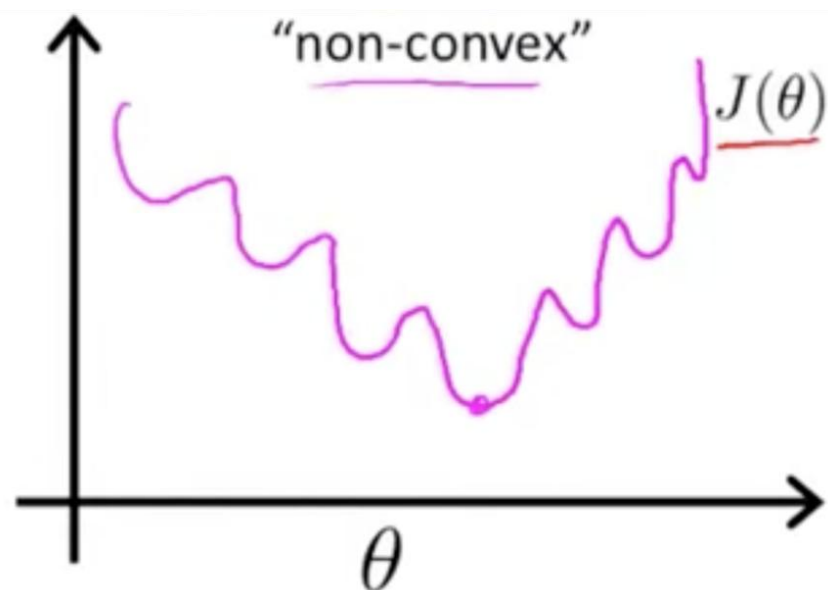
$$Cost(h_{\theta}(x), y) = \frac{1}{2} (h_{\theta}(x) - y)^2$$

Cost function for fitting a logistic regression model

$$\text{Cost}\left(h_{\theta}(x), y\right) = \frac{1}{2} \left(h_{\theta}(x) - y\right)^2$$

In logistic regression, if we use this cost function, it will be a non-convex function of the parameters θ

This is because $h_{\theta}(x)$ is no longer a linear function. It is a complex non-linear function (sigmoid function).



Lots of local minima means gradient descent may not find the global optimum

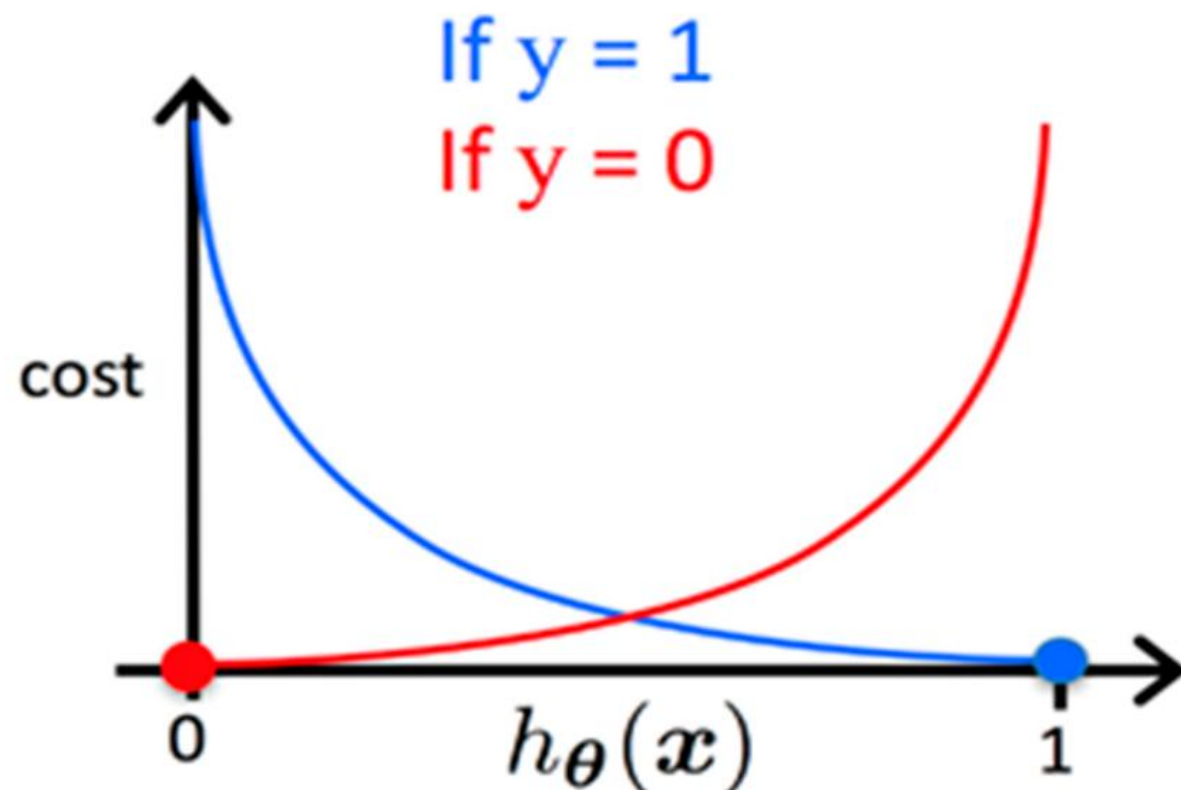
Cost function for fitting a logistic regression model

Thus a different cost function is required for logistic regression

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

This captures the intuition that if $h_{\theta}(x) = 0$ (predict $P(y=1|x; \theta) = 0$) but $y = 1$ this will penalize the learning algorithm with a massive cost

Following the previous example:
If the patient is told that the tumor is not malignant with full certainty ($h_{\theta}(x) = 0$), but the tumor was malignant in reality ($y=1$), then the cost, or the penalty is very high (∞)



The cost function needs to be simplified to apply gradient descent to find the optimal parameters

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

It can be expressed as:

$$\text{Cost}(h_{\theta}(x), y) = -y \log(h_{\theta}(x)) - (1 - y) \log(1 - h_{\theta}(x))$$

Since y takes only 2 values $\{0, 1\}$ substituting them shows that the simplified cost function is equivalent to the previous one

$$J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x), y)$$

$$J(\theta) = -\frac{1}{m} \left[\sum_{i=1}^m y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log (1 - h_{\theta}(x^{(i)})) \right]$$

To fit/optimize parameters (θ): Minimize $J(\theta)$ wrt θ

To make a prediction given a new x : Output $h_{\theta}(x)$ with the optimized parameters θ

$$h_{\theta}(x) = \frac{1}{1 + e^{-\theta^T x}}$$

Minimizing Cost Function using Gradient Descent

To minimize $J(\theta)$

- Use gradient descent
- Repeatedly update each parameter using a learning rate

$$\text{Repeat } \{ \theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta) \}$$

Repeat {

$$\theta_j := \theta_j - \alpha \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

} (simultaneously update all θ_j)

Minimizing Cost Function using Gradient Descent

- This equation is the same as the linear regression rule
 - The only difference is that the definition for the hypothesis has changed
- When implementing logistic regression with gradient descent, we have to update all the θ values (θ_0 to θ_n) simultaneously
 - Could use a for loop
 - Better would be a vectorized implementation
- Feature scaling for logistic regression helps run the gradient descent faster

Multi-class classification

Email foldering and tagging: work / friends / family / hobby

Medical report: not- ill / flu / cold

Weather: sunny / cloudy / rain / snow

$y \in \{ 1, 2, 3, \dots \}$

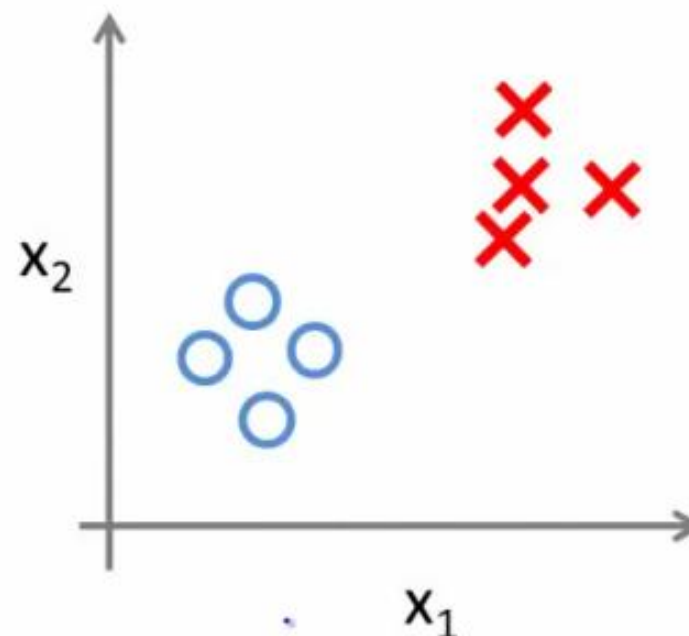
1: "class 1"

2: "class 2"

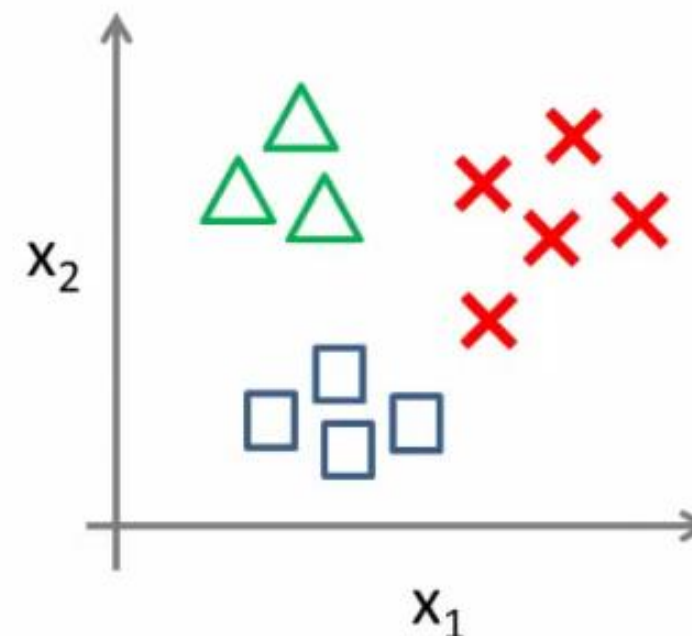
3: "class 3"

... so on.

Binary classification:

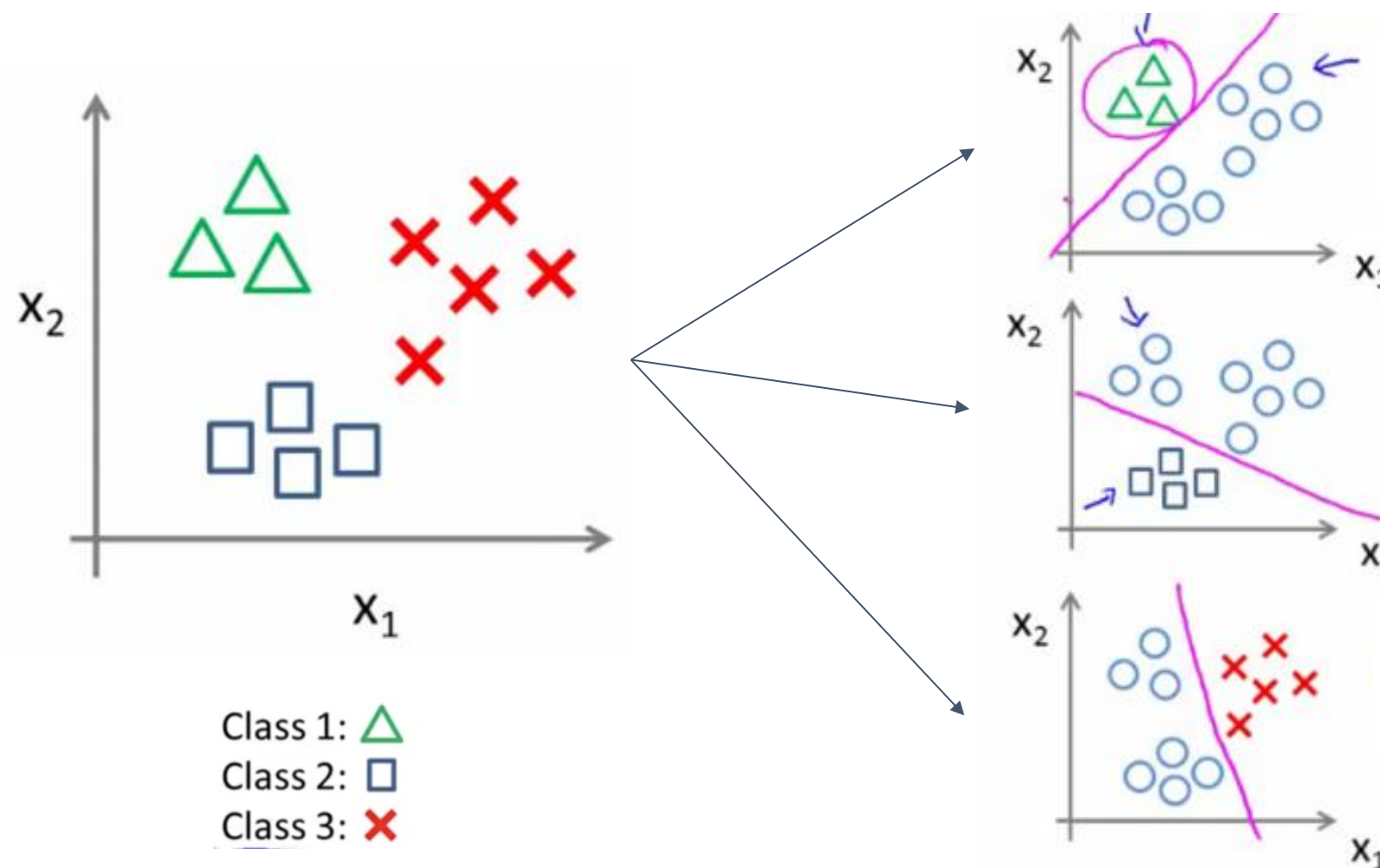


Multi-class classification:



Multi class classification: One-vs-All

- Train a logistic regression classifier $h_{\theta}^{(i)}(x)$ for each class i to predict the probability that $y = i$
- On a new input, x to make a prediction, pick the class i that maximizes the probability that $h_{\theta}^{(i)}(x) = 1$



- $h_{\theta}^1(x) = 1: P(y=1 \mid x_1; \theta)$

- $h_{\theta}^2(x) = 1: P(y=1 \mid x_2; \theta)$

- $h_{\theta}^3(x) = 1: P(y=1 \mid x_3; \theta)$

Suppose that you have trained a logistic regression classifier, and it outputs on a new example x a prediction $h_{\theta}(x) = 0.2$. This means (check all that apply):

- a. Our estimate for $P(y = 1|x;\theta)$ is 0.8
- b. Our estimate for $P(y = 0|x;\theta)$ is 0.8
- c. Our estimate for $P(y = 1|x;\theta)$ is 0.2
- d. Our estimate for $P(y = 0|x;\theta)$ is 0.2

Suppose that you have trained a logistic regression classifier, and it outputs on a new example x a prediction $h_{\theta}(x) = 0.2$. This means (check all that apply):

- a. Our estimate for $P(y = 1|x;\theta)$ is 0.8
- b. Our estimate for $P(y = 0|x;\theta)$ is 0.8**
- c. Our estimate for $P(y = 1|x;\theta)$ is 0.2**
- d. Our estimate for $P(y = 0|x;\theta)$ is 0.2

Which of the following are true (check all that apply):

- a. $J(\theta)$ will be a convex function, so gradient descent should converge to a global minimum
- b. Adding polynomial features (e.g., instead using $h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_2 + \theta_4 x_1 x_2 + \theta_5 x_2)$) could increase how well we can fit the training data
- c. The positive and negative examples cannot be separated using a straight line. So, gradient descent will fail to converge.
- d. Because the positive and negative examples cannot be separated using a straight line, linear regression will perform as well as logistic regression on this data.

Which of the following are true (check all that apply):

- a. **$J(\theta)$ will be a convex function, so gradient descent should converge to a global minimum**
- b. **Adding polynomial features (e.g., instead using $h_\theta(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_2 + \theta_4 x_1 x_2 + \theta_5 x_2)$) could increase how well we can fit the training data**
- c. The positive and negative examples cannot be separated using a straight line. So, gradient descent will fail to converge.
- d. Because the positive and negative examples cannot be separated using a straight line, linear regression will perform as well as logistic regression on this data.

For logistic regression, the gradient is given by

$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$. Which of these is a correct gradient descent update for logistic regression with a learning rate of α ? Check all that apply.

a. $\theta := \theta - \alpha \frac{1}{m} \sum_{i=1}^m (\theta^T x - y^{(i)}) x^{(i)}.$

b. $\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (\theta^T x - y^{(i)}) x_j^{(i)}$ (simultaneously update for all j).

c. $\theta := \theta - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}.$

d. $\theta := \theta - \alpha \frac{1}{m} \sum_{i=1}^m \left(\frac{1}{1+e^{-\theta^T x^{(i)}}} - y^{(i)} \right) x^{(i)}.$

For logistic regression, the gradient is given by $\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$. Which of these is a correct gradient descent update for logistic regression with a learning rate of α ? Check all that apply.

☐ $\theta := \theta - \alpha \frac{1}{m} \sum_{i=1}^m (\theta^T x - y^{(i)}) x^{(i)}.$

☐ $\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (\theta^T x - y^{(i)}) x_j^{(i)}$ (simultaneously update for all j).

☒ $\theta := \theta - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x^{(i)}.$

☒ $\theta := \theta - \alpha \frac{1}{m} \sum_{i=1}^m \left(\frac{1}{1+e^{-\theta^T x^{(i)}}} - y^{(i)} \right) x^{(i)}.$

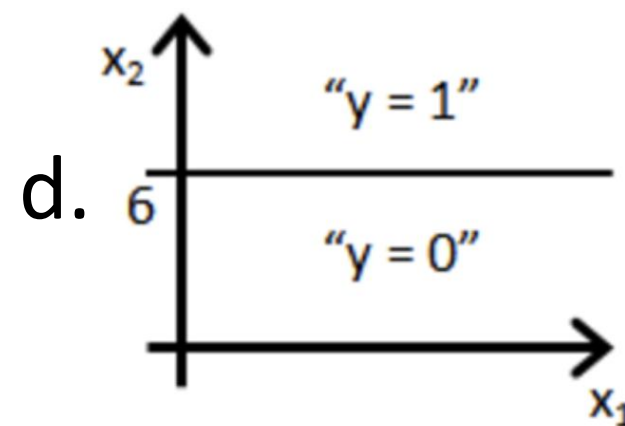
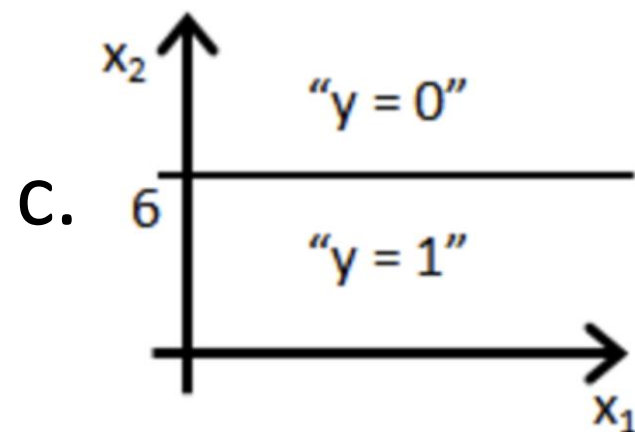
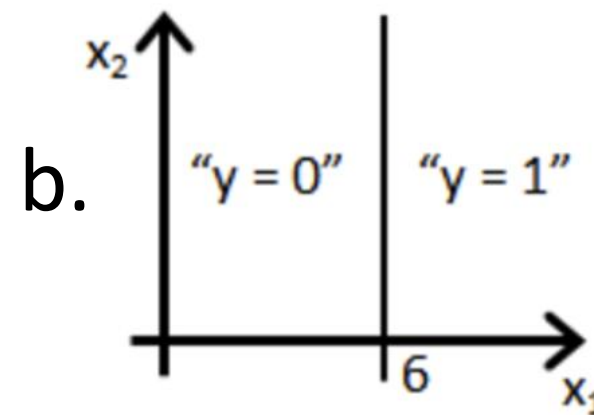
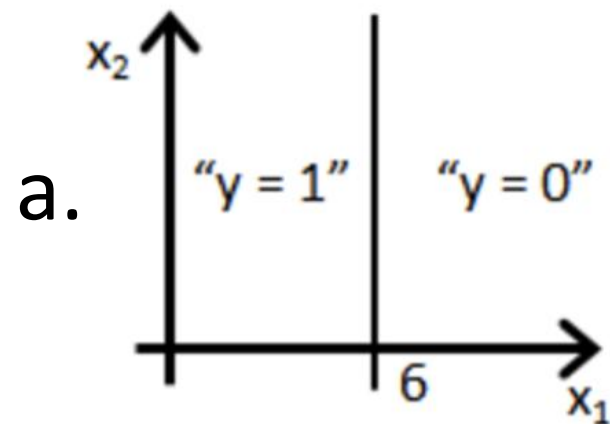
Which of the following statements are true (check all that apply):

- a. The cost function $J(\theta)$ for logistic regression trained with $m \geq 1$ examples is always greater than or equal to zero.
- b. Linear regression always works well for classification if you classify by using a threshold on the prediction made by linear regression.
- c. The sigmoid function is never greater than one
- d. For logistic regression, sometimes gradient descent will converge to a local minimum (and fail to find a global minimum). (This is the reason we prefer more advanced optimisation algorithms like conjugate gradient/BFGS/L-BFGS etc)

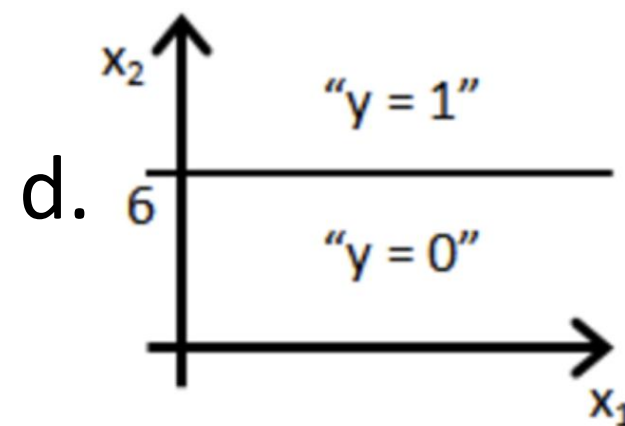
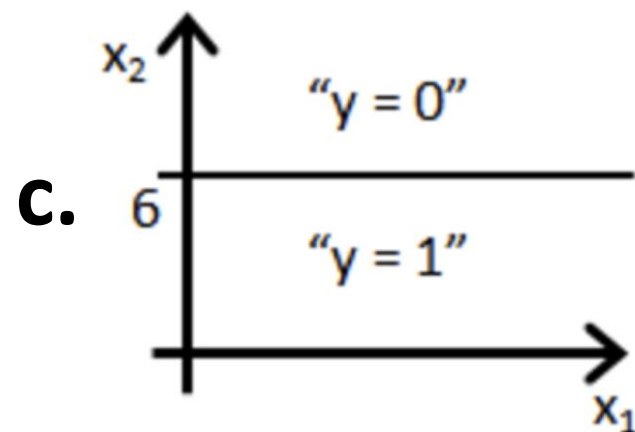
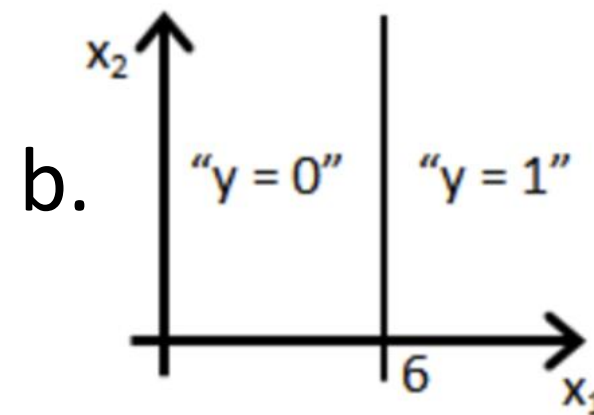
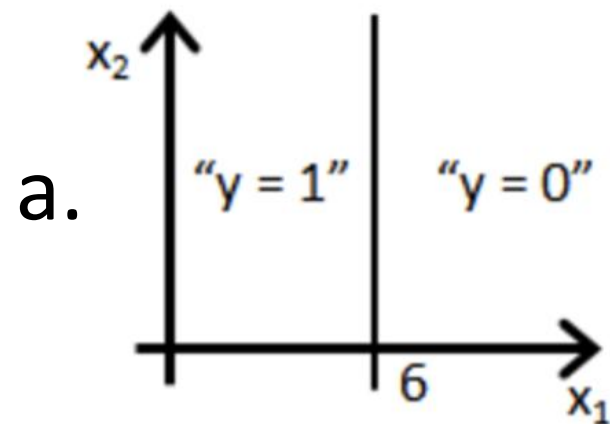
Which of the following statements are true (check all that apply):

- a. The cost function $J(\theta)$ for logistic regression trained with $m \geq 1$ examples is always greater than or equal to zero.**
- b. Linear regression always works well for classification if you classify by using a threshold on the prediction made by linear regression.
- c. The sigmoid function is never greater than one**
- d. For logistic regression, sometimes gradient descent will converge to a local minimum (and fail to find a global minimum). (This is the reason we prefer more advanced optimisation algorithms like conjugate gradient/BFGS/L-BFGS etc)

Suppose you train a logistic classifier $h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$. Suppose $\theta_0 = 6, \theta_1 = 0, \theta_2 = -1$. Which of the following figures represents the decision boundary found by your classifier?



Suppose you train a logistic classifier $h_{\theta}(x) = g(\theta_0 + \theta_1 x_1 + \theta_2 x_2)$. Suppose $\theta_0 = 6, \theta_1 = 0, \theta_2 = -1$. Which of the following figures represents the decision boundary found by your classifier?



Machine Learning

QUIZ Links



<https://www.sanfoundry.com/artificial-intelligence-ai-questions-answers-online-quiz/#:~:text=Explanation%3A%20In%20automatic%20vehicle%20set,an%20example%20of%20supervised%20learning.>

(<https://thecodingshef.com/all-unit-mcq-questions-of-machine-learning/>)

https://github.com/mGalarnyk/datasciencecoursera/blob/master/Stanford_Machine_Learning/Week3/week3quiz1.md



UE24CS352A: Machine Learning

Dr. Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu



UE24CS352A: **Machine Learning**

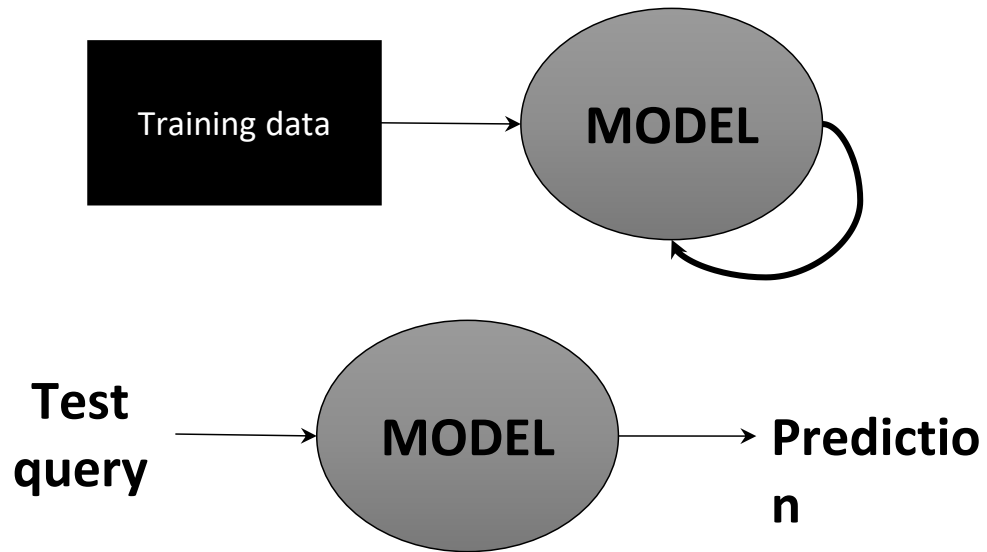
Dr. Shylaja S S

Professor

Department of Computer Science & Engineering

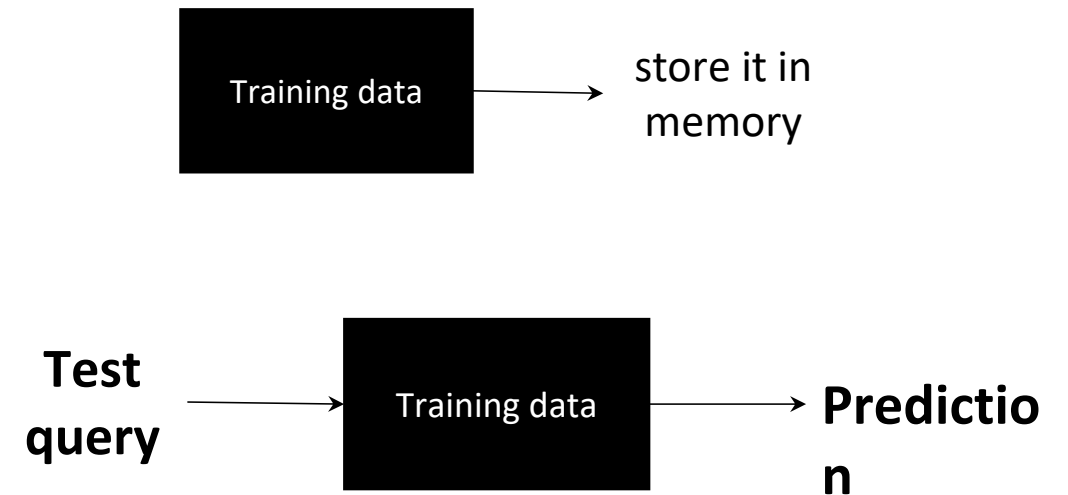
Acknowledgement : Dr. Rajanikanth K (Academic Advisor, PESU)
Teaching Assistant (Arya Rajiv Chaloli)

Eager Learning



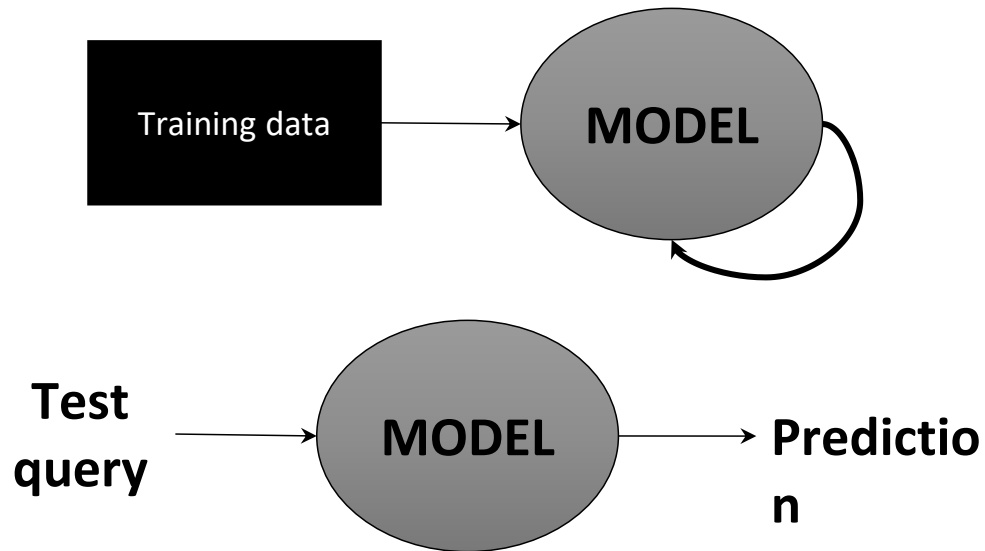
- Creates a model using the training data which is used across all test queries

Lazy Learning



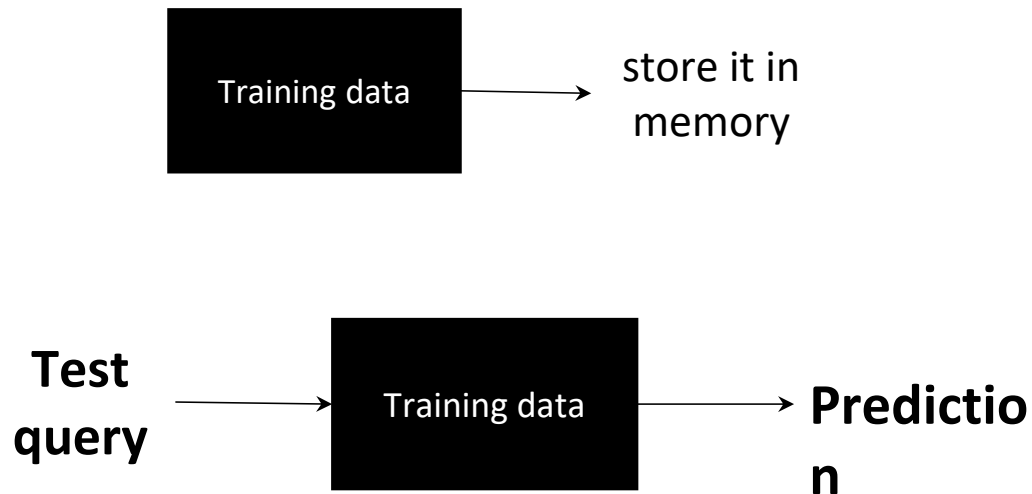
- For every test query, it processes the training data to make a prediction

Eager Learning



- Training data is used to create global model
- A generalized model is created based on training set
- On test query the global model is used to make the prediction
- Training data is not directly used to make prediction , instead the generated model is used
- Example: Decision Tree

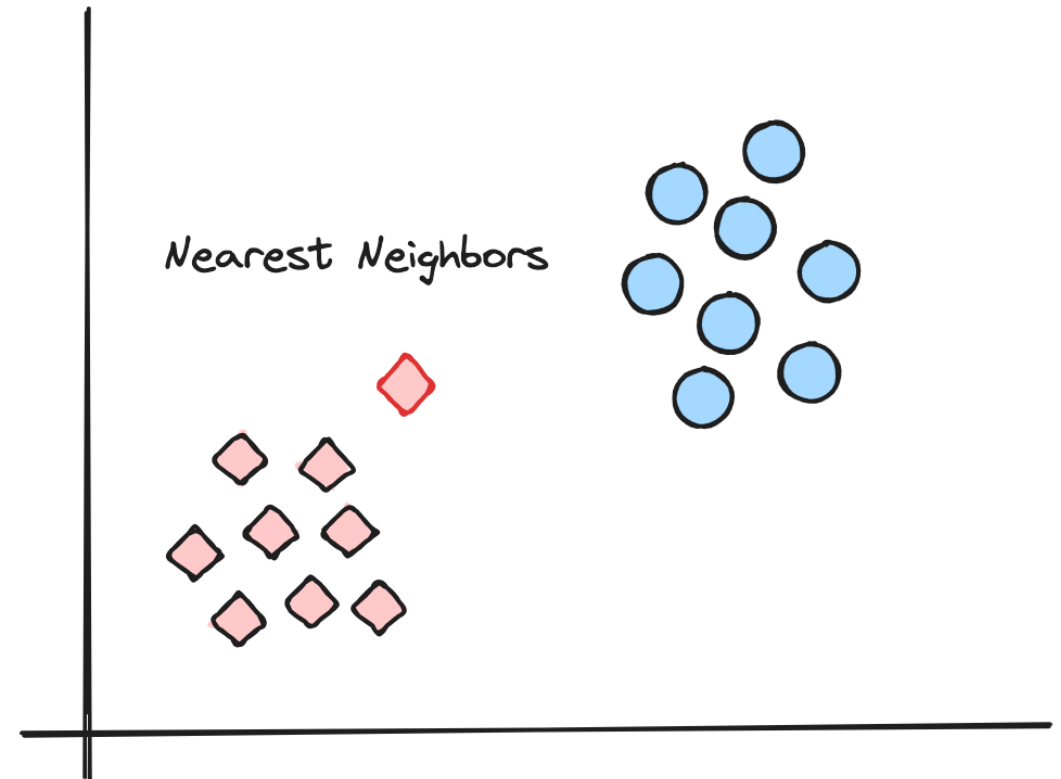
Lazy Learning



- No global model is created
- For every test instance a common procedure is run on training data to find the prediction value.
- Prediction uses training data directly
- Example: **Instance Based Learning** algorithms like KNN

- Instance-based learning (also called memory-based learning) is a family of learning algorithms that compare new problem instances with instances seen in training, which have been stored in memory.
- Because computation is postponed until a new instance is observed, these algorithms are sometimes referred to as "lazy."
- Examples of instance-based learning algorithms are the **k-nearest neighbors algorithm (our focus)**, kernel machines and RBF networks

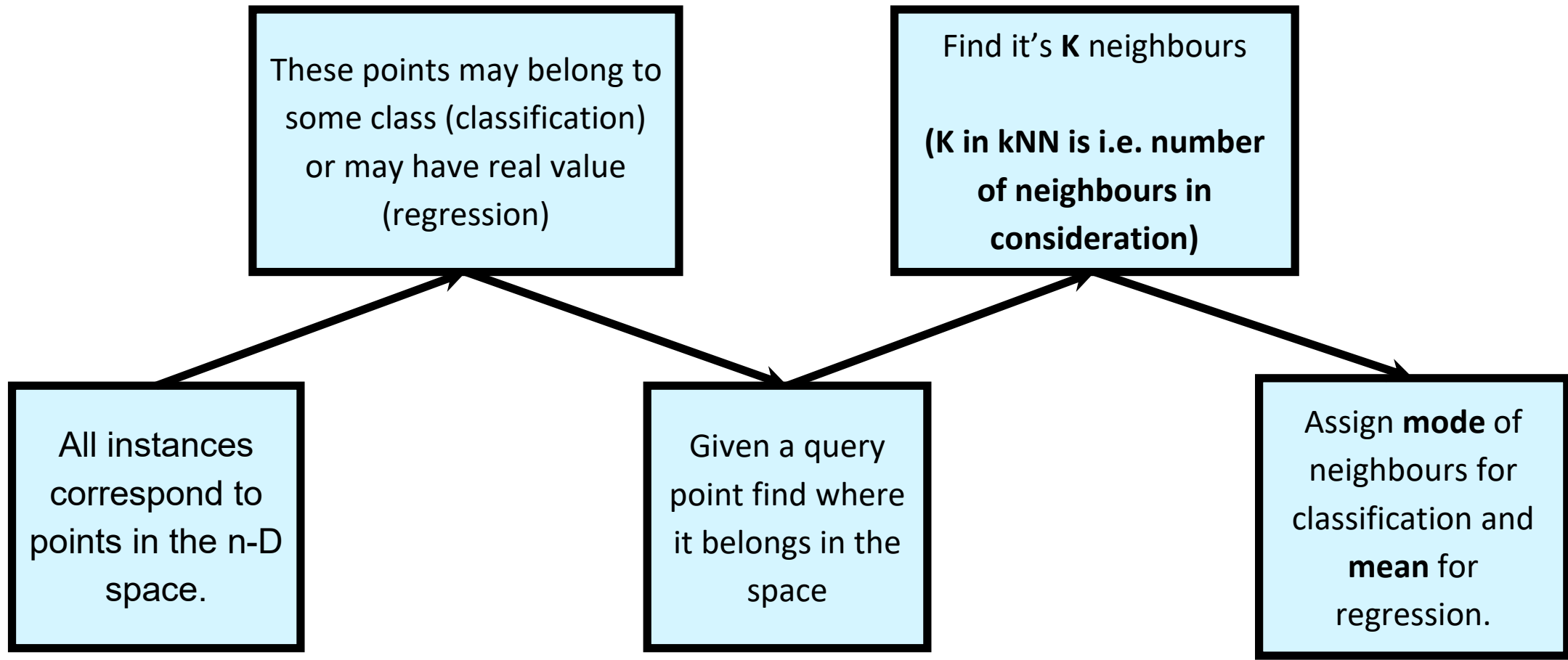
- A lazy learning algorithm
- Classify based on its neighbour
- The algorithm can be used for both classification and regression
- Works on the idea that similar people/things/subjects tend to stay together



The inductive bias of a learning algorithm is the set of assumptions that the learner uses to predict outputs of given inputs that it has not encountered.

Inductive Bias in kNN: The classification of an instance x will be most similar to the classification of the K other instances that are nearby.



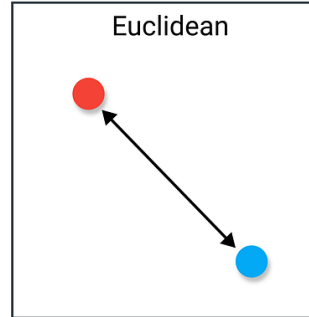


How to find neighbour? **Using distance measures**

Euclidean

$$\sqrt{\sum_{i=1}^d (x_i - y_i)^2}$$

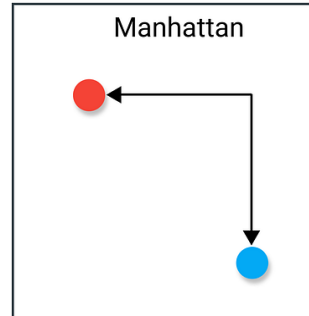
Euclidean



Manhattan

$$\sum_{i=1}^d |x_i - y_i|$$

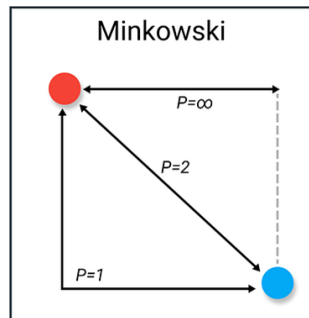
Manhattan



Minkowski

$$\left(\sum_{i=1}^d (|x_i - y_i|)^q \right)^{1/q}$$

Minkowski



where,

- d : number of attributes
- x_i and y_i : i^{th} attributes

We can manipulate the values of q and calculate different distances:

- $q = 1 \rightarrow$ Manhattan distance
- $q = 2 \rightarrow$ Euclidean distance

- In classification, the target function is discrete valued (m classes).
- Each of the data instances may belong to one of the m classes

We define Set V as the set of all classes, $V = \{v_1, v_2, v_3, \dots, v_m\}$.

- In d -dimensional space (R^d) where d is the number of features,

We can define the target function as $f : R^d \rightarrow V$

- Any data instance x_i will have a label $f(x_i)$ (- one of the class from set V).

KNN Algorithm for Discrete-Valued Target Function

◆ Training algorithm:

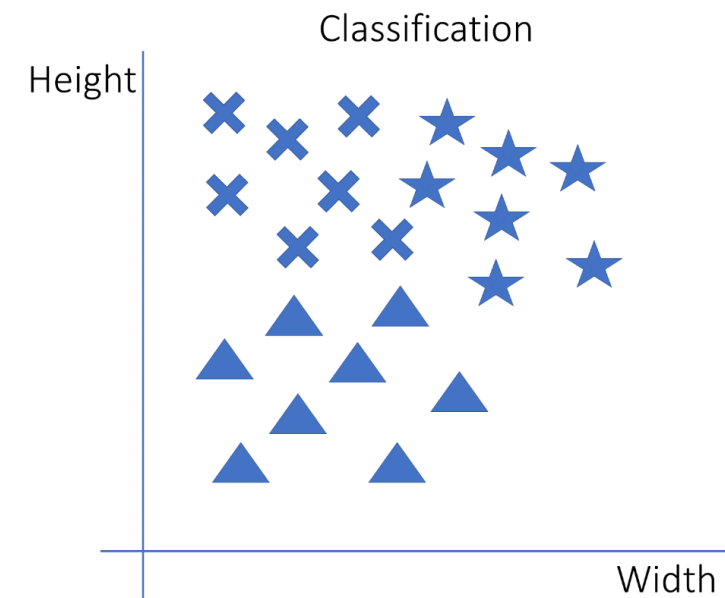
- Store each training example $\langle x, f(x) \rangle$

◆ Classification algorithm:

- Given x_q to be classified
- Find k nearest neighbor $x_1 \dots x_k$ of x_q

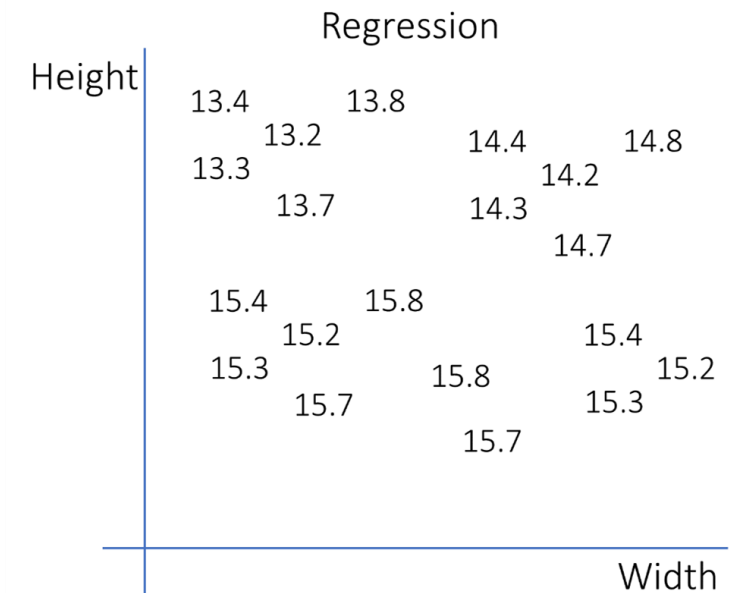
- $$\hat{f}(x_q) \leftarrow \arg \max_{v \in V} \sum_{i=1}^k \delta(v, f(x_i))$$

$$\delta(a, b) = \begin{cases} 1 & a = b \\ 0 & \text{otherwise} \end{cases}$$



- In regression, the target function is real valued (continuous value).
- In d-dimensional space (R^d) where d is the number of features,

We can define the target function as $f : R^d \rightarrow R$



KNN Algorithm for Real-Valued Target Function

- Store each training example $\langle x, f(x) \rangle$
- Find k nearest neighbor $x_1 \dots x_k$ of x_q

$$\hat{f}(x_q) \leftarrow \frac{\sum_{i=1}^k f(x_i)}{k}$$

Mean value of k nearest neighbors is returned as the approximated value of the query instance x_q .

Practice Problem - 1

Practice problem for K-NN Classification

Consider the following data set. Apply kNN algorithm to assess the risk for a patient whose BP is 100, sugar is 135, Haemoglobin is 12 and WBC count is 8 thousand. Take $k=3$ and use euclidean distance measure.

That means the query point is $x_q = (100, 135, 12, 8)$

#	B.P.	Sugar	Haemoglobin	WBC(in thousands)	Risk
A	100	120	12	6	No
B	110	130	14	5	Yes
C	120	110	11	7	Yes
D	100	140	13	7	No
E	115	140	11	6	Yes

#	B.P.	Sugar	Haemoglobin	WBC	Risk
A	100	120	12	6	No
B	110	130	14	5	Yes
C	120	110	11	7	Yes
D	100	140	13	7	No
E	115	140	11	6	Yes

we calculate distance between our query point $x_q = (100, 135, 12, 8)$ and every other point

Query	other points	Euclidean distance
x_q	A	15.13
x_q	B	11.74
x_q	C	32.04
x_q	D	5.19
x_q	E	15.96

#	B.P.	Sugar	Haemoglobin	WBC	Risk
A	100	120	12	6	No
B	110	130	14	5	Yes
C	120	110	11	7	Yes
D	100	140	13	7	No
E	115	140	11	6	Yes

Sort these distances and pick the 3 nearest neighbors(as $k = 3$)

Query	other points	Euclidean distance
x_q	D	5.19
x_q	B	11.74
x_q	A	15.13
x_q	E	15.96
x_q	C	32.04

#	B.P.	Sugar	Haemoglobin	WBC	Risk
A	100	120	12	6	No
B	110	130	14	5	Yes
C	120	110	11	7	Yes
D	100	140	13	7	No
E	115	140	11	6	Yes

our choice for the label of x_q is
mode of the risks of D, B, A

i.e mode(NO,YES,NO) = NO

Sort these distances and pick the
3 nearest neighbors(as $k = 3$)

Query	other points	Euclidean distance
x_q	D	5.19
x_q	B	11.74
x_q	A	15.13
x_q	E	15.96
x_q	C	32.04

Practice Problem - 2

Find the weight of ID 11. The value of k is 3. Use Euclidean distance.

ID	Height	Age	Weight
1	5	45	77
2	5.11	26	47
3	5.6	30	55
4	5.9	34	59
5	4.8	40	72
6	5.8	36	60
7	5.3	19	40
8	5.8	28	60
9	5.5	23	45
10	5.6	32	58
11	5.5	38	?

Practice problem for K-NN Regression

We find distance between the point id=11 and every other point

ID	Height	Age	Weight
1	5	45	77
2	5.11	26	47
3	5.6	30	55
4	5.9	34	59
5	4.8	40	72
6	5.8	36	60
7	5.3	19	40
8	5.8	28	60
9	5.5	23	45
10	5.6	32	58
11	5.5	38	?

Query pt	Data points	Distance
id 11	id 1	7.017
id 11	id 2	12.006
id 11	id 3	8.006
id 11	id 4	4.019
id 11	id 5	2.118
id 11	id 6	2.022
id 11	id 7	19.001
id 11	id 8	10.00
id 11	id 9	15
id 11	id 10	6.00

Sort the distances and choose 3 nearest neighbours

ID	Height	Age	Weight
1	5	45	77
2	5.11	26	47
3	5.6	30	55
4	5.9	34	59
5	4.8	40	72
6	5.8	36	60
7	5.3	19	40
8	5.8	28	60
9	5.5	23	45
10	5.6	32	58
11	5.5	38	?

Query pt	Data points	Distance
id 11	id 6	2.022
id 11	id 5	2.118
id 11	id 4	4.019
id 11	id 10	6.00
id 11	id 1	7.017
id 11	id 3	8.006
id 11	id 8	10.00
id 11	id 2	12.006
id 11	id 9	15
id 11	id 7	19.001

ID	Height	Age	Weight
1	5	45	77
2	5.11	26	47
3	5.6	30	55
4	5.9	34	59
5	4.8	40	72
6	5.8	36	60
7	5.3	19	40
8	5.8	28	60
9	5.5	23	45
10	5.6	32	58
11	5.5	38	?

Id	weight
id6	60
id5	72
id4	59

we choose mean of these 3
nearest neighbour as our answer

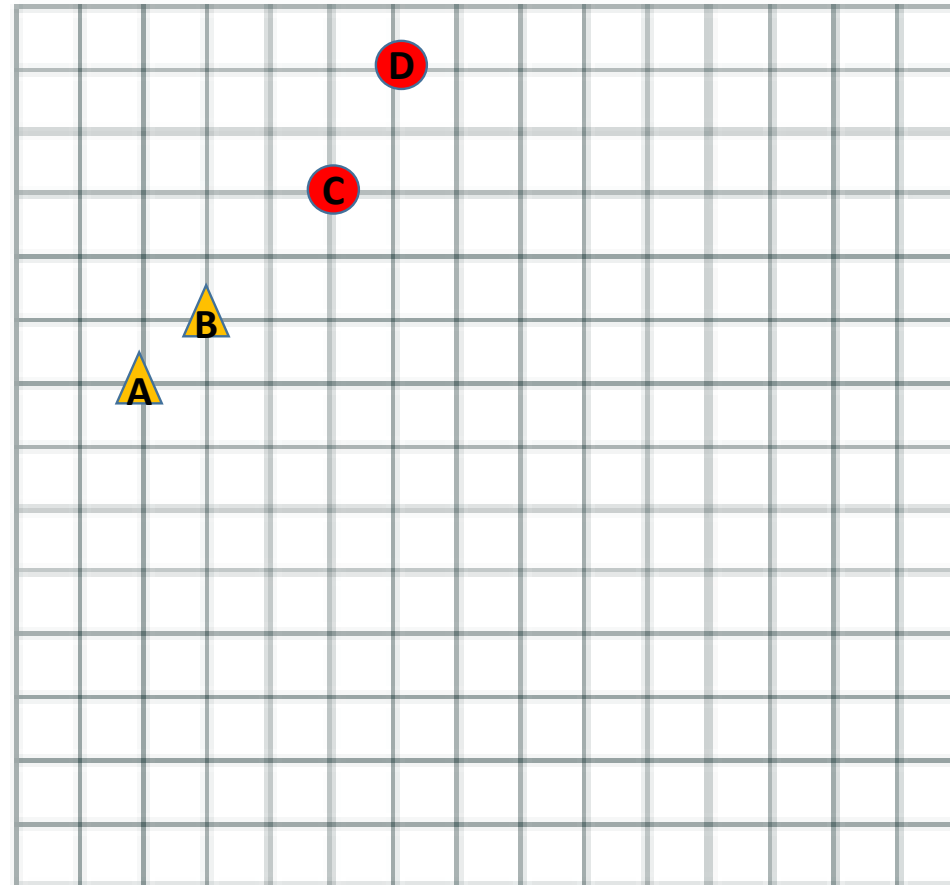
$$\text{answer} = \frac{60+72+59}{3} = 63.666$$

Choices a designer has to make while implementing KNN

- Decide on the features that are relevant for the problem?
---- Meaning of similarity
- What happens if the range of the features differ a lot?
---- Feature scaling techniques required (use normalization)
- How to find out how close/far away other points are?
---- Which distance measure to be used?
- On how many neighbors should I base my result on?
---- What is the appropriate value of **k** to be used for a given problem?

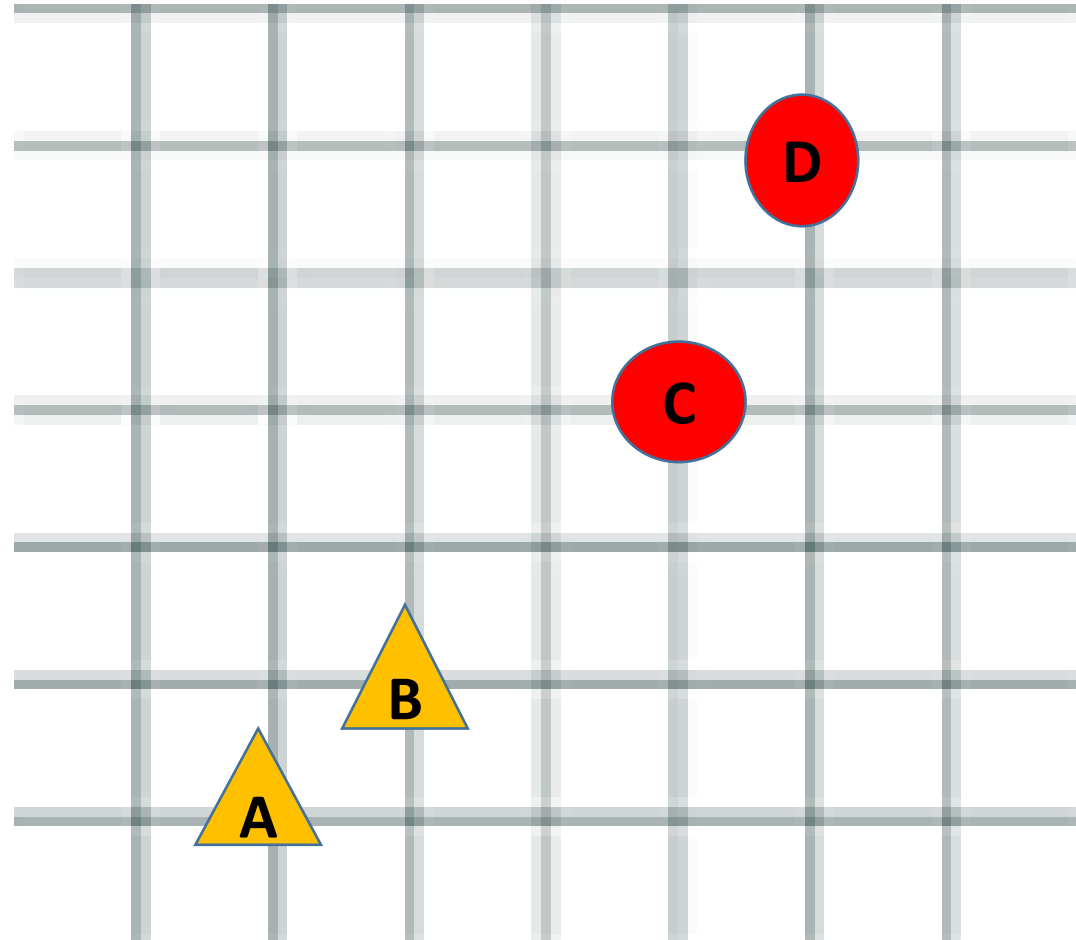
Let us consider the following points

Point	X	Y	CLASS
A	2	8	triangle
B	3	9	triangle
C	5	11	circle
D	6	13	circle



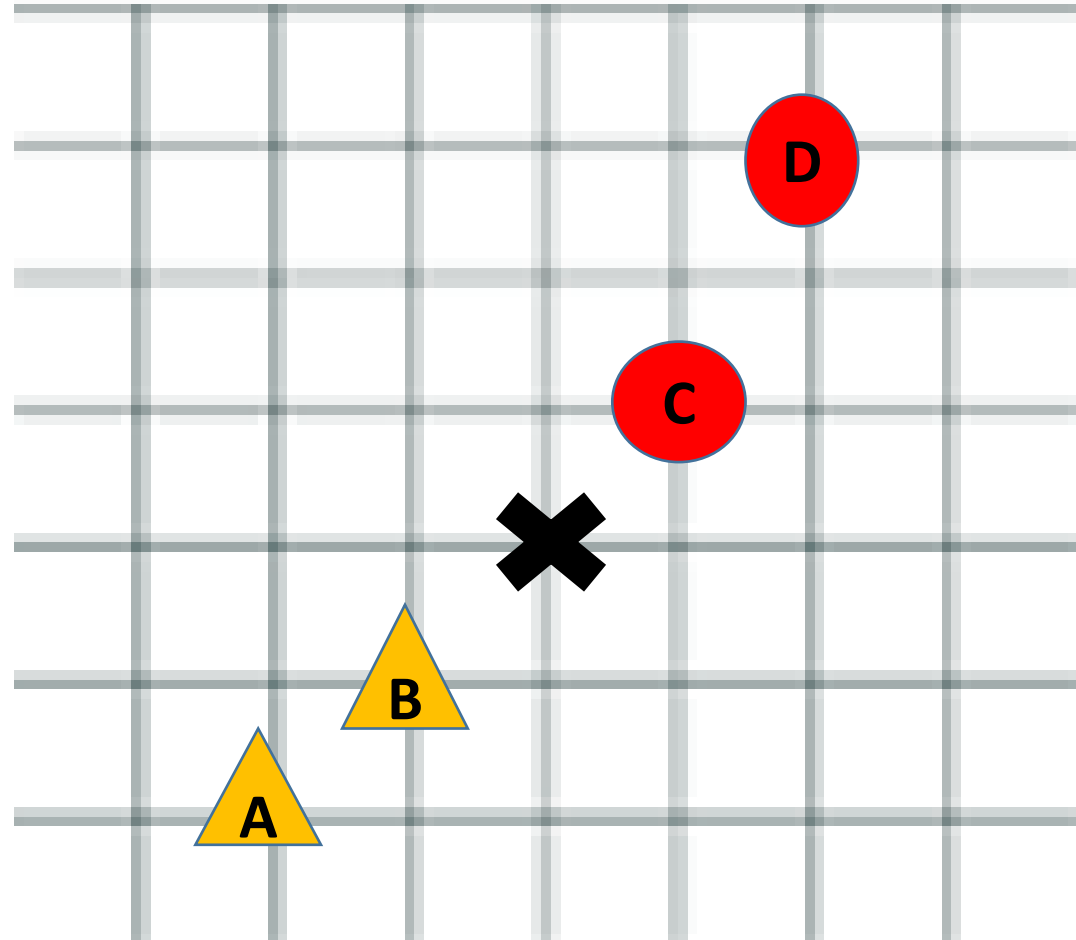
Taking a closer look at the points

Point	X	Y	CLASS
A	2	8	triangle
B	3	9	triangle
C	5	11	circle
D	6	13	circle



Let our query point to be $x_q = (4,10)$

Point	X	Y	CLASS
A	2	8	triangle
B	3	9	triangle
C	5	11	circle
D	6	13	circle



Let us calculate distance of query point x_q to all other points

Point	X	Y	CLASS
A	2	8	triangle
B	3	9	triangle
C	5	11	circle
D	6	13	circle

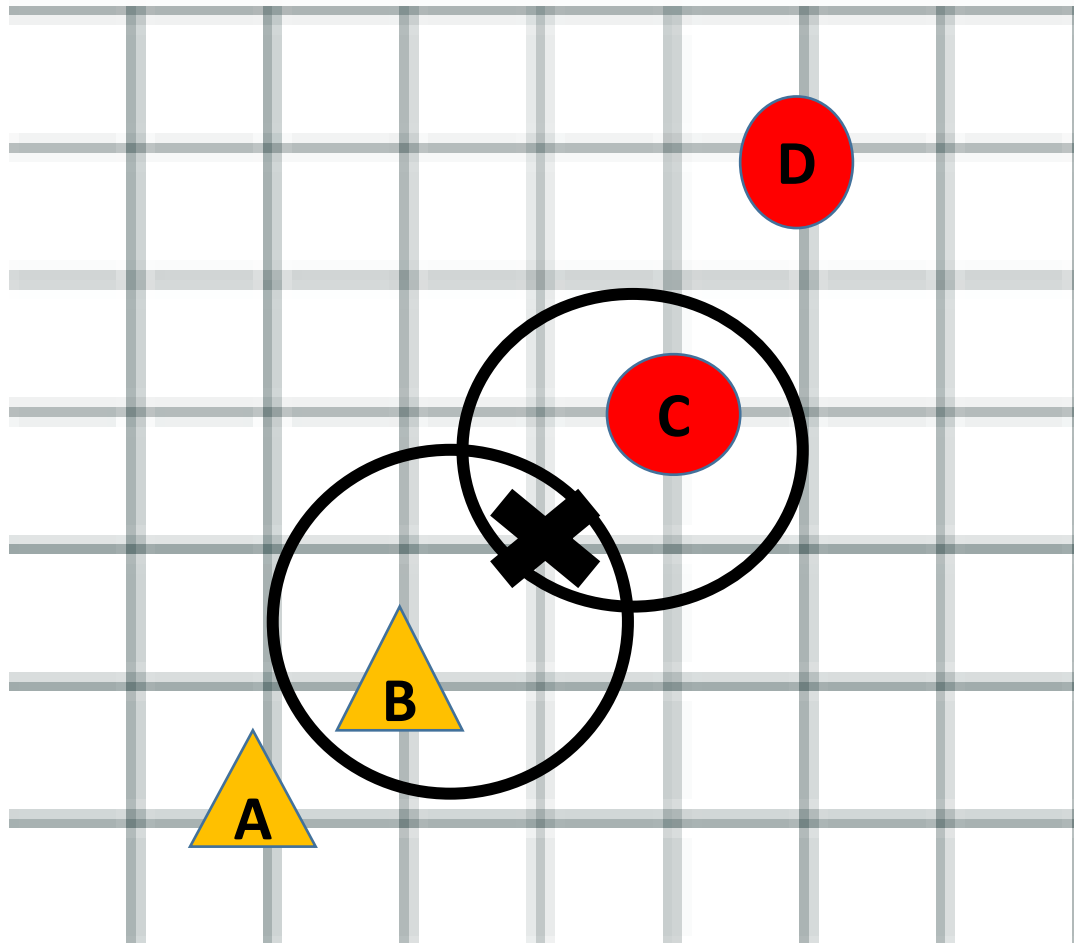
Training data

$$\text{Euclidean distance} : \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$$

Distance calculation

P1	P2	Distance
x_q	A	2.82
x_q	B	1.414
x_q	C	1.414
x_q	D	3.6055

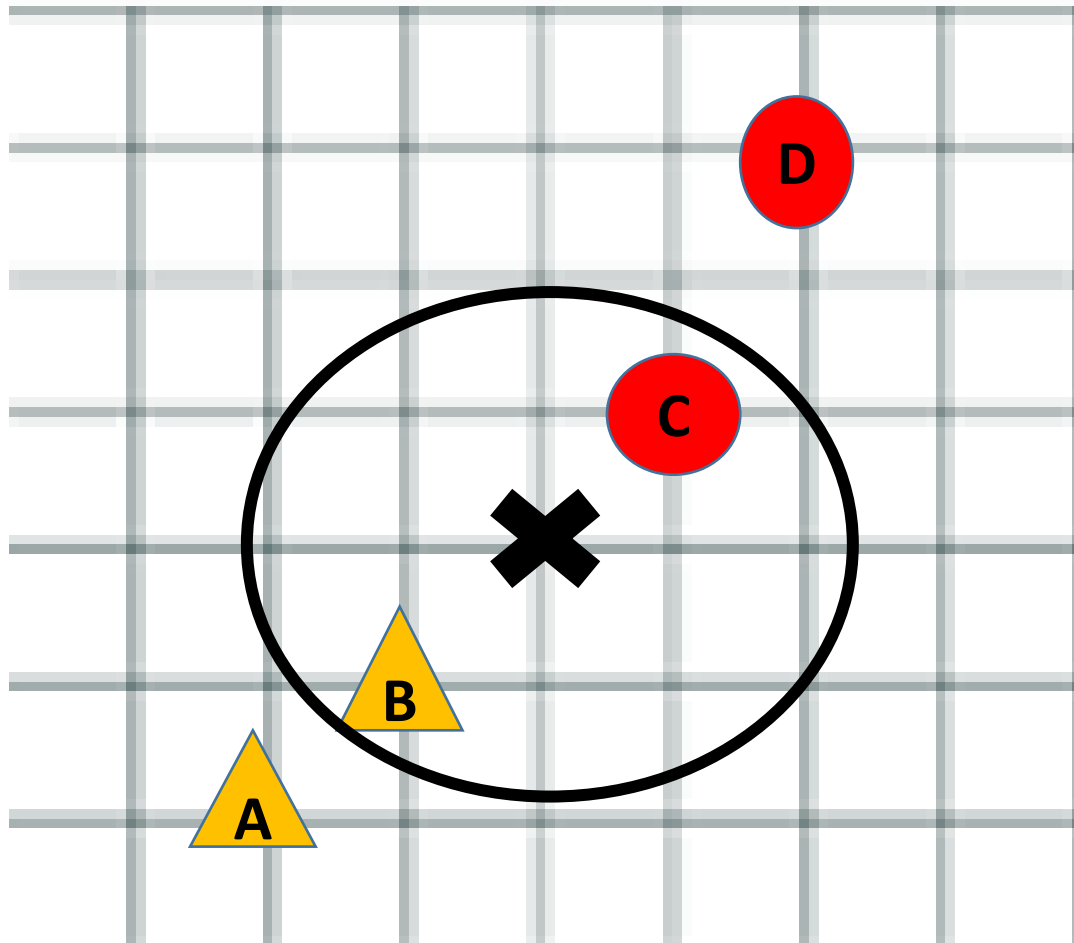
Let us try out different values of k



P1	P2	Distance	CLASS
x_q	A	2.82	triangle
x_q	B	1.414	triangle
x_q	C	1.414	circle
x_q	D	3.6055	circle

- Let's start with $k = 1$
- We have a dilemma in choosing neighbors.

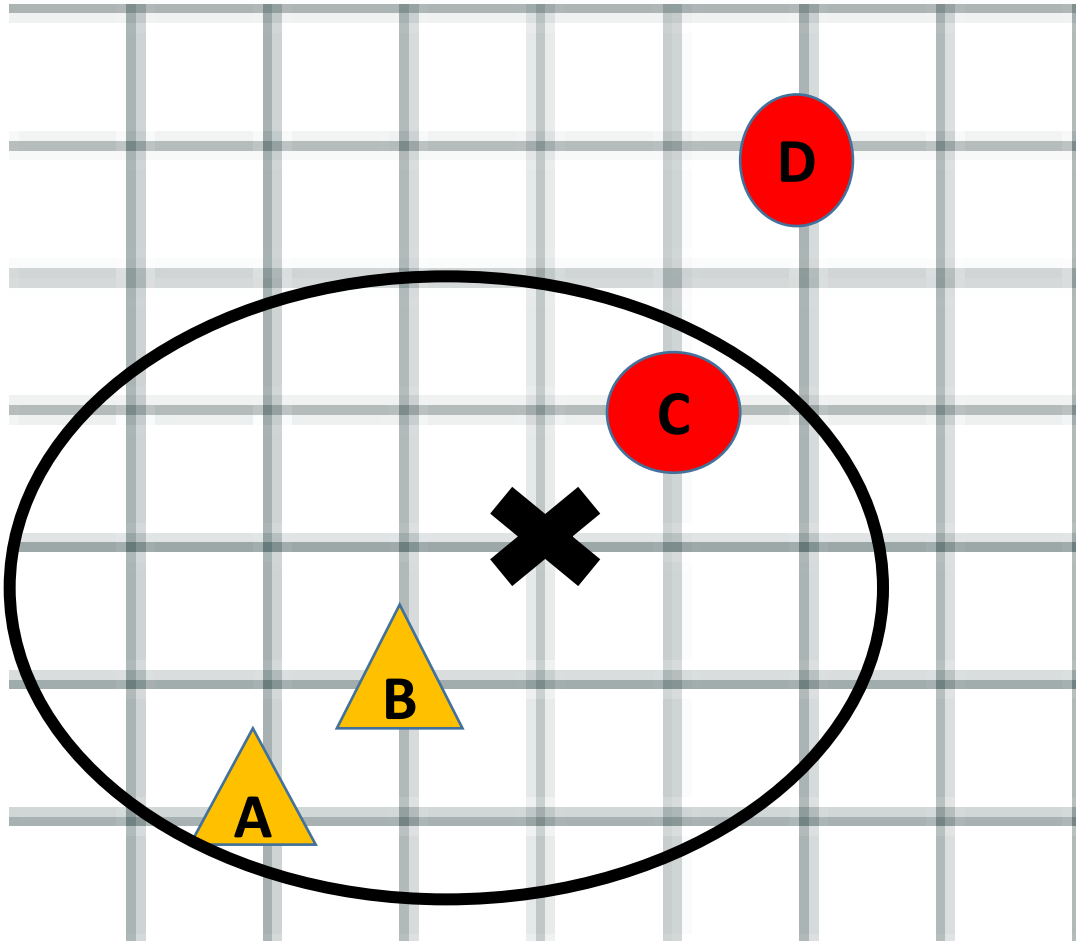
Let us try out different values of k



P1	P2	Distance	CLASS
x_q	A	2.82	triangle
x_q	B	1.414	triangle
x_q	C	1.414	circle
x_q	D	3.6055	circle

- Let's set $k = 2$.
- We have a dilemma while choosing mode.

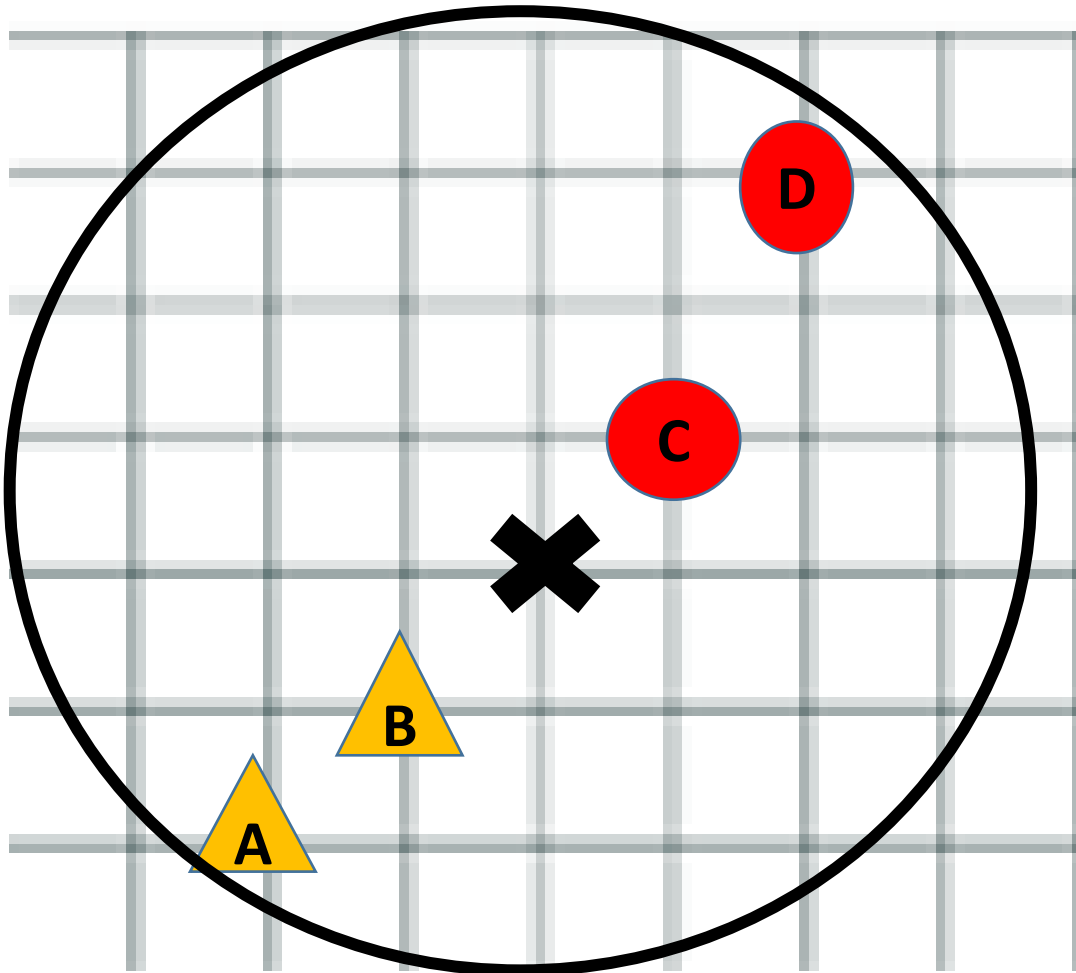
Let us try out different values of k



P1	P2	Distance	CLASS
x_q	A	2.82	triangle
x_q	B	1.414	triangle
x_q	C	1.414	circle
x_q	D	3.6055	circle

- Let's set $k = 3$
- There is no problem and we can assign the query point as a triangle.
- But we won't stop here!

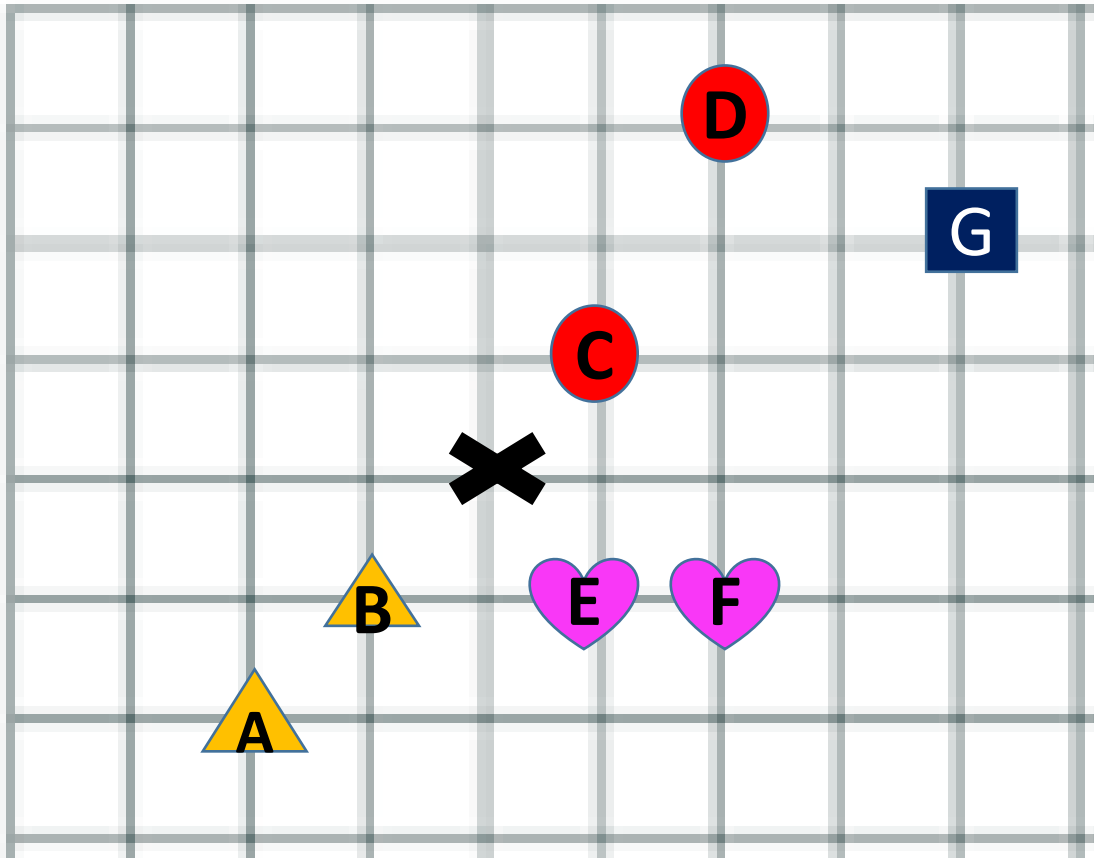
Let us try out different values of k



P1	P2	Distance	CLASS
x_q	A	2.82	triangle
x_q	B	1.414	triangle
x_q	C	1.414	circle
x_q	D	3.6055	circle

- Let's set $k=4$
- There is a problem again
- From our experiments, K should be odd if number of classes is even and even if number of classes is odd.

Assume the same query point, $x_q = (4,10)$



- Let's add two more classes with following data points.

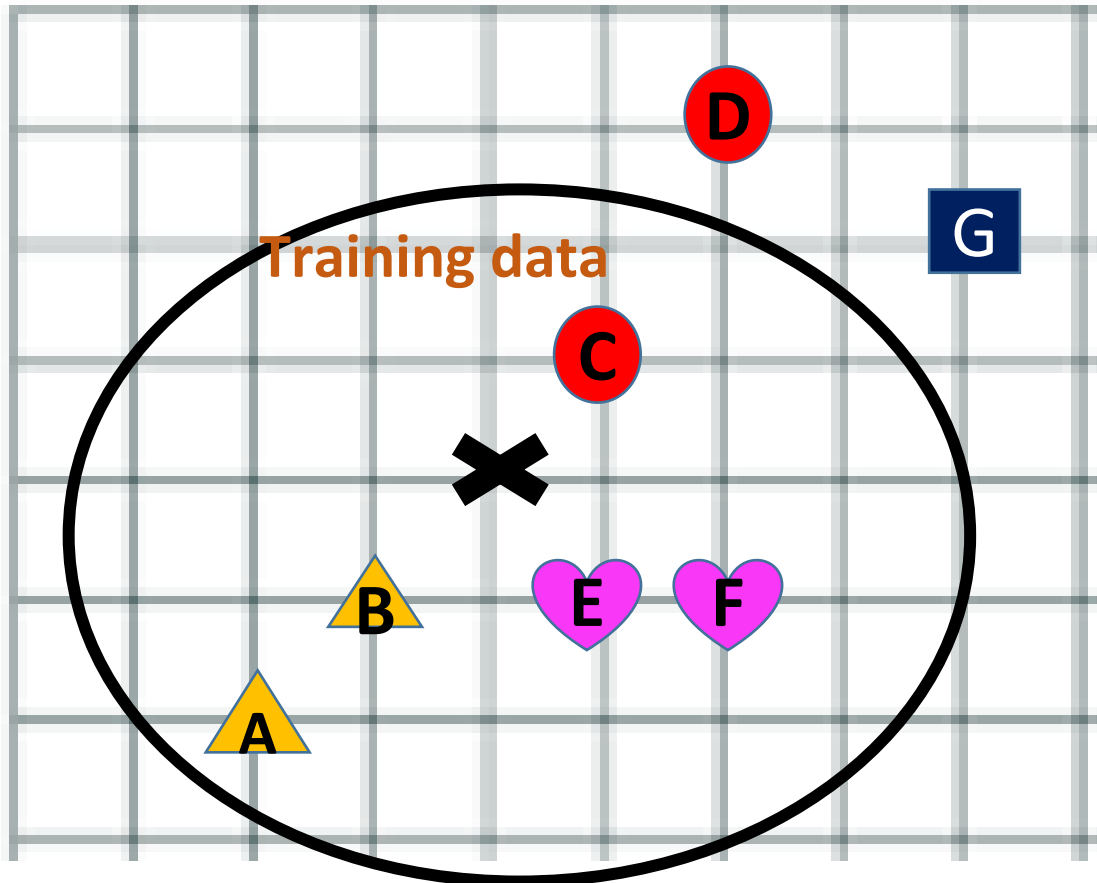
Point	X	Y	CLASS
A	2	8	triangle
B	3	9	triangle
C	5	11	circle
D	6	13	circle
E	5	9	heart
F	6	9	heart
G	8	12	square

Let's calculate the distance between the each point and query point $x_q = (4,10)$

Point	X	Y	CLASS
A	2	8	triangle
B	3	9	triangle
C	5	11	circle
D	6	13	circle
E	5	9	heart
F	6	9	heart
G	8	12	square

P1	P2	Distance
x_q	A	2.82
x_q	B	1.414
x_q	C	1.414
x_q	D	3.6055
x_q	E	1.414
x_q	F	2.23606
x_q	G	4.4721

Let us try out different values of k



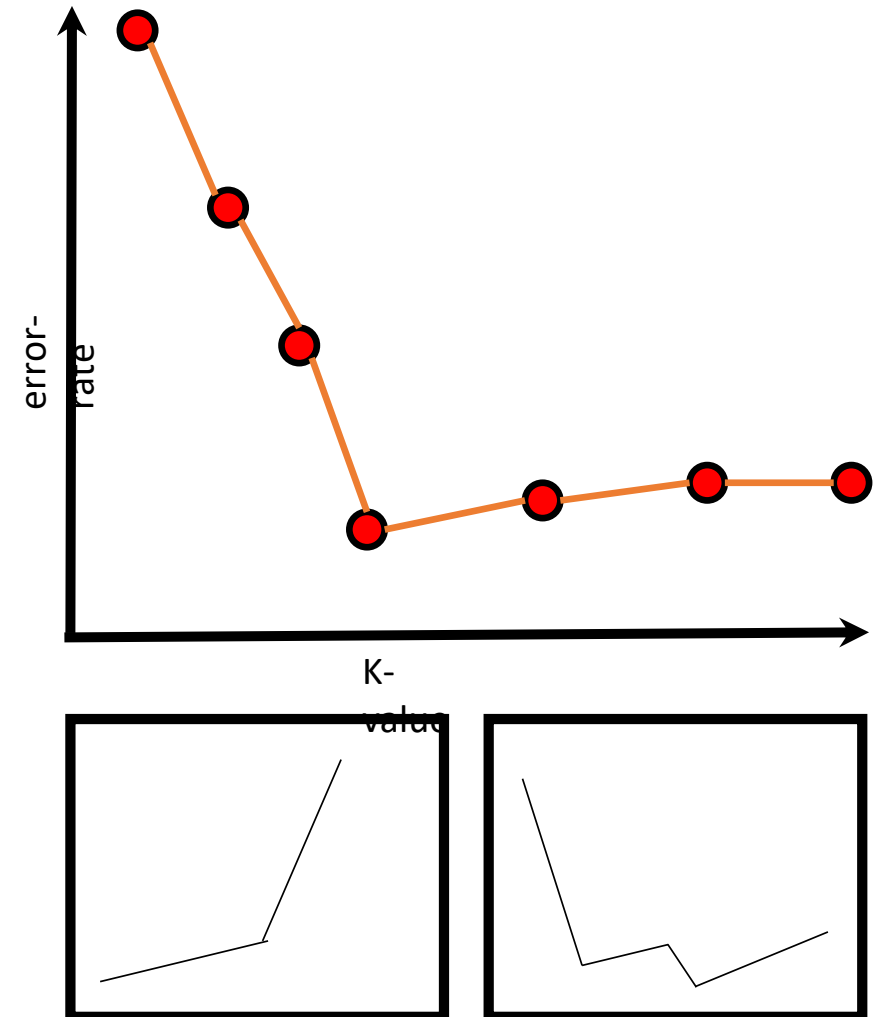
P1	P2	Distance	CLASS
x_q	A	2.82	triangle
x_q	B	1.414	triangle
x_q	C	1.414	circle
x_q	D	3.6055	circle
x_q	E	1.414	heart
x_q	F	2.23606	heart
x_q	G	4.4721	square

- The label has 4 classes, which is even. So from our previous statement, k should be odd.
- So let $k = 5$
- **We are stuck again!**

- In practice there is no right value for k to start with.
- However, you can start with
 $k = \text{number of classes} + 1$
- In case of tie decrease k by 1.
- Experimental results shows that **$k = \text{Sqrt}(n)$** , performs well too.
Where n is the size of dataset.

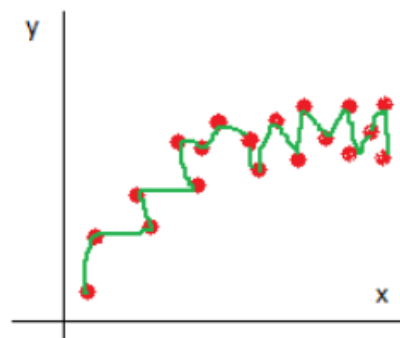
Elbow Method - another way of estimating the value of k

- Calculate the error rate for different values of K.
- Choose the optimal k value as the elbow value of the curve.
- Run the K-NN algorithm with the best K value found and redo the classification report and the confusion matrix.
- Any changes to the data will require you to redo the elbow method and re-define the best K
- Elbow method does not mean your curve should look exactly like an elbow, it can be of any type.

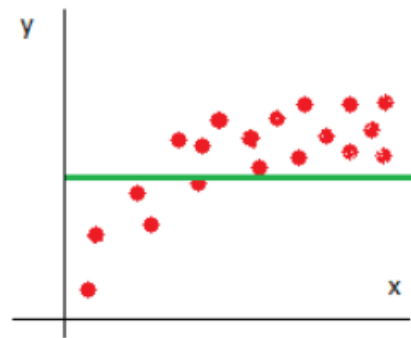


Bias-Variance Trade off wrt how new instances get classified

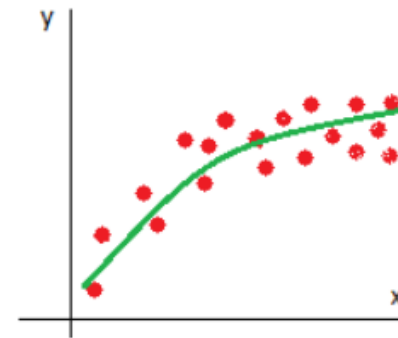
- **K is very low:** Could label the query points as the point near it as only small no. of points are considered. Chance for high variance aka **overfitting**.
- **K is very high:** Could mostly label the query points as the class label which has a majority in the given dataset. Chance for high bias aka **underfitting**.
- K is neither high nor low: Good trade-off without high bias and variance.



Overfitting (High Variance)



Underfitting (High Bias)



Just Right

Practical implementation of KNN

- For most of the practical purposes, the dataset is divided into production(training) and test data set (80:20)
 - some kind of pre-processing (learning) is being done
 - Not used as a naive (plain vanilla) version

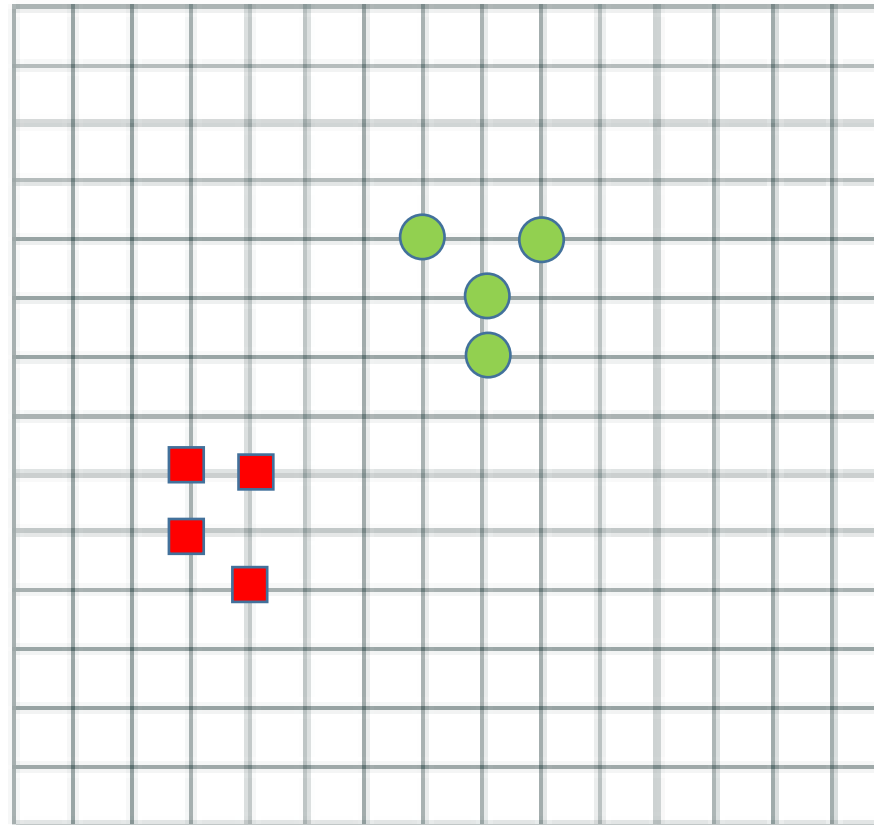
X	Y	Class
3	5	red
3	6	red
4	6	red
4	4	red
7	10	green
8	9	green
8	8	green
9	10	green
5	5	red
4	7	green

Machine Learning

Working algorithm for kNN Classification



- Upload the production data into memory space.
- Decide the distance measure.
(we will choose Euclidean)
- Decide the k value
(we will choose $k = 4$)
- For each data point in test data, find distance with respect to all neighbors and choose the nearest 4 neighbors and assign the prediction as **mode** of the neighbors.
- **Calculate the error and accuracy.**



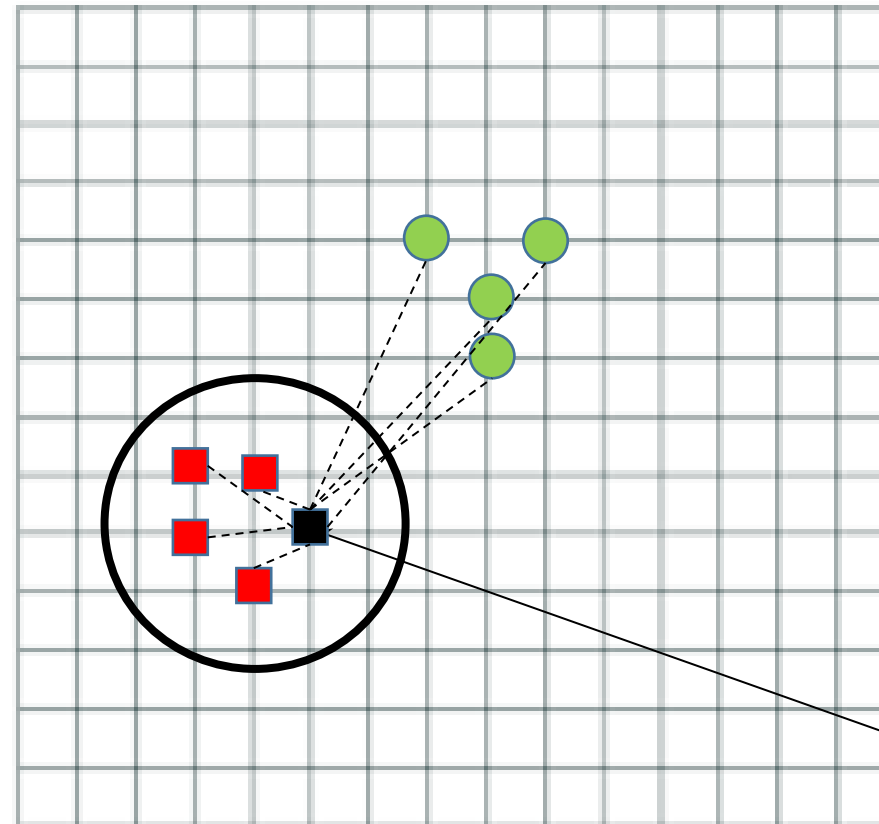
X	Y	Class
3	5	red
3	6	red
4	6	red
4	4	red
7	10	green
8	9	green
8	8	green
9	10	green

X	Y	Class
5	5	red
4	7	green

Machine Learning

Working algorithm for kNN Classification

TP	TN	FP	FN	Error = (FP + FN)/ All predictions
1	0	0	0	0



X	Y	Class
3	5	red
3	6	red
4	6	red
4	4	red
7	10	green
8	9	green
8	8	green
9	10	green

X	Y	Class
5	5	red
4	7	green

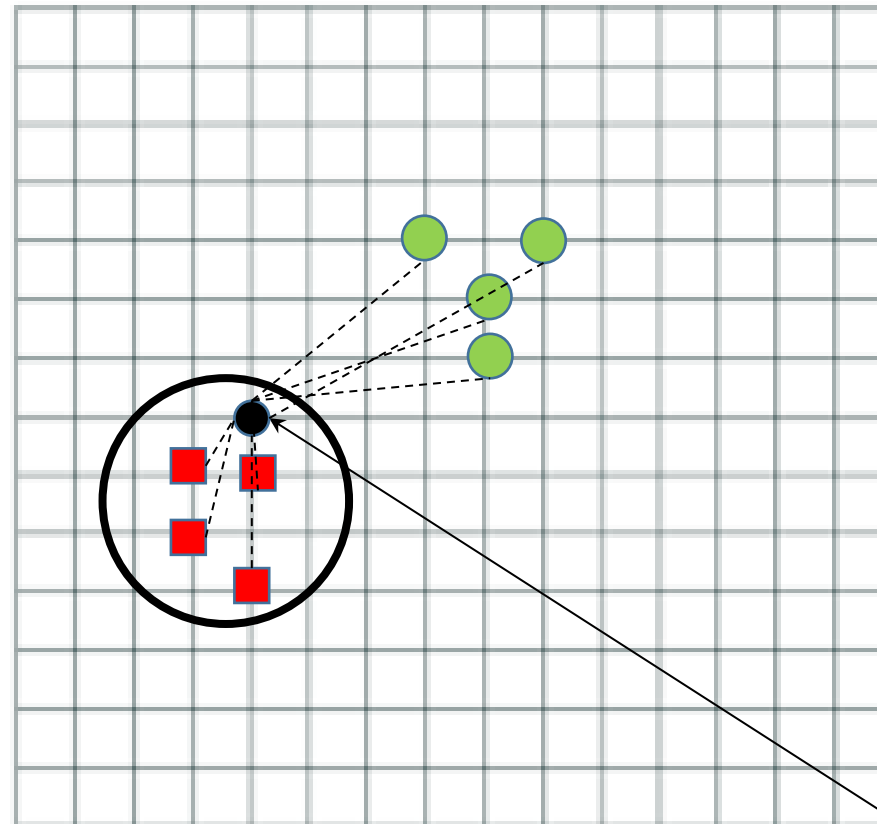
Machine Learning

Working algorithm for kNN Classification



TP	TN	FP	FN	Error = (FP + FN)/ All predictions
1	0	1	0	$\frac{1}{2} = 0.5$

This is a small example real data will be in 1000's



X	Y	Class
3	5	red
3	6	red
4	6	red
4	4	red
7	10	green
8	9	green
8	8	green
9	10	green

X	Y	Class
5	5	red
4	7	green

- **Collect the dataset**
- **Divide into production and test data set (80:20)**

X	Y	value
3	5	3.5
3	6	2.6
4	6	2.8
4	4	3.1
7	10	8.7
8	9	8.5
8	8	9.3
9	10	9.7
5	5	3.5
7	8	2.6

Machine Learning

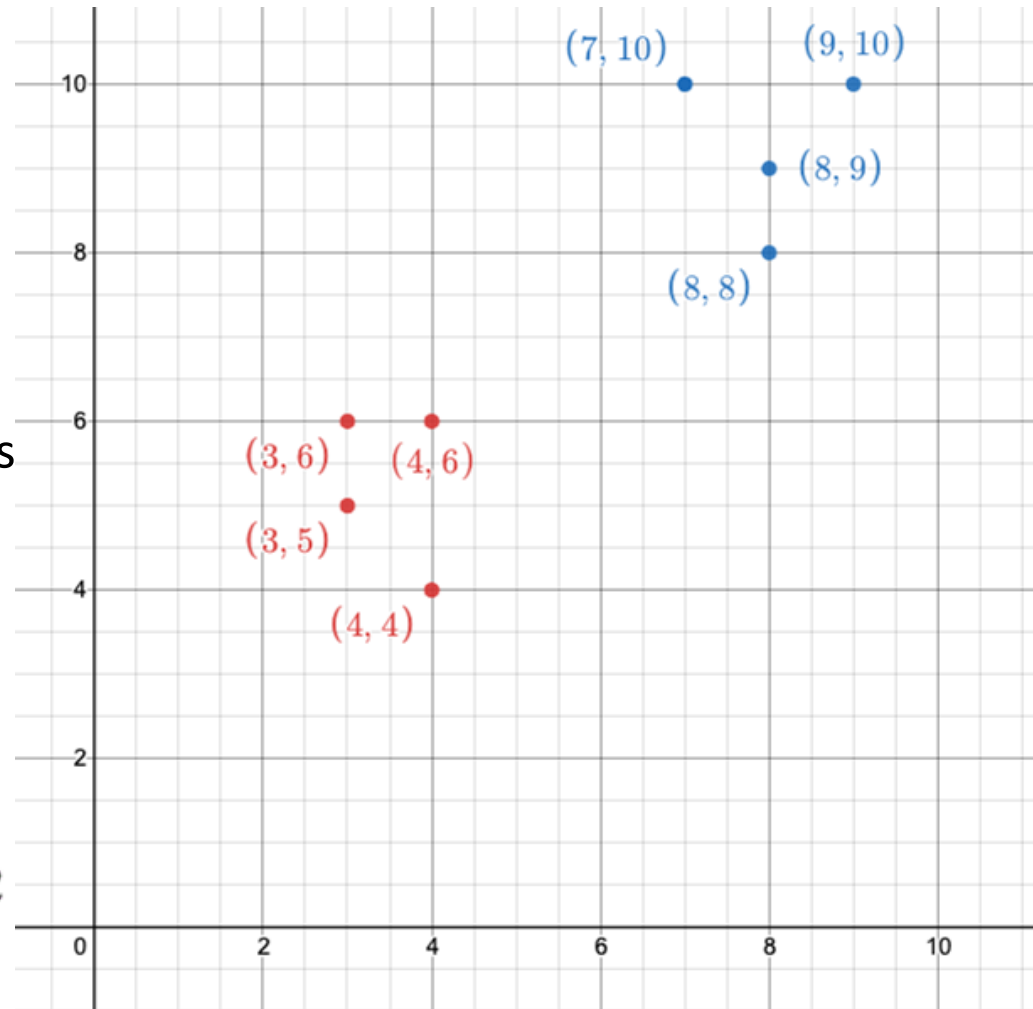
Working algorithm for kNN Regression



- Upload the production data into memory space
- Decide the distance measure
(we will choose Euclidean now)
- Decide the k value
(we will choose k = 4)
- For each data point in test data, find distance with respect to all neighbors and choose the nearest 4 neighbors and assign the prediction as **mean** of the neighbors.
- Calculate the error using MSE.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n \underbrace{(y_i - \hat{y}_i)}_{\text{predicted value} - \text{actual value}}^2$$

test set predicted value actual value



X	Y	value
3	5	3.5
3	6	2.6
4	6	2.8
4	4	3.1
7	10	8.7
8	9	8.5
8	8	9.3
9	10	9.7

X	Y	value
5	5	3.7
7	8	8.4

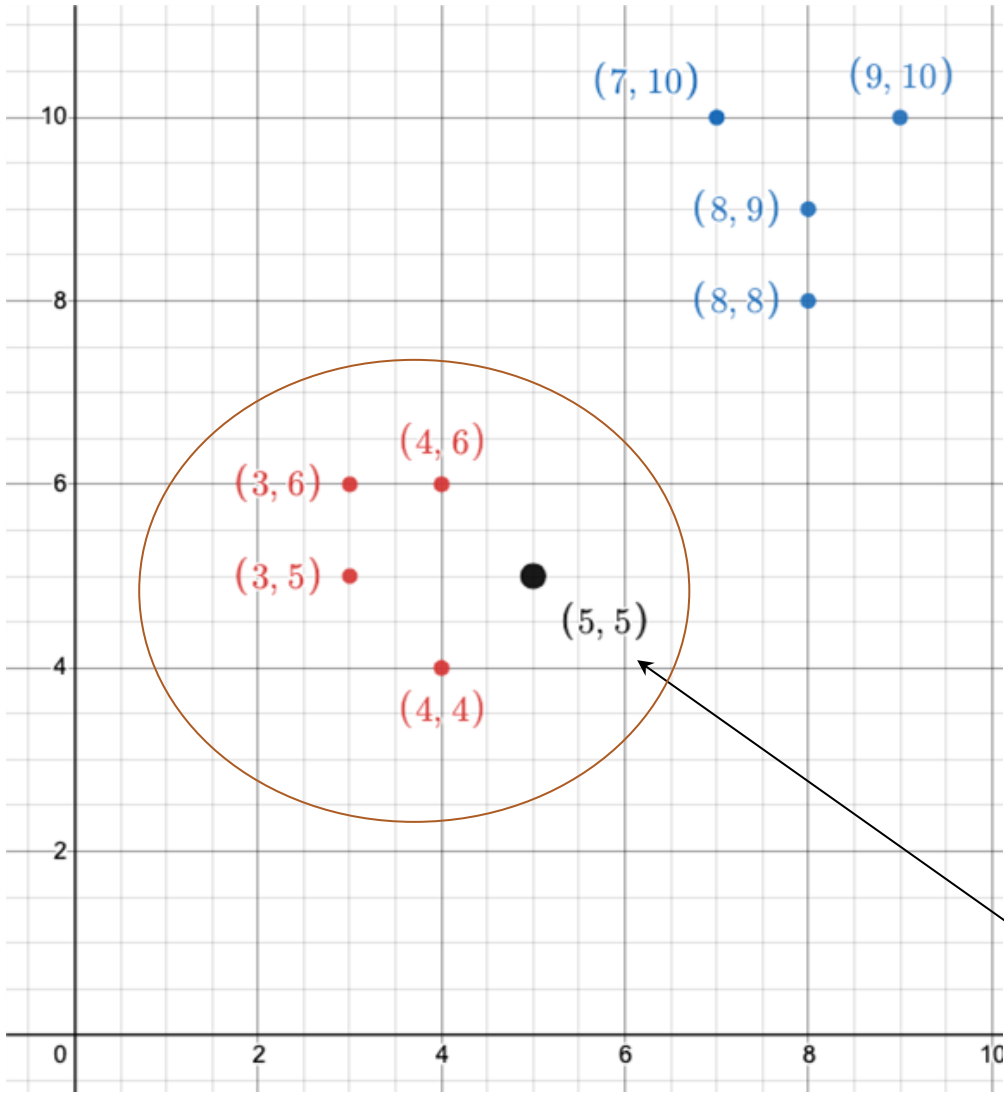
Machine Learning

Working algorithm for kNN Regression



$$\hat{y} = \frac{3.5 + 2.6 + 2.8 + 3.1}{4} = 3.0$$
$$e_1 = (3.7 - 3.0)^2 = 0.49$$

e1	0.49



X	Y	value
3	5	3.5
3	6	2.6
4	6	2.8
4	4	3.1
7	10	8.7
8	9	8.5
8	8	9.3
9	10	9.7

X	Y	value
5	5	3.7
7	8	8.4

Machine Learning

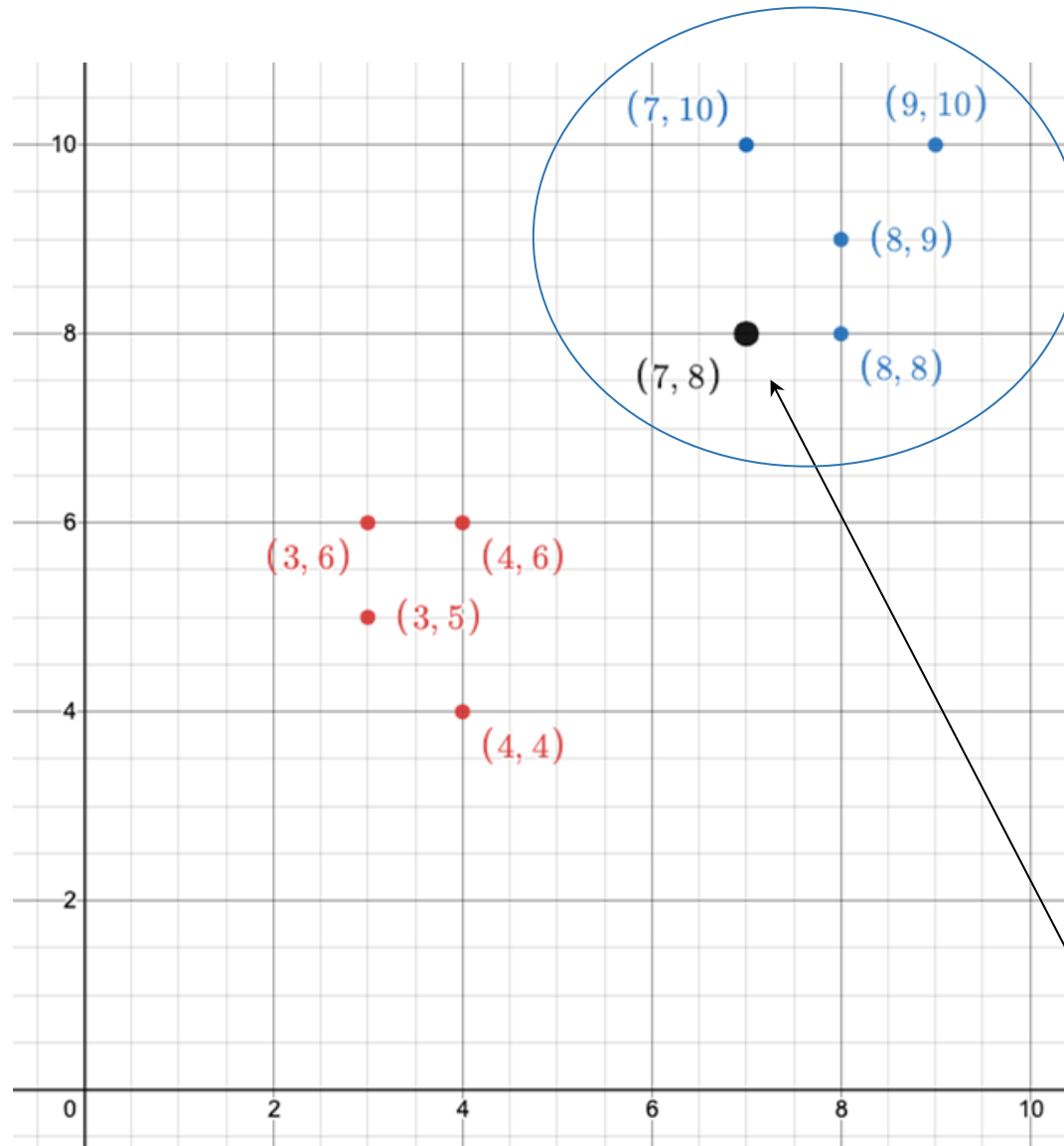
Working algorithm for kNN Regression



$$\hat{y} = \frac{8.7 + 8.5 + 9.3 + 9.7}{4} = 9.05$$

$$e_2 = (8.4 - 9.05)^2 = 0.4225$$

e1	0.49
e2	0.4225



X	Y	value
3	5	3.5
3	6	2.6
4	6	2.8
4	4	3.1
7	10	8.7
8	9	8.5
8	8	9.3
9	10	9.7
X	Y	value
5	5	3.7
7	8	8.4

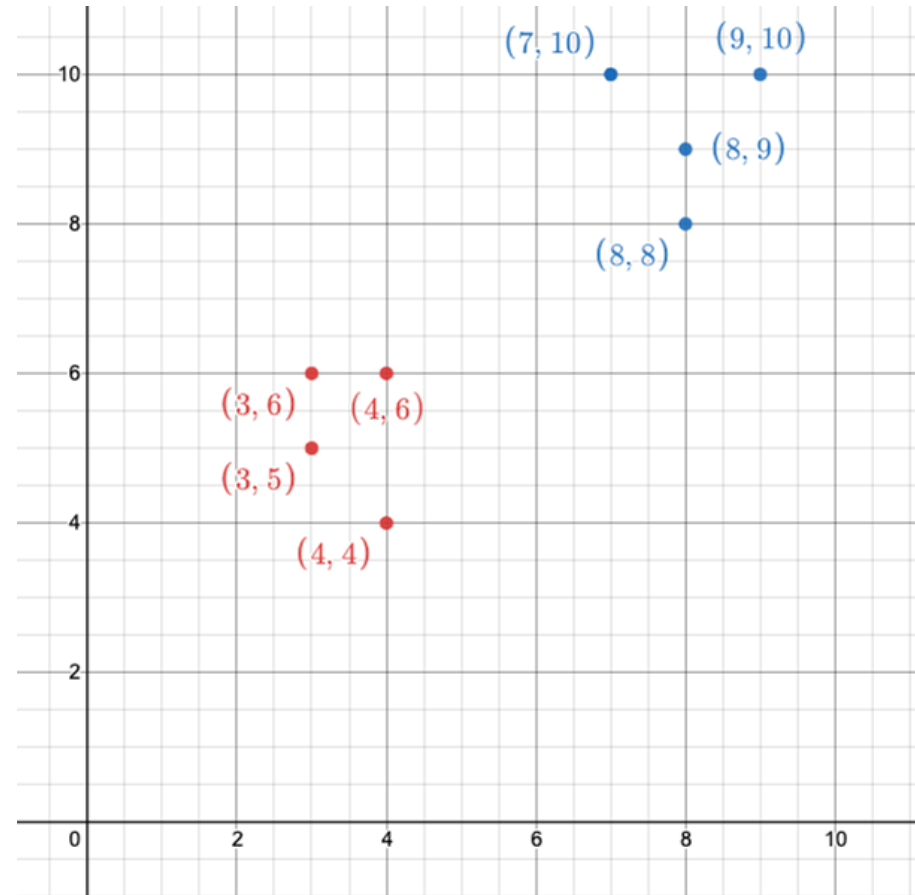
Machine Learning

Working algorithm for kNN Regression



e1	0.49
e2	0.4225

$$\text{total error} = \frac{1}{2}(e1 + e2) = \frac{0.49 + 0.4225}{2}$$
$$= 0.45625$$

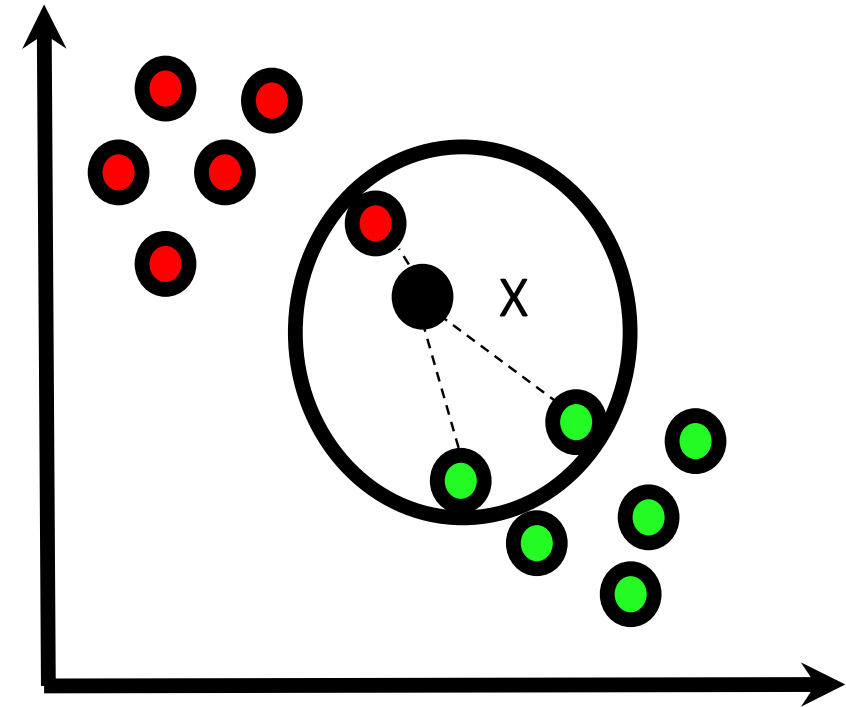


X	Y	value
3	5	3.5
3	6	2.6
4	6	2.8
4	4	3.1
7	10	8.7
8	9	8.5
8	8	9.3
9	10	9.7

X	Y	value
5	5	3.7
7	8	8.4

Let closer points have more influence

- Consider the given 2-D plot of a data set where the label is binary.
- Consider the query point x_q
- With vanilla kNN where $K = 3$:
 - --- x will be classified as green, when it's more likely to be red.



Machine Learning

Distance weighted kNN



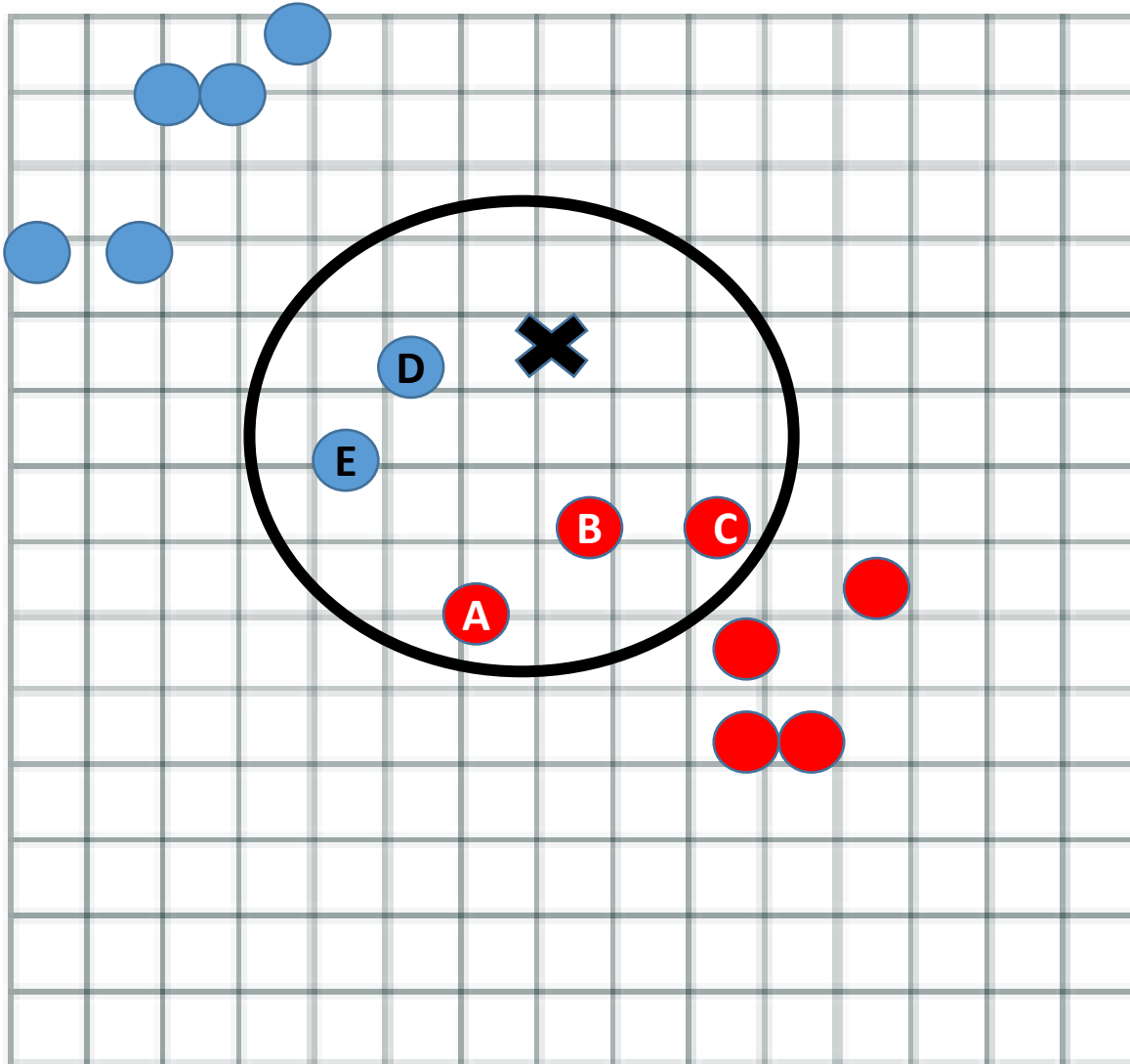
- Weigh the contribution of each of the k neighbour according to their distance to the query x_q
- Give greater weight to closer neighbour.

$$\hat{f}(x_q) \leftarrow \operatorname{argmax}_{v \in V} \sum_{i=1}^k w_i \delta(v, f(x_i))$$

$$w_i = \frac{1}{\operatorname{distance}(X_q, X_i)^m}$$

where m depends on how much you want to penalise the points that are far away from the query point.

where $\delta(a, b) = 1$ if $a = b$ and where $\delta(a, b) = 0$ otherwise.



- Consider the given plot.
- Assume the label is binary and the data instances either belong to class red or class blue.
- Assume the query point is marked as X.
- Let's say with $k = 5$ we get A, B, C, D, E as the closest points.

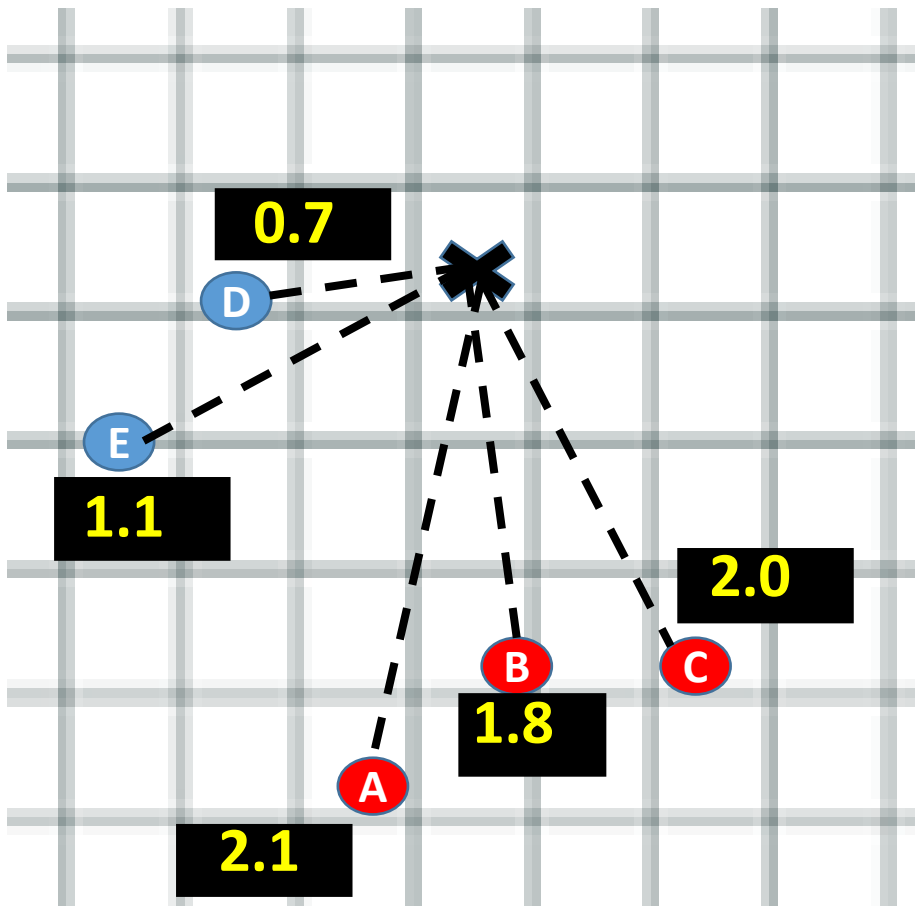
Question : What will the vanilla kNN classify the query point as:

Answer: Red

Machine Learning

An easy example of weighted KNN

- Let's say with $k = 5$ we get A, B, C, D, E as closest points with distances as depicted in the plot :



$$\begin{aligned}\text{Support for red} &= (1 / 2.1) * 1 + (1 / 1.8) * 1 + (1 / 2.0) * 1 \\ &= 0.476 + 0.556 + 0.5 \\ &= 1.532\end{aligned}$$

$$\begin{aligned}\text{Support for blue} &= (1 / 1.1) * 1 + (1 / 0.7) * 1 \\ &= 0.909 + 1.428 \\ &= 2.337\end{aligned}$$

$$\begin{aligned}\text{estimated class for X} &= \max(\text{red}-1.532, \text{blue}-2.337) \\ &= \text{blue}\end{aligned}$$

Advantage & Challenges of kNN

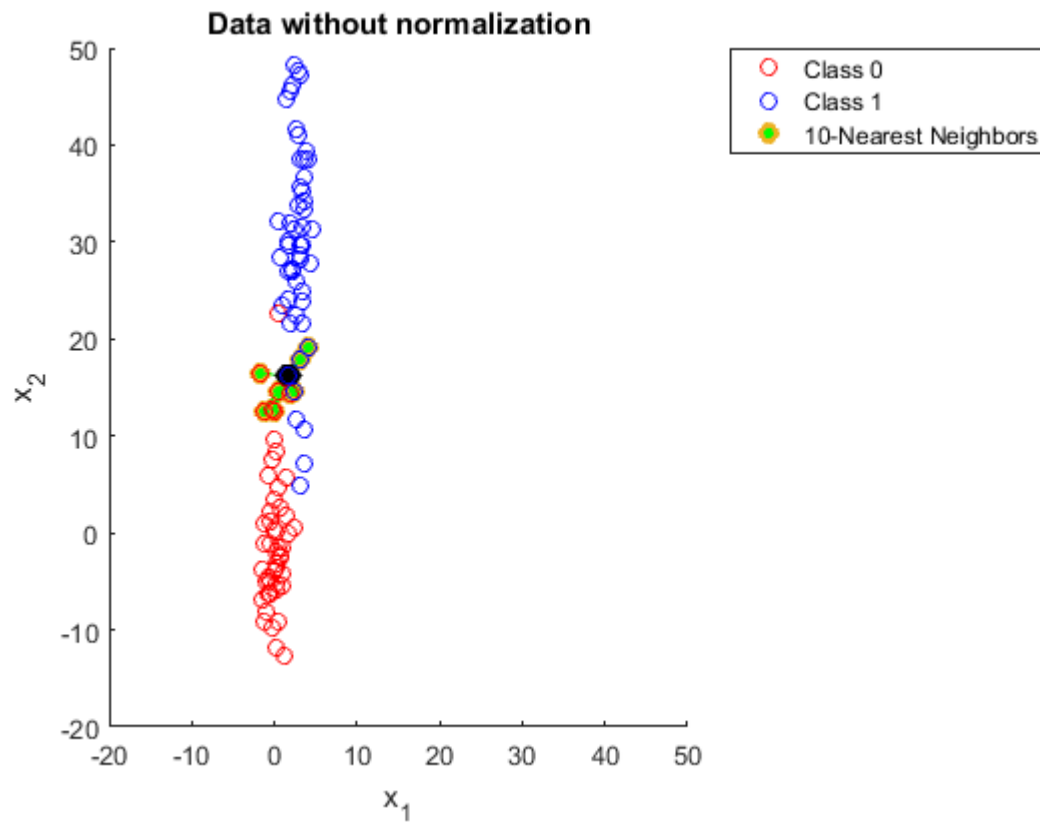
- Requires no training before making predictions. New data can be added seamlessly. It will not impact the accuracy of the algorithm.
- Only two parameters required i.e. k and the distance measure.
- Versatile - useful for both classification and regression.

- **Time**
Prediction is computationally expensive as we need to compute the distance between the query point and all other points. (N is in 1000s)
- **Space**
High memory requirement - lazy algorithm which stores all of the training data. (N is in 1000s)
- Does not work well with large datasets. Cost of calculating distance between the new point and each existing point is high.
- **It is sensitive to scaling.**
- The curse of dimensionality.

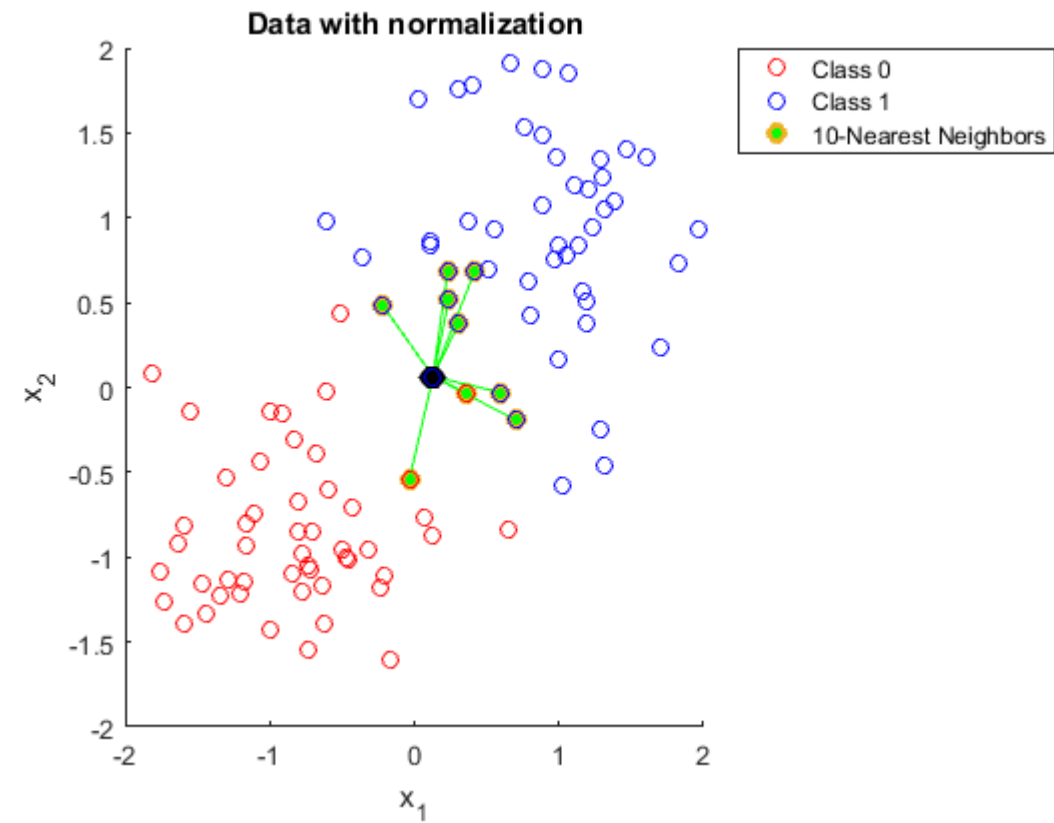
Machine Learning

Why do we scale?

Can you identify the nearest neighbors?



How about now?



Machine Learning

Effect of normalizing



Pre normalizing

Euclidean distance between 1 and 2=

$$[(100000 - 80000)^2 + (30 - 25)^2]^{1/2} = \mathbf{20000}$$

- High magnitude of income affected the distance between the two points.
- This will impact the performance as higher weightage is given to variables with higher magnitude.

ID	Age	Income(rupees)
1	25	80,000
2	30	100,000
3	40	90,000
4	30	50,000
5	40	110,000

Machine Learning

Effect of normalizing



Pre normalizing

Euclidean distance between 1 and 2=

$$[(100000 - 80000)^2 + (30 - 25)^2]^{1/2} = \mathbf{20000}$$

How to normalize?

$$x_i = \frac{x_i - \mu}{\sigma}$$

μ = Mean

σ = Standard Deviation

ID	Age	Income(rupees)
1	25	80,000
2	30	100,000
3	40	90,000
4	30	50,000
5	40	110,000

here,

$$\mu_{age} = 33, \sigma_{age} = 6$$

$$\mu_{income} = 86000, \sigma_{income} = 20591.26$$

Machine Learning

Effect of normalizing



Pre normalizing

Euclidean distance between 1 and 2=

$$[(100000 - 80000)^2 + (30 - 25)^2]^{1/2} = \mathbf{20000}$$

Post normalizing

Euclidean distance between 1 and 2 =

$$[(0.608 + 0.260)^2 + (-0.447 + 1.192)^2]^{1/2} = \mathbf{1.14}$$

- Distance is not biased towards the income variable anymore.
- Similar weightage given to both the variables.

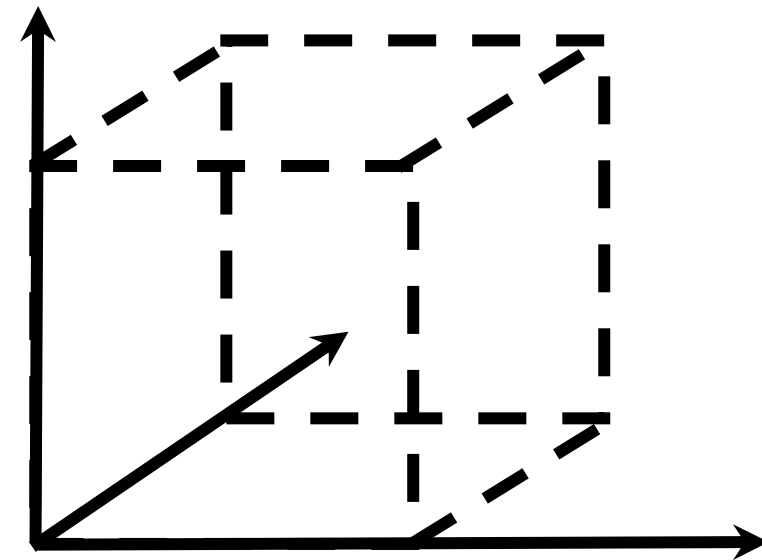
ID	Age	Income(rupees)
1	-1.192	-0.260
2	-0.447	0.608
3	1.043	0.173
4	-0.447	-1.563
5	1.043	1.042

Machine Learning

The curse of dimensionality



- The kNN classifier makes the assumption that similar points share similar labels.
- Unfortunately, in high dimensional spaces, points that are drawn from a probability distribution, tend to never be close together.
- We can illustrate this on a simple example.
- We will draw points uniformly at random within the unit cube.
- We will investigate how much space the k nearest neighbors of a test point inside this cube will take up.

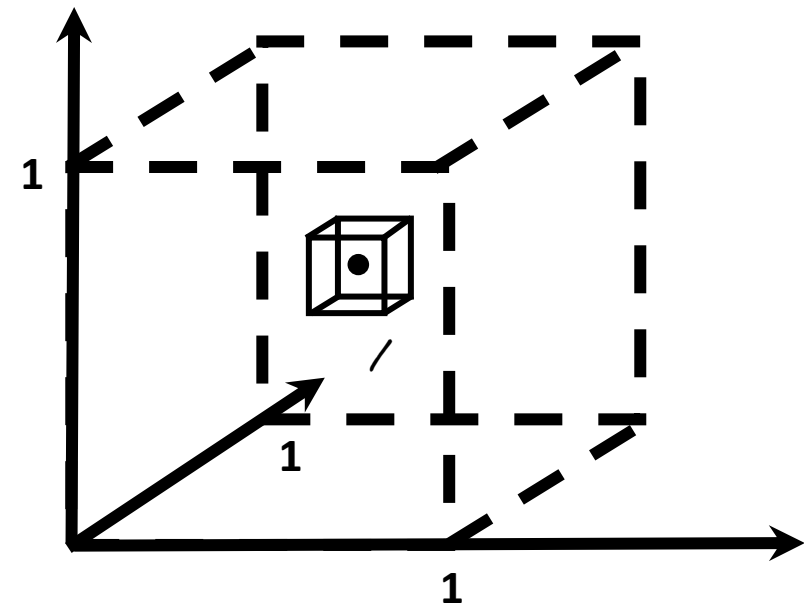


Machine Learning

The curse of dimensionality

- Formally, imagine the unit cube $[0,1]^d$
- All training data is sampled uniformly within this cube, i.e. $\forall i, x_i \in [0,1]^d$, and we are considering the **k=10** nearest neighbors of such a test point.
- Let ℓ be the edge length of the smallest hyper-cube that contains all k-nearest neighbor of a test point.
- Then $\ell^d \approx k/n$ and $\ell \approx (k/n)^{1/d}$
- If $n=1000$ (# data instances), how big is ℓ ?
- So as $d \gg 0$ almost the entire space is needed to find the 10-NN
- This breaks down the k-NN assumptions, because the k-NN are not particularly closer (and therefore more similar) than any other data points in the training set.
- Why would the test point share the label with those k-nearest neighbors, if they are not actually similar to it?

d	ℓ
2	0.1
10	0.63
100	0.955
1000	0.9954



Machine Learning

The curse of dimensionality

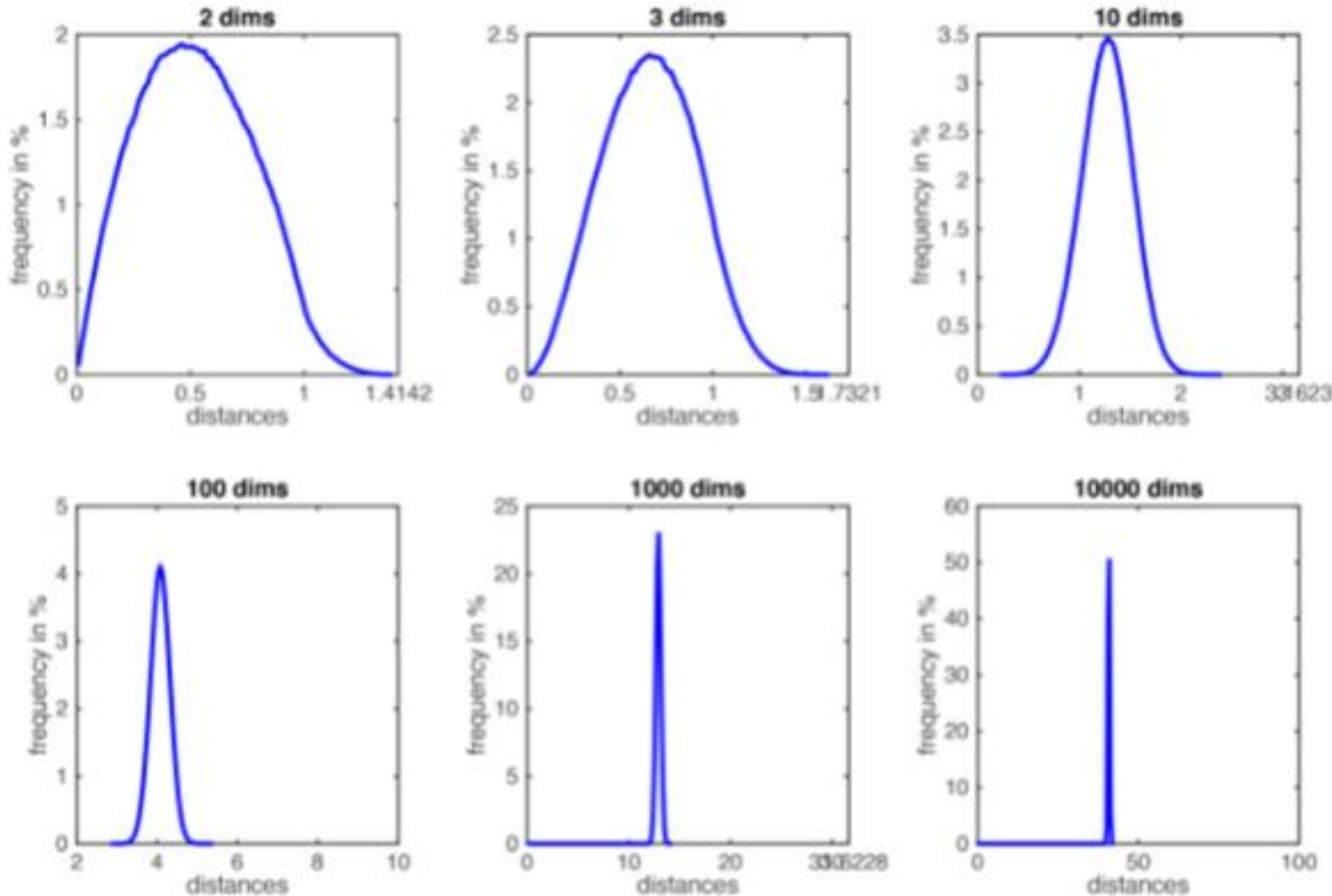


Figure demonstrating "the curse of dimensionality". The histogram plots show the distributions of all pairwise distances between randomly distributed points within d -dimensional unit squares. As the number of dimensions d grows, all distances concentrate within a very small range.

- One might think that one rescue could be to increase the number of training samples, n , until the nearest neighbors are truly close to the test point.
- How many data points would we need such that ℓ becomes truly small?
- Fix $\ell=1/10=0.1$
- $n=k/\ell^d=k*10^d$, which grows exponentially!
- For $d>100$ we would need far more data points than there are electrons in the universe...

- Assigning weights to the attributes when calculating distances. Let's say we're predicting the price of a house, we give higher weights to features like area and locality when compared to color.
- Iteratively, we leave out one of the attributes and test the algorithm. The exercise can then lead us to the best set of attributes.
- Dimensionality reduction using techniques like PCA.

How to handle different types of attributes?

- In case of Boolean values, then convert to 0 and 1.
- Non-binary characterizations:
 - Natural order among the data, then convert to numerical values based on order.
e.g. Educational attainment: HS, College, MS, PhD -> 1, 2, 3, 4
 - No order and > 1 category, then convert to one hot encoding.
e.g. Animals: Cat, Dog, Zebra -> (1, 0, 0), (0, 1, 0), (0, 0, 1)

	Cat	Dog	Zebra
	1	0	0
	0	1	0
	0	0	1

Let us say we have the following

Education	Place	Gender	Eligibility
High school	Banglore	M	Yes
College	Udupi	F	No
Phd	Mandya	M	No
Phd	Mandya	M	Yes
College	Udupi	F	No
High School	Mandya	M	No
Phd	Udupi	M	Yes
College	Banglore	F	Yes
Phd	Banglore	F	Yes
High School	Mandya	M	No

- Education, Place and Gender are attributes
- Eligibility is the target
- Education has natural order that is High school < College < Phd
- Place needs to be one-hot encoded
- Gender is binary

Machine Learning

How to handle categorical features?



The converted data looks something like this

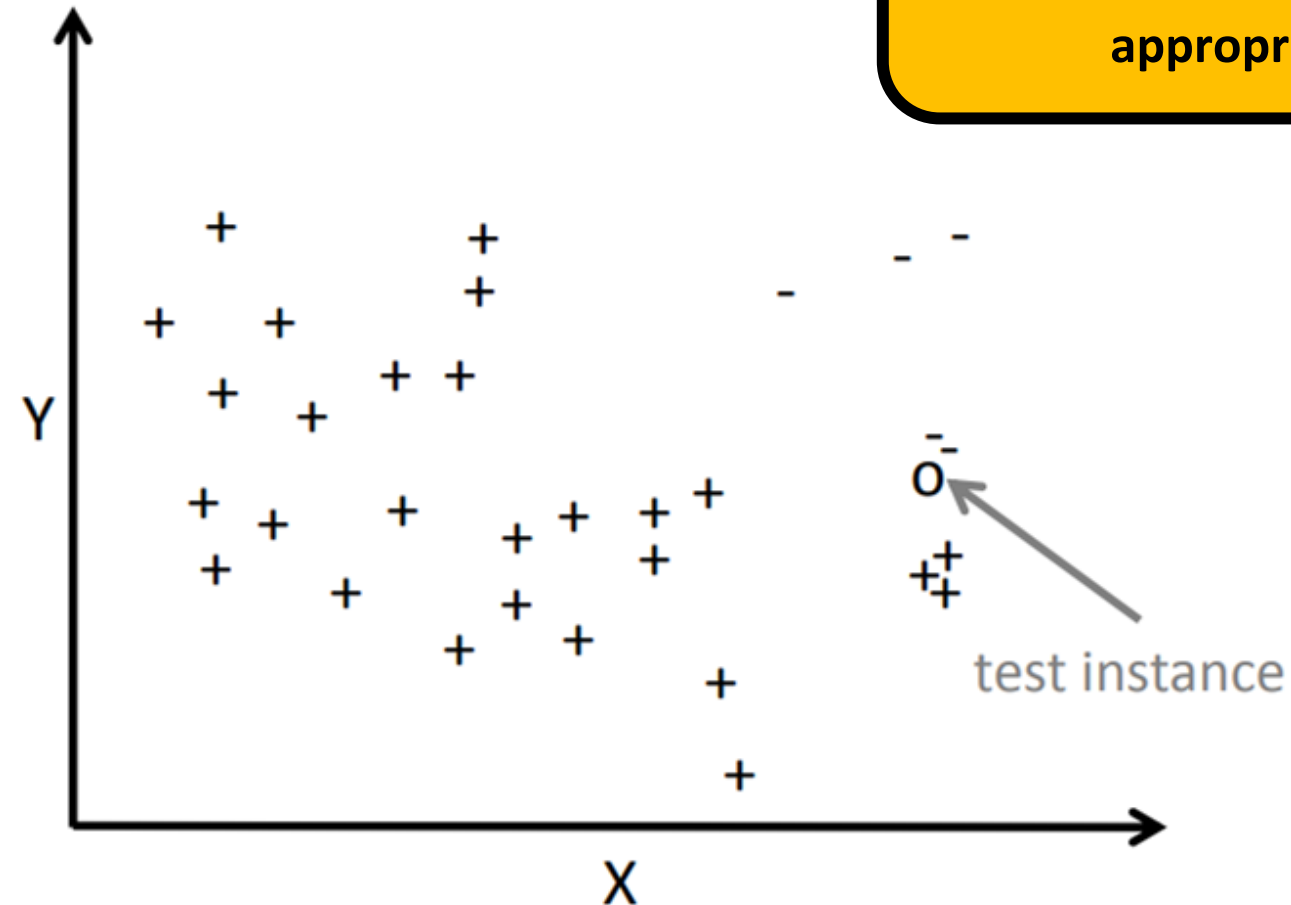
Education	Place	Gender	Eligibility	Education	P1	P2	Gender	Eligibility
High school	Banglore	M	Yes	1	0	0	1	Yes
College	Udupi	F	No	2	0	1	0	No
Phd	Mandya	M	No	3	1	0	1	No
Phd	Mandya	M	Yes	3	1	0	1	Yes
College	Udupi	F	No	2	0	1	0	No
High School	Mandya	M	No	1	1	0	1	No
Phd	Udupi	M	Yes	3	0	1	1	Yes
College	Banglore	F	Yes	2	0	0	0	Yes
Phd	Banglore	F	Yes	3	0	0	0	Yes
High School	Mandya	M	No	1	1	0	1	No

The converted data looks something like this

Education	P1	P2	Gender	Eligibility
1	0	0	1	Yes
2	0	1	0	No
3	1	0	1	No
3	1	0	1	Yes
2	0	1	0	No
1	1	0	1	No
3	0	1	1	Yes
2	0	0	0	Yes
3	0	0	0	Yes
1	1	0	1	No

- In Education 1,2,3 represents high school college and Phd respectively
- Place which had three types of value is broken into two attribute p1 and p2 where $(p1,p2)=(0,0)$ represents bangalore, $(p1,p2)=(0,1)$ represents Udupi and $(p1,p2)=(1,0)$ represents Mandya
- Since Gender is binary here , we can give 0 to Female and 1 to Male or the other way too

Setting K to a large value seems a good idea .
We get more votes!
For this dataset we set k value to 11 .Choose the most appropriate comment on this decision



A good idea since ,the dataset is not too big ,considering more neighbour would help

Not a good idea since class of a data point is determined more by its most nearest neighbour

Not a good decision,since there are 5 negative instances in the training set.Therefore $K > 10$ would have a majority of positive

None of the above

Machine Learning

Quiz



$$F(x, y) = \sum_{i=1}^n w_i f_i$$

where n is the number of points used to interpolate, f_i are the prescribed function values at the points (e.g., the dataset values), and w_i are the weight functions assigned to each point. The classical form of the weight function is:

$$w_i = \frac{h_i^{-p}}{\sum_{j=1}^n h_j^{-p}}$$

Do you think that the following is a good way to calculate the weights in weighted KNN

True

False

Which of the following statement is true about k-NN algorithm?

1. k-NN performs much better if all of the data have the same scale
- 2 .k-NN works well with a small number of input variables (p), but struggles when the number of inputs is very large
3. k-NN makes no assumptions about the functional form of the problem being solved

1 and 2

1 and 3

Only 1

All of the above

The above mentioned statements are assumptions of kNN algorithm

When you find noise in data which of the following option would you consider in k-NN?

I will increase the value of k

I will decrease the value of k

Noise can not be dependent on value of k

None of these

To be more sure of which classifications you make, you can try increasing the value of k.



UE24CS352A: Machine Learning

Dr. Shylaja S S

Department of Computer Science & Engineering

shylaja.sharath@pes.edu